K.C. Wang

Embedded and Real-Time Operating Systems

Springer

K.C. Wang
School of Electrical Engineering
and Computer Science
Washington State University
Pullman, WA
USA

ISBN 978-3-319-51516-8 ISBN 978-3-319-51517-5 (eBook)
DOI 10.1007/978-3-319-51517-5

Library of Congress Control Number: 2016961664

© Springer International Publishing AG 2017
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Printed on acid-free paper

This Springer imprint is published by Springer Nature
The registered company is Springer International Publishing AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Contents

1 Introduction 1
1.1 About This Book 1
1.2 Motivations of This Book 1
1.3 Objective and Intended Audience 1
1.4 Unique Features of This Book 2
1.5 Book Contents 3
1.6 Use This Book as a Textbook for Embedded Systems 4
1.7 Use This Book as a Textbook for Operating Systems 5
1.8 Use This Book for Self-study 5
References 5

2 ARM Architecture and Programming 7
2.1 ARM Processor Modes 8
2.2 ARM CPU Registers 8
2.2.1 General Registers 8
2.2.2 Status Registers 9
2.2.3 Change ARM Processor Mode 9
2.3 Instruction Pipeline 10
2.4 ARM Instructions 10
2.4.1 Condition Flags and Conditions 10
2.4.2 Branch Instructions 11
2.4.3 Arithmetic Operations 12
2.4.4 Comparison Operations 12
2.4.5 Logical Operations 12
2.4.6 Data Movement Operations 13
2.4.7 Immediate Value and Barrel Shifter 13
2.4.8 Multiply Instructions 14
2.4.9 LOAD and Store Instructions 14
2.4.10 Base Register 14
2.4.11 Block Data Transfer 14
2.4.12 Stack Operations 14
2.4.13 Stack and Subroutines 15
2.4.14 Software Interrupt (SWI) 15
2.4.15 PSR Transfer Instructions 15
2.4.16 Coprocessor Instructions 15
2.5 ARM Toolchain 16
2.6 ARM System Emulators 16
2.7 ARM Programming 17
2.7.1 ARM Assembly Programming Example 1 17
2.7.2 ARM Assembly Programming Example 2 19
2.7.3 Combine Assembly with C Programming 20

2.8 Device Drivers . . . 27
2.8.1 System Memory Map . . . 27
2.8.2 GPIO Programming . . . 27
2.8.3 UART Driver for Serial I/O . . . 29
2.8.4 Color LCD Display Driver . . . 33
2.9 Summary . . . 45
References . . . 46

3 Interrupts and Exceptions Processing . . . 47
3.1 ARM Exceptions . . . 47
3.1.1 ARM Processor Modes . . . 47
3.1.2 ARM Exceptions . . . 47
3.1.3 Exceptions Vector Table . . . 49
3.1.4 Exception Handlers . . . 49
3.1.5 Return from Exception Handlers . . . 50
3.2 Interrupts and Interrupts Processing . . . 51
3.2.1 Interrupt Types . . . 51
3.2.2 Interrupt Controllers . . . 51
3.2.3 Primary and Secondary Interrupt Controllers . . . 53
3.3 Interrupt Processing . . . 54
3.3.1 Vector Table Contents . . . 54
3.3.2 Hardware Interrupt Sequence . . . 54
3.3.3 Interrupts Control in Software . . . 54
3.3.4 Interrupt Handlers . . . 55
3.3.5 Non-nested Interrupt Handler . . . 55
3.4 Timer Driver . . . 56
3.4.1 ARM Versatile 926EJS Timers . . . 56
3.4.2 Timer Driver Program . . . 57
3.5 Keyboard Driver . . . 61
3.5.1 ARM PL050 Mouse-Keyboard Interface . . . 61
3.5.2 Keyboard Driver . . . 62
3.5.3 Interrupt-Driven Driver Design . . . 62
3.5.4 Keyboard Driver Program . . . 63
3.6 UART Driver . . . 67
3.6.1 ARM PL011 UART Interface . . . 67
3.6.2 UART Registers . . . 68
3.6.3 Interrupt-Driven UART Driver Program . . . 69
3.7 Secure Digital (SD) Cards . . . 73
3.7.1 SD Card Protocols . . . 74
3.7.2 SD Card Driver . . . 74
3.7.3 Improved SDC Driver . . . 80
3.7.4 Multi-sector Data Transfer . . . 82
3.8 Vectored Interrupts . . . 84
3.8.1 ARM PL190 Vectored Interrupts Controller (VIC) . . . 84
3.8.2 Configure VIC for Vectored Interrupts . . . 85
3.8.3 Vectored Interrupts Handlers . . . 86
3.8.4 Demonstration of Vectored Interrupts . . . 86
3.9 Nested Interrupts . . . 87
3.9.1 Why Nested Interrupts . . . 87
3.9.2 Nested Interrupts in ARM . . . 88
3.9.3 Handle Nested Interrupts in SYS Mode . . . 88
3.9.4 Demonstration of Nested Interrupts . . . 89

3.10 Nested Interrupts and Process Switch . . . 92
3.11 Summary . . . 92
References . . . 94

4 Models of Embedded Systems . . . 95
4.1 Program Structures of Embedded Systems . . . 95
4.2 Super-Loop Model . . . 95
4.2.1 Super-Loop Program Examples . . . 96
4.3 Event-Driven Model . . . 97
4.3.1 Shortcomings of Super-Loop Programs . . . 97
4.3.2 Events . . . 97
4.3.3 Periodic Event-Driven Program . . . 98
4.3.4 Asynchronous Event-Driven Program . . . 101
4.4 Event Priorities . . . 103
4.5 Process Models . . . 103
4.5.1 Uniprocessor Process Model . . . 103
4.5.2 Multiprocessor Process Model . . . 103
4.5.3 Real Address Space Process Model . . . 103
4.5.4 Virtual Address Space Process Model . . . 103
4.5.5 Static Process Model . . . 104
4.5.6 Dynamic Process Model . . . 104
4.5.7 Non-preemptive Process Model . . . 104
4.5.8 Preemptive Process Model . . . 104
4.6 Uniprocessor (UP) Kernel Model . . . 104
4.7 Uniprocessor (UP) Operating System Model . . . 104
4.8 Multiprocessor (MP) System Model . . . 105
4.9 Real-Time (RT) System Model . . . 105
4.10 Design Methodology of Embedded System Software . . . 105
4.10.1 High-Level Language Support for Event-Driven Programming . . . 105
4.10.2 State Machine Model . . . 105
4.10.3 StateChart Model . . . 110
4.11 Summary . . . 110
References . . . 111

5 Process Management in Embedded Systems . . . 113
5.1 Multitasking . . . 113
5.2 The Process Concept . . . 113
5.3 Multitasking and Context Switch . . . 114
5.3.1 A Simple Multitasking Program . . . 114
5.3.2 Context Switching . . . 116
5.3.3 Demonstration of Multitasking . . . 120
5.4 Dynamic Processes . . . 121
5.4.1 Dynamic Process Creation . . . 121
5.4.2 Demonstration of Dynamic Processes . . . 124
5.5 Process Scheduling . . . 124
5.5.1 Process Scheduling Terminology . . . 124
5.5.2 Goals, Policy and Algorithms of Process Scheduling . . . 125
5.5.3 Process Scheduling in Embedded Systems . . . 125
5.6 Process Synchronization . . . 126
5.6.1 Sleep and Wakeup . . . 126
5.6.2 Device Drivers Using Sleep/Wakeup . . . 127

5.7 Event-Driven Embedded Systems Using Sleep/Wakeup 129
5.7.1 Demonstration of Event-Driven Embedded System Using Sleep/Wakeup 131
5.8 Resource Management Using Sleep/Wakeup 132
5.8.1 Shortcomings of Sleep/Wakeup. 133
5.9 Semaphores 133
5.10 Applications of Semaphores 134
5.10.1 Semaphore Lock 134
5.10.2 Mutex lock. 135
5.10.3 Resource Management using Semaphore 135
5.10.4 Wait for Interrupts and Messages 135
5.10.5 Process Cooperation 135
5.10.6 Advantages of Semaphores. 136
5.10.7 Cautions of Using Semaphores 136
5.10.8 Use Semaphores in Embedded Systems 137
5.11 Other Synchronization Mechanisms. 139
5.11.1 Event Flags in OpenVMS 140
5.11.2 Event Variables in MVS 140
5.11.3 ENQ/DEQ in MVS 141
5.12 High-Level Synchronization Constructs 141
5.12.1 Condition Variables. 141
5.12.2 Monitors 142
5.13 Process Communication. 142
5.13.1 Shared Memory 142
5.13.2 Pipes 142
5.13.3 Signals 147
5.13.4 Message Passing 147
5.14 Uniprocessor (UP) Embedded System Kernel 152
5.14.1 Non-preemptive UP Kernel 152
5.14.2 Demonstration of Non-preemptive UP Kernel. 158
5.14.3 Preemptive UP Kernel 158
5.14.4 Demonstration of Preemptive UP Kernel 164
5.15 Summary 165
References. 167

6 Memory Management in ARM 169
6.1 Process Address Spaces 169
6.2 Memory Management Unit (MMU) in ARM 169
6.3 MMU Registers 170
6.4 Accessing MMU Registers. 171
6.4.1 Enabling and Disabling the MMU. 171
6.4.2 Domain Access Control 172
6.4.3 Translation Table Base Register 173
6.4.4 Domain Access Control Register. 173
6.4.5 Fault Status Registers 173
6.4.6 Fault Address Register. 174
6.5 Virtual Address Translations. 174
6.5.1 Translation Table Base 175
6.5.2 Translation Table 175
6.5.3 Level-One Descriptor 175
6.6 Translation of Section References 176

6.7 Translation of Page References . . . 176
6.7.1 Level-1 Page Table . . . 176
6.7.2 Level-2 Page Descriptors . . . 176
6.7.3 Translation of Small Page References . . . 177
6.7.4 Translation of Large Page References . . . 178
6.8 Memory Management Example Programs . . . 178
6.8.1 One-Level Paging Using 1 MB Sections . . . 178
6.8.2 Two-Level Paging Using 4 KB Pages . . . 183
6.8.3 One-Level Paging with High VA Space . . . 184
6.8.4 Two-Level Paging with High VA Space . . . 189
6.9 Summary . . . 190
Reference . . . 191

7 User Mode Process and System Calls . . . 193
7.1 User Mode Processes . . . 193
7.2 Virtual Address Space Mapping . . . 193
7.3 User Mode Process . . . 194
7.3.1 User Mode Image . . . 194
7.4 System Kernel Supporting User Mode Processes . . . 197
7.4.1 PROC Structure . . . 197
7.4.2 Reset Handler . . . 201
7.4.3 Kernel Code . . . 203
7.4.4 Kernel Compile-Link Script . . . 209
7.4.5 Demonstration of Kernel with User Mode Process . . . 209
7.5 Embedded System with User Mode Processes . . . 209
7.5.1 Processes in the Same Domain . . . 209
7.5.2 Demonstration of Processes in the Same Domain . . . 210
7.5.3 Processes with Individual Domains . . . 212
7.5.4 Demonstration of Processes with Individual Domains . . . 212
7.6 RAM Disk . . . 213
7.6.1 Creating RAM Disk Image . . . 215
7.6.2 Process Image File Loader . . . 216
7.7 Process Management . . . 218
7.7.1 Process Creation . . . 218
7.7.2 Process Termination . . . 219
7.7.3 Process Family Tree . . . 220
7.7.4 Wait for Child Process Termination . . . 220
7.7.5 Fork-Exec in Unix/Linux . . . 221
7.7.6 Implementation of Fork . . . 222
7.7.7 Implementation of Exec . . . 223
7.7.8 Demonstration of Fork-Exec . . . 225
7.7.9 Simple sh for Command Execution . . . 226
7.7.10 vfork . . . 228
7.7.11 Demonstration of vfork . . . 230
7.8 Threads . . . 231
7.8.1 Thread Creation . . . 231
7.8.2 Demonstration of Threads . . . 233
7.8.3 Threads Synchronization . . . 234
7.9 Embedded System with Two-Level Paging . . . 234
7.9.1 Static 2-Level Paging . . . 236
7.9.2 Demonstration of Static 2-Level Paging . . . 237
7.9.3 Dynamic 2-Level Paging . . . 238
7.9.4 Demonstration of 2-Level Dynamic Paging . . . 240

7.10 KMH Memory Mapping 241
7.10.1 KMH Using One-Level Static Paging 241
7.10.2 KMH Using Two-Level Static Paging 244
7.10.3 KMH Using Two-Level Dynamic Paging 245
7.11 Embedded System Supporting File Systems 245
7.11.1 Create SD Card Image 246
7.11.2 Format SD Card Partitions as File Systems 247
7.11.3 Create Loop Devices for SD Card Partitions 247
7.12 Embedded System with SDC File System 248
7.12.1 SD Card Driver Using Semaphore 248
7.12.2 System Kernel Using SD File System 252
7.12.3 Demonstration of SDC File System 253
7.13 Boot Kernel Image from SDC 253
7.13.1 SDC Booter Program 254
7.13.2 Demonstration of Booting Kernel from SDC 258
7.13.3 Booting Kernel from SDC with Dynamic Paging 259
7.13.4 Two-Stage Booting 260
7.13.5 Demonstration of Two-Stage Booting 261
7.14 Summary 262
References 264

8 General Purpose Embedded Operating Systems 265
8.1 General Purpose Operating Systems 265
8.2 Embedded General Purpose Operating Systems 265
8.3 Porting Existing GPOS to Embedded Systems 265
8.4 Develop an Embedded GPOS for ARM 266
8.5 Organization of EOS 266
8.5.1 Hardware Platform 266
8.5.2 EOS Source File Tree 267
8.5.3 EOS Kernel Files 267
8.5.4 Capabilities of EOS 268
8.5.5 Startup Sequence of EOS 268
8.5.6 Process Management in EOS 269
8.5.7 Assembly Code of EOS 271
8.5.8 Kernel Files of EOS 279
8.5.9 Process Management Functions 282
8.5.10 Pipes 282
8.5.11 Message Passing 283
8.5.12 Demonstration of Message Passing 285
8.6 Memory Management in EOS 285
8.6.1 Memory Map of EOS 285
8.6.2 Virtual Address Spaces 285
8.6.3 Kernel Mode Pgdir and Page Tables 285
8.6.4 Process User Mode Page Tables 286
8.6.5 Switch Pgdir During Process Switch 286
8.6.6 Dynamic Paging 286
8.7 Exception and Signal Processing 288
8.7.1 Signal Processing in Unix/Linux 288
8.8 Signal Processing in EOS 289
8.8.1 Signals in PROC Resource 289
8.8.2 Signal Origins in EOS 289
8.8.3 Deliver Signal to Process 289
8.8.4 Change Signal Handler in Kernel 290

8.8.5 Signal Handling in EOS Kernel . . . 290
8.8.6 Dispatch Signal Catcher for Execution in User Mode . . . 291
8.9 Device Drivers . . . 291
8.10 Process Scheduling in EOS . . . 292
8.11 Timer Service in EOS . . . 292
8.12 File System . . . 294
8.12.1 File Operation Levels . . . 294
8.12.2 File I/O Operations . . . 296
8.12.3 EXT2 File System in EOS . . . 302
8.12.4 Implementation of Level-1 FS . . . 304
8.12.5 Implementation of Level-2 FS . . . 310
8.12.6 Implementation of Level-3 FS . . . 316
8.13 Block Device I/O Buffering . . . 318
8.14 I/O Buffer Management Algorithm . . . 318
8.14.1 Performance of the I/O Buffer Cache . . . 321
8.15 User Interface . . . 321
8.15.1 The INIT Program . . . 321
8.15.2 The Login Program . . . 322
8.15.3 The sh Program . . . 322
8.16 Demonstration of EOS . . . 323
8.16.1 EOS Startup . . . 323
8.16.2 Command Processing in EOS . . . 323
8.16.3 Signal and Exception Handling in EOS . . . 323
8.17 Summary . . . 326
References . . . 327

9 Multiprocessing in Embedded Systems . . . 329
9.1 Multiprocessing . . . 329
9.2 SMP System Requirements . . . 329
9.3 ARM MPcore Processors . . . 331
9.4 ARM Cortex-A9 MPcore Processor . . . 331
9.4.1 Processor Cores . . . 331
9.4.2 Snoop Control Unit (SCU) . . . 332
9.4.3 Generic Interrupt Controller (GIC) . . . 332
9.5 GIC Programming Example . . . 332
9.5.1 Configure the GIC to Route Interrupts . . . 332
9.5.2 Explanations of the GIC Configuration Code . . . 339
9.5.3 Interrupt Priority and Interrupt Mask . . . 340
9.5.4 Demonstration of GIC Programming . . . 340
9.6 Startup Sequence of ARM MPcores . . . 340
9.6.1 Raw Startup Sequence . . . 341
9.6.2 Booter Assisted Startup Sequence . . . 341
9.6.3 SMP Booting on Virtual Machines . . . 341
9.7 ARM SMP Startup Examples . . . 341
9.7.1 ARM SMP Startup Example 1 . . . 342
9.7.2 Demonstration of SMP Startup Example 1 . . . 346
9.7.3 ARM SMP Startup Example 2 . . . 347
9.7.4 Demonstration of ARM SMP Startup Example 2 . . . 348
9.8 Critical Regions in SMP . . . 349
9.8.1 Implementation of Critical Regions in SMP . . . 349
9.8.2 Shortcomings of XCHG/SWAP Operations . . . 350
9.8.3 ARM Synchronization Instructions for SMP . . . 350

9.9 Synchronization Primitives in SMP . . . 351
9.9.1 Spinlocks . . . 351
9.9.2 Spinlock Example . . . 352
9.9.3 Demonstration of SMP Startup with Spinlock . . . 353
9.9.4 Mutex in SMP . . . 353
9.9.5 Implementation of Mutex Using Spinlock . . . 354
9.9.6 Demonstration of SMP Startup with Mutex Lock . . . 355
9.10 Global and Local Timers . . . 357
9.10.1 Demonstration of Local Timers in SMP . . . 359
9.11 Semaphores in SMP . . . 359
9.11.1 Applications of Semaphores in SMP . . . 360
9.12 Conditional Locking . . . 361
9.12.1 Conditional Spinlock . . . 361
9.12.2 Conditional Mutex . . . 361
9.12.3 Conditional Semaphore Operations . . . 362
9.13 Memory Management in SMP . . . 362
9.13.1 Memory Management Models in SMP . . . 363
9.13.2 Uniform VA Spaces . . . 363
9.13.3 Non-uniform VA Spaces . . . 365
9.13.4 Parallel Computing in Non-uniform VA Spaces . . . 368
9.14 Multitasking in SMP . . . 371
9.15 SMP Kernel for Process Management . . . 372
9.15.1 The ts.s file . . . 373
9.15.2 The t.c file . . . 379
9.15.3 Kernel.c file . . . 381
9.15.4 Device Driver and Interrupt Handlers in SMP . . . 384
9.15.5 Demonstration of Process Management in SMP . . . 387
9.16 General Purpose SMP Operating System . . . 389
9.16.1 Organization of SMP_EOS . . . 389
9.16.2 SMP_EOS Source File Tree . . . 390
9.16.3 SMP_EOS Kernel Files . . . 390
9.16.4 Process Management in SMP_EOS . . . 391
9.16.5 Protect Kernel Data Structures in SMP . . . 392
9.16.6 Deadlock Prevention in SMP . . . 394
9.16.7 Adapt UP Algorithms for SMP . . . 395
9.16.8 Device Driver and Interrupt Handlers in SMP . . . 396
9.17 SMP_EOS Demonstration System . . . 397
9.17.1 Startup Sequence of SMP_EOS . . . 397
9.17.2 Capabilities of SMP_EOS . . . 397
9.17.3 Demonstration of SMP_EOS . . . 398
9.18 Summary . . . 399
References . . . 399

10 Embedded Real-Time Operating Systems . . . 401
10.1 Concepts of RTOS . . . 401
10.2 Task Scheduling in RTOS . . . 401
10.2.1 Rate-Monotonic Scheduling (RMS) . . . 402
10.2.2 Earliest-Deadline-First (EDF) Scheduling . . . 402
10.2.3 Deadline-Monotonic Scheduling (DMS) . . . 403
10.3 Priority Inversion . . . 403
10.4 Priority Inversion Prevention . . . 404
10.4.1 Priority Ceiling . . . 404
10.4.2 Priority Inheritance . . . 404

10.5 Survey of RTOS . . . 404
10.5.1 FreeRTOS . . . 404
10.5.2 MicroC/OS (μC/OS) . . . 405
10.5.3 NuttX . . . 405
10.5.4 VxWorks . . . 406
10.5.5 QNX . . . 407
10.5.6 Real-time Linux . . . 407
10.5.7 Critique of Existing RTOS . . . 409
10.6 Design Principles of RTOS . . . 412
10.6.1 Interrupts Processing . . . 412
10.6.2 Task Management . . . 412
10.6.3 Task Scheduling . . . 412
10.6.4 Synchronization Tools . . . 412
10.6.5 Task Communication . . . 412
10.6.6 Memory Management . . . 412
10.6.7 File System . . . 412
10.6.8 Tracing and Debugging . . . 413
10.7 Uniprocessor RTOS (UP_RTOS) . . . 413
10.7.1 Task Management in UP_RTOS . . . 413
10.7.2 Task Synchronization in UP_RTOS . . . 414
10.7.3 Task Scheduling in UP_RTOS . . . 414
10.7.4 Task Communication in UP_RTOS . . . 414
10.7.5 Protection of Critical Regions . . . 415
10.7.6 File System and Logging . . . 415
10.7.7 UP_RTOS with Static Periodic Tasks and Round-Robin Scheduling . . . 416
10.7.8 UP_RTOS with Static Periodic Tasks and Preemptive Scheduling . . . 426
10.7.9 UP_RTOS with Dynamic Tasks Sharing Resources . . . 432
10.8 Multiprocessor RTOS (SMP_RTOS) . . . 440
10.8.1 SMP_RTOS Kernel for Task Management . . . 440
10.8.2 Adapt UP_RTOS to SMP . . . 456
10.8.3 Nested Interrupts in SMP_RTOS . . . 458
10.8.4 Preemptive Task Scheduling in SMP_RTOS . . . 461
10.8.5 Task Switch by SGI . . . 461
10.8.6 Timesliced Task Scheduling in SMP_RTOS . . . 463
10.8.7 Preemptive Task Scheduling in SMP_RTOS . . . 465
10.8.8 Priority Inheritance . . . 468
10.8.9 Demonstration of the SMP_RTOS System . . . 470
10.9 Summary . . . 474
References . . . 474

Index . . . 477

Interrupts and Exceptions Processing

3

In every computer system, the CPU is designed to continually execute instructions. An exception is an event recognized by the CPU, which diverts the CPU from its normal executions to do something else, called exception processing. An interrupt is an external event, which diverts the CPU from its normal executions to do interrupt processing. In a broader sense, interrupts are special kinds of exceptions. The only difference between exceptions and interrupts is that the former may originate from the CPU itself but the latter always originate from external sources. Interrupts are essential to every computer system. Without interrupts, a computer system would be unable to respond to external events, such as user inputs, timer events and requests for service from I/O devices, etc. Most embedded systems are designed to respond to external events and handle such events when they occur. For this reason, interrupts and interrupts processing are especially important to embedded systems. In this chapter, we shall discuss exceptions, interrupts and interrupts processing in ARM based systems.

In Chap. 2, we developed simple drivers for the LCD display and UARTs. The LCD is a memory mapped device, which does not use interrupts. UARTs support interrupts but the simple UART driver uses polling, not interrupts, for I/O. The main disadvantage of I/O by polling is that it does not use the CPU efficiently. While the CPU is doing I/O by polling, it is constantly busy and can't do anything else. In a computer system, I/O should be done by interrupts whenever possible. In this Chapter, we shall show how to apply the principle of interrupts processing to design and implement interrupt-driven device drivers.

3.1 ARM Exceptions

3.1.1 ARM Processor Modes

The ARM processor has seven different operating modes, which are determined by the 5 mode bits [4:0] in the current processor status register (CSPR) (ARM Architecture 2016; ARM Processor Architecture 2016). Table 3.1 shows the seven modes of the ARM processor.

Among the seven modes, only the User mode is non-privileged. All other modes are privileged. An unusual feature of the ARM architecture is that, while the CPU is in a privileged mode, it can change to any other mode, by simply altering the mode bits in the CPSR. When the CPU is in the un-privileged User mode, the only way to change to a privileged mode is through exceptions, interrupts, or the SWI instruction. Each privileged mode has its own banked registers, except System mode, which shares the same set of registers with the User mode, e.g. they have the same stack pointer (R13) and the same link register (R14).

3.1.2 ARM Exceptions

An exception is an event recognized by the processor, which diverts the processor from its normal executions to handle the exception. In a general sense, interrupts are also exceptions. In ARM, there are seven exception types (excluding the Reserved type) (ARM Processor Architecture 2016), which are shown in Table 3.2.

When an exception occurs, the ARM processor does the following.

© Springer International Publishing AG 2017
K.C. Wang, *Embedded and Real-Time Operating Systems*,
DOI 10.1007/978-3-319-51517-5_3

Table 3.1 ARM Processor Modes

Mode	ModeBits	Mode Usage
USR	0x10	for running tasks in user mode
FIQ	0x11	for fast interrupt processing
IRQ	0x12	for ordinary interrupts processing
SVC	0x13	supervisor mode for OS kernel
ABT	0x17	for handling prefetch or data aborts
UND	0x1B	for handling undefined instructions
SYS	0x1F	privileged system mode

Table 3.2 ARM Exceptions

Name	Vector	Description	Mode	Priority
Reset	0x00	reset	SVC	1
UND	0x04	undefined instruction	UND	6
SWI	0x08	software interrupt	SVC	6
PAB	0x0C	prefetch abort	ABT	5
DAB	0x10	data exception abort	ABT	2
Reserved	0x14	reserved	-	-
IRQ	0x18	interrupt request	IRQ	4
FIQ	0x1C	fast interrupt request	FIRQ	3

(1). Copy CPSR into SPSR for the mode in which the exception is to be handled.
(2). Change CPSR mode bits to the appropriate mode, map in the banked registers and disable interrupts. IRQ is always disabled, FIQ is disabled only when an FIQ occurs and on reset.
(3). Set the LR_mode register to the return address.
(4). Set the Program Counter (PC) to the vector address of the exception. This forces a branch to the appropriate exception handler.

If multiple exceptions occur simultaneously, they are handled in accordance with the priority order shown in Table 3.2. The following is a list of the exception events and how they are handled by the ARM processor.

- A Reset event occurs when the processor is powering up. This is the highest priority event and shall be taken whenever it is signaled. Upon entry to the reset handler, the CPSR is in SVC mode and both IRQ and FIQ interrupts are masked out. The task of the reset handler is to initialize the system. This includes setting up stacks of various modes, configuring the memory and initializing device drivers, etc.
- A Data Abort (DAB) events occur when the memory controller or MMU indicates that an invalid memory address has been accessed. For example, if there is no physical memory for an address, or the processor does not have access permission to a region of memory, the data abort exception is raised. Data aborts have the second highest priority. This means that the processor will handle data abort exceptions first, before handling any interrupts.
- A FIQ interrupt occurs when an external peripheral sets the FIQ pin to nFIQ. A FIQ interrupt is the highest priority interrupt. Upon entry to the FIQ handler, both IRQ and FIQ interrupts are disabled. This means that while handling an FIQ interrupt, no other interrupts can occur unless they are explicitly enabled by the software. In ARM based systems, FIQ is usually used to handle interrupts from a single interrupt source of extreme urgency. Allowing multiple FIQ sources would defeat the purpose of FIQ.

- An IRQ interrupt occurs when an external peripheral device sets the IRQ pin. An IRQ interrupt is the second highest priority interrupt. The processor will handle an IRQ interrupt if there is no FIQ interrupt or data abort exception. Upon entry to the IRQ handler, IRQ interrupts are masked out. The CSPR's I-bit should remain set until the current interrupt source has been cleared.
- A Pre-fetch Abort (PFA) event occurs when an attempt to load an instruction results in a memory fault. The exception occurs if the instruction reaches the execution stage of the pipeline and none of the higher exceptions/interrupts have been raised. Upon entry to the PFA handler, IRQ is disabled but FIQ remains enabled, so that any FIQ interrupt will be taken immediately while processing a PFA exception.
- A SWI interrupt occurs when the SWI instruction has been fetched and decoded successfully and none of the other higher priority exceptions/interrupts have been raised. Upon entry to the SWI handler, the CPSR is set to SVC mode. SWI interrupts are usually used to implement system calls from User mode to an OS kernel in SVC mode.
- An Undefined Instruction event occurs when an instruction not in the ARM/Thumb instruction set has been fetched and decoded successfully, and none of the other exceptions/interrupts have been flagged. In an ARM based system with coprocessors, the ARM processor will poll the coprocessors to see if they can handle the instruction. If no coprocessor claims the instruction, then an undefined instruction exception is raised. SWI and Undefined Instruction have the same priority since they cannot occur at the same time. In other words, the instruction being executed can not be both a SWI and an undefined instruction. In practice, undefined instructions can be used to provide software breakpoints when debugging ARM programs.

3.1.3 Exceptions Vector Table

The ARM processor uses a vector table to handle exceptions and interrupts. Table 3.3 shows the ARM vector table contents.

The vector table defines the entry points of exceptions and interrupts handlers. The vector table is located at the physical address 0. Many ARM based systems begins execution from a flash memory or ROM, which may be remapped to 0xFFFF0000 during booting. If the initial vector table is not at 0 in SRAM, it must be copied to SRAM before remap it to 0x00000000. This is normally done during system initialization.

3.1.4 Exception Handlers

Each vector table entry contains an ARM instruction (B, BL or LDR), which causes the processor to load the PC with the entry address of an exception handler routine. The following code segment shows the typical vector table contents, in which each LDR instruction loads the PC with the entry address of an exception handler function. For the reserved vector entry (0x14), a branch to itself loop is sufficient since the exception can never occur.

Table 3.3 ARM Vector Table

Address	Exception	Mode
0x00	Reset	SVC
0x04	Undefined instruction	UND
0x08	Software interrupt (SWI)	SVC
0x0C	Prefetch abort (PFA)	ABT
0x10	Data abort	ABT
0x14	Reserved	N/A
0x18	IRQ	IRQ
0x1C	FIQ	FIQ

```
0x00  LDR PC, reset_handler_addr
0x04  LDR PC, undef_handler_addr
0x08  LDR PC, swi_handler_addr
0x0C  LDR PC, prefetch_abort_handler_addr
0x10  LDR PC, data_abort_handler_addr
0x14  B .
0x18  LDR PC, irq_handler_addr
0x1C  LDR PC, fiq_handler_addr
reset_handler_addr:          .word reset_handler
undef_handler_addr:          .word undef_handler
swi_handler_addr:            .word swi_handler
prefetch_abort_handler_addr: .word prefetch_abort_handler
data_abort_handler_addr:     .word data_abort_handler
irq_handler_addr:            .word irq_handler
fiq_handler_addr:            .word fiq_handler
```

When a vector uses LDR to load the handler entry address, the handler will be called indirectly. LDR must load a constant located within 4 kB from the vector table but it can branch to a full 32-bit address. A B (branch) instruction will go directly to the handler, but it can only branch to a 24-bit address. Since the FIQ vector is the last entry in the vector table, the FIQ handler code can be placed at the FIQ vector location directly, allowing the FIQ handler to be executed quickly.

3.1.5 Return from Exception Handlers

Assume that exception/interrupt handlers are entered by LDR PC instructions, as in

```
LDR PC, handler_entry_address
```

Upon entry to an exception/interrupt handler routine, the processor automatically stores the return address in the link register r14 of the current mode. Due to the instruction pipeline in the ARM processor, the return address stored in the link register includes an offset, which must be subtracted from the link register in order to return to the correct location priori to the exception or interrupt. Table 3.4 shows the program counter offsets of different exceptions.

For Data Abort, the return address is PC-8, which points to the original instruction that caused the exception. For interrupts and Prefetch Abort, the return address is PC-4 since the instruction to be executed before the exception is at current PC-4. For SWI and Undefined instruction, the return address is the current LR because the ARM processor has already executed the SWI or undefined instruction. To most beginners, the non-uniform treatment of the IRQ and SWI return addresses is often a source of confusion. The 'S' suffix at the end of the MOV instruction specifies that, if the destination registers involve loading the PC, the CPSR shall be restored from the saved SPSR also.

A typical method of return from an interrupt handler is to execute the following instruction at the end of the interrupt handler

Table 3.4 Program Counter Offsets

Exception	Offset	Return PC
Reset	n/a	n/a
Data Abort	-8	SUBS pc,lr,#8
FIQ	-4	SUBS pc,lr,#4
IRQ	-4	SUBS pc,lr,#4
Prefetch Abort	-4	SUBS pc,lr,#4
SWI	0	MOVS pc,lr
Undefined	0	MOVS pc,lr

```
SUBS pc, r14_irq, #4
```

which loads the PC with r14_irq – 4, assuming that r14_irq has not been altered in the interrupt handler. Alternatively, the link register can be adjusted at the beginning of the interrupt handler, as in.

```
SUB  lr, lr, #4
   <handler code>
MOVS pc, lr
```

A more widely used method is as follows.

```
SUB  lr, lr, #4            // subtract 4 form LR
STMFD sp_irq!,{r0-r12,lr}  // push r0-r12,LR
    <handler code>
LDMFD sp_irq!,{r0-r12,pc}^ // pop  r0-r12,PC,SPSR
```

The interrupt handler first subtracts 4 from the link register, saves it in the stack, then it executes the handler code. When the handler finishes, it returns to the interrupted point by the LDMFD instruction with the ^ symbol, which loads PC with the saved LR and restores SPSR, causing return to the previous mode priori to the interrupt. Instead of subtracting 4 from the link register manually, an interrupt handler may be written in C with the interrupt attribute, as in

```
void __attribute__((interrupt))handler()
{
   // actual handler code
}
```

In that case, the compiler generated code will adjust the link register automatically.

3.2 Interrupts and Interrupts Processing

3.2.1 Interrupt Types

The ARM processor accepts only two external interrupt requests, FIQ and IRQ. Both are level-sensitive active low signals to the processor. For an interrupt to be accepted by the CPU, the appropriate interrupt mask (I or F) bit in the CPSR must be cleared to 0. FIQ has higher priority than IRQs, so that FIQ will be handled first when multiple interrupts occur. Handling an FIQ causes IRQs and subsequent FIQ to be disabled, preventing them from being taken until after the FIQ handler exits or explicitly enables them. This is usually done by restoring the CPSR from the SPSR at the end of the FIQ handler.

The FIQ vector is the last entry in the vector table. The FIQ handler code can be placed directly at the vector location and run sequentially from that address. This avoids a branch instruction and its associated delay. If the system has a cache memory, the vector table and FIQ handler might all be locked down in one block within the cache. This is important because FIQ is designed to handle the interrupt as quickly as possible. Each privileged mode has its own banked registers r13, r14 and SPSR. The FIQ mode has 5 extra banked register (r8–r12), which can be used to contain information between calls in the FIQ handler, further increasing the execution speed of the FIQ handler.

3.2.2 Interrupt Controllers

An ARM based system usually supports only one FIQ interrupt source but it may support many IRQ requests from different sources. In order to support multiple IRQ interrupts, an interrupt controller is necessary, which sorts out different IRQ sources and presents only one IRQ request to the CPU. Most ARM boards include a Vectored Interrupt Controller (VIC), which is either the ARM PL190 or PL192 (ARM PL190, PL192 2016). A VIC provides the following functions.

. Prioritize the interrupt sources
. Support vectored interrupts

3.2.2.1 ARM PL190/192 Interrupt Controller

Figure 3.1 shows the block diagram of the PL190 VIC. It supports 16 vectored interrupts. The PL192 VIC is similar but it supports 32 vectored interrupts.

3.2.2.2 Vectored and Non-vectored IRQs

The VIC takes all the interrupt requests from different sources and arranges them into three categories, FIQ, vectored IRQ and non-vectored IRQ. FIQ has the highest priority. In the case of the PL190 VIC, it takes 16 interrupt requests and prioritizes them within sixteen vectored IRQ slots, denoted by IRQ0-IRQ15. Each vectored interrupt has a vector address, denoted by VectAddr0-VectAddr15.

3.2.2.3 Interrupt Priorities

Among the vectored interrupts, IRQ0 has the highest priority and IRQ15 has the lowest priority. Non-vectored IRQ's have the lowest priority. The VIC OR's the requests from both the vectored and non-vectored IRQ's to generate the IRQ signal to the ARM core, which is shown as nVICIRQ line in Fig. 3.1.

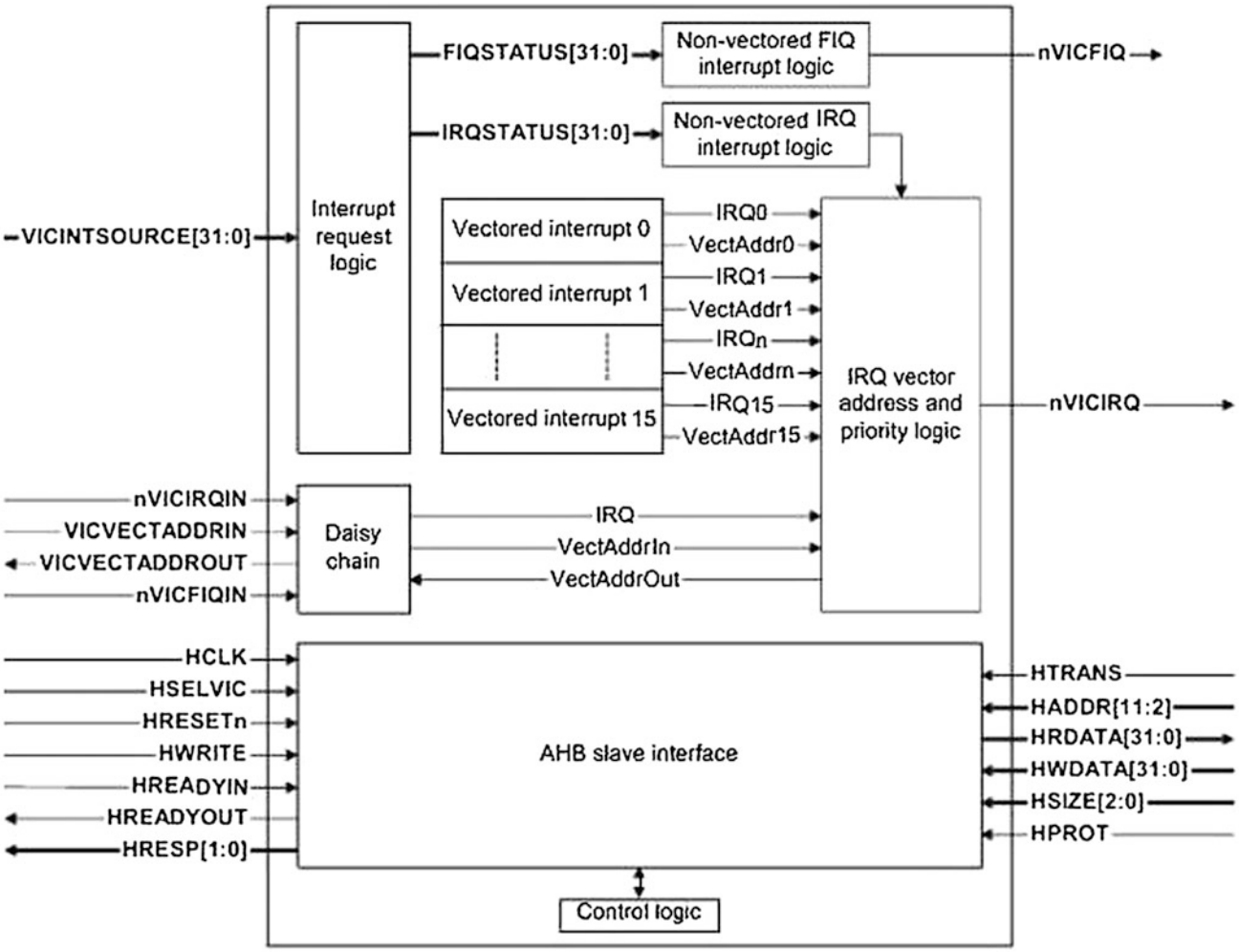


Fig. 3.1 ARM PL190 VIC

3.2.3 Primary and Secondary Interrupt Controllers

Some ARM boards may contain more than one VIC. For example, the ARM926EJ-S board has two VICs, a primary VIC (PIC) and a secondary VIC (SIC), which are shown in Fig. 3.2. Most inputs to the PIC are dedicated to high priority interrupts, such as timers, GPIO and UARTs. Low priority interrupt sources, such as USB, Ethernet, keyboard and mouse, are fed to the SIC. Some of these interrupts may be routed to the PIC at IRQs 21 to 26. Lower priority interrupts, such as touch-screen, keyboard and mouse are collectively routed to IRQ 31 of the PIC.

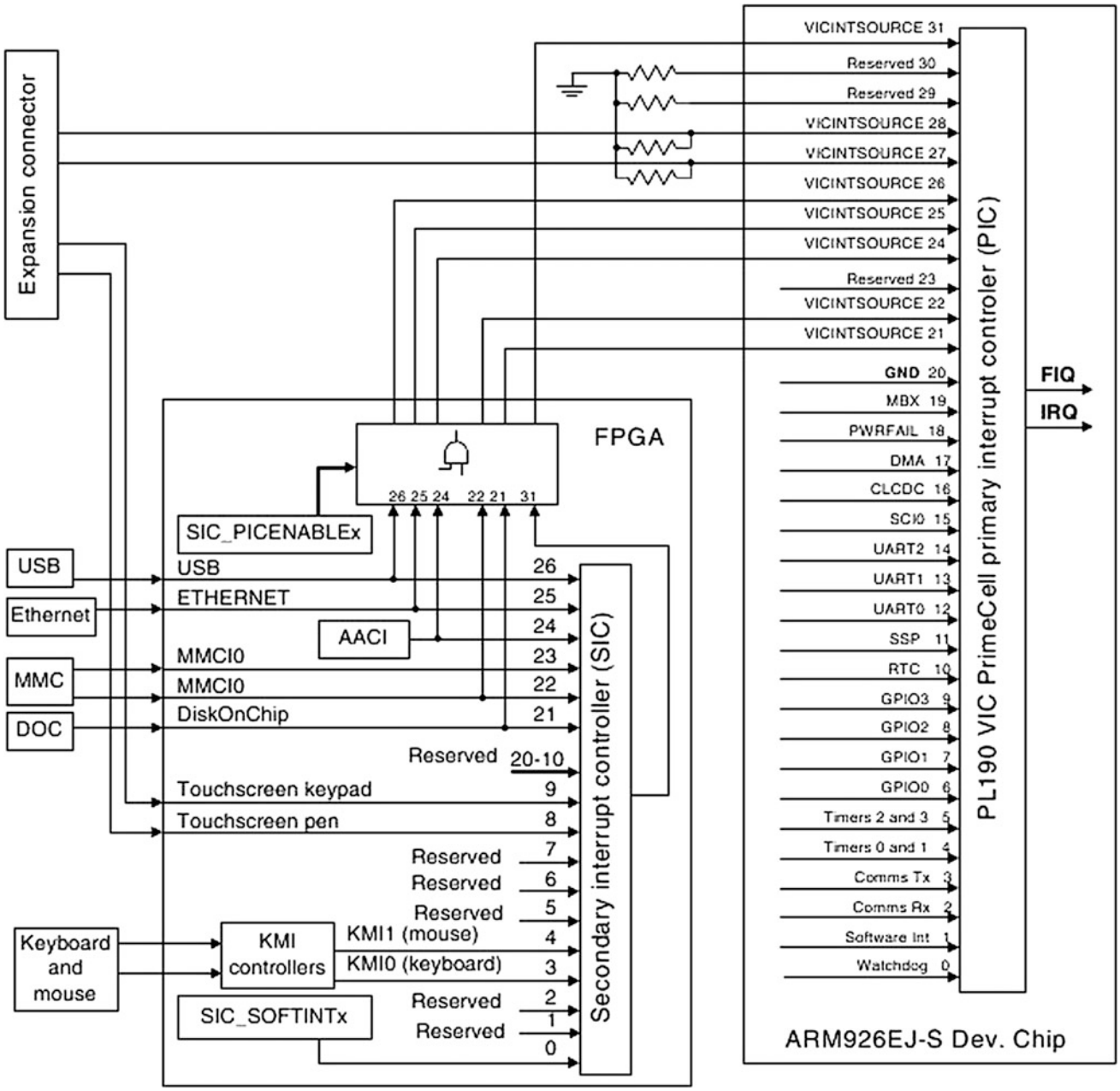


Fig. 3.2 VICs in ARM926EJ-S Board

3.3 Interrupt Processing

3.3.1 Vector Table Contents

Interrupt vectors are in the exception vector table. Each interrupt vector location contains an instruction which loads the PC with the entry address of an interrupt handler. For FIQ and IRQ interrupts, the vector contents are

```
0x18:  LDR PC, irq_handler_addr
0x1C:  LDR PC, fiq_handler_addr
irq_handler_addr: .word irq_handler
fiq_handler_addr: .word fiq_handler
```

3.3.2 Hardware Interrupt Sequence

When a FIQ interrupt occurs, the processor does the following.

1. LR_fiq = address of the next instruction to be executed + 4.
2. SPSR_fiq = CPSR.
3. CPSR[4:0] = 0x11 (FIQ mode).
4. CPSR[5] = 0 (Execute in ARM state).
5. CPSR[7-6] = 11 (Disable IRQ and FIQ interrupts).
6. PC = 0x1C (execute FIQ handler).

Similarly, when an IRQ interrupt occurs, the processor does the following.

1. LR_irq = address of the next instruction to be executed + 4.
2. SPSR_irq = CPSR.
3. CPSR[4:0] = 0x12 (IRQ mode).
4. CPSR[5] = 0 (Execute in ARM state).
5. CPSR[7-6] = 01 (Disable IRQ but keep FIQ enabled).
6. PC = 0x18 (execute IRQ handler).

3.3.3 Interrupts Control in Software

When discussing interrupts and interrupts processing, there are some commonly used terms which warrant clarification.

3.3.3.1 Enable/Disable Interrupts

Each device has a control register or, in some cases, a separate interrupt control register, which can be programmed to either allow or disallow the device to generate interrupt requests. If a device is to use interrupts, the device interrupt control register must be configured with interrupts enabled. If needed, device interrupts can be disabled explicitly. Thus, the terms enable/disable interrupts should be applied only to devices.

3.3.3.2 Interrupt Masking

When a device raises an interrupt to the CPU, the CPU may either accept or not accept the interrupt immediately, depending on the interrupt masking bits in the CPU's status register. For an IRQ interrupt, the ARM CPU accepts the interrupt if the I bit in the CPSR register is 0, meaning that the CPU has IRQ interrupts unmasked or mask-in. It does not accept the interrupt while the CPSR's I bit is 1, meaning that the CPU has IRQ interrupts masked out. Masked out interrupts are not lost. They are kept pending until the CPSR's I bit is changed to 0, at which time the CPU will accept the interrupt. Thus, when applied to the CPU, enable/disable interrupts really mean mask-in/mask-out interrupts. In most literatures these terms are used interchangeably but the reader should be aware of their differences.

3.3.3.3 Clear Device Interrupt Request

When the CPU accepts an IRQ interrupt, it starts to execute the interrupt handler for that device. At the end of the interrupt handler it must clear the interrupt request, which causes the device to drop its interrupt request, allowing it to generate the next interrupt. This is usually done by accessing some of the device interface registers. For example, reading the data register of an input device clears the device interrupt request. For some output devices, it may be necessary to disable the device interrupt explicitly when there is no more data to output.

3.3.3.4 Send EOI to Vectored Interrupt Controller

In a system with multiple interrupt sources, a Vectored Interrupt Controller (VIC) is usually used to prioritize the device interrupts, each with a dedicated vector address. At the end of handling the current interrupt, the interrupt handler must inform the VIC that it has finished processing the current interrupt (of the highest priority), allowing the VIC to re-prioritize pending interrupts requests. This is referred to as sending an End-of-Interrupt (EOI) to the interrupt controller. For the ARM PL190, this is done by writing an arbitrary value to the VIC's vector address register at base +0x30.

The ARM processor has a simple way to enable/disable (unmask/mask) interrupts while in privileged mode. The following code segments show how to enable/disable IRQ interrupts of the ARM processor.

```
lock:
       MRS  r1, CPSR        ; read CPSR into r1
       ORR  r1, r1, #0x80   ; set bit-7(I bit) to 1
       MSR  CPSR, r1        ; write r1 back to CPSR

unlock:
       MRS  r1, CPSR        ; read CPSR into r1
       BIC  r1, r1, #0x80   ; clear bit-7 (I bit) in r1
       MSR  CPSR r1         ; write r1 back to CPSR
```

To enable IRQ interrupts, first copy CPSR into a working register, clear the I_bit (7) in the working register to 0. Then copy the updated register back to CPSR, which enables IRQ interrupts. Similarly, setting CPSR's I_bit disables IRQ interrupts. Similar code segments can be used to enable/disable FIQ interrupts (by clear/set bit-6 in CPSR).

3.3.4 Interrupt Handlers

Interrupt handlers are also known as Interrupt Service Routines (ISRs). Interrupt handlers can be classified into three different types.

3.3.4.1 Interrupt Handler Types

. Non-nested interrupt handler: handles one interrupt at a time. Interrupts are not enabled until execution of the current ISR has finished.
. Nested interrupt handler: While inside an ISR, enable IRQ to allow IRQ interrupts of higher priorities to occur. This implies that while executing the current ISR, it may be interrupted to execute another ISR of higher priority.
. Re-entrant interrupt handler: Enable IRQ as soon as possible, allowing the same ISR to be executed again.

3.3.5 Non-nested Interrupt Handler

When the ARM CPU accepts an IRQ interrupt, it enters the IRQ mode with IRQ interrupts masked out, preventing it from accepting other IRQ interrupts. The CPU sets PC to point to the IRQ entry in the vector table and executes that instruction. The instruction loads PC with the entry address of the interrupt handler, causing execution to enter the interrupt handler. The interrupt handler first saves the execution context before the interrupt. Then it determines the interrupt source and calls the appropriate ISR. After servicing the interrupt, it restores the saved context and sets the PC to point back to the next

instruction prior to the interruption. Then it returns to the original place of the interruption. The simplest interrupt handler serves only one interrupt at a time. While executing the interrupt handler, IRQ interrupts are masked out until control is returned back to the interrupted point. The algorithm and control flow of a non-nested interrupt handler is as follows.

```
non_nested_interrupt_handler()
{
   // IRQ interrupt masked out in CPSR
   1. Save Context
   2. Determine interrupt source
   3. Call ISR to handle the interrupt
   4. Restore Context
   5. Restore CPSR by SPSR and return to interrupt point
}
```

The following code segment shows the organization of a simple IRQ interrupt handler. It assumes that an IRQ mode stack has been set up properly, which is typically done in the reset handler during system initialization.

```
SUB   lr, lr, #4
STMFD sp_irq!, {r0-r12, lr}
 {specific interrupt service routine code}
LDMFD sp_irq!, {r0-r12, pc}^
```

The first instruction adjusts the link register (r14) for return to the interrupted point. The STMFD instruction saves the context at the point of interruption by pushing CPU registers that must be preserved onto the stack. The time taken to execute a STMFD or LDMFD instruction is proportionally to the number of registers being transferred. To reduce interrupt processing latency, a minimum number of registers should be saved. When writing ISR in a high-level programming language, such as C, it is important to know the calling convention of the compiler generated code as this will affect the decision on which registers should be saved on the stack. For instance, the ARM compiler generated code preserves r4–r11 during function calls, so there is no need to save these registers unless they are going to be used by the interrupt handler. Once the registers have been saved, it is now safe to call C functions to process the interrupt. At the end of the interrupt handler the LDMFD instruction restores the saved context and return from the interrupt handler. The '^' symbol at the end of the LDMFD instruction means that the CPSR will be restored from the saved SPSR. As noted in the Chap. 2 on ARM instructions, the '^' restores the saved SPSR only if the PC is loaded at the same time. Otherwise, it only restores the bank registers of the previous mode, excluding the saved SPSR. This special feature can be used to access User mode registers while in a privileged mode.

The organization of the simple interrupt handler is suitable for handling both FIQ and IRQ interrupts one at a time without interrupt nesting. After saving the execution context, the interrupt handler must determine the interrupt source. In a simple ARM system that does not use vectored interrupts, the interrupt source is in the interrupt status register, denoted by IRQstatus, located at a known (memory-mapped) address. To determine the interrupt source, simply read the IRQstatus register and scan the contents for any bit or bits that are set. Each non-zero bit stands for an active interrupt request. The interrupt handler may scan the bits in a specific order, which determines the interrupt processing priority in software.

With the above background information on interrupts and interrupts processing, we are ready to write some real programs that use interrupts. In the following, we shall show how to write interrupt handlers for I/O devices. Device drivers using interrupts are called **interrupt-driven device drivers**. Specifically, we shall show how to implement interrupt-driven drivers for timer, keyboard, UART and Secure Digital Cards (SDC).

3.4 Timer Driver

3.4.1 ARM Versatile 926EJS Timers

The ARM Versatile 926EJS board contains two ARM SB804 dual timer modules [ARM Timers 2004]. Each timer module contains two timers, which are driven by the same clock. The base addresses of the timers are at

```
Timer0: 0x101E2000, Timer1: 0x101E2020
Timer2: 0x101E3000, Timer3: 0x101E3020
```

Timer0 and Timer1 interrupt at IRQ4. Timer2 and Timer3 interrupt at IRQ5, both on the primary Vectored Controller (VIC). To begin with, we shall not use vectored interrupts. Vectored interrupts will be discussed later. From a programming point of view, the most important timer registers are the control and counter registers. The following lists the meanings of the timer control register bits.

```
Bit   --------- Function -------   ----- Setting=0x66 --------
 7    timer disable/enable         0 set to 1 in timer_start()
 6    free running/periodic mode   1 for periodic mode
 5    interrupt disable/enable     1 for interrupt enabled
 4    NOT used                     0 (use 0 for default)
 3:2  divider:00=1,01=8,10=256     01 for divide by 8
 1    16/32-bit counter value      1 for 32-bit counter
 0    wraparound/oneshot mode      0 for wraparound mode
-----------------------------------------------------------
```

3.4.2 Timer Driver Program

The sample program C3.1 demonstrates a timer driver for the ARM Versatile board. The program consists of the following components.

(**1**). **t.ld file:** The linker script file specifies reset_handler as the entry address. It defines two 4 KB areas as SVC and IRQ stacks.

```
ENTRY(reset_handler)
SECTIONS
{
  . = 0x10000;         /* loading address */
  .text : { ts.o *(.text) }
  .data : { *(.data) }
  .bss : { *(.bss) }
  . = ALIGN(8);
  . = . + 0x1000;      /* 4kB of SVC stack space */
  svc_stack_top = .;
  . = . + 0x1000;      /* 4kB of IRQ stack space */
  irq_stack_top = .;
}
```

(**2**). **ts.s file:** The assembly code file, ts.s, defines the entry point and the reset code.

```
/***************** ts.s file of C3.1 ******************/
.text
.code 32
.global reset_handler, vectors_start, vectors_end
```

```
reset_handler:
  LDR sp, =svc_stack_top   // set SVC mode stack
  BL copy_vectors          // copy vector table to address 0
  MSR cpsr, #0x92          // to IRQ mode
  LDR sp, =irq_stack_top   // set IRQ mode stack
  MSR cpsr, #0x13          // go back to SVC mode with IRQ on
  BL main                  // call main() in C
  B .                      // loop if main ever return

irq_handler:
  sub lr, lr, #4
  stmfd sp!, {r0-r12, lr}   // stack r0-r12 and lr
  bl IRQ_handler            // call IRQ_hanler() in C
  ldmfd sp!, {r0-r12, pc}^ // return

vectors_start:
  LDR PC, reset_handler_addr
  LDR PC, undef_handler_addr
  LDR PC, swi_handler_addr
  LDR PC, prefetch_abort_handler_addr
  LDR PC, data_abort_handler_addr
  B .
  LDR PC, irq_handler_addr
  LDR PC, fiq_handler_addr
  reset_handler_addr:          .word reset_handler
  undef_handler_addr:          .word undef_handler
  swi_handler_addr:            .word swi_handler
  prefetch_abort_handler_addr: .word prefetch_abort_handler
  data_abort_handler_addr:     .word data_abort_handler
  irq_handler_addr:            .word irq_handler
  fiq_handler_addr:            .word fiq_handler
vectors_end:
```

When the program starts, QEMU loads the executable image, t.bin, to 0x10000. Upon entry to ts.s, execution begins from the label reset_handler. First, it sets the SVC mode stack pointer and calls copy_vector() in C to copy the vectors to address 0. It switches to IRQ mode to set up the IRQ stack pointer. Then it switches back to SVC mode with IRQ interrupts enabled and calls main() in C. The main program normally runs in SVC mode. It enters IRQ mode only to handle interrupts. Since we are not using FIQ, nor trying to deal with any exceptions at this moment, all other exception handlers (in exceptions.c file) are while(1) loops.

(**3**). **t.c file:** this file contains main(), copy_vector() and IRQ_handler() in C

```
#include "defines.h"    // LCD, TIMER and UART addresses
#include "string.c"     // strcmp, strlen, etc
#include "timer.c"      // timer handler file
#include "vid.c"        // LCD driver file
#include "exceptions.c" // other exception handlers

void copy_vectors(void){// copy vector table in ts.s to 0x0
    extern u32 vectors_start, vectors_end;
    u32 *vectors_src = &vectors_start;
    u32 *vectors_dst = (u32 *)0;
    while (vectors_src < &vectors_end)
      *vectors_dst++ = *vectors_src++;
}
```

```
void timer_handler();
TIMER *tp[4];            // 4 TIMER structure pointers

void IRQ_handler()       // IRQ interrupt handler in C
{
    // read VIC status registers to determine interrupt source
    int vicstatus = VIC_STATUS;
    // VIC status BITs: timer0,1=4, uart0=13, uart1=14
    if (vicstatus & (1<<4)){         // bit4=1:timer0,1
      if (*(tp[0]->base+TVALUE)==0)  // timer 0
         timer_handler(0);
      if (*(tp[1]->base+TVALUE)==0)  // timer 1
         timer_handler(1);
    }
    if (vicstatus & (1<<5)){         // bit5=1:timer2,3
      if (*(tp[2]->base+TVALUE)==0)  // timer 2
         timer_handler(2);
      if (*(tp[3]->base+TVALUE)==0)  // timer 3
         timer_handler(3);
    }
}

int main()
{
   int i;
   color = RED;              // int color in vid.c file
   fbuf_init();              // initialize LCD driver
   printf("main starts\n");
   /* enable VIC for timer interrupts */
   VIC_INTENABLE  = 0;
   VIC_INTENABLE |= (1<<4);  // timer0,1 at VIC.bit4
   VIC_INTENABLE |= (1<<5);  // timer2,3 at VIC.bit5
   timer_init();
   for (i=0; i<4; i++){      // start all 4 timers
      tp[i] = &timer[i];
      timer_start(i);
   }
   printf("Enter while(1) loop, handle timer interrupts\n");
}
```

(**4**). **timer.c file:** timer.c implements the timer handlers

```
// timer register u32 offsets from base address
#define TLOAD   0x0
#define TVALUE  0x1
#define TCNTL   0x2
#define TINTCLR 0x3
#define TRIS    0x4
#define TMIS    0x5
#define TBGLOAD 0x6
```

```
typedef volatile struct timer{
  u32 *base;            // timer's base address; as u32 pointer
  int tick, hh, mm, ss; // per timer data area
  char clock[16];
}TIMER;

volatile TIMER timer[4]; //4 timers; 2 per unit; at 0x00 and 0x20

void timer_init()
{
  int i; TIMER *tp;
  printf("timer_init()\n");
  for (i=0; i<4; i++){
    tp = &timer[i];
    if (i==0) tp->base = (u32 *)0x101E2000;
    if (i==1) tp->base = (u32 *)0x101E2020;
    if (i==2) tp->base = (u32 *)0x101E3000;
    if (i==3) tp->base = (u32 *)0x101E3020;
    *(tp->base+TLOAD) = 0x0;   // reset
    *(tp->base+TVALUE)= 0xFFFFFFFF;
    *(tp->base+TRIS)  = 0x0;
    *(tp->base+TMIS)  = 0x0;
    *(tp->base+TLOAD) = 0x100;
    // CntlReg=011-0010=|En|Pe|IntE|-|scal=01|32bit|0=wrap|=0x66
    *(tp->base+TCNTL) = 0x66;
    *(tp->base+TBGLOAD) = 0x1C00; // timer counter value
    tp->tick = tp->hh = tp->mm = tp->ss = 0; // initialize wall clock
    strcpy((char *)tp->clock, "00:00:00");
  }
}

void timer_handler(int n) {
    int i;
    TIMER *t = &timer[n];
    t->tick++;        // Assume 120 ticks per second
    if (t->tick==120){
      t->tick = 0; t->ss++;
      if (t->ss == 60){
         t->ss = 0; t->mm++;
         if (t->mm == 60){
            t->mm = 0; t->hh++; // no 24 hour roll around
         }
      }
      t->clock[7]='0'+(t->ss%10); t->clock[6]='0'+(t->ss/10);
      t->clock[4]='0'+(t->mm%10); t->clock[3]='0'+(t->mm/10);
      t->clock[1]='0'+(t->hh%10); t->clock[0]='0'+(t->hh/10);
    }
    color = n;               // display in different color
    for (i=0; i<8; i++){
        kpchar(t->clock[i], n, 70+i); // to line n of LCD
    }
    timer_clearInterrupt(n); // clear timer interrupt
}

void timer_start(int n) // timer_start(0), 1, etc.
{
```

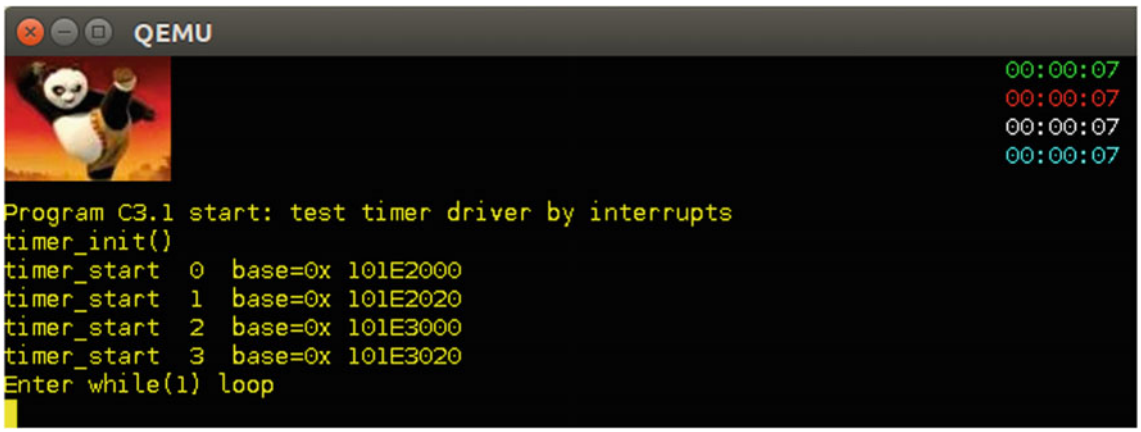


Fig. 3.3 Outputs of timer interrupt handlers

```
  TIMER *tp = &timer[n];
  kprintf("timer_start %d base=%x\n", n, tp->base);
  *(tp->base+TCNTL) |= 0x80;    // set enable bit 7
}

int timer_clearInterrupt(int n) // timer_start(0), 1, etc.
{
  TIMER *tp = &timer[n];
  *(tp->base+TINTCLR) = 0xFFFFFFFF;
}
void timer_stop(int n)       // stop a timer
{
  TIMER *tp = &timer[n];
  *(tp->base+TCNTL) &= 0x7F; // clear enable bit 7
}
```

Figure 3.3 shows the LCD screen of running the program C3.1. Each timer displays a wall clock at the right-top corner of the screen. In the LCD driver, the scroll-up limit is set to a line below the logo and the wall clocks, so that they will not be affected during scroll-up. The wall clocks are updated on each second. As exercises, the reader may change the starting values of the wall clocks to display local time in different time zones, or change the timer counter values to generate timer interrupts with different frequencies.

3.5 Keyboard Driver

3.5.1 ARM PL050 Mouse-Keyboard Interface

The ARM Versatile board includes an ARM PL050 Mouse-Keyboard Interface (MKI) which provides support for a mouse and a PS/2 compatible keyboard [ARM PL050 MKI 1999]. The keyboard's base address is at 0x1000600. It has several 32-bit registers, which are at offsets from the base address.

```
Offset   Register         Bits Assignment
------  ----------   ------------------------------
 0x00    Control     bit 5=0(AT) 4=IntEn  2=Enable
 0x04    Status      bit 4=RXF   3=RXBUSY
 0x08    Data        input scan code
 0x0C    ClkDiv      (a value between 0-15)
 0x10    IntStatus   bit 0=RX interrupt
-------------------------------------------------
```

3.5.2 Keyboard Driver

In this section, we shall develop a simple interrupt-driven driver for the ARM versatile keyboard. In order to use interrupts, the keyboard's control register must be initialized to 0x14, i.e. bit 2 enables the keyboard and bit 4 enables Rx (input) interrupts. The keyboard interrupts at IRQ3 on the Secondary VIC, which is routed to IRQ31 on the Primary VIC. Instead of ACSII code, the keyboard generates scan codes. A complete listing of scan codes is included in the keyboard driver. Translation of scan code to ASCII is done by mapping tables in software. This allows the same keyboard to be used for different languages. For each key typed, the keyboard generates two interrupts; one when the key is pressed and another one when the key is released. The scan code of key release is 0x80 + the scan code of key press, i.e. bit7 is 0 for key press and 1 for key release. When the keyboard interrupts, the scan code is in the data register (0x08). The interrupt handler must read the data register to get the scan code, which also clears the keyboard interrupt. Some special keys generate escape key sequences, e.g. the UP arrow key generates 0xE048, where 0xE0 is the escape key itself. The following shows the mapping tables for translating scan codes into ASCII. The keyboard has 105 keys. Scan codes above 0x39 (57) are special keys, which cannot be mapped directly, so they are not shown in the key maps. Such special keys are recognized by the driver and handled accordingly. Figure 3.4 shows the key mapping tables.

3.5.3 Interrupt-Driven Driver Design

Every interrupt-driven device driver consists of three parts; a lower-half part, which is the interrupt handler, an upper-half part, which is called by the application program, and a common data area containing a buffer for data and control variables for synchronization, which are shared by the lower and upper parts. Figure 3.5 shows the organization of the keyboard driver. The top part of the figure shows kbd_init(), which initialize the KBD driver when the system starts. The middle part shows the control and data flow path from the KBD device to a program. The bottom part shows the lower-half, input buffer, and the upper-half organization of the KBD driver.

When the main program starts, it must initialize the keyboard driver control variables. When a key is pressed, the KBD generates an interrupt, causing the interrupt handler to be executed. The interrupt handler fetches the scan code from KBD data port. For normal key presses, it translates the scan code into ASCII, enters the ASCII char into an input buffer, buf[N],

```
#define NSCAN 58
/* Scan codes to ASCII for unshifted keys */
char unshift[NSCAN] = { // NSCAN=58
  0, 033,'1','2','3','4','5','6','7','8','9','0','-','=','\b','\t',
 'q','w','e','r','t','y','u','i','o','p','[',']', '\r', 0,'a','s',
 'd','f','g','h','j','k','l',';', 0,  0,  0,  0, 'z', 'x','c','v',
 'b','n','m',',','.','/', 0, '*', 0, ' ' };
/* Scan codes to ASCII for shifted keys */
char shift[NSCAN] = {
  0, 033,'!','@','#','$','%','^','&','*','(',')','_','+','\b','\t',
 'Q','W','E','R','T','Y','U','I','O','P','{','}', '\r', 0,'A','S',
 'D','F','G','H','J','K','L',':', 0, '~', 0,  '|','Z','X','C','V',
 'B','N','M','<','>','?', 0, '*', 0, ' ' };
```

Fig. 3.4 Key mapping tables

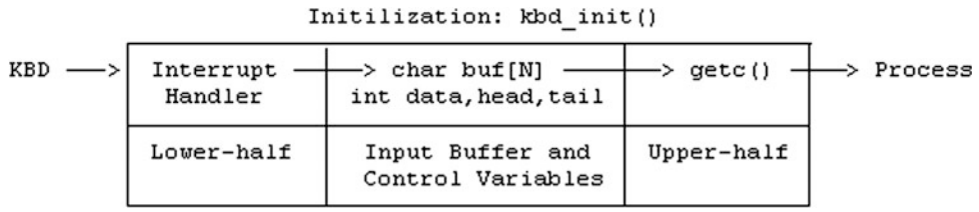


Fig. 3.5 KBD driver organization

and notifies the upper-half of the input char. When the program side needs an input char, it calls getc() of the upper-half driver, trying to get a char from buf[N]. The program waits if there is no char in buf[N]. The control variable, data, is used to synchronize the interrupt handler and the main program. The choice of the control variable depends on the synchronization tool used. The following shows the C code of a simple KBD driver. The driver handles only lower case keys. Extending the driver to handle upper case and special keys is left as an exercise in the Problem section.

3.5.4 Keyboard Driver Program

The sample program C3.2 demonstrates a simple interrupt-driven keyboard driver. It consists of the following components.

(**1**). **t.ld file:** The linker script file is the same as in C3.1.
(**2**). **ts.s file:** The ts.s file is almost the same as in C3.1, except that it adds the lock and unlock functions for enable/disable IRQ interrupts.

```
/********* ts.s file ***********/
.text
.code 32
.global reset, vectors_start, vectors_end
.global lock, unlock
reset_handler:   // entry point of program
// set SVC stack
  LDR sp, =svc_stack_top
// copy vector table to address 0
  BL copy_vectors
// go in IRQ mode to set IRQ stack
  MSR cpsr, #0x92
  LDR sp, =irq_stack_top
// go back in SVC mode with IRQ interrupts enabled
  MSR cpsr, #0x13
// call main() in SVC mode
  BL main
  B .
irq_handler:
  sub lr, lr, #4
  stmfd sp!, {r0-r12, lr}  // stack ALL registers
  bl  IRQ_handler          // call IRQ_hanler() in C
  ldmfd sp!, {r0-r3, r12, pc}^ // return
lock:                // mask out IRQ interrupts
  MRS r0, cpsr
  ORR r0, r0, #0x80   // set I bit means MASK out IRQ interrupts
  MSR cpsr, r0
  mov pc, lr
```

```
unlock:               // mask in IRQ interrupts
  MRS r0, cpsr
  BIC r0, r0, #0x80   // clr I bit means MASK in IRQ interrupts
  MSR cpsr, r0
mov pc, lr
vectors_start:
   // vector table: same as before
vectors_end:
```

(**3**). **t.c file:** This file implements the main(), copy_vector() and IRQ_handler() functions. For simplicity, we shall only use timer0 to display a single wall clock.

```
/*************** t.c file *************/
#include "defines.h"
#include "string.c"
void timer_handler();
#include "kbd.c"
#include "timer.c"
#include "vid.c"
#include "exceptions.c"

void copy_vectors(){ // same as before }
void IRQ_handler()       // IRQ interrupt handler in C
{
  // read VIC status registers to find out which interrupt
  int vicstatus = VIC_STATUS;
  // VIC status BITs: timer0,1=4, uart0=13, uart1=14
  if (vicstatus & (1<<4)){    // bit4=1:timer0,1
     timer_handler(0);        // timer0 only
  }
  if (vicstatus & (1<<31)){   // PIC.bit31= SIC interrupts
     if (sicstatus & (1<<3)){ // SIC.bit3 = KBD interrupt
        kbd_handler();
     }
  }
}

int main()
{
  int i;
  char line[128];
  color = RED;   // int color in vid.c file
  fbuf_init();   // initialize LCD display
  /* enable VIC interrupts: timer0 at IRQ3, SIC at IRQ31 */
  VIC_INTENABLE  = 0;
  VIC_INTENABLE |= (1<<4);    // timer0,1 at PIC.bit4
  VIC_INTENABLE |= (1<<5);    // timer2,3 at PIC.bit5
  VIC_INTENABLE |= (1<<31);   // SIC to PIC.bit31
  /* enable KBD IRQ on SIC */
  SIC_INTENABLE = 0;
  SIC_INTENABLE |= (1<<3);    // KBD int=SIC.bit3
  timer_init();   // initialize timer
  timer_start(0); // start timer0
```

```
  kbd_init();     // initialize keyboard driver
  printf("C3.2 start: test KBD and TIMER drivers\n");
  while(1){
     color = CYAN;
     printf("Enter a line from KBD\n");
     kgets(line);
     printf("line = %s\n", line);
  }
}
```

In order to use interrupts, the devices must be configured to generate interrupts. This is done in the device initialization code, in which each device is initialized with interrupts enabled. In addition, both the Primary and Secondary Interrupt Controllers (PIC and SIC) must be configured to enable the device interrupts. The keyboard interrupts at IRQ3 on the SIC, which is routed to IRQ31 on the PIC. To enable keyboard interrupts, both bit 3 of the SIC_INTENABLE and bit 31 of the VIC_INTENABLE registers must be set to 1.

These are done in main(). After initializing the devices for interrupts, the main program executes a while(1) loop, in which it prompts for an input line from the KBD and prints the line to the LCD display.

(**4**). **kbd.c file:** The kbd.c file implements a simple keyboard driver.

```
/******** kbd.c file *****************/
#include "keymap"
/******** KBD register byte offsets; for char *base *****/
#define KCNTL 0x00 // 7-6- 5(0=AT)4=RxIntEn 3=TxIntEn
#define KSTAT 0x04 // 7-6=TxE 5=TxBusy 4=RXFull 3=RxBusy
#define KDATA 0x08 // data register;
#define KCLK  0x0C // clock divisor register;   (not used)
#define KISTA 0x10 // interrupt status register;(not used)

typedef volatile struct kbd{ // base = 0x10006000
  char *base;      // base address of KBD, as char *
  char buf[128];   // input buffer
  int head, tail, data, room; // control variables
}KBD;
volatile KBD kbd;  // KBD data structure

int kbd_init()
{
  KBD *kp = &kbd;
  kp->base = (char *)0x10006000;
  *(kp->base+KCNTL) = 0x14; // 00010100=INTenable, Enable
  *(kp->base+KCLK)  = 8;    // PL051 manual says a value 0 to 15
  kp->data = 0; kp->room = 128; // counters
  kp->head = kp->tail = 0;  // index to buffer
}

void kbd_handler()           // KBD interrupt handler in C
{
  u8 scode, c;
  int i;
  KBD *kp = &kbd;
  color = RED;               // int color in vid.c file
  scode = *(kp->base+KDATA); // read scan code in data register
  if (scode & 0x80)          // ignore key releases
```

```
    return;
  c = unsh[scode];           // map scan code to ASCII
  if (c != '\r')
     printf("kbd interrupt: c=%x %c\n", c, c);
  kp->buf[kp->head++] = c;  // enter key into CIRCULAR buf[ ]
  kp->head %= 128;
  kp->data++; kp->room--;   // update counters
}

int kgetc() // main program calls kgetc() to return a char
{
  char c;
  KBD *kp = &kbd;
  unlock();                 // enable IRQ interrupts
  while(kp->data <= 0);     // wait for data; READONLY
  lock();                   // disable IRQ interrupts
    c = kp->buf[kp->tail++];// get a c and update tail index
    kp->tail %= 128;
    kp->data--; kp->room++; // update with interrupts OFF
  unlock();                 // enable IRQ interrupts
  return c;
}

int kgets(char s[ ])        // get a string from KBD
{
  char c;
  while((c=kgetc()) != '\r'){
    *s++ = c;
  }
  *s = 0;
  return strlen(s);
}
```

(5). Interrupt Processing Sequence

(5).1. When an interrupt occurs, the CPU follows the IRQ vector at 0x18 to enter

```
irq_handler:
    sub lr, lr, #4              // adjust lr
    stmfd sp!, {r0-r12, lr}      // save regs in IRQ stack
    bl IRQ_handler               // call IRQ_hanler() in C
    ldmfd sp!, {r0-r12, pc}^     // return
```

irq_handler first subtract 4 form the link register lr_irq. The adjusted lr is the correct return address to the point of interruption. It pushes registers r0–r12 and lr into the IRQ stack. Then it call IRQ_handler() in C. Upon return from IRQ_handler(), it pops the stack, which loads PC with the saved lr and also restores the SPSR, causing control return to the original point of interruption. To speed up interrupt processing, irq_handler may only save registers that must be preserved. Since our interrupt handler is written in C, not in assembly, it suffices to save only r0–r3, r12 (stack frame pointer) and lr (link register).

(5).2. IRQ_handler(): IRQ_handler() first reads the status registers of both PIC and SIC. The interrupt handler must scan the bits of the status register to determine the interrupt source. Each bit = 1 in the status register represents an active interrupt. The scanning order should follow the interrupt priority order, i.e. from bit 0 to bit 31.

```
    vicstatus = VIC_STATUS;
  sicstatus = SIC_STATUS;
  // VIC status BITs: timer0=4, uart0=13, uart1=14, SIC=31: KBD at 3
  if (vicstatus & (1<<4))     // bit 4 => timer interrupt
     timer_handler(0);
  if (vicstatus & (1<<31)){   // SIC interrupts=bit_31=>KBD at bit 3
     if (sicstatus & (1<<3)){ // SIC bit 3 => KBD interrupt
        kbd_handler();
     }
  }
```

(5).3. kbd_handler(): The kbd_handler() reads the scan code from the KBD data register, which clears the KBD interrupt. It ignores any key release, so the driver can only handle lower case keys and no special keys at all. For each key pressed, it prints a "kbd interrupt key" message to the LCD display. As noted before, we have adapted the generic printf() function for formatted printing to the LCD screen. Then, it maps the scan code to a (lower case) ASCII char and enters the char into the input buffer. The control variable, data, represents the number of chars in the input buffer.
(5).4. kgetc() and kgets() functions: kgetc() is for getting an input char from the keyboard. kgets() is for getting an input line ended with the \r key. The simple KBD driver is intended mainly to illustrate the design principle of an interrupt-driven input device driver. The driver's buffer and control variables form a critical region since they are accessed by both the main program and KBD interrupt handler. When the interrupt handler executes, the main program is logically not executing. So the main program can not interfere with the interrupt handler. However, while the main program executes, interrupts may occur, which diverts the program to execute the interrupt handler, which may interfere with the main program. For this reason, when a program calls kgetc(), which may modify the shared variables in the driver, it must mask out interrupts to prevent keyboard interrupts from occurring. In kgetc(), the main program first enables interrupts, which is optional if the program is already running with interrupts enabled. Then it loops until the variable kp->data is nonzero, meaning that there are chars in the input buffer. Then it disables interrupts, gets a char from the input buffer and updates the shared variables. The code segment which ensures shared variables can only be updated by one execution entity at a time is commonly known as a **Critical Region or Critical Section** (Silberschatz et al. 2009; Stallings 2011; Wang 2016). Finally, it enables interrupts and returns a char. On the ARM CPU it is not possible to mask out only the keyboard interrupts. The lock() operation masks out all IRQ interrupts, which is a little overkill but it gets the job done. Alternatively, we may write to the keyboard's control register to explicitly disable/enable keyboard interrupts. The disadvantage is that it must access memory mapped locations, which is much slower than masking out interrupts via the CPU's CPSR register.

Figure 3.6 shows the KBD driver using interrupts. As the figure shows, the main program only prints complete lines, but each input key generates an interrupt and prints a "kbd interrupt" message, along with the input char in ASCII.

3.6 UART Driver

3.6.1 ARM PL011 UART Interface

The ARM Versatile board supports four PL011 UART devices for serial I/O (ARM PL011 2005). Each UART device has a base address in the system memory map.

```
UART0: 0x101F1000
UART1: 0x101F2000
UART2: 0x101F3000
UART3: 0x10009000
```

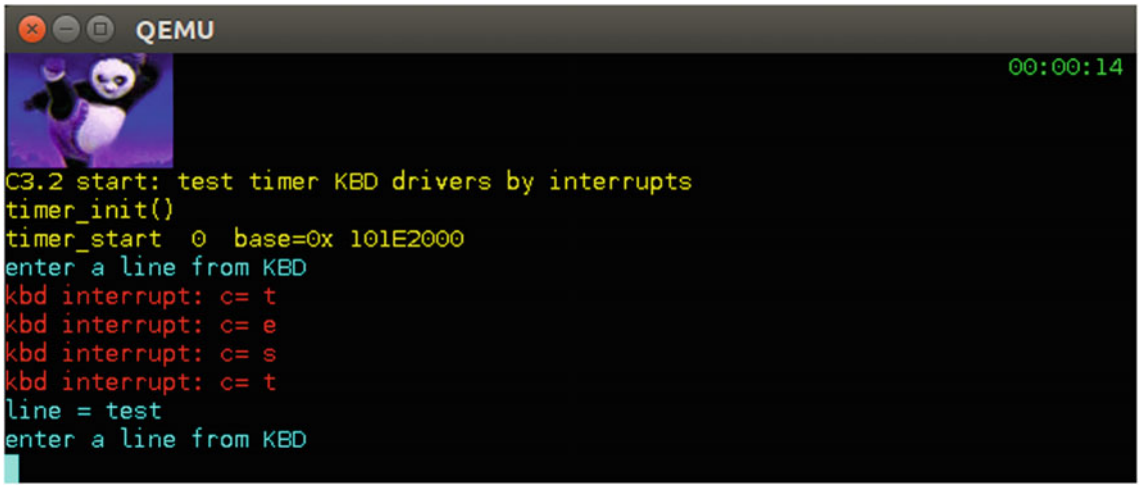


Fig. 3.6 KBD driver using interrupts

The first 3 UARTs, UART0 to UART2 are adjacent in the system memory map. They interrupt at IRQ12 to IRQ14 on the primary PIC. UART4 in located at 0x10000900 and it interrupts at IRQ6 on the SIC. In general, a UART must be initialized by the following steps.

(1). Write a divisor value to the baud rate register for a desired baud rate. The ARM PL011 technical reference manual lists the following integer divisor values (based on 7.38 MHz UART clock) for the commonly used baud rates:

```
0x4=1152000, 0xC=38400, 0x18=192000, 0x20=14400, 0x30=9600
```

(2). Write to Line Control register to specify the number of bits per char and parity, e.g. 8 bits per char with no parity, etc.
(3). Write to Interrupt Mask register to enable/disable RX and TX interrupts

When using the emulated ARM Versatilepb board under QEMU, it seems that QEMU automatically uses default values for both baud rate and line control parameters, making steps (1) and (2) either optional or unnecessary. In fact, it is observed that writing any value to the integer divisor register (0x24) would work, but the reader should be aware this is not the norm for UARTs in real systems. In this case, we only need to program the Interrupt Mask register (if using interrupts) and check the Flag register during serial I/O.

3.6.2 UART Registers

Each UART interface contains several 32-bit registers. The most important UART registers are

```
Offset    Name           Function
------   --------  ------------------------------
 0x00    UARTDR    data register: for read/write chars
 0x04    UARTSR    receive status/clear error
 0x18    UARTFR    TxEmpty, RxFull, etc.
 0x24    UARIBRD   set baud rate
 0x2c    UARTLCR   line control register
 0x30    UARTCR    control register
 0x38    UARTIMSC  interrupt mask for TX and RX
 0x40    UARTMIS   interrupt status
```

Some of the UART registers are already explained in the program C2.3 of Chap. 2. Here, we shall focus on the registers that are related to interrupts. The ARM UART interface supports many kinds of interrupts. For simplicity, we shall only consider Rx (Receiving) and Tx (Transmitting) interrupts for data transfer and ignore such interrupts as modem status and error conditions. To allow UART Rx and Tx interrupts, bits 4 and 5 of the interrupt mask register (UARTIMSC) must be set to 1. When a UART interrupts, the masked interrupt status register (UARTMIS) contains the interrupt identification, e.g. bit 4 = 1 if it it's an Rx interrupt and bit 5 = 1 if it's a Tx interrupt. Depending on the interrupt type, the interrupt handler can branch to a corresponding ISR to handle the interrupt.

3.6.3 Interrupt-Driven UART Driver Program

This section shows the design and implementation of an interrupt-driven UART driver for serial I/O. In order to keep the diver simple, we shall only use UART0 and UART1 but the same code is also applicable to other UARTs. The UART driver program is denoted by C3.3, which is organized as follows.

3.6.3.1 The uart.c File

This file implements the UART driver using interrupts. Each UART is represented by a UART structure. It contains the UART base address, a unit number, an input buffer, an output buffer and control variables. Both buffers are circular, with head pointers for entering chars and tail pointers for removing chars. Among the control variables, data is the number of chars in the buffer and room is the number of empty spaces in the buffer. For outputs, txon is a flag indicating whether the UART is already in the transmission state. To use interrupts, the UART Interrupt Mask Set/Clear register (IMSC) must be set up properly. The UART supports many kinds of interrupts. For simplicity, we shall only consider TX (bit 5) and RX (bit 4) interrupts. In uart_init(), the C statement

```
*(up->base+IMSC) |= 0x30; // bits 4,5 = 1
```

sets bits 4 and 5 of IMSC to 1, which enable TX and RX interrupts of the UART. For simplicity, we shall only use UART0 and UART1. UART0 interrupts at IRQ12 and UART1 interrupts at IRQ13, both on the primary VIC. In order to allow UART interrupts, bits 12 and 13 of the PIC must be set to 1. These are done in main() by the statements

```
VIC_INTENABLE |= (1<<12); // UART0 at bit12
VIC_INTENABLE |= (1<<13); // UART1 at bit13
```

A special feature of the ARM PL011 UART is that it supports FIFO buffers in hardware for both send and receive operations. It can be programmed to raise interrupts when the FIFO buffers are at different levels between FULL and EMPTY. In order to keep the UART driver simple, we shall not use the hardware FIFO buffers. They are disabled by the statement

```
*(up->base+CNTL) &= ~0x10; // disable UART FIFO
```

This makes the UART operate in single char mode. Also, TX interrupt is triggered only after writing a char to the data register. When the system has no more outputs to a UART, it must disable the TX interrupt. The following shows the uart.c driver code.

```
/************* uart.c file ****************/
#define UDR   0x00
#define UDS   0x04
#define UFR   0x18
#define CNTL  0x2C
#define IMSC  0x38
#define MIS   0x40
#define SBUFSIZE 128
```

```
typedef volatile struct uart{
    char *base; // base address; as char *
    int n;      // uart number 0-3
    char inbuf[SBUFSIZE];
    int indata, inroom, inhead, intail;
    char outbuf[SBUFSIZE];
    int outdata, outroom, outhead, outtail;
    volatile int txon; // 1=TX interrupt is on
}UART;
UART uart[4]; // 4 UART structures
int uart_init()
{
    int i; UART *up;
    for (i=0; i<4; i++){ // uart0 to uart2 are adjacent
    up = &uart[i];
    up->base = (char *)(0x101F1000 + i*0x1000);
    *(up->base+CNTL) &= ~0x10; // disable UART FIFO
    *(up->base+IMSC) |= 0x30;
    up->n = i; UART ID number
    up->indata = up->inhead = up->intail = 0;
    up->inroom = SBUFSIZE;
    up->outdata = up->outhead = up->outtail = 0;
    up->outroom = SBUFSIZE;
    up->txon = 0;
}
 uart[3].base = (char *)(0x10009000); // uart3 at 0x10009000
}
void uart_handler(UART *up)
{
 u8 mis = *(up->base + MIS); // read MIS register
 if (mis & (1<<4)) // MIS.bit4=RX interrupt
 do_rx(up);
 if (mis & (1<<5)) // MIS.bit5=TX interrupt
 do_tx(up);
}
int do_rx(UART *up) // RX interrupt handler
{
 char c;
 c = *(up->base+UDR);
 printf("rx interrupt: %c\n", c);
 if (c==0xD)
 printf("\n");
 up->inbuf[up->inhead++] = c;
 up->inhead %= SBUFSIZE;
 up->indata++; up->inroom--;
}
int do_tx(UART *up) // TX interrupt handler
{
 char c;
 printf("TX interrupt\n");
 if (up->outdata <= 0){ // if outbuf[ ] is empty
 *(up->base+MASK) = 0x10; // disable TX interrupt
 up->txon = 0; // turn off txon flag
 return;
 }
```

```
 c = up->outbuf[up->outtail++];
 up->outtail %= SBUFSIZE;
 *(up->base+UDR) = (int)c; // write c to DR
 up->outdata--; up->outroom++;
}

int ugetc(UART *up)           // return a char from UART
{
  char c;
  while(up->indata <= 0); // loop until up->data > 0 READONLY
  c = up->inbuf[up->intail++];
  up->intail %= SBUFSIZE;
  // updating variables: must disable interrupts
  lock();
    up->indata--; up->inroom++;
  unlock();
  return c;
}
int uputc(UART *up, char c)   // output a char to UART
{
  kprintf("uputc %c ", c);
  if (up->txon){ //if TX is on, enter c into outbuf[]
     up->outbuf[up->outhead++] = c;
     up->outhead %= 128;
     lock();
      up->outdata++; up->outroom--;
     unlock();
     return;
  }
  // txon==0 means TX is off => output c & enable TX interrupt
  // PL011 TX is riggered only if write char, else no TX interrupt
  int i = *(up->base+UFR);         // read FR
  while( *(up->base+UFR) & 0x20 ); // loop while FR=TXF
  *(up->base+UDR) = (int)c;        // write c to DR
  UART0_IMSC |= 0x30; // 0000 0000: bit5=TX mask bit4=RX mask
  up->txon = 1;
}
int ugets(UART *up, char *s)       // get a line from UART
{
  kprintf("%s", "in ugets: ");
  while ((*s = (char)ugetc(up)) != '\r'){
    uputc(up, *s++);
  }
 *s = 0;
}
int uprints(UART *up, char *s)     // print a line to UART
{
  while(*s)
    uputc(up, *s++);
}
```

3.6.3.2 Explanations of the UART Driver Code

(1). uart_handler(UART *up): The uart interrupt handler reads the MIS register to determine the interrupt type. It's a RX interrupt if bit 4 of MIS is 1. It's a TX interrupt if bit 5 of MIS is 1. Depending on the interrupt type, it calls do_rx() or do_tx () to handle the interrupt.
(2). do_rx(): This is an input interrupt. It reads the ASCII char from the data register, which clears the RX interrupt of the UART. Then it enters the char into the circular input buffer and increment the data variable by 1. The data variable represents the number of chars in the input buffer.
(3). do_tx(): This is an output interrupt, which is triggered by writing the last char to the output data register and transmission of that char has finished. The handler checks whether there are any chars in the output buffer. If the output buffer is empty, it disables the UART TX interrupt and returns. If the TX interrupt is not disabled, it would cause an infinite sequence of TX interrupts. If the output buffer is not empty, it takes a char from the buffer and writes it to the data register for transmission out.
(4). ugetc(): ugetc is for the main program to get a char from a UART port. Its logic and synchronization with the RX interrupt handler are the same as kgetc() of the keyboard driver. So we shall not repeat them here again.
(5). uputc(): uputc() is for the main program to output a char to a UART port. If the UART port is not transmitting (the txon flag is off), it writes the char to the data register, enables TX interrupt and sets the txon flag. Otherwise, it enters the char into the output buffer, updates the data variable and returns. The TX interrupt handler will output the chars from the output buffer on each successive interrupt.
(6). Formatted printing: uprintf(UART *up, char *fmt,…) is for formatted printing to a UART port. It is based on uputc().

3.6.3.3 Demonstration of KBD and UART Drivers

The t.c file contains the IRQ_handler() and the main() function. The main() function first initializes the UART devices and the VIC for interrupts. Then it tests the UART driver by issuing serial I/O on the UART terminals. For each IRQ interrupt, the IRQ_handler() determines the interrupt source and calls an appropriate handler to handle the interrupt. For clarity, UART related code are shown in bold faced lines.

```
/*************** C3.3. t.c file ******************/
#include "defines.h"
#include "string.c"
#include "uart.c"  // UART driver
#include "kbd.c"   // keyboard driver
#include "timer.c" // timer
#include "vid.c"   // LCD display driver
#include "exceptions.c"

void copy_vectors(void){ // same as before }

void IRQ_handler()       // IRQ interrupt handler in C
{
  // read VIC status registers to find out which interrupt
  int vicstatus = VIC_STATUS;
  // VIC status BITs: timer0,1=4,uart0=13,uart1=14
  if (vicstatus & (1<<5)){         // bit5:timer2,3
     if (*(tp[2]->base+TVALUE)==0) // timer 2
        timer_handler(2);
     if (*(tp[3]->base+TVALUE)==0) // timer 3
        timer_handler(3);
  }
  if (vicstatus & (1<<4)){         // bit4=1:timer0,1
     timer_handler(0);
  }
  if (vicstatus & (1<<12))         // bit12=1: uart0
     uart_handler(&uart[0]);
```

```
  if (vicstatus & (1<<13))         // bit13=1: uart1
     uart_handler(&uart[1]);
  if (vicstatus & (1<<31)){   // PIC.bit31= SIC interrupts
     if (sicstatus & (1<<3)){ // SIC.bit3 = KBD interrupt
        kbd_handler();
     }
  }
}

int main()
{
   char line[128];
   UART *up;
   KBD  *kp;
   fbuf_init();   // initialize LCD display
   printf("C3.4 start: test KBD TIMER UART drivers\n");

   /* enable timer0,1, uart0,1 SIC interrupts */
   VIC_INTENABLE = 0;
   VIC_INTENABLE |= (1<<4);  // timer0,1 at bit4
   VIC_INTENABLE |= (1<<5);  // timer2,3 at bit5
   VIC_INTENABLE |= (1<<12); // UART0 at bit12
   VIC_INTENABLE |= (1<<13); // UART1 at bit13
   VIC_INTENABLE |= (1<<31); // SIC to VIC's IRQ31
   SIC_INTENABLE = 0;
   SIC_INTENABLE |= (1<<3);  // KBD int=bit3 on SIC
   kbd_init();      // initialize keyboard
   uart_init();     // initialize UARTs
   up = &uart[0];   // test UART0 I/O
   kp = &kbd;
   timer_init();
   timer_start(0);  // timer0 only
   while(1){
     kprintf("Enter a line from KBD\n");
     kgets(line);
     kprintf("Enter a line from UART0\n");
     uprints("Enter a line from UARTS\n\r");
     ugets(up, line);
     uprintf("%s\n", line);
   }
}
```

Figure 3.7 shows the outputs of the interrupt-driven KBD and UART drivers. The figure shows that UART inputs are by rx interrupts and outputs are by tx interrupts.

3.7 Secure Digital (SD) Cards

For most embedded systems, the primary mass storage devices are Secure Digital (SD) cards (SDC 2016) due to their compact size, low power consumption and compatibility with other kinds of mobile devices. Many embedded systems may not have any mass storage device to provide file system support, but they usually start up from either a flash memory card or a SD card. A good example is the Raspberry Pi (Raspberry_Pi 2016). It requires a SD card to boot up an operating system, which is usually a version of Linux, called Raspbian, adapted for the ARM architecture. Most ARM based systems include the ARM PrimeCell PL180/ PL181 multimedia card interface (ARM PL180 1998; ARM PL181 2001) to provide support for

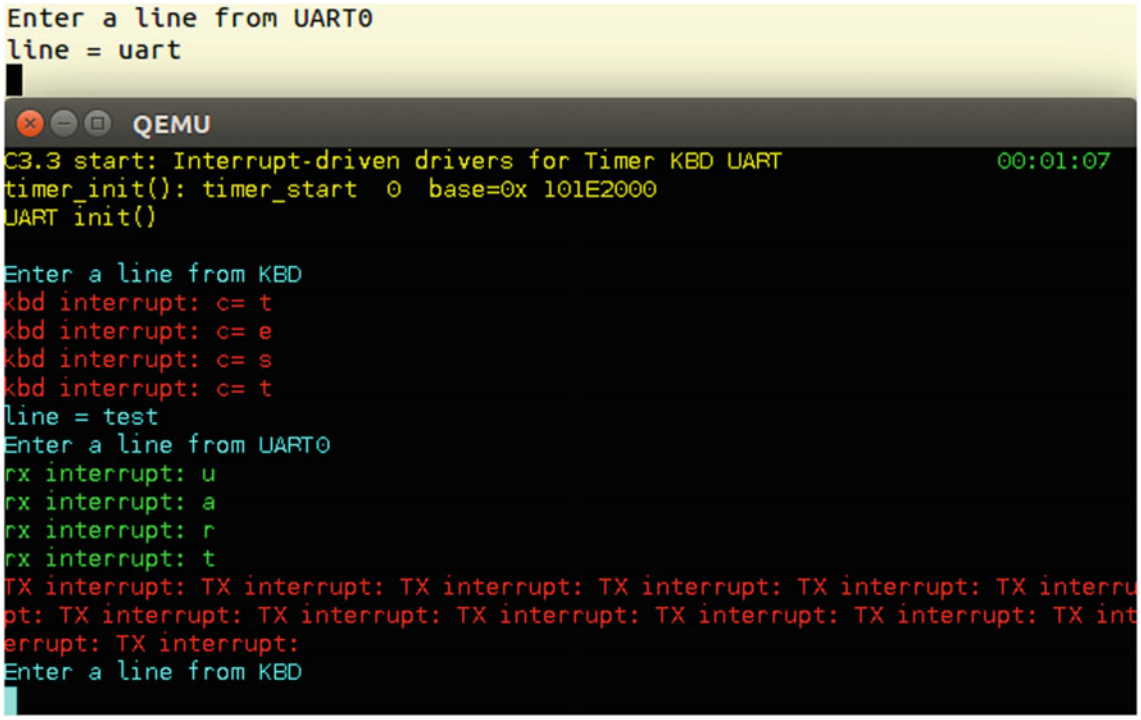


Fig. 3.7 Demonstration of KBD and UART drivers

both multimedia and SD cards. The emulated ARM Versatilepb virtual machine under QEMU includes the PL180 multimedia interface also but it only supports SD cards.

3.7.1 SD Card Protocols

The simplest SD card protocol is the Serial Peripheral Interface (SPI) (SPI 2016). The SPI protocol requires the host machine to have a SPI port, which is available in many ARM based systems. For host machines without SPI ports, SD cards must use the native SD protocol [SD specification 2016], which is more capable and therefore more complex than the SPI protocol. QEMU's multimedia interface supports SD cards in native mode but not in SPI mode. For this reason, we shall develop a SD card driver that operates in the native SD mode.

3.7.2 SD Card Driver

The sample program C3.4 implements an interrupt-driven SD card driver. It demonstrates the SD driver by writing to the sectors of a SD card and then reading back the sectors to verify the results. The program consists of the following components.

(**1**). **sdc.h:** This header file defines the PL180 Multi-Media Card (MMC) registers and bit masks. For the sake of brevity, we only show the PL180 registers.

```
/** PL180 registers from BASE **/
#define   POWER          0x00
#define   CLOCK          0x04
#define   ARGUMENT       0x08
#define   COMMAND        0x0C
#define   RESPCOMMAND    0x10
#define   RESPONSE0      0x14
#define   RESPONSE1      0x18
```

```
#define   RESPONSE2     0x1C
#define   RESPONSE3     0x20
#define   DATATIMER     0x24
#define   DATALENGTH    0x28
#define   DATACTRL      0x2C
#define   DATACOUNT     0x30
#define   STATUS        0x34
#define   STATUS_CLEAR  0x38
#define   MASK0         0x3C
#define   MASK1         0x40
#define   CARD_SELECT   0x44
#define   FIFO_COUNT    0x48
#define   FIFO          0x80
/** more ID registers NOT USED **/
```

(**2**). **sdc.c file:** This file implements the SDC driver.

```
/*********** sdc.c ARM PL180 SDC driver **********/
#include "sdc.h"
u32 base;
#define printf kprintf

int delay(){ int i; for (i=0; i<100; i++); }

int do_command(int cmd, int arg, int resp)
{
  *(u32 *)(base + ARGUMENT) = (u32)arg;
  *(u32 *)(base + COMMAND)  = 0x400 | (resp<<6) | cmd;
  delay();
}
int sdc_init()
{
  u32 RCA = (u32)0x45670000; // QEMU's hard-coded RCA
  base    = (u32)0x10005000; // PL180 base address
  printf("sdc_init : ");
  *(u32 *)(base + POWER) = (u32)0xBF; // power on
  *(u32 *)(base + CLOCK) = (u32)0xC6; // default CLK

  // send init command sequence
  do_command(0,  0,   MMC_RSP_NONE);// idle state
  do_command(55, 0,   MMC_RSP_R1);  // ready state
  do_command(41, 1,   MMC_RSP_R3);  // argument must not be zero
  do_command(2,  0,   MMC_RSP_R2);  // ask card CID
  do_command(3,  RCA, MMC_RSP_R1);  // assign RCA
  do_command(7,  RCA, MMC_RSP_R1);  // transfer state: must use RCA
  do_command(16, 512, MMC_RSP_R1);  // set data block length

  // set interrupt MASK0 registers bits = RxAvail|TxEmpty
  *(u32 *)(base + MASK0) = (1<<21)|(1<<18); //0x00240000;
  printf("done\n");
}

// shared variables between SDC driver and interrupt handler
volatile char *rxbuf, *txbuf;
```

```
volatile int  rxcount, txcount, rxdone, txdone;
int sdc_handler()
{
  u32 status, err;
  int i;
  u32 *up;

  // read status register to find out RxDataAvail or TxBufEmpty
  status = *(u32 *)(base + STATUS);

  if (status & (1<<21)){ // RxDataAvail: read data
    printf("SDC RX interrupt: );
    up = (u32 *)rxbuf;
    err = status & (DCRCFAIL | DTIMEOUT | RXOVERR);
    while (!err && rxcount) {
       printf("R%d ", rxcount);
       *(up) = *(u32 *)(base + FIFO);
       up++;
       rxcount -= sizeof(u32);
       status = *(u32 *)(base + STATUS);
       err = status & (DCRCFAIL | DTIMEOUT | RXOVERR);
    }
    rxdone = 1;
  }
  else if (status & (1<<18)){ // TxBufEmpty: send data
    printf("SDC TX interrupt: );
    up = (u32 *)txbuf;
    status_err = status & (DCRCFAIL | DTIMEOUT);

    while (!status_err && txcount) {
       printf("W%d ", txcount);
       *(u32 *)(base + FIFO) = *up;
       up++;
       txcount -= sizeof(u32);
       status = *(u32 *)(base + STATUS);
       status_err = status & (DCRCFAIL | DTIMEOUT);
    }
    txdone = 1;
  }
  //printf("write to clear register\n");
  *(u32 *)(base + STATUS_CLEAR) = 0xFFFFFFFF;
  printf("SDC interrupt handler return\n");
}

int get_sector(int sector, char *buf)
{
  u32 cmd, arg;

  //printf("get_sector %d %x\n", sector, buf);
  rxbuf = buf; rxcount = 512; rxdone = 0;
  *(u32 *)(base + DATATIMER) = 0xFFFF0000;
  // write data_len to datalength reg
  *(u32 *)(base + DATALENGTH) = 512;

  //printf("dataControl=%x\n", 0x93);
  // 0x93=|9|0011|=|9|DMA=0,0=BLOCK,1=Host<-Card,1=Enable
```

```
  *(u32 *)(base + DATACTRL) = 0x93;
  cmd = 17;       // CMD17 = READ single block
  arg = (sector*512);
  do_command(cmd, arg, MMC_RSP_R1);
  while(rxdone == 0);
  printf("get_sector return\n");
}

int put_sector(int sector, char *buf)
{
  u32 cmd, arg;

  //printf("put_sector %d %x\n", sector, buf);
  txbuf = buf; txcount = 512; txdone = 0;
  *(u32 *)(base + DATATIMER) = 0xFFFF0000;
  *(u32 *)(base + DATALENGTH) = 512;

  cmd = 24;      // CMD24 = Write single block
  arg = (u32)(sector*512);
  do_command(cmd, arg, MMC_RSP_R1);

  //printf("dataControl=%x\n", 0x91);
  // 0x91=|9|0001|=|9|DMA=0,BLOCK=0,0=Host->Card,1=Enable
  *(u32 *)(base + DATACTRL) = 0x91; // Host->card
  while(txdone == 0);
  printf("put_sector return\n");
}
```

The SDC driver consists of 4 major parts, which are explained below.

(1). sdc_init(): This is called from the main program to initialize the SDC. When writing SDC drivers, the most crucial step is the SD card initialization, which comprises the following steps.

. send CMD0 to get the SDC into idle state
. send CMD8 to determine SDC type and voltage range
. send CMD55 followed by CMD41 to get the SDC into ready state
. send CMD2 to get Card Identification (CID)
. send CMD3 to get/set Card Relative Address (RCA)
. send CMD7 to select the SDC by RCA to transfer data state.
. set Interrupt Mask register MASK0 bits to enable Rx and Tx interrupts

In general, every command except CMD0 expects a response of different type. After sending a command, the responses are in the response registers, but they are ignored in the SD driver. It is observed that in the emulated PL180 MMC of QEMU, the Relative Card Address (RCA) assigned to the SDC is hard-coded as 0x4567. In fact, each CMD3 command increases the RCA by 0x4567. The reason for this rather peculiar behavior is probably due to a typo in the PL180 emulator of QEMU. It should just set the RCA once by RCA = 0x4567, rather than RCA += 0x4567, which increments it by 0x4567 on each CMD3 command. After initialization, the driver may issue a CMD17 to read a block or a CMD24 to write a block. For SDC, the default block (sector) size is 512 bytes. The SDC also supports read/write multiple sectors by CMD18 and CMD25, respectively. Data transfer may use one of three different schemes: polling, interrupts or DMA. DMA is suitable for transferring large amounts of data. In order to keep the driver code simple, the SDC driver only uses interrupts, not DMA, to read/write one sector of 512 bytes at a time. It will be extended to read/write multi-sectors later. SDC interrupts are enabled by setting bits in the Interrupt Mask registers MASK0. In sdc_init(), the interrupt mask bits are set to RxDataAvail (bit 21) and TxBufEmpty (bit 18). The MMC will generate a SDC interrupt when either there are data in the input buffer or there are rooms in the output buffer. It disables and ignores other kinds of SDC interrupts, e.g. interrupts due to error conditions.

(2). get_sector(int sector, char *buf): get_sector() is for reading a sector of 512 bytes from the SDC. The algorithm of get_sector() is as follows.
. Set global rxbuf = buf and rxdone = 0 for the Rx interrupt handler to use;
. Set DataTimer to default, set DataLength to 512;
. Set DataCntl to 0x93 (block size = 2**9, respR1, SDC to Host, and Enable);
. Send CMD17 with argument = sector*512 (byte address on SDC);
. (Busy) wait for the Rx interrupt handler to finish reading data;

Upon receiving a CMD17, the PL180 MultiMedia Controller (MMC) starts to transfer data from SDC to its internal input buffer. The MMC has a 16x32 bits FIFO input data buffer. When data becomes available, it generates an Rx interrupt, causing SDC_handler() to be executed, which actually transfers data from the MMC to rxbuf. After sending CMD17, the main program busily waits for a volatile rxdone flag, which will be set by the interrupt handler when data transfer completes.

(3). put_sector(int sector, char *buf): put_sector() is for writing a block of data to the SDC. The algorithm of put_sector() is as follows.

. Set global txbuf=buf and txdone=0 for the Tx interrupt handler to use;
. Set DataTimer to default and DataLength to 512;
. Send CMD24 with argument=sector*512 (byte address on SDC);
. Set DataCntl to 0x91 (block size=2**9, respR1, Host to SDC, Enable);
. (Busy) wait for the Tx interrupt handler to finish writing data;

Upon receiving a CMD24, the PL180 MMC starts to transfer data. The MMC has a 16x32 bits FIFO output data buffer. If the Tx buffer is empty, it generates an SDC interrupt, causing SDC_handler() to be executed, which actually transfers data from buf to the MMC. After sending CDM24, the main program busily waits for a volatile txdone flag, which will be set by the interrupt handler when data transfer completes.

(4). sdc_handler(): This is the SDC interrupt handler. It first checks the status register to determine the interrupt source. If it is an RxDataAvail interrupt (bit 21 is set), it transfers data from the MMC controller to rxbuf by a loop.

```
while (!err && rxcount) {
  printf("R%d ", rxcount);
  *(up) = *(u32 *)(base + FIFO);
  up++;
  rxcount -= sizeof(u32);
  status = *(u32 *)(base + STATUS);
  err = status & (SDI_STA_DCRCFAIL | SDI_STA_DTIMEOUT |
                  SDI_STA_RXOVERR);
}
```

Barring any errors, each iteration of the loop reads a u32 (4 bytes) from the MMC's FIFO input buffer, decrements rxcount by 4 until rxcount reaches 0. Then it sets the rxdone flag to 1, allowing the main program in get_sector() to continue.

If the SDC interrupt is a Tx interrupt (bit 18 is set), it writes data from txbuf to the MMC's FIFO by a loop.

```
while (!err && txcount){
   printf("W%d ", txcount);
   *(u32 *)(base + FIFO) = *up;
   up++;
   txcount -= sizeof(u32);
   status = *(u32 *)(base + STATUS);
   err = status & (SDI_STA_DCRCFAIL | SDI_STA_DTIMEOUT);
}
```

Barring any errors, each iteration of the loop writes a u32 (4 bytes) to the MMC's FIFO output buffer, decrements txcount by 4 until txcount reaches 0. Then it sets the txdone flag to 1, allowing the main program in put_sector() to continue.

(**3**). **t.c file:** The t.c file is the same as in C3.3, except for the added code for SDC initialization and testing. For clarity, the modified lines of t.c are shown in bold face letters.

```
#include "defines.h"
#include "string.c"
#define printf kprintf
char *tab = "0123456789ABCDEF";
#include "uart.c"
#include "kbd.c"
#include "timer.c"
#include "vid.c"
#include "exceptions.c"
#include "sdc.c"

void copy_vectors(void) { // same as before }

void IRQ_handler()
{
   int vicstatus, sicstatus;
   int ustatus, kstatus;
   // read VIC SIV status registers to find out which interrupt
   vicstatus = VIC_STATUS;
   sicstatus = SIC_STATUS;
   // VIC status BITs: timer0=4, uart0=13, uart1=14, SIC=31: KBD at 3
   if (vicstatus & (1<<4)){  // bit 4: timer0
      timer_handler(0);
   }
   if (vicstatus & (1<<12)){ // Bit 12: UART0
      uart_handler(&uart[0]);
   }
   if (vicstatus & (1<<13)){ // bit 13: UART1
      uart_handler(&uart[1]);
   }
   if (vicstatus & (1<<31)){ // SIC interrupts=bit_31 on VIC
      if (sicstatus & (1<<3)){  // KBD at IRQ3 of SIC
         kbd_handler();
      }
      if (sicstatus & (1<<22)){ // SDC at IRQ22 of SIC
         sdc_handler();
      }
   }
}

char rbuf[512], wbuf[512];
char *line[2] = {"THIS IS A TEST LINE", "this is a test line"};

int main()
{
   int i, sector, N;
   fbuf_init();
   kbd_init();
   uart_init();
```

```
   /* enable timer0,1, uart0,1 SIC interrupts */
   VIC_INTENABLE = (1<<4);   // timer0,1 at bit4
   VIC_INTENABLE |= (1<<12); // UART0 at bit12
   VIC_INTENABLE |= (1<<13); // UART1 at bit13
   VIC_INTENABLE |= (1<<31); // SIC to VIC's IRQ31
   /* enable KBD and SDC IRQ */
   SIC_INTENABLE = (1<<3);   // KBD int=bit3 on SIC
   SIC_INTENABLE |= (1<<22); // SDC int=bit22 on SIC
   SIC_ENSET = (1<<3);       // KBD int=3 on SIC
   SIC_ENSET |= (1<<22);     // SDC int=22 on SIC
   timer_init();
   timer_start(0);
   /* Code for testing UART and KBD drivers are omitted */
   printf("test SDC DRIVER\n");
   sdc_init();
   N = 1;        // Write|Read N sectors of SDC
   for (sector=0; sector < N; sector++){
     printf("WRITE sector %d: ", sector);
     memset(wbuf, ' ', 512);  // blank out wbuf
     for (i=0; i<12; i++)     // write lines to wbuf
         strcpy(wbuf+i*40, line[sector % 2]);
     put_sector(sector, wbuf);
   }
   printf("\n");

   for (sector=0; sector < N; sector++){
     printf("READ  sector %d\n", sector);
     get_sector(sector, rbuf);
     for (i=0; i<512; i++){
       printf("%c", rbuf[i]);
     }
     printf("\n");
   }
   printf("in while(1) loop: enter keys from KBD or UART\n");
   while(1);
}
```

The ARM Versatile user manual specifies that the MMCI0 interrupts at IRQ22 on both the VIC and SIC. However, in the PL180 emulated by QEMU, it actually interrupts at IRQ22 of the SIC, which is routed to IRQ31 of the VIC. The reason for this discrepancy is unknown. Other than this minor discrepancy, the emulated PL180 works as expected. Figure 3.8 shows the outputs of running the SDC driver program C3.4.

3.7.3 Improved SDC Driver

In the SDC driver, the interrupt handler performs all the data transfer on a single interrupt. Since data transfer from the MMC to the SDC may be slow, the interrupt handler must execute the data transfer loop many times while waiting for the MMC to become ready to provide or accept data. The drawback of this scheme is that it is essentially the same as, or even worse than, I/O by polling. In general, an interrupt handler should be completed as soon as possible. Any excessive checking and waiting inside an interrupt handler must be avoided or eliminated. It is therefore desirable to minimize the number of interrupts and maximize the amount of data transfer on each interrupt. This leads us to an improved SDC driver. In the improved SDC driver, we program the MMC to generate interrupts only when the Rx FIFO is full or the Tx FIFO is empty. For each interrupt, we transfer 16 u32 data on each interrupt. The following code segments show the improved SDC driver, in which the modifications are shown in bold faced lines.

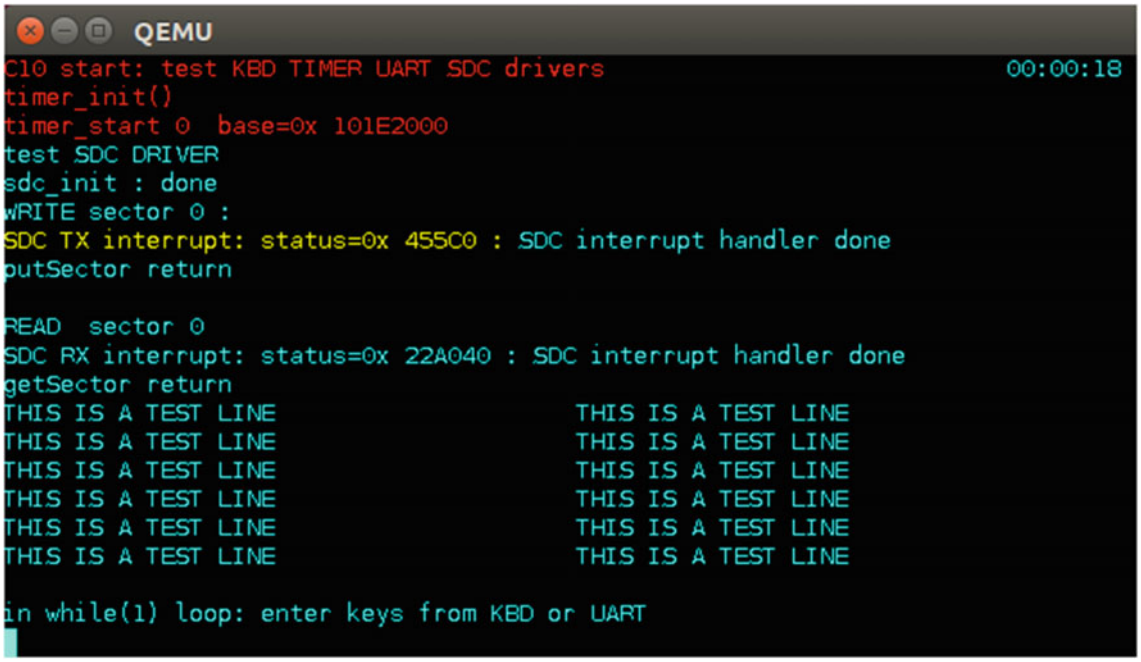


Fig. 3.8 Outputs of SDC driver program

```
// shared variables between SDC driver and interrupt handler
volatile char *rxbuf, *txbuf;
volatile int  rxcount, txcount, rxdone, txdone;

int sdc_handler()
{
  u32 status, status_err;
  int i;
  u32 *up;
  // read status register to find out TXempty or RxFull interrupt
  status = *(u32 *)(base + STATUS);

  if (status & (1<<17)){ // RxFull: read 16 u32 at a time;
     printf("RX interrupt: ");
     up = (u32 *)rxbuf;
     status_err = status & (DCRCFAIL | DTIMEOUT | RXOVERR);
     if (!status_err && rxcount) {
        printf("R%d ", rxcount);
        for (i = 0; i < 16; i++)
           *(up + i) = *(u32 *)(base + FIFO);
        up += 16;
        rxcount -= 64;
        rxbuf += 64;
        status = *(u32 *)(base + STATUS); // clear Rx interrupt
     }
     if (rxcount == 0)
         rxdone = 1;
  }
  else if (status & (1<<18)){ // TXempty: write 16 u32 at a time
    printf("TX interrupt: ");
```

```
    up = (u32 *)txbuf;
    status_err = status & (DCRCFAIL | DTIMEOUT);
    if (!status_err && txcount) {
       printf("W%d ", txcount);
       for (i = 0; i < 16; i++)
           *(u32 *)(base + FIFO) = *(up + i);
       up += 16;
       txcount -= 64;
       txbuf += 64;            // advance txbuf for next write
       status = *(u32 *)(base + STATUS); // clear Tx interrupt
    }
    if (txcount == 0)
        txdone = 1;
  }
  //printf("write to clear register\n");
  *(u32 *)(base + STATUS_CLEAR) = 0xFFFFFFFF;
  // printf("SDC interrupt handler done\n");
}
```

Figure 3.9 shows the outputs of running the improved SDC driver program. As the figure shows, each interrupt transfers 16 4-byte data, so that the byte transfer count decrements by 64 on each interrupt. As a further improvement, the reader may program the MMC to generate interrupts when the Rx FIFO is half-full and Tx FIFO is half-empty. In that case, each interrupt can transfer 8 4-byte data. It improves the data transfer rate at the expense of more interrupts and hence more overhead due to interrupt processing.

3.7.4 Multi-sector Data Transfer

The above SD drivers transfer one sector (512 bytes) of data at a time. An embedded system may support file systems, which usually use 1 or 4 kB file block size. In that case, it would be more efficient to transfer data from/to SD cards in multi-sectors that matches file block size. The following code segments show the modified SD driver which transfers data in multi-sectors.

To read multi-sectors, issue the command CMD18. To write multi-sectors, issue the command CMD25. In both cases, the data length is the file block size. For multi-sector data transfers, data transmission must be terminated by a stop transmission command CMD12, which is issued in the interrupt handler when the byte count (rxcount or txcount) reaches 0. The modified lines of the driver are shown in bold face. In the code segment, FBLK_SIZE is defined as 4096. Each get_block()/put_block() call read/write a (file) block of 4 KB data.

```
// shared variables between SDC driver and interrupt handler
#define FBLK_SIZE 4096
volatile char *rxbuf, *txbuf;
volatile int  rxcount, txcount, rxdone, txdone;

int sdc_handler()
{
  u32 status, err, *up;
  int i;
  // read status register to find out TXempty or RxAvail
  status = *(u32 *)(base + STATUS);
  if (status & (1<<17)){ // RxFull: read 16 u32 at a time;
     up = (u32 *)rxbuf;
     err = status & (DRCFAIL | DTIMEOUT | RXOVERR);
     if (!err && rxcount){
        for (i = 0; i < 16; i++)
```

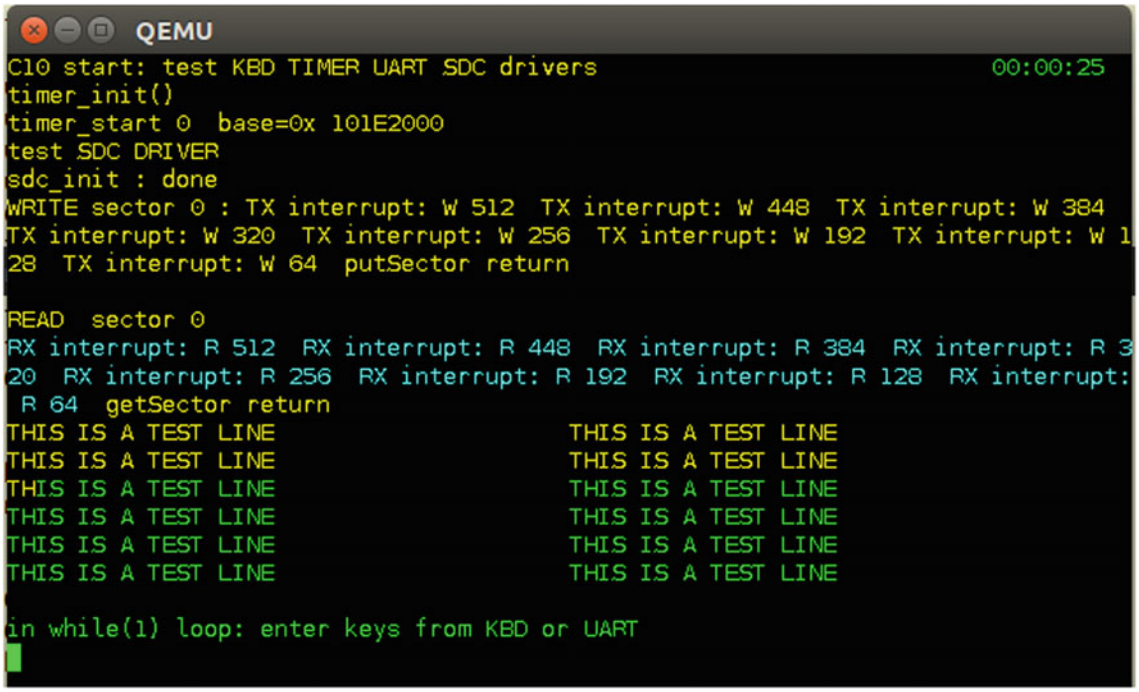


Fig. 3.9 Outputs of the improved SDC driver

```
            *(up + i) = *(u32 *)(base + FIFO);
        up += 16;
        rxcount -= 64;
        rxbuf += 64;
        status = *(u32 *)(base + STATUS); // read status
     }
     if (rxcount == 0){
        do_command(12, 0, MMC_RSP_R1); // stop transmission
        rxdone = 1;
     }
  }
  else if (status & (1<<18)){ // TXempty: write 16 u32 at a time
    up = (u32 *)txbuf;
    err = status & (DCRCFAIL | DTIMEOUT);
    if (!err && txcount) {
       for (i = 0; i < 16; i++)
           *(u32 *)(base + FIFO) = *(up + i);
       up += 16;
       txcount -= 64;
       txbuf += 64;            // advance txbuf for next write
       status = *(u32 *)(base + STATUS); // read status
    }
    if (txcount == 0){
       do_command(12, 0, MMC_RSP_R1); // stop transmission
       txdone = 1;
    }
  }
  *(u32 *)(base + STATUS_CLEAR) = 0xFFFFFFFF;
```

```
}
   int get_block(int blk, char *buf)
{
  u32 cmd, arg;
  rxbuf = buf; rxcount = FBLK_SIZE; rxdone = 0;
  *(u32 *)(base + DATATIMER) = 0xFFFF0000;
  // write data_len to datalength reg
  *(u32 *)(base + DATALENGTH) = FBLK_SIZE;
  // 0x93=|9|0011|=|9|DMA=0,0=BLOCK,1=Host<-Card,1=Enable
  *(u32 *)(base + DATACTRL) = 0x93; // Card to Host

  cmd = 18;       // CMD18 = READ multi-sectors
  arg = (blk*FBLK_SIZE);
  do_command(cmd, arg, MMC_RSP_R1);

  while(rxdone == 0);
  printf("get_block return\n");
}

int put_block(int blk, char *buf)
{
  u32 cmd, arg;
  txbuf = buf; txcount = FBLK_SIZE; txdone = 0;
  *(u32 *)(base + DATATIMER) = 0xFFFF0000;
  *(u32 *)(base + DATALENGTH) = FBLK_SIZE;

  cmd = 25;      // CMD25 = Write multi-sectors
  arg = (u32)(blk*fBLK_SIZE);
  do_command(cmd, arg, MMC_RSP_R1);

  // 0x91=|9|0001|=|9|DMA=0,0=BLOCK,0=Host->Card,1=Enable
  *(u32 *)(base + DATACTRL) = 0x91; // Host to Card

  while(txdone == 0);
  printf("put_block return\n");
}
```

The reader may replace the SDC driver in the sample program C3.4 with the above code and test run the program to verify multi-sector data transfers. The reader may also change FBLK_SIZE to suit other block sizes, which must be a multiple of sector size (512).

3.8 Vectored Interrupts

So far, all the example programs use non-vectored interrupts. The disadvantage of non-vectored interrupts is that the interrupt handler must scan the interrupt status register for non-zero bits to determine the interrupt sources, which is time-consuming. In many other computer systems, such as the Intel x86 based PCs, interrupts are vectored by hardware. In the vectored interrupt scheme, each interrupt is assigned a vector number determined by the interrupt priority. When an interrupt occurs, the CPU can get the vector number of the interrupt from the interrupt controller hardware and uses it to invoke a corresponding interrupt service routine directly. The ARM PL190 Vectored Interrupt Controller (VIC) also has this capability. In this section, we show how to program the PL190 VIC for vectored interrupts processing.

3.8.1 ARM PL190 Vectored Interrupts Controller (VIC)

The PL190 VIC of the ARM Versatile/926EJ-S board supports vectored interrupts. The VIC technical manuals [ARM PL190 2004] contain the following information on how to program the VIC for vectored interrupts.

. VectorAddr Register (0x30): The VectorAddr register contains the ISR address of current active IRQ. At the end of current ISR, write a value to this register to clear the current interrupt. The PL192 VIC has the additional capability of prioritizing the IRQ sources by writing values 0–15 to the IntPriority registers. At the end of the current ISR, writing to the VectorAddr register allows the VIC to re-prioritize pending IRQs.
. DefaultVecAddr Register (0x34): This register contains the ISR address of a default interrupt, e.g. for any spurious interrupt.
. VectorAddress Registers [0–15] (0x100-0x13C): Each of these registers contains the ISR address of IRQ0 to IRQ15. The PL192 has 32 vectorAddress registers for 32 ISRs.
. VectorControl Registers [0–15] (0x200-0x23C): Each of these registers contains the interrupt source (bits 4–1) and an Enable bit (bit 5).

In order to use vectored interrupts, each device must be enabled for interrupts at both the device level and also on the VIC. We demonstrate vectored interrupts by the sample program C3.5 for the following devices.

```
. Timer0:     VectorInt0
. UART0:      VectorInt1
. UART1:      VectorInt2
. Keyboard:   VectorInt3
```

The code segments used to program the VIC for vectored interrupts are listed below.

3.8.2 Configure VIC for Vectored Interrupts

C3.5.1 : The function vecotrInt_init() configures the VIC for vectored interrupts.

```
int vectorInt_init() / Use vectored interrupts of PL190 VIC
{
  printf("vectorInterrupt_init()\n");

  /*********** set up vectored interrupts *****************
  (1). write to vectoraddr0 (0x100) with ISR of timer0
               vectoraddr1 (0x104) with ISR of UART0
               vectoraddr2 (0x108) with ISR of UART1
               vectoraddr3 (0x10C) with ISR of KBD
  ******************************************************/
  *((int *)(VIC_BASE_ADDR+0x100)) = (int)timer0_handler;
  *((int *)(VIC_BASE_ADDR+0x104)) = (int)uart0_handler;
  *((int *)(VIC_BASE_ADDR+0x108)) = (int)uart1_handler;
  *((int *)(VIC_BASE_ADDR+0x10C)) = (int)kbd_handler;
  // (2). write to intControlRegs  = E=1|IRQ#=1xxxxx
  *((int *)(VIC_BASE_ADDR+0x200)) = 0x24;  //100100 at IRQ 4
  *((int *)(VIC_BASE_ADDR+0x204)) = 0x2C;  //101100 at IRQ 12
  *((int *)(VIC_BASE_ADDR+0x208)) = 0x2D;  //101101 at IRQ 13
  *((int *)(VIC_BASE_ADDR+0x20C)) = 0x3F;  //111111 at IRQ 31
  // (3). write 0's to IntSelectReg to generate IRQ interrupts
  //(any bit=1 generates FIQ interrupt)
  *((int *)(VIC_BASE_ADDR+0x0C)) = 0;
}
```

3.8.3 Vectored Interrupts Handlers

C3.5.2. Rewrite IRQ_handler() for Vectored Interrupts. When using vectored interrupts, any IRQ interrupt still comes to the IRQ_handler() as usual. However, we must rewrite IRQ_handler() to use vectored interrupts. Upon entry to IRQ_handler(), we must read the VectorAddr register to acknowledge the interrupt first. Unlike the non-vectored interrupt case, there is no need to read the status registers to determine the interrupt source. Instead, we can get the address of the current IRQ handler from the vectorAddr register directly. Then simply invoke the handler by its entry address. In addition, the interrupt source can also be determined from the VectorStatus register. Upon return from the handler, send EOI to the VIC controller by writing a (any) value to the vectorAddr register, allowing it to re-prioritize pending interrupt requests. The following shows the modified IRQ_handler() function for vectored interrupts.

```
void IRQ_handler( )
{
  void *(*f)( );     // f as a function pointer
  int status = *(int *)(VIC_BASE_ADDR+0x30);
  f =(void *)*((int *)(VIC_BASE_ADDR+0x30));
  f();               // call the ISR function
  *((int *)(VIC_BASE_ADDR+0x30)) = 1; // write to vectorAddr as EOI
}
```

3.8.4 Demonstration of Vectored Interrupts

C3.5.3. t.c file: For the sake of brevity, we only show the relevant code of main() that are used to test the various device interrupts.

```
int main()
{
   fbuf_init();  // LCD display
   kbd_init();   // KBD
   uart_init();  // UARTs
   timer_init(); // Timer
   up = &uart[0];
   kp = &kbd;
   /* enable timer0,1, uart0,1 SIC interrupts */
   VIC_INTENABLE = 0;
   VIC_INTENABLE |= (1<<4);  // timer0,1 at bit4
   VIC_INTENABLE |= (1<<12); // UART0 at bit12
   VIC_INTENABLE |= (1<<13); // UART1 at bit13
   VIC_INTENABLE |= (1<<31); // SIC to VIC's IRQ31
   /* enable KBD IRQ from SIC */
   SIC_INTENABLE = 0;
   SIC_INTENABLE |= (1<<3);  // KBD int=bit3 on SIC
   printf("Program start: test Vectored Interrupts\n");
   vectorInt_init(); // must do this AFTER driver_init()
   timer_start(0);   // start Timer0
   printf("test UART0 I/O: enter text from UART 0\n");
   while(1){
     ugets(up, line);
     uprintf("  line=%s\n", line);
     if (strcmp(line, "end")==0)
        break;
   }
```

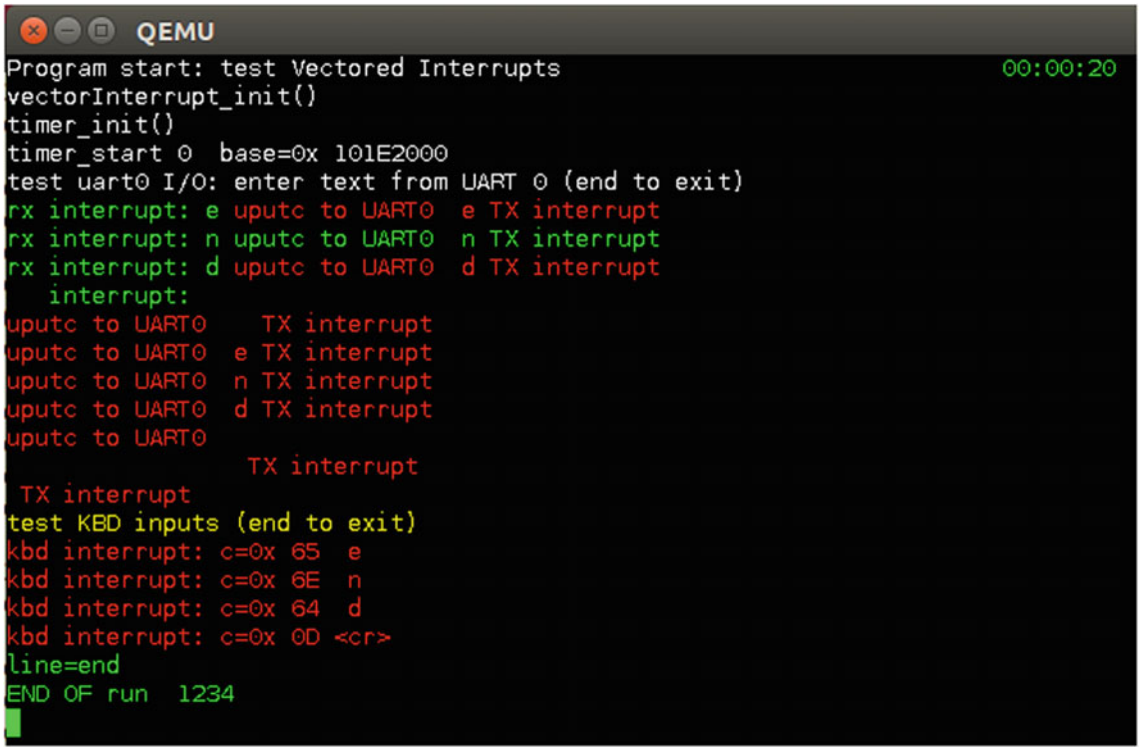


Fig. 3.10 Demonstration of vectored interrupts

```
   printf("test UART1 I/O: enter text from UART 1\n");
   up = &uart[1];
   while(1){
     ugets(up, line);
     ufprintf(up, "  line=%s\n", line);
     if (strcmp(line, "end")==0)
        break;
   }
   printf("test KBD inputs\n"); // print to LCD
   while(1){
      kgets(line);
      printf("line=%s\n", line);
      if (strcmp(line, "end")==0)
         break;
   }
   printf("END OF run %d\n", 1234);
}
```

Figure 3.10 shows the sample outputs of running the program C3.5, which demonstrates timer, UART and KBD drivers using vectored interrupts.

3.9 Nested Interrupts

3.9.1 Why Nested Interrupts

In the above example programs, C3.1–C3.5, which demonstrate interrupts processing, every IRQ interrupt handler begins execution with IRQ interrupts disabled (masked out). IRQ interrupts are not enabled (masked-in) until the interrupt handler

has finished. This implies that interrupts can only be handled one at a time. The disadvantage of this scheme is that it may lead to **interrupt priority inversion**, in which processing a low priority interrupt may block or delay processing of higher priority interrupts. Interrupt priority inversion may increase the system response time to interrupts, which is undesirable in embedded systems with critical timing requirements. To remedy this, embedded systems should allow nested interrupts. In the nested interrupts scheme, a higher priority interrupt may preempt the processing of lower priority interrupts, i.e. before the current interrupt handler finishes, it can accept and handle interrupts of higher priorities, thereby reducing interrupts processing latency and improving the system response to interrupts.

3.9.2 Nested Interrupts in ARM

The ARM processor is not designed to support nested interrupts efficiently, due to the following properties of the ARM processor architecture.

When the ARM CPU accepts an IRQ interrupt, it switches to IRQ mode, which has its own banked registers lr_irq, sp_irq, spsr and cpsr. The CPU saves the return address (with an +4 offset) into lr_irq, saves the previous mode CPSR into spsr and enters the interrupt handler to handle the current interrupt. In order to support nested interrupts, the interrupt handler must unmask interrupts at some point in time to allow interrupts of higher priorities to occur. However, this creates two problems.

(1). Accepting another interrupt may corrupt the link register: Assume that, after enabling IRQ interrupts, the interrupt handler calls an ISR to handle the current interrupt, as in

```
irq_handler:
    // find out the IRQ source to determine its ISR
    // enable IRQ interrupts
    bl  ISR
HERE:
```

When calling the ISR to handle the current interrupt, the link register lr_irq contains the return address to the label HERE. While executing the ISR, if another interrupt occurs, the CPU would re-enter irq_handler, which changes lr_irq to the return address of the new interrupt point. This corrupts the original link register lr_irq, causing the ISR to return to the wrong address when it finishes.

(2). Over-writing saved CPSR: When the ARM processor accepts an interrupt, it saves the CPSR of the interrupted point in the (banked) SPSR_irq, The saved SPSR may be in USER or SVC mode if the interrupted code was executing in USER or SVC mode. While executing an ISR in IRQ mode, if another interrupt occurs the CPU would over-write the SPSR_irq with the CPSR in IRQ mode, which would cause the first ISR to return to the wrong mode when it finishes.

It is fairly easy to deal with the problem in (2). Upon entry to the interrupt handler but before enabling IRQ for further interrupts, we can save SPSR into the IRQ mode stack. When the ISR finishes, we restore the saved SPSR from the stack. However, if we allow nested interrupts in IRQ mode, there is no way to overcome the problem in (1). It will cause an infinite loop since every ISR would return to the beginning of the interrupt handler again. The only way to alleviate this problem is to steer the CPU away from IRQ mode. For this reason, ARM introduced the SYS mode, which is a privileged mode but has a different link register than the IRQ mode. Assume that, before enabling further IRQ interrupts, we switch the CPU to SYS mode and call an ISR in SYS mode. If another interrupt occurs, it would alter the IRQ mode link register lr_irq but not the link register lr_sys in SYS mode. This allows the ISR to return to the correct address when it finishes. So the scheme of handling nested interrupt is as follows.

3.9.3 Handle Nested Interrupts in SYS Mode

irq_handler: // entry point of IRQ interrupts
(1). Save context of interrupted point and lr into IRQ mode stack

(2). Save SPSR into IRQ mode stack
(3). Switch to SYS mode with IRQ disabled
(4). Determine interrupt source and clear interrupt.
(5). Enable IRQ interrupts
(6). Call ISR in SYS mode for the current IRQ interrupt
(7). Switch back to IRQ mode with IRQ disabled
(8). Restore saved SPSR form IRQ mode stack
(9). Return by the saved context and lr in IRQ mode stack

Many ARM processors require 8-byte aligned stacks. When changing the CPU to SYS mode, it may be necessary to check and adjust the SYS mode stack for proper alignment first. Since the SYS mode stack begins at an 8-byte boundary, we may assume that the checking and adjustment is unnecessary. The following list the irq_handler code which implements the above algorithm.

```
irq_handler:
  stmfd sp!, {r0-r3, r12, lr} // save context in IRQ stack
  mrs   r12, spsr             // copy spsr into r12
  stmfd sp!, {r12}            // save SPSR in IRQ stack
  msr cpsr, #0x9F        // switch to SYS mode with IRQ disabled
  ldr r1, =vectorAddr    // read vectorAddr reg to ack interrupt
  ldr r0, [r1]
  msr   cpsr, #0x1F      // enable IRQ in SYS mode

  bl    IRQ_handler      // handle current IRQ in SYS mode

  msr   cpsr, #0x92      // switch back to IRQ mode with IRQ disabled
  ldmfd sp!, {r12}       // get saved SPSR from IRQ stack
  msr   spsr, r12        // restore spsr
  ldr  r1, =vectorAddr   // write to vectorAddr as EOI
  str  r0, [r1]
  ldmfd sp!, {r0-r3, r12, lr} // restore saved context from IRQ stack
  subs pc, r14, #4            // return to interrupted point
```

ARM recommends handling nested interrupts in SYS mode (Nesting Interrupts 2011) but it can also be done in SVC mode, which is used in the demonstration program.

3.9.4 Demonstration of Nested Interrupts

C3.6.1. ts.s file: First, we show the modified irq_handler for nested interrupts. Instead of SYS mode, it handles nested interrupts in SVC mode. The get_cpsr() function returns the processor mode in CPSR. It is for displaying the current mode of the CPU.

```
.set vectorAddr, 0x10140030  // VIC vectorAddress register
irq_handler:
  stmfd sp!, {r12, lr}  // save r12, lr in IRQ stack
  mrs   r12, spsr       // copy spsr into r12
  stmfd sp!, {r12}      // save spsr in IRQ stack
  msr cpsr, #0x93       // switch to SVC mode with IRQ disabled
  stmfd sp!, {r0-r3, r12, lr} // save context in SVC mode stack
  ldr r1, =vectorAddr
  ldr r0, [r1]          // read VIC vectAddr to ACK interrupt
  // bl enterINT        // see comments
  msr cpsr, #0x13       // enable IRQ in SVC mode
```

```
  bl IRQ_handler         // handle IRQ in SVC mode
  ldmfd sp!, {r0-r3, r12, lr} // restore context from SVC stack
  msr cpsr, #0x92        // to IRQ mode with IRQ disabled
  ldmfd sp!, {r12}       // pop saved spsr
  msr spsr, r12          // restore spsr
  // bl exitINT          // see comment
  ldr  r1, =vectorAddr   // issue EOI to VIC
  str  r0, [r1]
  ldmfd sp!, {r12, r14}  // return
  subs pc, r14, #4
get_cpsr:                // return CPSR
  mrs r0, cpsr
  mov pc, lr
```

C3.6.2. t.c file: The t.c file is the same as in C3.5, except for the added functions enterINT() and exitINT(). In the C3.5 program, which uses vectored interrupts, the interrupts priorities are (from high to low)

Timer0, UART0, UART1, KBD

Without nested interrupts, each interrupt is processed from start to finish without preemption. With nested interrupts, processing a low priority interrupt may be preempted by a higher priority interrupt. In order to demonstrate this, we add the following code to the C3.6 program.

(1). In the irq_handler code, before enabling IRQ interrupts and calling ISR for the current interrupt, we let the interrupt handler call enterINT(), which reads the VICstatus register to determine the interrupt source. If it's a KBD interrupt, it sets a volatile global inKBD flag to 1, clears a volatile global tcount to 0 and prints an enterKBD message. If it's a timer interrupt in the middle of handling a KBD interrupt, it increments tcount by 1.
(2). When an ISR returns to irq_handler, it calls exitINT(). If the current interrupt is from KBD, it prints the tcount value and resets tcount to 0.

The tcount value represents the number of high priority timer interrupts serviced while executing the low priority KBD handler. The reader may un-comment the statements labeled (1) and (2) in the irq_handler code to verify the effect of nested (timer) interrupts. The following lists the code of enterINT() and exitINT().

```
volatile int status, inKBD, tcount;

int enterINT()
{
  status = *((int *)(VIC_BASE_ADDR)); // read VICstatus register
  if (status & (1<<31)){ // KBD
     tcount = 0;
     inKBD = 1;
     printf("enterKBD ");
  }
  if ((status & (1<<4)) && (inKBD){ // timer0 inside KBD handler
     tcount++;              // tcount=number of timer interrupts
  }
}
int exitINT()
{
  if (status & 0x80000000){ // KBD
     printf("exitKBD=%d\n", tcount); // show tcount in KBD handler
     tcount=0;              // reset tcount to 0
  }
}
```

```
void IRQ_handler( )
{
  void *(*f)();                            // f as a function pointer
  f =(void *)*((int *)(VIC_BASE_ADDR+0x30)); // read ISR address
  (*f)( );                                 // call the ISR function
  //*((int *)(VIC_BASE_ADDR+0x30)) = 1;    // write EOI to vectorAddr
}

int main()
{
   char line[128];
   UART *up; kp = &kbd;
   fbuf_init();
   kbd_init();
   uart_init();
   // enable timer0,1 uart0,1 SIC and KBD interrupts as in C8
   vectorInt_init(); // same as in C8
   timer_init();
   timer_start(0);

   printf("Program C3.6 start: test NESTED Interrupts\n");
   up = &uart[0];
   printf("test uart0 I/O: enter text from UART 0\n");
   while(1){
     ugets(up, line);
     uprintf("  line=%s\n", line);
     if (strcmp(line, "end")==0)
      break;
   }
   printf("test UART1 I/O: enter text from UART 1\n");
   up = &uart[1];
   while(1){
     ugets(up, line);
     ufprintf(up, "  line=%s\n", line);
     if (strcmp(line, "end")==0)
      break;
   }
   printf("test KBD inputs\n"); // print to LCD
   while(1){
      kgets(line);
      printf("line=%s\n", line);
      if (strcmp(line, "end")==0)
         break;
   }
   printf("END OF run %d\n", 1234);
}
```

Figure 3.11 shows the outputs of running the example program C3.6, which demonstrates nested interrupts. As the figure shows, many timer interrupts may occur while handling a single KBD interrupt.

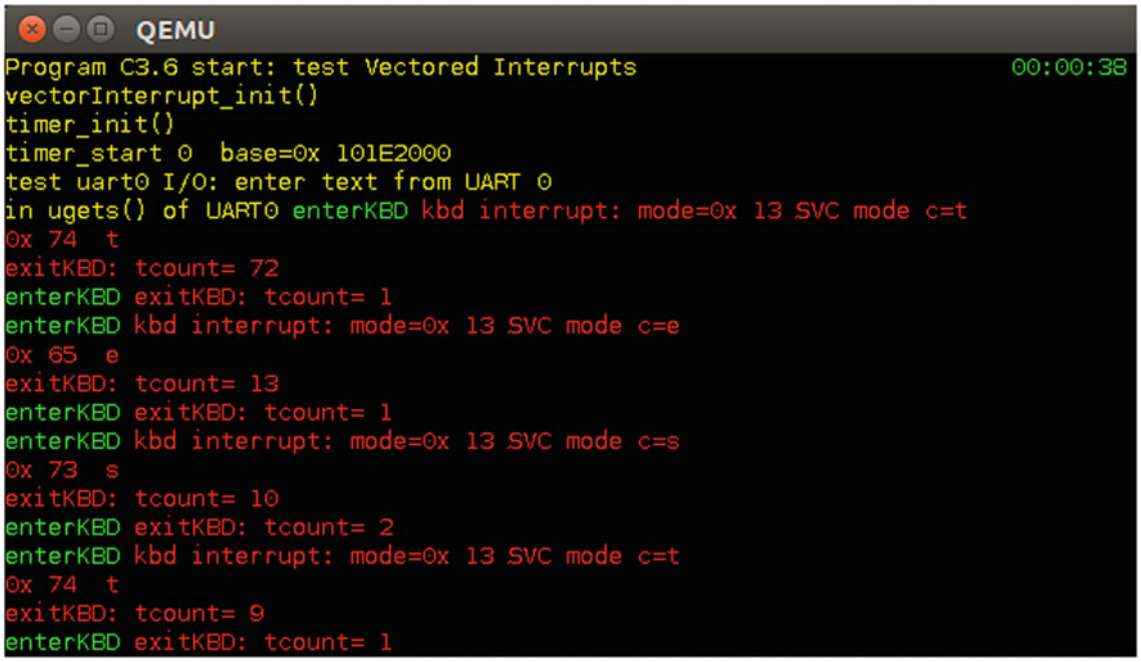


Fig. 3.11 Demonstration of nested interrupts

3.10 Nested Interrupts and Process Switch

The ARM scheme of handling nested interrupts in SYS or SVC mode works only if every interrupt handler returns to the original point of interruption, so that no process switch occurs after an interrupt. It would not work if an interrupt may cause context switch to a different process. This is because part of the context belonging to the switched out process still remains in the IRQ stack, which may be over-written when the new process handles another interrupt. If this happens, the switched out process would never be able to resume running again due to corrupted or lost execution context. In order to prevent this, the IRQ stack contents must be transferred to the SVC stack of the switched out process (and reset the IRQ stack pointer to prevent it from growing out of bounds). A possible way to avoid transferring stack contents is to allocate a separate IRQ stack for each process. But this would require a lot of memory spaces dedicated to processes as IRQ stacks, which basically nullifies the advantage of using a single IRQ stack for interrupt processing. Also, in the ARM architecture it is not possible to use the same memory area as both the SVC and IRQ stacks of a process due to the separate stack pointers in SVC and IRQ modes. These necessitate transferring IRQ stack contents during context switch, which seems to be an inherent weakness of the ARM processor architecture in terms of multitasking.

3.11 Summary

Interrupts and interrupts processing are essential to embedded systems. This chapter covers exceptions and interrupts processing. It describes the operating modes of ARM processors, exception types and exception vectors. It explains the functions of interrupt controllers and the principles of interrupts processing in detail. Then it applies the principles of interrupts processing to the design and implementation of interrupt-driven device drivers, which include drivers for timers, keyboard, UART and SD cards, and demonstrates the device drivers by example programs. It explained the advantages of vectored interrupts, showed how to configure the VIC for vectored interrupts, and demonstrated vectored interrupts processing. It also explained the principles of nested interrupts and demonstrated nested interrupts processing by example programs.

List of Sample Programs

C3.1: Timer driver
C3.2: KBD driver
C3.3: UART driver
C3.4: SDC driver: R/W sector
Improved SDC driver: R/W 64 long per interrupt
Multi-sector R/W for file BLKSIZE
C3.5: Vectored interrupts for Timer, UART, KBD
C3.6: Nested interrupts for KBD and Timer

Problems

1. In the example program C3.1, the vector table is at the end of the ts.s file.
(1). In the ts.s file, comment out the line

BL copy_vector

Re-compile and run the program again. The program should NOT work. Explain why?

(2). Move the vector table to the beginning of the ts.s file, as in

```
vectors_start:
    // vector table
vectors_end:
reset_handler:
```

Change the entry point in t.ld to vectors_start. Recompile and run the program again. It should also work. Explain why?

2. In the example 3.2, the irq_handler save all the registers in the IRQ stack. Modify it to save a MINIMUM number of registers. Figure out which registers must be saved to make the program still work.
3. Modify the KBD driver in program C3.2 to support upper case letters, and special control keys, Control-C and Control-D.
4. Modify the UART driver program C3.3 to support UART2 and UART3.
5. Modify the UART driver program C3.3 to support internal FIFO buffers of the UART.
6. The ARM VIC interrupt controller assigns fixed IRQ priorities in the order of IRQ0 (high) to IRQ31 (low). Vectored interrupts allows reordering of the IRQ priorities. In the example program C3.5, the priorities of vectored interrupts are assigned in their original order. Modify vectorInt_init() to assign different vectored interrupt priorities, e.g. in the order of KBD, UART1, UART0, timer0, from high to low. Test whether vectored interrupts still work. Discuss the implications of such an assignment of interrupt priorities.
7. The Example program C3.6 handles nested interrupts in SVC mode. Modify it to handle nested interrupts in SYS mode, as suggested by ARM.
8. In the example program C3.6, which supports nested interrupts, there are two lines in the irq_handler:

```
LDR  r1, =vectorAddr
LDR  r0, [r1]     // read VIC vectAddr to ACK interrupt
```

Comment out these lines to see what would happen and explain why?

9. In embedded systems, the SD card is often used as a booting device for booting up an operating system. During booting, the booter code may read sectors from the SD card by polling since it must wait for data. Instead of using interrupts, rewrite the SD driver using polling.
10. Rewrite the SD driver by using DMA to transfer large amounts of data.

References

ARM Architecture: https://en.wikipedia.org/wiki/ARM_architecture, 2016.
ARM Processor Architecture: http://www.arm.com/products/processors/instruction-set-architectures, 2016.
ARM PL011: ARM PrimeCellUART (PL011) Technical Reference Manual, http://infocenter.arm.com/help/topic/com.arm.doc.ddi0183, 2005.
ARMPL180: ARM PrimeCell Multimedia Card Interface (PL180) Technical Reference Manual, ARM Information Center, 1998.
ARMPL181: ARM PrimeCell Multimedia Card Interface (PL181) Technical Reference Manual, ARM Information Center, 2001.
ARM PL190: PrimeCell Vectored Interrupt Controller (PL190), http://infocenter.arm.com/help/topic/com.arm.doc.ddi0181e/DDI0181.pdf, 2004.
ARM Timers: ARM Dual-Timer Module (SP804) Technical Reference Manual, Arm Information Center, 2004.
ARM PL050: ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Technical Reference Manual, ARM Information Center, 1999.
Nesting Interrupts: Nesting Interrupts, ARM Information Center, 2011.
SD specification: Simplified Version of SD Host Controller Spec, https://www.sdcard.org/developers/overview/host_controller/simple_spec.
SDC: Secure Digital cards: SD Standard Overview - SD Association https://www.sdcard.org/developers/overview, 2016.
SPI: Serial Peripheral Interface, https://en.wikipedia.org/wiki/Serial_Peripheral_Interface_Bus, 2016, 2016.
Raspberry_Pi: https://www.raspberrypi.org/products/raspberry-pi-2-model-b, 2016.
Silberschatz, A., P.A. Galvin, P.A., Gagne, G, "Operating system concepts, 8th Edition", John Wiley & Sons, Inc. 2009.
Stallings, W. "Operating Systems: Internals and Design Principles (7th Edition)", Prentice Hall, 2011.
Wang, K.C., "Design and Implementation of the MTX Operating Systems, Springer International Publishing AG, 2015.

Models of Embedded Systems

4

4.1 Program Structures of Embedded Systems

In the early days, most embedded systems were designed for specific applications. An embedded system usually consists of a microcontroller, which is used to monitor a few sensors and generate signals to control a few external devices, such as to turn on LEDs or activates relays and servo motors to control a robot, etc. For this reason, control programs of early embedded systems are also very simple. They are written in the form of either a super-loop or event-driven program structure. However, as computing power and demand for multi-functional systems increase in recent years, embedded systems have undergone a tremendous leap in both applications and complexity. To cope with the ever increasing demands for extra functionality and the resulting system complexity, the super-loop and event driven program structures are no longer adequate. Modern embedded systems need more powerful software. As of now, many embedded systems are in fact high-powered computing machines capable of running full-fledged operating systems. A good example is smart phones, which use the ARM core with gig bytes internal memory and multi-gig bytes micro SD card for storage and run adapted versions of Linux, such as Android (Android 2016). The current trend in embedded systems design is clearly moving in the direction of developing multi-functional operating systems suitable for the mobile environment. In this chapter, we shall discuss the various program structures and programming models that are suitable for current and future embedded systems.

4.2 Super-Loop Model

A super-loop is a program structure composed of an infinite loop, with all the tasks of the system contained in the loop. The general form of a super-loop program is

```
main()
 {
    system_initialization();
    while(1){
       Check_Device_Status();
       Process_Device_Data();
       Output_Response();
    }
 }
```

After system initialization, the program executes an infinite loop, in which it checks the status of a system component, such as that of an input device. When the device indicates there are input data, it collects the input data, processes the data and generates outputs as response. Then it repeats the loop.

© Springer International Publishing AG 2017
K.C. Wang, *Embedded and Real-Time Operating Systems*,
DOI 10.1007/978-3-319-51517-5_4

4.2.1 Super-Loop Program Examples

We illustrate the super-loop program structure by examples. In the first example program, denoted by C4.1, we assume that an embedded system controls an UART for I/O. Our goal here is to develop a control program which continually checks whether there is any input from the UART port. Whenever a key is pressed, it gets the input key, processes the input and generates an output, such as to turn on an LED, flip a switch, etc. When running the program on an emulated ARM virtual machine, which does not have any LED or switch, we shall simply echo the input key to simulate processing the input and generating an output response.

In the program, it checks a UART for inputs. For each alphabetical key in lowercase, it converts the key to uppercase and displays the key to the UART port. In addition, it also handles the return key by outputting a new line char to produce the right visual effects. The program's assembly code is the same as those in Chap. 3. We only show the C code of the program.

Example Program C4.1: Super-loop Program

```
/************* t.c file of a super-loop program C4.1 *************/
#define UDR  0x00
#define UFR  0x18
char *ubase;
int main()
{
    char c;
    ubase = (char *)0x101F1000;     // 1. initialize UART0 base address
    while(1){
      if (*(ubase + UFR) & 0x40){   // 2. check UART0 for RxFull
        c = *(ubase + UDR);         // 3. get UART0 input key
         if (c > = 'a' && c <= 'z') //    if lowercase key
            c += ('A' - 'a');       //    convert to uppercase
         *(ubase + UDR) = c;        // 4. generate output
         if (c == '\r')
            *(ubase + UDR) = '\n';
      }
    }
}
```

When running the C4.1 program on an ARM virtual machine under QEMU, it echoes each alphabetical key to UART0 in uppercase. Instead of a single device, the program can be generalized to monitor and control several devices, all in the same loop. We demonstrate this technique by the next program, C4.2, which monitors and controls two devices, a UART and a keyboard.

Example Program C4.2: The program monitors and controls 2 devices in a super-loop.

```
/*********** t.c file of Program C4.2 ***********/
#include "keymap"
#define KSTAT 0x04
#define KDATA 0x08
#define UDR   0x00
#define UFR   0x18
char *ubase,*kbase;
int uart_init(){ ubase = (char *)0x101F1000; }
int kbd_init() { kbase = (char *)0x10006000; }
int main()
{
   char c, scode;
   uart_init(); kbd_init();         // 1. initialization
```

```
   while(1){                          // super-loop
     if (*(ubase + UFR) &  0x40){     // 2. if UART RxFull
        c = *(ubase + UDR);           // 3. handle UART input
        if (c >= 'a' && c <= 'z')
            c += ('A' - 'a');
        *(ubase + UDR) = c;
        if (c == '\r')
           *(ubase + UDR) = '\n';
     }
     if (*(kbase + KSTAT) & 0x10){  // 4. if KBD RxFull
       scode = *(kbase + KDATA);    // 5. handle KBD key (press)
       if (scode & 0x80)
          continue;
       c = unsh[scode];
       *(ubase + UDR) = c;          // echo KBD key to UART
       if (c == '\r')
          *(ubase + UDR) = '\n';
     }
   }
}
```

When running the program C4.2, it echoes UART0 inputs in uppercase, and keyboard inputs in lowercase, all to UART0.

4.3 Event-Driven Model

4.3.1 Shortcomings of Super-Loop Programs

The drawback of the super-loop program model is that the program must continually check the status of each and every device, even if the device is not ready. This not only wastes CPU time but also causes excessive power consumption. In an embedded system, it is often more desirable to reduce power consumption than to increase CPU utilization. Rather than continually checking the status of every device, an alternative way is to wait for the device to become ready. For example, in the program C4.1, we may replace the checking device status statements with a busy-wait loop, as in

$$\text{while}(\text{device_has_no_data});$$

But this does not remedy the problem since the CPU is still continually executing the busy-wait loop. Another drawback of this scheme is that the program would be unable to respond to any KBD inputs while it is waiting for UART inputs, and vice versa.

4.3.2 Events

In a programming environment, an event is something that is generated by a source and recognized by a recipient, causing the latter to take action to handle the event. Instead of continually checking for inputs, an embedded system can be designed to be event-driven, i.e. it takes action only in response to events. For this reason, **event-driven systems** (Cheong et al. 2003; Dunkels et al. 2006) are also called **reactive systems**. Events can be synchronous, i.e. they occur in a predictable manner, or asynchronous, i.e. they may occur at any time and in any order. Examples of synchronous events are periodic events from a timer, e.g. when the timer count has reached a certain value. Examples of asynchronous events are user inputs, such as press a key, click a mouse button and flip a switch, etc. Because of their unpredictability, the simple super-loop program structure is unsuited to dealing with asynchronous events. In the event-driven programming model, the main program may executes in a loop or sits in an idle state, waiting for any event to occur. When an event occurs, an event catcher recognizes the event and notifies the main program, causing it to take an appropriate action to handle the event. In an embedded system, events are usually associated with interrupts from hardware devices. In this case, an **event-driven program** becomes a simple

interrupt-driven program. We illustrate the interrupt-driven program structure by two examples. The first example handles periodic events and the second example handles asynchronous events.

4.3.3 Periodic Event-Driven Program

In this example, we assume that an embedded system consists of a timer and a display device, e.g. an LCD. The timer is programmed to generate 60 interrupts per second. The system must react to the following periodic timing events: On each second, it displays a wall clock in hh:mm:ss format to the LCD. Every 5 s, it displays a message string, also to the LCD. There are two possible ways to implement a control program that meets these periodic timing requirements. If the tasks to be performed are short, they can be performed by the timer interrupt handler directly. In this case, after initialization, the main program can execute an idle loop. To reduce power consumption, the idle loop can use Wait-For-Interrupt (WFI) or equivalent instruction, which puts the CPU into a power-saving state, waiting for interrupts. The ARM926EJ-S board does not support the WFI instruction but most ARM Cortex-5 processors (ARM Cortex-5 2010) implement a WFI mode by writing to the coprocessor CP15. The following shows the program code of the first version of Example C4.3. The tasks performed by the timer interrupt handler are shown in bold faced lines.

Version 1 of C4.3:

```
/************* timer.c file *****************/
typedef volatile struct timer{
  u32 *base;            // timer's base address
  int tick, hh, mm, ss; // timer data area
  char clock[16];
}TIMER;
volatile TIMER timer;   // timer structure

void timer_init()
{// initialize timer0 to generate 60 interrupts per second
 // timer.clock=00:00:00
}

void timer_handler() {
  TIMER *t = &timer;
  t->tick++;          // assume 60 ticks per second
  if (t->tick == 60){ // update ss, mm, hh
     t->tick = 0; t->ss++;
     if (t->ss == 60){
        t->ss = 0; t->mm++;
        if (t->mm == 60){
           t->mm = 0; t->hh++; t->hh %= 24;
        }
     }
  }
  if (t->tick == 0){ // on each second, display a wall clock
     for (i=0; i<8; i++)
         unkpchar(t->clock[i], 0, 70+i);
     t->clock[7]='0'+(t->ss%10); t->clock[6]='0'+(t->ss/10);
     t->clock[4]='0'+(t->mm%10); t->clock[3]='0'+(t->mm/10);
     t->clock[1]='0'+(t->hh%10); t->clock[0]='0'+(t->hh/10);
     for (i=0; i<8; i++)
         kpchar(t->clock[i], 0, 70+i); // kputchr(char, row, col)
  }
```

```
  if ((t->ss % 5) == 0) // every 5 seconds, display string
     printf("5 seconds event\n");
  timer_clearInterrupt();
}
/************** t.c file ******************/
#include "timer.c"
#include "vid.c"
#include "interrupts.c"
void IRQ_handler()
{
  int vicstatus;
  // read VIC SIV status registers to find out which interrupt
  vicstatus = VIC_STATUS;
  if (vicstatus & (1<<4)){   // timer0=bit4
     timer_handler();
  }
}
int main()
{
   fbuf_init();             // LCD driver
   VIC_INTENABLE = (1<<4);  // timer0 enable=bit4
   printf("Timer Event Program start\n");
   timer_init();
   timer_start();
   printf("Enter while(1) loop\n", 1234);
   while(1){
     asm("MOV r0, #0; MCR p15,0,R0,c7,c0,4"); // CPU enter WFI state
     printf("CPU out WFI state\n");
   }
}
```

In general, an interrupt handler should be as short as possible. If the periodic tasks are long, e.g. longer than a timer tick, it is undesirable to perform the timer dependent tasks inside the interrupt handler, unless the system supports nested interrupts. As shown in Chap. 3, the ARM architecture can not handle nested interrupts directly. In this case, it would be better to let the main program perform all the tasks. As before, the main program executes in a loop. When there are no events from the timer, it enters a power-saving state, waiting for the next interrupt. When a timer event occurs, the timer interrupt handler simply sets a global volatile flag variable. After waking up from the power-saving state, the main program can check the flag variables to take appropriate actions.

In the second version of the example program C4.3, the task of displaying the wall clock is re-written as a function, which is executed by the main program on each second. The following shows the second version of the C4.3 program.

Version 2 of C4.3: main program perform periodic tasks.

```
/************ new timer.c file **********/
volatile int one_second=0; five_seconds=0; // flags
void timer_handler() {
   TIMER *t = &timer;
   t->tick++;
   if (t->tick == 60){
      t->tick = 0; t->ss++;
      if (t->ss == 60){
         t->ss = 0; t->mm++;
```

```
         if (t->mm == 60){
            t->mm = 0; t->hh++;
         }
      }
   }
   if (t->tick == 0)       // on each second
      one_second = 1;      // turn on one_second flag
   if ((t->ss % 5)==0)     // every 5 seconds
      five_seconds = 1;    // turn on five_seconds flag
   timer_clearInterrupt();
}

char clock[16] = "00:00:00";
/***************** t.c file ***************/
int wall_clock(TIMER *t)
{
   int i, ss, mm, hh;
   int_off();
    ss = t->ss; mm = t->mm; hh = t->hh; // copy from timer struct
   int_on();
   for (i=0; i<8; i++)
       unkpchar(clock[i], 0, 70+i);
   clock[7]='0'+(ss%10); clock[6]='0'+(ss/10);
   clock[4]='0'+(mm%10); clock[3]='0'+(mm/10);
   clock[1]='0'+(hh%10); clock[0]='0'+(hh/10);
   for (i=0; i<8; i++)
       kpchar(clock[i], 0, 70+i); // kputchr(char, row, col)
}
int main()
{
   fbuf_init();
   VIC_INTENABLE = (1<<4);  // timer0 at bit4
   printf("C4.3: Periodic Events Program\n");
   timer_init();
   timer_start();
   while(1){
     if (one_second){
        wall_clock(&timer);
        one_second = 0;
     }
     if (five_seconds){
        printf("five seconds event\n");
        five_seconds = 0;
     }
     asm("MOV r0, #0; MCR p15,0,R0,c7,c0,4"); // enter WFI mode
     printf("CPU come out WFI state\n");
   }
}
```

Figure 4.1 shows the outputs of running the C4.3 program, which demonstrates periodic events.

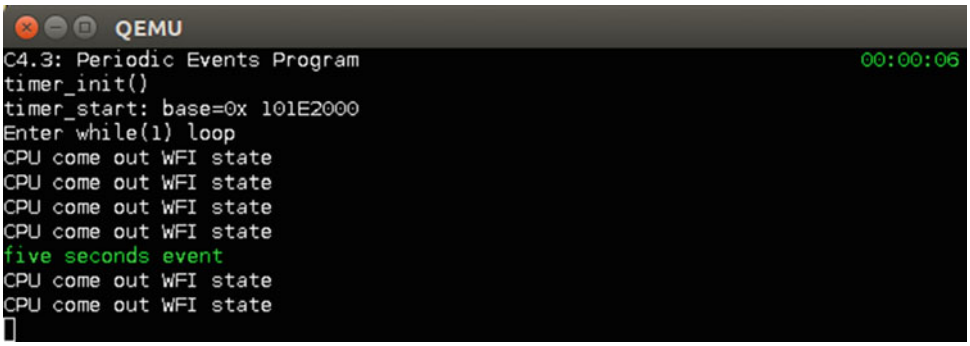


Fig. 4.1 Periodic Event-Driven Program

4.3.4 Asynchronous Event-Driven Program

Asynchronous events are non-periodic in nature, which may occur at any time and in any order. The next program, denoted by C4.4, demonstrates non-periodic or asynchronous events. In this example, we assume that the program monitors and controls two input devices; a UART and a keyboard. The main program tries to get an input line form either the UART or the KBD and echoes the line. The program is a condensed version of the example program C3.3 of Chap. 3, which implements interrupt-driven UART and KBD drivers. First, it initializes the (volatile) global flag variables, uline and kline to 0. Then it repeatedly checks the flag variables, which will be set by the interrupt handlers. When keys are pressed on the UART terminal or the keyboard, the UART or KBD interrupt handler will get the keys, echo them and enter them into an input buffer. When an ENTER key is pressed on either device, the interrupt handler turns on the corresponding flag variable to signal the occurrence of an event, allowing the main program to continue. When the main program detects an event, it extracts a line from the device driver's input buffer and echoes it to the LCD (for KBD inputs) or the UART. Then, it clears the flag variable and continues the loop. Due to the asynchronous nature of events, it is necessary to disable interrupts when clearing the flag variables in order to prevent race conditions between the main program and the interrupt handler.

```
/** Condensed uart.c, kbd.c and t.c files of Program C4.4. **/
volatile int uline=0, kline=0; // GLOBAL flag variables

void uart_handler(UART *up)
{
  u8 mis = *(up->base + MIS);    // read UART MIS register
  if (mis & 0x10) do_rx(up);
  else            do_tx(up);
}
int do_rx(UART *up)
{
  // get and echo input char c; enter c into inbuf[ ](same as before)
  if (c=='\r'){  // has an input line now
     uprintf("UART interrupt handler: turn on uline flag!\n");
     uline = 1;
  }
}
int do_tx(UART *up){ // output a char from outbuf[ ]; }
int ugetc(UART *up){ // return a char from inbuf[]; }
int uputc(UART *up, char c){ // enter char into outbuf[]; }
int ugets(UART *up, char *s){// get a line from inbuf[]; }
int uprints(UART *up, char *s) { // print a line; }
int uprintf(char *fmt, ...){// formatted print to UART; }

// kbd.c file
int kbd_init(){ // KBD initialization }
```

```
void kbd_handler()
{
  // get and echo KBD key press, enter key into buf[ ](same as before)
  if (c=='\r'){
     kprintf("KBD interrupt handler: turn on kline flag!\n");
     kline = 1;
  }
}
/************ t.c file of main() function *************/
#include "uart.c"          // UART driver
#include "kbd.c"           // KBD driver
#include "vid.c"           // LCD display driver
#include "exceptions.c"

void IRQ_handler()
{ // determine IRQ source; invoke UART or KBD interrupt handler; }

int main()
{
   char line[128];
   fbuf_init();
   uart_init();  // set UART base and enable UART interrupts
   kbd_init();   // set KBD base and enable KBD interrupts
   // code to enable UART and KBD interrupts on VIC and SIC
   while(1){// main program: check event flags; handle events
     if (uline){
        ugets(line);
        uprintf("UART: line=%s\n", line);
        lock(); uline = 0; unlock(); // reset uline to 0
     }
     if (kline){
        gets(line);
        color = GREEN;
        printf("KBD: line=%s\n", line);
        lock(); kline = 0; unlock(); // reset kline to 0
     }
     asm("MOV r0, #0; MCR p15,0,R0,c7,c0,4"); // enter WFI mode
   }
}
```

Figure 4.2 shows the outputs of running the C4.4 program, which demonstrates asynchronous events.

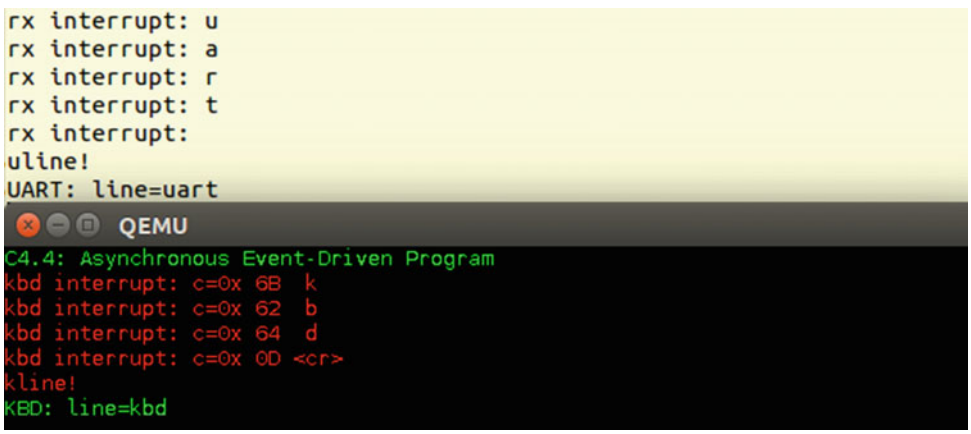


Fig. 4.2 Asynchronous Event-Driven Program

4.4 Event Priorities

In an event-driven system, some events may be more urgent than others. Events can be assigned different priorities according to their urgency and importance. Events should be handled in accordance with their priority order. There are several ways to prioritize the events. First, interrupt related events can be assigned different priorities by an interrupt controller. The event processing order of the main program should be consistent with the interrupts priorities. Second, event handlers can be implemented as independent execution entities called **processes** or **tasks**, which can be scheduled to run by priority. We shall explain the process programming model in the next section. Here, we briefly justify the need for processes first. The example programs of C4.3 and C4.4 have three main shortcomings. First, for fast response to interrupts, interrupt handlers should be as short as possible. This is especially important to timer interrupts in order not to lose timer ticks or require nested interrupts. Even for periodic events, they should be handled by the main program, not in the timer interrupt handler. Second, while in the power-saving state, the main program still needs to come up to execute the loop on each and every interrupt, even though the awaited events, e.g. a complete input line, have not occurred yet. It would be better if the program can come out of the power-saving state only when needed. This way, it does not have to poll every event when it runs. Third, events are not limited to user inputs or device interrupts. They may originate from other execution entities in the system as a means of synchronization and communication. In order to accomplish these, it is necessary to incorporate the notion of processes or tasks into the system. This leads us to the process model for embedded systems.

4.5 Process Models

In the process model, an embedded system comprises many concurrent processes. A process is an execution entity which can be scheduled to run, suspended from running (and yield CPU to other processes), and resumed to run again, etc. Each process is an independent execution unit designed to perform a specific task. Depending on the execution environment of the processes, the process model can be classified into several sub-models.

4.5.1 Uniprocessor Process Model

A uniprocessor (UP) system consists of only one CPU. In a UP system, processes run on the same CPU concurrently through multitasking.

4.5.2 Multiprocessor Process Model

A Multiprocessor (MP) system consists of a multiple number of CPUs, including multi-core processors. In a MP system, processes may run on different CPUs in parallel. In addition, each CPU or processor core may also run processes through multitasking. MP systems will be covered in Chap. 9.

4.5.3 Real Address Space Process Model

In the real address space model, the system either is not equipped with, or does not utilize, the memory management hardware due to timing constraints. Without memory management hardware to provide address mapping, all processes run in the same real address space of the system kernel. The drawback of this model is the lack of memory protection. Its main advantages are simplicity, less hardware resource requirements and high efficiency.

4.5.4 Virtual Address Space Process Model

In the virtual address space model, the system uses memory management hardware to provide each process with a unique virtual address space through address mapping. Processes may run in either kernel mode or user mode. While in kernel

mode, all processes share the same address space of the kernel. While in user mode, each process has a distinct virtual address space that is isolated and protected from other processes.

4.5.5 Static Process Model

In the static process model, all processes are created when the system starts, and they remain in the system permanently. Each process may be periodic or event-driven. Process scheduling is usually by static process priority without preemption, i.e. each process runs until it gives up the CPU voluntarily.

4.5.6 Dynamic Process Model

In the dynamic process model, processes can be created dynamically to perform specific tasks on demand. When a process completes its task, it terminates and releases all the resources back to the system for reuse.

4.5.7 Non-preemptive Process Model

In the non-preemptive process model, each process runs until it gives up the CPU voluntarily, e.g. when a process goes to sleep, suspends itself or explicitly yields CPU to another process.

4.5.8 Preemptive Process Model

In the preemptive process model, CPU can be taken away from a process to run another process at any time.

The above classifications of process models are not all mutually exclusive. Depending on the application, an embedded system may be designed as a mixture of appropriate process models. For instance, most existing embedded systems can be classified as the following types.

4.6 Uniprocessor (UP) Kernel Model

In this model, the system has only one CPU with no memory management hardware for address mapping. All processes run in the same address space of a kernel. Processes can be either static or dynamic. Processes are scheduled by static priority without preemption. Most simple embedded systems fit this model. The resulting system is equivalent to the non-preemptive kernel of an operating system. We shall discuss this system model in more detail in Chap. 5.

4.7 Uniprocessor (UP) Operating System Model

This is an extension of the UP Kernel model. In this model, the system uses memory management hardware to support address mapping, thus providing each process with a unique virtual address space. Each process runs in either kernel mode or a separate user mode. While in kernel mode, all processes run in the same address space of the kernel. While in user mode, each process executes in a private address space, which is isolated and protected from other processes. Processes share common data objects only in the protected kernel space. While in kernel mode, a process runs until it gives up the CPU voluntarily without preemption. While in user mode, a process can be preempted to yield CPU to other process of higher priority. Such a system is equivalent to a general purpose UP operating system. We shall discuss general purpose OS in more detail in Chap. 7.

4.8 Multiprocessor (MP) System Model

In this model, the system consists of a multiple number of CPUs or processor cores, which share the same physical memory. In a MP system, processes may run on different CPUs in parallel. Compared with UP systems, MP systems require advanced concurrent programming techniques and tools for process synchronization and protection. We shall discuss MP system in Chap. 9.

4.9 Real-Time (RT) System Model

In general, all embedded systems are designed with some timing requirements, such as quick responses to interrupts and short interrupt processing completion time, etc. An embedded system may only use these timing requirements as guidelines but it does not guarantee that the requirements are always attainable. In contrast, an embedded system intended for real-time applications must meet very stringent timing requirements, such as guaranteed minimal response time to interrupts and completing every requested service within a prescribed time limit. This environment is equivalent to a real-time system. We shall discuss real-time embedded systems in Chap. 10.

4.10 Design Methodology of Embedded System Software

As embedded systems becoming ever more complex, the traditional software design for embedded systems by the ad hoc approach is no longer adequate. As a result, there are many formal design methodologies proposed for embedded system software design, which include

4.10.1 High-Level Language Support for Event-Driven Programming

This design approach focuses on using the events and exceptions support of high-level programming languages, such as JAVA and C++, as a model to develop event-driven programs for embedded systems. A representative work in this area is the task model for event-driven programming (Fischer et al. 2007).

4.10.2 State Machine Model

This design method treats embedded system software as a finite state machine (FSM) (Edwards et al. 1997; Gajski et al. 1994). A **finite state machine (FSM)** is a system

```
FSM = {S, X, O, f}, where S = a finite set of states,
                          X = a finite set of inputs,
                          O = a finite set of outputs,
                          f is a state transition function, which maps S X I into S X O.
```

For each pair of (state, input) = (s, x), f(s, x) = (s', o), where s' is the next state of s, and o is the output generated during the state transition. A FSM is **fully specified** if f(s, x) is defined for every pair of (s, x). A FSM is **deterministic** if, for every pair of (s, x), f(s, x) is unique. A FSM is of the **Mealy model** (Katz and Borriello 2005) if the outputs depend on inputs. A FSM is of the **Moore model** if the outputs depend only on states.

The state machine design method models the specifications of embedded systems by fully specified and deterministic Mealy model FSMs, which allow for formal verification of the resulting systems. It also exploits programming language features to translate state machines into program code. We illustrate the state machine design model by an example.

Example Program C4.5: Assume that comment lines in C programs begin with two adjacent / symbols and end on the same line. Design an embedded system which takes C program source files as inputs and removes comment lines from the C programs. Design and implementation of such a system based on the FSM model consists of three steps.

Step 1: Construct a FSM State Table: The system can be modeled by a FSM with 5 states.

S0 = initial state, which has not seen any input
S1 = has not seen any / symbol yet
S2 = has seen the first / symbol
S3 = has seen 2 adjacent // symbols
S4 = final or termination state

Although each input is a single char, we shall classify the input chars into different cases, which are treated as distinct inputs to the system. Thus, we define inputs as

x1 = '/'
x2 = '\n'
x3 = not in {'/', \n', EOF}
x4 = EOF (end-of-file)

While the system is in a state, each input causes a state transition to a next state and generates an output (string). A FSM can be represented by a **state table**, which specifies the state transitions and outputs due to each input. For this example, the initial state table of the FSM is shown in Table 4.1, in which the output symbol - denotes null string.

In the state table, S0 is the initial state, which represents the condition that the system has not seen any input yet, and S4 is the final or termination state, in which the system has completed its task and halts. The initial state table is constructed in accordance with the problem specification. Starting from the initial state S0, if the input is a '/', it goes to state S2, which represents the condition that the system has seen the first '/', and generates a null output string. This is because this '/' may be the start of a comment line. If so, it should not be emitted as part of the output. If the input is a '\n', it goes to state S1 and generates an output string "\n". If the input is not '/' or '\n' or EOF, it goes to S1 also and generates an output string containing the same input char. If the input is EOF, it goes to the final state S4 with a null output string and terminates. While in the state S2, if the input x is not a '/' or\n or EOF, it goes back to S1 and generates the output string "/x". This is because a '/' followed by an ordinary char is not a comment line, which must be part of the output string. Other entries of the state table are constructed in a similar way.

Step 2: State Table Minimization: When constructing an initial state table from a problem specification, depending on how the states are defined as perceived by the system designer, the number of states may be more than actually needed. Thus, the initial state table may not be minimal. For instance, if we regard the final state S4 as a default condition for the system to terminate, then S4 is redundant, which can be eliminated. An initial state table may also contain many states that are actually equivalent. The second step in the FSM design model is to minimize the state table by eliminating redundant and equivalent states. In order to do this, we first clarify what is meant by equivalent states.

(1) **Equivalence relation**: An equivalent relation R is a binary relation applied to a set of objects, which is

Table 4.1 Initial State Table of FSM

State	x1=/	x2=\n	x3=others	x4=EOF
S0	S2/-	S1/"x2"	S1/"x3"	S4/-
S1	S2/-	S1/"x2"	S1/"x3"	S4/-
S2	S3/-	S1/"/x2"	S1/"/x3"	S4/-
S3	S3/-	S1/"x2"	S3/-	S4/-
S4	S4/-	S4/-	S4/-	S4/-

Reflexive: for any object x, x R x is true.
Symmetric: for any objects x, y, x R y implies y R x.
Transitive: for any objects x, y, z, x R y and y R z imply x R z.

For example, the = relation of real numbers is an equivalence relation. Similarly, for any integer N >0, the modulo-N (% N) relation of nonnegative integers is also an equivalence relation.

(2) **Equivalent class**: An equivalence relation can be used to partition (divide) a set into equivalent classes such that all the objects in the same class are equivalent. As a result, each equivalent class can be represented by a single object of that class.

Example: When applying the % 10 relation to the set of nonnegative integers, it partitions the set into the equivalent classes {0}–{9}. Each class {i} consists of all integers which yield a remainder of i when divided by 10. We may use 0–9 to represent the equivalent classes {0}–{9}.

(3). **Equivalent states**: In a FSM, two states Si and Sj are equivalent if for every input x,

their outputs are identical and their next states are equivalent.

Note that the definition of equivalent states only requires that, for each input their outputs must be identical but not their next states, which only need to be equivalent. This seems to create a chicken-egg problem, but we can handle it easily, as will be shown shortly.

(4) **State table minimization**: This step tries to reduce the number of states in a FSM state table to a minimum. While it may be hard to find all the equivalent states in a state table directly, it is very easy to spot state pairs that can not be equivalent based on their outputs. In elementary logic, we know that

$$\text{if A then B is } \textbf{equivalent} \text{ to if [not B] then [not A]}$$

When trying to prove something like "if A then B", we may either attack from the front by showing that "if A is true, then B must be true", or from the rear by showing that "if B is not true, then A can not be true" The strategy of attacking from the rear is often known as **proof by contradiction**, which is used in almost all proofs in the computability theory in computer science. So, rather than trying to identify equivalent states in a state table, we shall use a strategy which tries to identify and eliminate all non-equivalent states. The scheme is implemented by an Implication Chart, which is a table containing all pairs of states in a state table. In the implication chart, each cell corresponds to a state-pair (Si, Sj). Since an implication chart is symmetrical and all diagonal cells (Si, Si) are obviously equivalent states, it suffices to show only the lower half of the chart, without the diagonal cells. The algorithm of identifying non-equivalent state-pairs in an implication chart is as follows.

(1) Use the outputs of the states to cross out any cell (Si, Sj) that can not be equivalent.
(2) For each non-crossed out cell (Si, Sj), examine their next state pairs (Si', Sj') under each input. Cross out the cell (Si, Sj) if any of their next state-pair (Si', Sj') has been crossed out.
(3) Repeat (2) until there are no state-pair cells that can be crossed out

When the algorithm ends, each non-crossed out cell (Si, Sj) identifies a pair of equivalent states. Then, use the transitive property of equivalent state-pairs to construct equivalent classes.

For our example, we first construct an implication chart and cross out all the cells of the state-pairs that are non-equivalent. For example, it is obvious that S0 and S2 can not be equivalent since their outputs are not the same for every input. So we cross out the cell of (S0, S2). For the same reason, we can cross out the cell of (S0, S3). Likewise, we can cross out the cells of (S1, S2), (S1, S3) and (S2, S3). Table 4.2 shows the implication cart after applying (1) to cross out the cells of non-equivalent state-pairs.

Table 4.2 Initial Implication Chart of FSM

State	S0	S1	S2
S1			
S2	X	X	
S3	X	X	X

Table 4.3 Implication Chart of FSM

State	S0	S1	S2
S1	S2-S2 S1-S1 S1-S1		
S2	X	X	
S3	X	X	X

From the initial implication chart of Table 4.2, we fill each non-crossed out cell with next state pairs, which is shown in the cell of (S0, S1) of Table 4.3.

Then we apply step (2) of the algorithm, trying to cross out any cells containing state pairs that are already crossed out. In this case, there are none. So the algorithm terminates. The final implication chart of Table 4.3 reveals that (S0, S1) are equivalent states, which can be combined into a single state.

The process of identifying and eliminating equivalent states in state tables is known as the **FSM minimization** problem, which has been thoroughly studied in the design of finite state machines (Katz and Borriello 2005). It suffices to say that we can always reduce a fully specified and deterministic FSM state table to a minimal form, which is unique up to isomorphism (by renaming the states). Furthermore, the algorithm requires only polynomial computing time. For this example, the minimal state table is shown in Table 4.4, which has only 3 non-equivalent states.

Table 4.4 Minimal State Table of FSM

State	x1=/	x2=\n	x3 !={/,\n}
S1	S2/-	S1/"x2"	S1/"x3"
S2	S3/-	S1/"/x2"	S1/"/x3"
S3	S3/-	S1/"x2"	S3/-

The **state diagram** of a FSM is a directed graph, in which each node represents a state and an arc form Si to Sj, denoted by Si->Sj, represents a state transition from state Si to stat Sj. The arc is marked with all the input/output pairs that cause the state transition. State tables and state diagrams are equivalent in the sense that they convey exactly the same information. The reader may draw a state diagram for the state table shown in Table 4.4. This is left as an exercise.

Step 3: Translate State Table/State Diagram into Code: A state table or state diagram can be translated into C code almost directly. Using the switch–case statements of C, each state corresponds to a distinct case in an outer switch statement, and each input corresponds to a distinct case in an inner switch statement. We illustrate the translation by a complete C program, which simulates the intended embedded system.

```
/*************** Example Program C4.5 *****************/
#include <stdio.h>
int main()
{
  int c;                      // input char
  int state = 1;              // initial current state = S1
  FILE *fp = fopen("cprogram.c", "r"); // input is a C program

  while((c=fgetc(fp))!= EOF){  // if EOF: terminate
    switch(state){             // switch based on current state
      case 1:                  // state S1
        switch(c){             // next states/outputs
          case '/' : state = 2;                   break;
          case '\n': state = 1; printf("%c", c);  break;
          default  : state = 1; printf("%c", c);  break;
         };                                      break;
      case 2:                  // state S2
        switch(c){             // next states/outputs
          case '/' : state = 3;                   break;
          case '\n': state = 1; printf("/%c", c); break;
          default:   state = 1; printf("/%c", c); break;
        };                                       break;
      case 3:                  // state S3
        switch(c){             // next states/outputs
          case '/' : state = 3;                   break;
          case '\n': state = 1; printf("%c", c);  break;
          default  : state = 3;                   break;
      };
    }
  }
}
```

The reader may compile and run the above C4.5 program under Linux on C source files that use // as comment lines. The outputs should show that it removes all comment lines from C source files. The reader may also consult Problem 4.2 to handle a minor design flaw of the program.

It is noted that, when translating a FSM state table or state diagram into code, the resulting C code may not be very pretty nor efficient (in terms of code size), but the translation process is almost mechanical, which can be automated if needed. It makes the coding step an engineering endeavor rather than an art of programming. This is the strongest asset of the FSM model. However, the FSM model does have its limitations in that the number of states can not be too large. Whereas it may be quite easy to handle problems with only a few states, it would be too difficult to manage state tables or state diagrams with hundreds of states. For this reason, it is impractical and nearly impossible to design and implement a complete operating system by the FSM model.

4.10.3 StateChart Model

The StateChart model (Franke B 2016) is based on the finite state machine model. It adds concurrency and communications among the concurrent execution entities. It is intended for modeling complex embedded systems with concurrent tasks. Since this model involves the advanced concepts of concurrent processes, process synchronization and inter-process communication, we shall not discuss it any further.

4.11 Summary

This chapter covers models of embedded systems. It explained the simple super-loop system model and pointed out its shortcomings. It discussed the event-driven model and demonstrated the periodic and asynchronous event-driven system models by example programs. Then it justified the need for processes or tasks in embedded systems, and discussed the various process models. Lastly, it introduced some of the formal design methodologies for embedded systems, and it illustrated the FSM model by a detailed design and implementation example.

PROBLEMS

1. In the example program C4.3, after initializing the system the main() function executes a while(1) loop:

```
int main()
{
   // initial1zation
   while(1){
     asm("MOV r0, #0; MCR p15,0,R0,c7,c0,4");
     printf("CPU out of WFI state\n");
  }
}
```

   (1) Comment out the asm line. Run the program again to see what would happen.
   (2) In terms of CPU power consumption, what difference does the asm statement make?

2. In the Example Program C4.5, it assumes that a comment line starts with 2 adjacent / symbols to the end of line. However, string constants enclosed in matched pairs of double quotes may contain any number of / symbols but they are not comment lines, e.g. printf("this // is not a /// comment line\n"); Modify the state table of program C4.5 to handle this case. Translate the modified state table or state diagram into C code and run the modified program to test whether it works correctly.

3. Assume that comment blocks in C programs begins with /* and ends with */. Nested comment blocks are not allowed, which should result in an error. Write a C program, which detects and removes comment blocks from C program source files.

(1) Model the program as a FSM and construct a state diagram for the FSM.
(2) Write C code to implement the FSM as an event-driven system.

References

Android, https://en.wikipedia.org/wiki/Android_(operating_system), 2016.
ARM Cortex-5, ARM Cortex-A5 Processor, ARM Information Center, 2010.
Cheong, E, Liebman, J, Liu, J, Zhao, F, "TinyGALS: A programming model for event-driven embedded systems", ACM symposium on Applied Computing, 2003.
Dunkels, A, Schmidt, O, Voigt, T, Ali, "MProtothreads: simplifying event-driven programming of memory-constrained embedded systems", SenSys '06 Proc. of the 4th international conference on Embedded networked sensor systems, 2006.
Edwards, S, Lavagno, L, Lee, E.A, Sangiovanni-Vincentelli, A, "Design of Embedded Systems: Formal Models, Validation, and Synthesis", Proc. of the IEEE, Vol. 85, No.3, March, 1997, PP366–390.
Franke, B, "Embedded Systems Lecture 4: Statecharts, University of Edinburgh.
Fischer, J, Majumdar, R, Millstein, T, "Tasks: Language Support for Event-driven.
Programming", ACM SIGPLAN 2007 Workshop on PEPM, 2007.
Gajski, DD, Vahid, F, Narayan, S, Gong, J, "Specification and design of embedded systems", PTR Prentice Hall, 1994.
Katz, R. H. and G. Borriello, "Contemporary Logic Design", 2nd Edition, Pearson, 2005.

Process Management in Embedded Systems

5

5.1 Multitasking

In general, multitasking refers to the ability of performing several independent activities at the same time. For example, we often see people talking on their mobile phones while driving. In a sense, these people are doing multitasking, although a very dangerous kind. In computing, multitasking refers to the execution of several independent tasks at the same time. In a single CPU or uniprocessor (UP) system, only one task can execute at a time. Multitasking is achieved by multiplexing the CPU's execution time among different tasks, i.e. by switching the CPU from one task to another. If the switch is fast enough, it gives the illusion that all the tasks are executing simultaneously. This logical parallelism is called concurrency. In a multiprocessor (MP) system, tasks can execute on different CPUs in parallel in real time. In addition, each processor may also do multitasking by executing different tasks concurrently. Multitasking is the basis of all operating systems, as well as the foundation of concurrent programming in general. For simplicity, we shall consider uniprocessor (UP) systems first. MP systems will be covered later in Chap. 9 on Multiprocessor Systems.

5.2 The Process Concept

A multitasking system supports concurrent executions of many processes. The heart of a multitasking system is a control program, known as the operating system (OS) kernel, which provides functions for process management. In a multitasking system, processes are also called tasks. For all practical purposes, the terms process and task can be used interchangeably. First, we define an execution image as a memory area containing the execution's code, data and stack. Formally, **a process is the execution of an image**. It is a sequence of executions regarded as a single entity by the OS kernel for using system resources. System resources include memory space, I/O devices and, most importantly, CPU time. In an OS kernel, each process is represented by a unique data structure, called the Process Control Block (PCB) or Task Control Block (TCB), etc. In this book, we shall simply call it the PROC structure. Like a personal record, which contains all the information of a person, a PROC structure contains all the information of a process. In a single CPU system, only one process can be executing at a time. The OS kernel usually uses a global PROC pointer, running or current, to point at the PROC that is currently executing. In a real OS, the PROC structure may contain many fields and quite large. To begin with, we shall define a very simple PROC structure to represent processes.

```
typedef struct proc{
        struct proc *next;
        int    *ksp;
        int    kstack[1024];
}PROC;
```

In the PROC structure, the next field is a pointer pointing to the next PROC structure. It is used to maintain PROCs in dynamic data structures, such as link lists and queues. The ksp field is the saved stack pointer of a process when it is not executing, and kstack is the execution stack of a process. As we expand the OS kernel, we shall add more fields to the PROC structure later.

© Springer International Publishing AG 2017
K.C. Wang, *Embedded and Real-Time Operating Systems*,
DOI 10.1007/978-3-319-51517-5_5

5.3 Multitasking and Context Switch

5.3.1 A Simple Multitasking Program

We begin to demonstrate multitasking by a simple program. The program is denoted by C5.1. It consists of a ts.s file in ARM assembly code and a t.c file in C.

(1). ts.s file: the ts.s file defines the program's entry point reset_handler, in which it
(1). set the SVC stack pointer to the high end of proc0.kstack[ ].
(2). add a tswitch() function in assembly code for task switching.

```
// ——————————————— ts.s file of C5.1 ———————————————————————
.global main, proc0, procsize  // imported from C code
.global reset_handler, tswitch, scheduler, running


reset_handler:
  LDR r0, =proc0     // r0->proc0
  LDR r1, =procsize  // r1 ->procsize
  LDR r2, [r1, #0]   // r2 = procsize
  ADD r0, r0, r2     // r0 -> high end of proc0
  MOV sp, r0         // sp -> high end of proc0
  BL  main           // call main() in C


// ADD tswitch() function for task switch
tswitch:
SAVE:
  STMFD sp!, {r0-r12, lr}
  LDR r0, =running   // r0=&running
  LDR r1, [r0, #0]   // r1->runningPROC
  STR sp, [r1, #4]   // running->ksp = sp
FIND:
  BL scheduler        // call scheduler() in C
RESUME:
  LDR r0, =running
  LDR r1, [r0, #0]   // r1->running PROC
  LDR sp, [r1, #4]   // restore running->ksp
  LDMFD sp!, {r0-r12, lr} // restore register
  MOV pc, lr         // return
```

(2). t.c file: The t.c file includes the LCD and keyboard drivers for I/O. It defines the PROC structure type, a PROC structure proc0 and a running PROC pointer pointing to the current executing PROC.

```
/************* t.c file of C5.1 *****************/
#include "vid.c"             // LCD display driver
#include "kbd.c"             // KBD driver
#define SSIZE  1024          // stack size per PROC
typedef struct proc{         // process structure
        struct proc *next;   // next PROC pointer
        int *ksp;            // saved sp when PROC is not running
        int  kstack[SSIZE];  // process kernel mode 4KB stack
}PROC;                       // PROC is a type
int  procSize = sizeof(PROC);
```

```
PROC proc0, *running;          // proc0 structure and running pointer
int scheduler(){ running = &proc0; }
main()                         // called from ts.s
{
  running = &proc0;            // set running PROC pointer
  printf("call tswitch()\n");
   tswitch();                  // call tswitch()
  printf("back to main()\n");
}
```

Use the ARM toolchain (2016) to compile-link ts.s and t.c to generate a binary executable t.bin as usual. Then run t.bin on the Versatilepb VM (Versatilepb 2016) under QEMU, as in

qemu-system-arm –M versatilepb –m 128M **–kernel t.bin**

During booting, QEMU loads t.bin to 0x10000 and jumps to there to execute the loaded image. When execution starts in ts.s, it sets the SVC mode stack pointer to the high end of proc0. This makes proc0's kstack area as the initial stack. Up to this point, the system has no notion of any process because there is none. The assembly code calls main() in C. When control enters main(), we have an image in execution. By the definition of process, which is the execution of an image, we have a process in execution, although the system still does not know which process is executing. In main(), after setting running = &proc0, the system is now executing the process proc0. This is how a typical OS kernel starts to run an initial process when it begins. The initial process is handcrafted or created by brute force. Starting from main(), the run-time behavior of the program can be traced and explained by the execution diagram of Fig. 5.1, in which the key steps are labeled **(1)** to **(6)**.

At (1), it lets running point to proc0, as shown on the right-hand side of Fig. 5.1. Since we assume that running always points at the PROC of the current executing process, the system is now executing the process proc0.
At (2), it calls tswitch(), which loads LR(r14) with the return address and enters tswitch.
At (3), it executes the SAVE part of tswitch(), which saves CPU registers into stack and saves the stack pointer sp into proc0.ksp.
At (4), it calls scheduler(), which sets running to point at proc0 again. For now, this is redundant since running already points at proc0. Then it executes the RESUME part of tswitch().
At (5), it sets sp to proc0.ksp, which is again redundant since they are already the same. Then it pops the stack, which restores the saved CPU registers.
At (6), it executes MOV pc, lr at the end of RESUME, which returns to the calling place of tswitch().

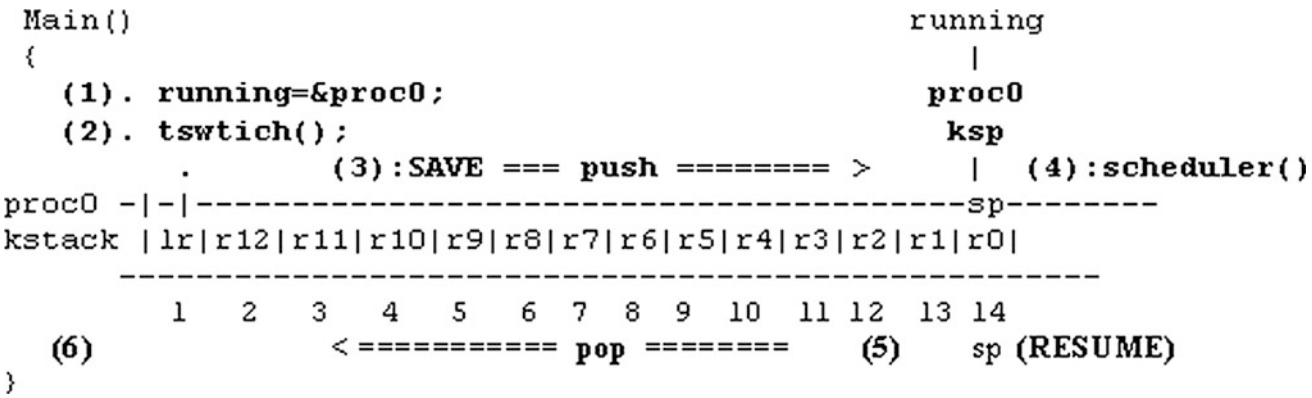


Fig. 5.1 Execution diagram of proc0

5.3.2 Context Switching

Besides printing a few messages, the program seems useless since it does practically nothing. However, it is the basis of all multitasking programs. To see this, assume that we have another PROC structure, proc1, which called tswitch() and executed the SAVE part of tswitch() before. Then proc1's ksp must point to its stack area, which contains saved CPU registers and a return address from where it called tswitch(), as shown in Fig. 5.2.

In scheduler(), if we let running point to proc1, as shown in the right-hand side of Fig. 5.2, the RESUME part of tswitch () would change sp to proc1's ksp. Then the RESUME code would operate on the stack of proc1. This would restore the saved registers of proc1, causing proc1 to resume execution from where it called tswitch() earlier. This changes the execution environment from proc0 to proc1.

Context Switching :

Changing the execution environment of one process to that of another is called context switching, which is the basic mechanism of multitasking.

With context switching, we can create a multitasking environment containing many processes. In the next program, denoted by C5.2, we define NPROC = 5 PROC structures. Each PROC has a unique pid number for identification. The PROCs are initialized as follows.

```
running -> P0 -> P1 -> P2 -> P3 -> P4 ->
           |                           |
           <---------------------------<--
```

P0 is the initial running process. All the PROCs form a circular link list for simple process scheduling. Each of the PROCs, P1 to P4, is initialized in such a way that it is ready to resume running from a body() function. Since the initialization of the PROC stack is crucial, we explain the steps in detail. Although the processes never existed before, we may pretend that they not only existed before but also ran before. The reason why a PROC is not running now is because it called tswitch() to give up CPU earlier. If so, the PROC's ksp must point to its stack area containing saved CPU registers and a return address, as shown in Fig. 5.3, where the index -i means SSIZE-i.

Since the PROC never really ran before, we may assume that its stack was initially empty, so that the return address, rPC=LR, is at the very bottom of the stack. What should be the rPC? It may point to any executable code, e.g. the entry address of a body() function. What about the "saved" registers? Since the PROC never ran before, the register values do not matter, so they can all be set to 0. Accordingly, we initialize each of the PROCs, P1 to P4, as shown in Fig. 5.4.

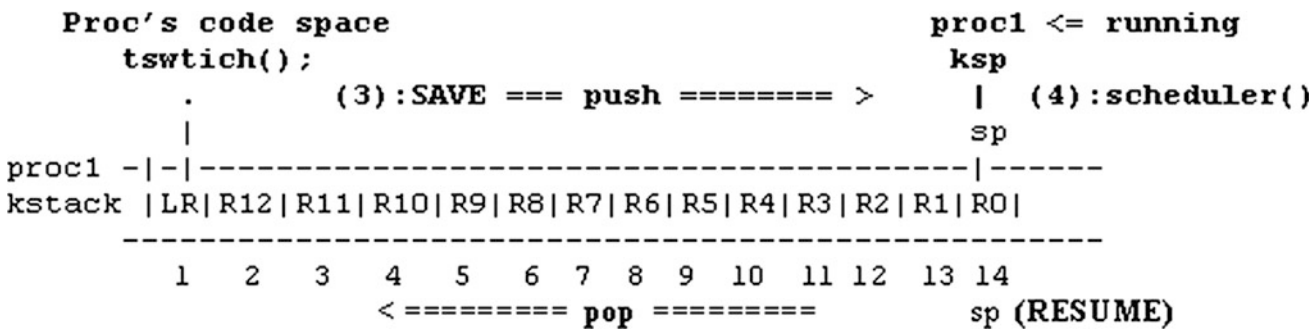


Fig. 5.2 Execution diagram of proc1

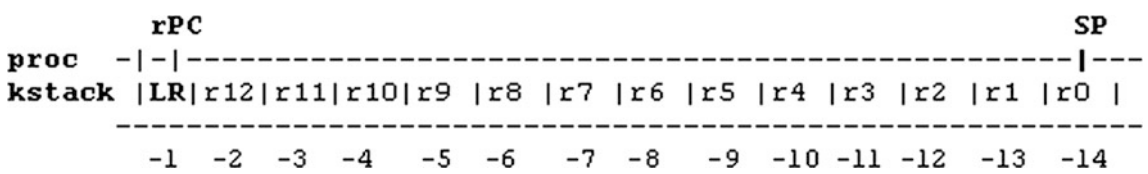


Fig. 5.3 Process stack contents

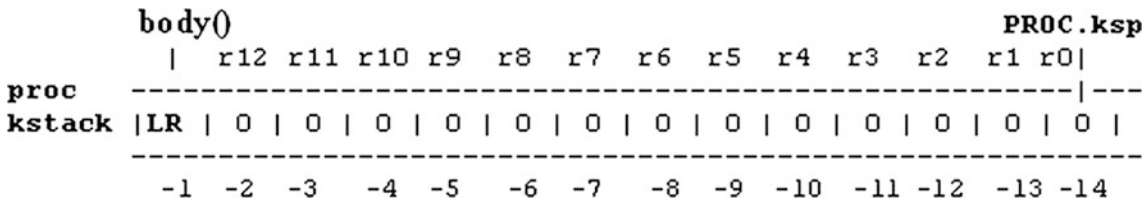


Fig. 5.4 Initial stack contents of process

With this setup, when a PROC becomes running, i.e. when running points to the PROC, it would execute the RESUME part of tswitch(),

```
LDMFD sp!, {r0-r12, lr}
MOV pc, lr
```

which restores the "saved" CPU registers, followed by MOV pc, lr, causing the process to execute the body() function.

After initialization, P0 calls tswitch() to switch process. In tswitch(), P0 saves CPU registers into its own stack, saves the stack pointer in its PROC.ksp and calls scheduler(). We modify the scheduler() function by letting running point to the next PROC, i.e.

```
running = running->next;
```

So P0 switches to P1. P1 begins by executing the RESUME part of tswitch(), causing it to resume to the body() function. While in body(), the running process prints its pid and prompts for an input char. Then it calls tswitch() to switch to the next process, etc. Since the PROCs are in a circular link list, they will take turn to run. The following lists the assembly and C code of C5.2.

(1). ts.s file of C5.2

```
.global vectors_start, vectors_end
.global main, proc, procsize
.global tswitch, scheduler, running
.global lock, unlock
reset_handler:
// set SVC sp to proc[0] high end
  LDR r0, =proc
  LDR r1, =procsize
  LDR r2, [r1, #0]
  ADD r0, r0, r2
  MOV sp, r0
// copy vector table to address 0
  BL copy_vectors
// go in IRQ mode to set IRQ stack
  MSR cpsr, #0x92
  LDR sp, =irq_stack
// go back to SVC mode with IRQ enabled
  MSR cpsr, #0x13
// call main() in SVC mode
  BL main
  B .
```

```
tswitch:
  mrs   r0, cpsr       // SVC mode, IRQ interrupts off
  orr   r0, r0, #0x80
  msr   cpsr, r0
  stmfd sp!, {r0-r12, lr}
  LDR r0, =running     // r0=&running
  LDR r1, [r0, #0]     // r1->runningPROC
  str sp, [r1, #4]     // running->ksp = sp
  bl scheduler
  LDR r0, =running
  LDR r1, [r0, #0]     // r1->runningPROC
  lDR sp, [r1, #4]
  mrs   r0, cpsr       // SVC mode, IRQ interrupts on
  bic   r0, r0, #0x80
  msr   cpsr, r0
  ldmfd sp!, {r0-r12, pc}


irq_handler:
  sub lr, lr, #4
  stmfd sp!, {r0-r12, lr}
  bl irq_chandler
  ldmfd sp!, {r0-r12, pc}^
lock:                  // disable IRQ interrupts
  MRS r0, cpsr
  ORR r0, r0, #0x80    // set IRQ mask bit
  MSR cpsr, r0
  mov pc, lr
unlock:                // enable IRQ interrupts
  MRS r0, cpsr
  BIC r0, r0, #0x80    // clear IRQ mask bit
  MSR cpsr, r0
  mov pc, lr
vectors_start:         // vector table
  LDR PC, reset_handler_addr
  LDR PC, undef_handler_addr
  LDR PC, swi_handler_addr
  LDR PC, prefetch_abort_handler_addr
  LDR PC, data_abort_handler_addr
  B .
  LDR PC, irq_handler_addr
  LDR PC, fiq_handler_addr
reset_handler_addr:          .word reset_handler
undef_handler_addr:          .word undef_handler
swi_handler_addr:            .word swi_handler
prefetch_abort_handler_addr: .word prefetch_abort_handler
data_abort_handler_addr:     .word data_abort_handler
irq_handler_addr:            .word irq_handler
fiq_handler_addr:            .word fiq_handler
vectors_end:
```

The assembly code is the same as in C5.1, except for the following modifications:

. the initial SVC mode stack pointer is set to proc[0]'s kstack
. copy vector table to address 0
. set IRQ mode stack and install IRQ handler (for keyboard driver)
. call main() in SVC mode with IRQ interrupts enabled

(2). Device Drivers of C5.2: The LCD driver is the same as in C5.1. The keyboard driver is the same as in Sect. 3.5.4 of Chap. 3.

(3). t.c file of C5.2:

```
/************ t.c file of C5.2 **********/
#include "vid.c"      // LCD driver
#include "kbd.c"      // KBD driver
#define  NPROC    5
#define  SSIZE 1024
typedef struct proc{
  struct proc *next;
  int    *ksp;
  int    pid;
  int    kstack[SSIZE];
}PROC;
PROC proc[NPROC], *running;
int procsize = sizeof(PROC);
int body()
{
  char c;
  printf("proc %d resume to body()\n", running->pid);
  while(1){
    printf("proc %d in body() input a char [s] : ", running->pid);
    c = kgetc(); printf("%c\n", c);
    tswitch();
  }
}
int kernel_init()
{
  int i, j;
  PROC *p;
  printf("kernel_init()\n");
  for (i=0; i<NPROC; i++){
    p = &proc[i];
    p->pid = i;
    p->status = READY;
    for (j=1; j<15; j++) // initialize proc.kstack and saved ksp
        p->kstack[SSIZE-j] = 0;      // all saved regs = 0
    p->kstack[SSIZE-1] = (int)body;  // resume point = body
    p->ksp = &(p->kstack[SSIZE-14]); // saved ksp
    p->next = p + 1;                 // point to next PROC
  }
  proc[NPROC-1].next = &proc[0];     // circular PROC list
  running = &proc[0];
}
```

```
int scheduler()
{
  printf("proc %d in scheduler\n", running->pid);
  running = running->next;
  printf("next running = %d\n", running->pid);
}
int main()
{
   char c;
   fbuf_init();            // initialize LCD driver
   kbd_init();             // initialize KBD driver
   printf("Welcome to WANIX in Arm\n");
   kernel_init();
   while(1){
     printf("P0 running input a key : ");
     c = kgetc(); printf("%c\n", c);
     tswitch();
   }
}
```

5.3.3 Demonstration of Multitasking

Figure 5.5 shows the outputs of running the C5.2 multitasking program. It uses the process pid to display in different colors, just for fun.

Before continuing, it is worth noting the following.

(1). In the C5.2 multitasking program, none of processes, P1 to P4, actually calls the body() function. What we have done is to convince each process that it called tswitch() from the entry address of body() to give up CPU earlier, and that is where it shall resume to when it begins to run. Thus, we can fabricate an initial environment for each process to start. The process has no choice but to obey. This is the power (and joy) of systems programming.

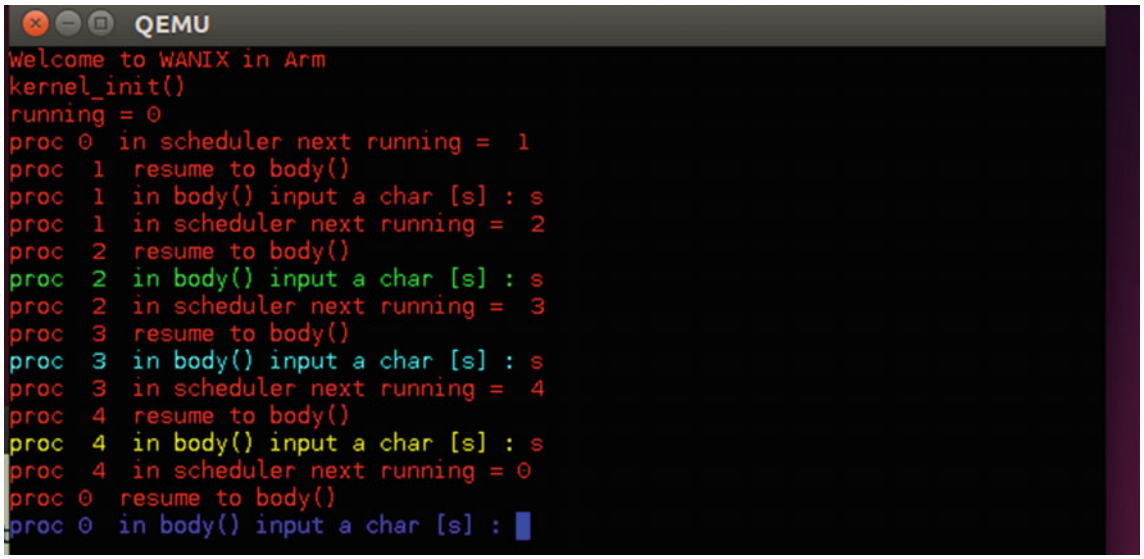


Fig. 5.5 Demonstration of multitasking

(2). All the processes, P1 to P4, execute the same body() function but each executes in its own environment. For instance, while executing the body() function, each process has its own local variable c in the process stack. This shows the difference between processes and functions. A function is just a piece of passive code, which has no life. Processes are executions of functions, which makes the function code alive.
(3). When a process first enters the body() function, the process stack is logically empty. As soon as execution starts, the process stack will grow (and shrink) by the function calling sequence as described in Sect. 2.7.3.2 of Chap. 2.
(4). The per-process kstack size is defined as 4KB. This implies that the maximal length of function call sequence (and the associated local variable spaces) of every process must never exceed the kstack size. Similar remarks also apply to other privileged mode stacks, e.g. the IRQ mode stack for interrupts processing. All of these are under the planning and control of the kernel designer. So stack overflow should never occur in kernel mode.

5.4 Dynamic Processes

In the program C5.2, P0 is the initial process. All other processes are created statically by P0 in kernel_init(). In the next program, denoted by C5.3, we shall show how to create processes dynamically.

5.4.1 Dynamic Process Creation

(1). First, we add a status and a priority field to the PROC structure, and define the PROC link lists: freeList and readyQueuee, which are explained below.

```
#define NPROC    9
#define FREE     0
#define READY    1
#define SSIZE 1024
typedef struct proc{
   struct proc *next;    // next PROC pointer
   int    *ksp;          // saved sp when NOT running
   int    pid;           // process ID
   int    status;        // FREE|READY, etc.
   int    priority;      // priority value
   int    kstack[SSIZE]; // process kernel mode stack
}PROC;
PROC proc[NPROC], *running, *freeList, *readyQueue;
```

. **freeList** = a (singly) link list containing all FREE PROCs. When the system starts, all PROCs are in the freeList initially. When create a new process, we allocate a free PROC from freeList. When a process terminates, we deallocate its PROC and release it back to the freeList for reuse.
. **readyQueue** = a priority queue of PROCs that are ready to run. PROCs with the same priority are ordered First-in-first-out (FIFO) in the readyQueue.

(2). In the queue.c file, we implement the following functions for list and queue operations.

```
/*************** queue.c file ***************/
PROC *get_proc(PROC **list){ return a PROC pointer from list }
int put_proc(PROC **list, PROC *p){ // enter p into list }
int enqueue(PROC **queue, PROC *p){ // enter p into queue by priority }
PROC *dequeue(PROC **queue){ // remove & return first PROC from queue }
int printList(PROC *p){ // print list elements }
```

(3). In the kernel.c file, kernel_init() initializes the kernel data structures, such as the freeList and readyQueue. It also creates P0 as the initial running process. The function

int pid = kfork(int func, int priority)

creates a new process to execute a function func() with the specified priority. In the example program, every new process begins execution from the same body() function. When a task has completed its work, it may terminate by the function

void kexit()

which releases its PROC structure back to the freeList for reuse. The function scheduler() is for process scheduling. The following lists the C code of kernel.c and t.c files.

```
/************** kernel.c file of Program C5.3 **********/
int kernel_init()
{
  int i, j;
  PROC *p;
  printf("kernel_init()\n");
  for (i=0; i<NPROC; i++){
    p = &proc[i];
    p->pid = i;
    p->status = FREE;
    p->next = p + 1;
  }
  proc[NPROC-1].next = 0;
  freeList = &proc[0];     // all PROCs in freeList
  readyQueue = 0;          // readyQueue empty
  // create P0 as initial running process
  p = get_proc(&freeList);
  p->priority = 0;         // P0 has the lowest priority 0
  running = p;
  printf("running = %d\n", running->pid);
  printList(freeList);
}
int body()  // process code
{
  char c;
  color = running->pid;
  printf("proc %d resume to body()\n", running->pid);
  while(1){
    printf("proc %d in body() input a char [s|f|x] : ", running->pid);
    c = kgetc(); printf("%c\n", c);
    switch(c){
      case 's': tswitch();           break;
      case 'f': kfork((int)body, 1); break;
      case 'x': kexit();             break;
    }
  }
}
// kfork() creates a new process to exceute func with priority
int kfork(int func, int priority)
{
  int i;
  PROC *p = get_proc(&freeList);
```

```
  if (p==0){
     printf("no more PROC, kfork failed\n");
     return -1;                    // return -1 for FAIL
  }
  p->status = READY;
  p->priority = priority;
  // set kstack for proc resume to execute func()
  for (i=1; i<15; i++)
      p->kstack[SSIZE-i] = 0;      // all "saved" regs = 0
  p->kstack[SSIZE-1] = func;       // resume execution address
  p->ksp = &(p->kstack[SSIZE-14]); // saved ksp
  enqueue(&readyQueue, p);         // enter p into readyQueue
  printf("%d kforked a new proc %d\n", running->pid, p->pid);
  printf("freeList = "); printList(readyQueue);
  return p->pid;
}
void kexit()   // called by process to terminate
{
   printf("proc %d kexit\n", running->pid);
   running->status = FREE;
   put_proc(running);
   tswitch();  // give up CPU
}
int scheduler()
{
  if (running->status == READY)
     enqueue(&readyQueue, running);
  running = dequeue(&readyQueue);
}

/************ t.c file of Program C5.3 ***********/
#include "type.h"       // PROC type and constants
#include "string.c"     // string opertion functions
#include "queue.c"      // list and queue operation functions
#include "vid.c"        // LCD driver
#include "kbd.c"        // KBD driver
#include "exceptions.c"
#include "kernel.c"
void copy_vectors(void){// same as before }
void IRQ_handler(){     // handles only KBD interrupts }

int main()
{
   fbuf_init();         // initialize LCD display
   kbd_init();          // initialize KBD driver
   printf("Welcome to Wanix in ARM\n");
   kernel_init();       // initialize kernel, create and run P0
   kfork((int)body, 1); // P0 create P1 in readyQueue
   while(1){
     while(readyQueue==0); // P0 loops if readyQueue empty
     tswitch();
   }
}
```

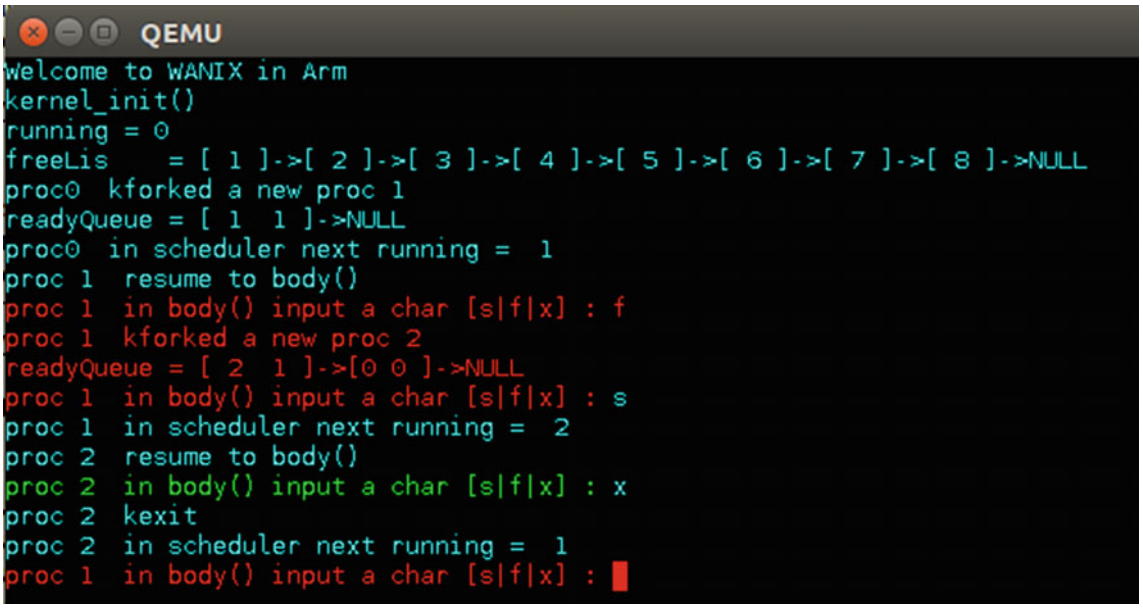


Fig. 5.6 Demonstration of dyamic process

In the t.c file, it first initializes the LCD display and the KBD driver. Then it initializes the kernel to run the initial process P0, which has the lowest priority 0. P0 creates a new process P1 and enters it into readyQueue. Then P0 calls tswitch() to switch process to run P1. Every new process resumes to execute the body() function. While a process runs, the user may enter 's' to switch process, 'f' to create a new process and 'x' to terminate, etc.

5.4.2 Demonstration of Dynamic Processes

Figure 5.6 shows the screen of running the C5.3 program. As the figure shows, a 'f' input causes P1 to kfork a new process P2 in the readyQueue. A 's' input causes P1 to switch process to run P2, which resumes to execute the same body() function. While P2 runs, the reader may enter commands to let P2 switch process or kfork a new process, etc. While a process runs, a 'x' input causes the process to terminate.

5.5 Process Scheduling

5.5.1 Process Scheduling Terminology

In a multitasking operating system, there are usually many processes ready to run. The number of runnable processes is in general greater than the number of available CPUs. Process scheduling is to decide when and on which CPU to run the processes in order to achieve an overall good system performance. Before discussing process scheduling, we first clarify the following terms, which are usually associated with process scheduling.

(1). I/O-bound vs. compute-bound processes:

A process is considered as I/O-bound if it suspends itself frequently to wait for I/O operations. I/O-bound processes are usually from interactive users who expect fast response time. A process is considered as compute-bound if it uses CPU time extensively. Compute-bound processes are usually associated with lengthy computations, such as compiling programs and numerical computations, etc.

(2). Response time vs. throughput:

Response time refers to how fast a system can respond to an event, such as entering a key from the keyboard. Throughput is the number of processes completed per unit time.

(3). Round-robin vs. dynamic priority scheduling:

In round-robin scheduling, processes take turn to run. In dynamic priority scheduling, each process has a priority, which changes dynamically (over time), and the system tries to run the process with the highest priority.

(4). Preemption vs. non-preemption:

Preemption means the CPU can be taken away from a running process at any time. Non-preemption means a process runs until it gives up CPU by itself, e.g. when the process finishes, goes to sleep or becomes blocked.

(5). Real-time vs. time-sharing:

A real-time system must respond to external events, such as interrupts, within a minimum response time, often in the order of a few milliseconds. In addition, the system may also need to complete the processing of such events within a specified time limit. In a time-sharing system, each process runs with a guaranteed time slice so that all processes receive their fair share of CPU time.

5.5.2 Goals, Policy and Algorithms of Process Scheduling

Process scheduling is intended to achieve the following goals.

.high utilization of system resources, especially CPU time,
.fast response to interactive or real-time processes,
.guaranteed completion time of real-time processes,
.fairness to all processes for good throughput, etc.

It is easy to see that some of goals are conflicting to one another. For example, fast response time and high throughput usually cannot be achieved at the same time. A scheduling policy is a set of rules, by which a system tries to achieve all or some of the goals. For a general purpose operating system, the scheduling policy is usually trying to achieve good overall system performance by striving for a balance among the conflicting goals. For embedded and real-time systems, the emphases are usually on fast response to external events and guaranteed process execution time. A scheduling algorithm is a set of methods that implements a scheduling policy. In an OS kernel, the various components, i.e. data structures and code used to implement the scheduling algorithm, are collectively known as the **process scheduler**. It is worth noting that in most OS kernel there is not a single piece of code or module that can be identified as the scheduler. The functions of a scheduler are implemented in many places inside the OS kernel, e.g. when a running process suspends itself or terminate, when a suspended process becomes runnable again and, most notably, in the timer interrupt handler.

5.5.3 Process Scheduling in Embedded Systems

In an embedded system, processes are created to perform specific tasks. Depending on the importance of the task, each process is assigned a priority, which is usually static. Processes run either periodically or in response to external events. The primary goal of process scheduling is to ensure quick response to external events and guarantee process execution time. Resource utilization and throughput are relatively unimportant. For these reasons, the process scheduling policy is usually based on process priority or by round-robin for processes with the same priority. In most simple embedded systems, processes usually execute in the same address space. In this case, the scheduling policy is usually non-preemptive. Each process runs until it gives up the CPU voluntarily, e.g. when the process goes to sleep, becomes suspended or explicitly

yields control to another process. Preemptive scheduling is more complex due to the following reasons. With preemption, many processes may run concurrently in the same address space. If a process is in the middle of modifying a shared data object, it must not be preempted unless the shared data object is protected in a critical region. Otherwise, the shared data object may be corrupted by other processes. Protection of critical regions will be discussed in the next section on process synchronization.

5.6 Process Synchronization

When multiple processes execute in the same address space, they may access and modify shared (global) data objects. Process synchronization refers to the rules and mechanisms used to ensure the integrity of shared data objects in a concurrent processes environment. There are many kinds of process synchronization tools. For a detailed list of such tools, their implementation and usage, the reader may consult (Wang 2015). In the following, we shall discuss some simple synchronizing tools that are suitable for embedded systems. In addition to discussing the principles of process synchronization, we shall also show how to apply them to the design and implementation of embedded systems by example programs.

5.6.1 Sleep and Wakeup

The simplest mechanism for process synchronization is the sleep/wakeup operations, which are used in the original Unix kernel. When a process must wait for something, e.g. a resource, that is currently unavailable, it goes to sleep to suspend itself and give up the CPU, allowing the system to run other processes. When the needed resource becomes available, another process or an interrupt handler wakes up the sleeping processes, allowing them to continue. Assume that each PROC structure has an added event field. The algorithms of sleep/wakeup are as follows.

```
sleep(int event)
{
  record event value in running PROC.event;
  change running PROC status to SLEEP;
  switch process;
}
wakeup(int event)
{
  for each PROC *p do{
     if (p->status==SLEEP && p->event==event){
        change p->status to READY;
        enter p into readyQueue;
     }
   }
}
```

In order for the mechanism to work, sleep() and wakeup() must be implemented properly. First, each operation must be atomic (indivisible) from the process point of view. For instance, when a process executes sleep(), it must complete the sleep operation before someone else tries to wake it up. In a non-preemptive UP kernel, only one process runs at a time, so processes can not interfere with one another. However, while a process runs, it may be diverted to handle interrupts, which may interfere with the process. To ensure the atomicity of sleep and wakeup, it suffices to disable interrupts. Thus, we may implement sleep() and wakeup() as follows.

```
int sleep(int event)
{
    int SR = int_off();  // disable IRQ and return CPSR
    running->event = event;
    running->status = SLEEP;
```

```
        tswitch();          // switch process
        int_on(SR);         // restore original CPSR

    }
    int wakeup(int event)
    {
        int SR = int_off();  // disable IRQ and return CPSR
        for each PROC *p do{
            if (p->status==SLEEP && p->event==event){
               p->status = READY;
               enqueue(&readyQueue, p);
            }
        }
        int_on(SR);        // restore original CPSR
    }
```

Note that wakeup() wakes up ALL processes, if any, that are sleeping on an event. If no process is sleeping on the event, wakeup has no effect, i.e. it amounts to a NOP and does nothing. It is also worth noting the interrupt handlers can never sleep or wait (Wang 2015). They can only issue wakeup calls to wake up sleeping processes.

5.6.2 Device Drivers Using Sleep/Wakeup

In Chap. 3, we developed several device drivers using interrupts. The organization of these device drivers exhibits a common pattern. Every interrupt-driven device driver consists of three parts; a lower-half part, which is the interrupt handler, an upper-half part, which is called by a main program, and a data area containing an I/O buffer and control variables, which are shared by the lower and upper parts. Even with interrupts, the main program still must use busy-wait loops to wait for data or room in the I/O buffer, which is essentially the same as polling. In a multitasking system, I/O by polling does not use the CPU effectively. In this section, we shall show how to use processes and sleep/wakeup to implement interrupt-driven device drivers without busy-wait loops.

5.6.2.1 Input Device Drivers

In Chap. 3, the KBD driver uses interrupts but the upper-half uses polling. When a process needs an input key, it executes a busy-wait loop until the interrupt handler puts a key into the input buffer. Our goal here is to replace the busy-wait loop with sleep/wakeup. First, we show the original driver code by polling. Then, we modify it to use sleep/wakeup for synchronization.

(1). KBD structure: The KBD structure is the middle part of the driver. It contains an input buffer and control variables, e.g. data = number of keys in the input buffer.

```
typedef struct kbd{     // base = 0x10006000
  char *base;           // base address of KBD, as char *
  char buf[BUFSIZE];    // input buffer size=128 bytes
  int head, tail, data; // control variables; data=0 initially
}KBD; KBD kbd;
```

(2). kgetc(): This is the (base function) of the upper-half of the KBD driver.

```
int kgetc() // main program calls kgetc() to return a char
{
  char c;
  KBD *kp = &kbd;
  unlock();                 // enable IRQ interrupts
  while(kp->data == 0);     // busy-wait for data;
```

```
  lock();                   // disable IRQ interrupts
    c = kp->buf[kp->tail++];// get a char and update tail index
    kp->tail %= BUFSIZE;
    kp->data--;             // update data with interrupts OFF
  unlock();                 // enable IRQ interrupts
  return c;
}
```

We assume that the main program is now running a process. When a process needs an input key, it calls kgetc(), trying to get a key from the input buffer. Without any means of synchronization, the process must rely on a busy-wait loop

while (kp->data == 0); // busy-wait for data;

which continually checks the data variable for any key in the input buffer.

(3). kbd_handler(): The interrupt handler is the lower-half of the KBD driver.

```
kbd_handler()
{
  struct KBD *kp = &kbd;
  char scode = *(kp->base+KDATA); // read scan code in data register
  if (scode & 0x80)          // ignore key releases
     return;
  if (data == BUFSIZE)       // if input buffer FULL
     return;                 // ignore current key
  c = unsh[scode];           // map scan code to ASCII
  kp->buf[kp->head++] = c;   // enter key into CIRCULAR buf[ ]
  kp->head %= BUFSIZE;
  kp->data++;                // inc data counter by 1
}
```

For each key press, the interrupt handler maps the scan code to a (lowercase) ASCII char, stores the char in the input buffer and updates the counting variables data. Again, without any means of synchronization, that's all the interrupt handler can do. For instance, it can not notify the process of available keys directly. Consequently, the process must check for input keys by continually polling the driver's data variable. In a multitasking system, the busy-wait loop is undesirable. We can use sleep/wakeup to eliminate the busy-wait loop in the KBD driver as follows.

(1). KBD structure: no need to change.
(2). kgetc(): rewrite kgetc() to let process sleep for data if there are no keys in the input buffer. In order to prevent race conditions between the process and the interrupt handler, the process disables interrupts first. Then it checks the data variable and modifies the input buffer with interrupts disabled, but it must enable interrupts before going to sleep. The modified kgetc () function is

```
int kgetc() // main program calls kgetc() to return a char
{
  char c;
  KBD *kp = &kbd;
  while(1){
    lock();                 // disable IRQ interrupts
    if (kp->data==0){       // check data with IRQ disabled
       unlock();            // enable IRQ interrupts
       sleep(&kp->data);    // sleep for data
```

```
    }
  }
  c = kp->buf[kp->tail++]; // get a c and update tail index
  kp->tail %= BUFSIZE;
  kp->data--;              // update with interrupts OFF
  unlock();                // enable IRQ interrupts
  return c;
}
```

(3). kbd_handler(): rewrite KBD interrupt handler to wake up sleeping processes, if any, that are waiting for data. Since process cannot interfere with interrupt handler, there is no need to protect the data variables inside the interrupt handler.

```
kbd_handler()
{
  struct KBD *kp = &kbd;
  scode = *(kp->base+KDATA); // read scan code in data register
  if (scode & 0x80)          // ignore key releases
     return;
  if (kp->data == BUFSIZE)   // ignore key if input buffer FULL
  c = unsh[scode];           // map scan code to ASCII
  kp->buf[kp->head++] = c;   // enter key into CIRCULAR buf[ ]
  kp->head %= BUFSIZE;
  kp->data++;                // update counter
  wakeup(&kp->data);         // wakeup sleeping process, if any
}
```

5.6.2.2 Output Device Drivers

An output device driver also consists of three parts; a lower-half, which is the interrupt handler, an upper-half, which is called by process to output data, and a middle part containing data buffer and control variables, which are shared by the lower and upper halves. The major difference between an output device driver and an input device driver is that the roles of process and interrupt handler are reversed. In an output device driver, process writes data to the data buffer. If the data buffer is full, it goes to sleep to wait for rooms in the data buffer. The interrupt handler extracts data from the buffer and outputs them to the device. Then it wakes up any process that is sleeping for rooms in the data buffer. A second difference is that for most output devices the interrupt handler must explicitly disable the device interrupts when there are no more data to output. Otherwise, the device will keep generating interrupts, resulting in an infinite loop. The third difference is that it is usually acceptable for several processes to share the same output device, but an input device can only allow one active process at a time. Otherwise, processes may get random inputs from the same input device.

5.7 Event-Driven Embedded Systems Using Sleep/Wakeup

With dynamic process creation and process synchronization, we can implement event-driven multitasking systems without busy-wait loops. We demonstrate such a system by the example program C5.4.

Example Program C5.4: We assume that the system hardware consists of three devices; a timer, a UART and a keyboard. The system software consists of three processes, each controls a device. For convenience, we also include an LCD for displaying outputs from the timer and keyboard processes.

Upon starting up, each process waits for a specific event. A process runs only when the awaited event has occurred. In this case, events are timer counts and I/O activities. At each second, the timer process displays a wall clock on the LCD. Whenever an input line is entered from the UART, the UART process gets the line and echoes it to the serial terminal. Similarly, whenever an input line is entered from the KBD, the KBD process gets the line and echoes it to the LCD. As before, the system runs on an emulated ARM virtual machine under QEMU. The system's startup sequence is identical to that of C5.3. We shall only show how to set up the system to run the required processes and their reactions to events. The system operates as follows.

(1). Initialization:
copy vectors, configure VIC and SIC for interrupts;
run the initial process P0, which has the lowest priority 0;
initialize drivers for LCD, timer, UART and KBD; start the timer;
(2). Create tasks: P0 call kfork(NAME_task, priority) to create the timer, UART and KBD processes and enter them into the readyQueue. Each process executes its own NAME_task() function with a (static) priority, ranging from 3 to 1.
(3). Then P0 executes a while(1) loop, in which it switches process whenever the readyQueue is non-empty.
(4). Each process resumes to execute its own NAME_code() function, which is an infinite loop. Each process calls sleep (event) to sleep on a unique event value (address of the device data structure).
(5). When an event occurs, the device interrupt handler calls wakeup(event) to wake up the corresponding process. Upon waking up, each process resumes running to handle the event. For example, the timer interrupt handler no longer displays the wall clock. It is performed by the timer process on each second.

The following lists the C code of the Example program C5.4.

```
/********** C code of Example Program C5.4 ***********/
#include "type.h"
#include "vid.c"         // LCD driver
#include "kbd.c"         // KBD driver
#include "uart.c"        // UART driver
#include "timer.c"       // timer driver
#include "exceptions.c"  // exception handlers
#include "queue.c"       // queue functions
#include "kernel.c"      // kernel for task management

int copy_vectors() { // copy vectors as before }
int irq_chandler() { // invoke IRQ handler of timer, UART, KBD }

int timer_handler(){ // on each second: kwakeup(&timer); }
int uart_handler() { // on input line:  kwakeup(&uart); }
int kbd_handler()  { // on input line:  kwakeup(&kbd); }

int i, hh, mm, ss;   // globals for timer_handler
char clock[16] = {"00:00:00"};

int timer_task()   // code of timer_task
{
  while(1){
    printf("timer_task %d running\n", running->pid);
    ksleep((int)&timer);
    // use timer tick to update ss, mm, hh; then display wall clock
    clock[7]='0'+(ss%10); clock[6]='0'+(ss/10);
    clock[4]='0'+(mm%10); clock[3]='0'+(mm/10);
    clock[1]='0'+(hh%10); clock[0]='0'+(hh/10);
    for (i=0; i<8; i++){
        kpchar(clock[i], 0, 70+i);
    }
  }
}

int uart_task()   // code of uart_task
{
  char line[128];
  while(1){
    uprintf("uart_task %d sleep for line from UART\n", running->pid);
    ksleep((int)&uart);
```

```
    uprintf("uart_task %d running\n", running->pid);
    ugets(line);
    uprintf("line = %s\n", line);
  }
}

int kbd_task()      // code of kbd_task
{
  char line[128];
  while(1){
    printf("KBD task %d sleep for a line from KBD\n", running->pid);
    ksleep((int)&kbd);
    printf("KBD task %d running\n", running->pid);
    kgets(line);
    printf("line = %s\n", line);
  }
}

int main()
{
   fbuf_init();    // LCD driver
   uart_init();    // UART driver
   kbd_init();     // KBD driver
   printf("Welcome to Wanix in ARM\n");
   // initialize VIC interrupts: same as before
   timer_init();   // timer driver
   timer_start();
   kernel_init();  // initialize kernel and run P0
   printf("P0 create tasks\n");
   kfork((int)timer_task, 3);  // timer task
   kfork((int)uart_task,  2);  // uart task
   kfork((int)kbd_task,   1);  // kbd task
   while(1){ // P0 runs whenever no task is runnable
     if (readyQueue)
        tswitch();
   }
}
```

5.7.1 Demonstration of Event-Driven Embedded System Using Sleep/Wakeup

Figure 5.7 shows the output screens of running the example program C5.4, which demonstrates an event-driven multi-tasking system. As the Fig. 5.7 shows, the timer task displays a wall clock on the LCD on each second. The uart task prints a line to UART0 only when there is an input line from the UART0 port, and the kbd task prints a line to the LCD only when there is an input line from the keyboard. While these tasks are sleeping for their awaited events, the system is running the idle process P0, which is diverted to handles all the interrupts. As soon as a task is woken up and entered into the readyQueue, P0 switches process to run the newly awakened task.

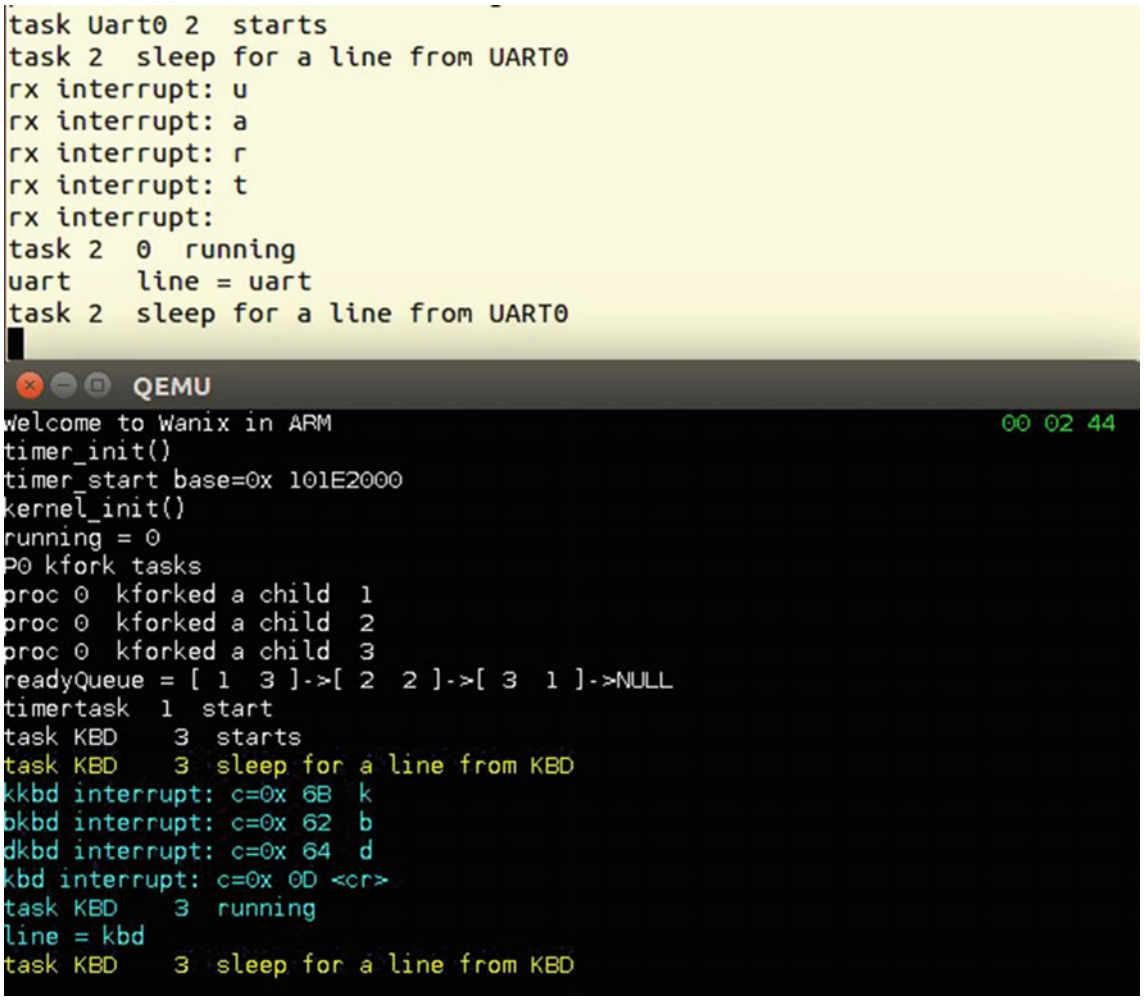


Fig. 5.7 Event-driven multitasking system using sleep/wakeup

5.8 Resource Management Using Sleep/Wakeup

In addition to replacing busy-wait loops in device drivers, sleep/wakeup may also be used for general process synchronization. A typical usage of sleep/wakeup is for resource management. A resource is something that can be used by only one process at a time, e.g. a memory region for updating, a printer, etc. Each resource is represented by a res_status variable, which is 0 if the resource is FREE, and nonzero if it's BUSY. Resource management consists of the following functions

int acquire_resource(); // acquire a resource for exclusive use
int release_resource(); // release a resource after use

When a process needs a resource, it calls acquire_resource(), trying to get a resource for exclusive use. In acquire_resource(), the process tests res_status first. If res_status is 0, the process sets it to 1 and returns OK for success. Otherwise, it goes to sleep, waiting for the resource to become FREE. While the resource is BUSY, any other process calling acquire_resource() would go to sleep on the same event value also. When the process which holds the resource calls release_recource(), it clears res_status to 0 and issues wakeup(&res_status) to wakeup ALL processes that are waiting for the resource. Upon waking up, each process must try to acquire the resource again. This is because when an awakened process runs, the resource may no longer be available. The following code segment shows the resource management algorithm using sleep/wakeup.

```
                    int res_status = 0;       // resource initially FREE
-------------------------------------------------------------------------------
int acquire_resource()     |   int release_resource()
{                          |   {
    while(1){              |
      int SR = int_off();  |       int SR = int_off();
      if (res_status==0){  |
         res_status = 1;   |       res_status = 0;
         break;            |
      }                    |
      sleep(&res_status);  |       wakeup(&res_status);
    }                      |
    int_on(SR);            |       int_on(SR);
    return OK;             |       return OK;
}                          |   }
-------------------------------------------------
```

5.8.1 Shortcomings of Sleep/Wakeup

Sleep and wakeup are simple tools for process synchronization, but they also have the following shortcomings.

. An event is just a value. It does not have any memory location to record the occurrence of an event. Process must go to sleep first before another process or an interrupt handler tries to wake it up. The sleep-first-wakeup-later order can always be achieved in a UP system, but not necessarily in MP systems. In a MP system, processes may run on different CPUs simultaneously (in parallel). It is impossible to guarantee the execution order of the processes. Therefore, sleep/wakeup are suitable only for UP systems.
. When used for resource management, if a process goes to sleep to wait for a resource, it must retry to get the resource again after waking up, and it may have to repeat the sleep-wakeup-retry cycles many times before succeeding (if ever). The repeated retry loops means poor efficiency due to excessive overhead in context switching.

5.9 Semaphores

A better mechanism for process synchronization is the semaphore, which does not have the above shortcomings of sleep/wakeup. A **(counting)** semaphore is a data structure

```
typedef struct semaphore{
   int spinlock;  // spin lock, needed only in MP systems
   int value;     // initial value of semaphore
   PROC *queue    // FIFO queue of blocked processes
}SEMAPHORE;
```

In the semaphore structure, the spinlock field is to ensure any operation on a semaphore can only be performed as an atomic operation by one process at a time, even if they may run in parallel on different CPUs. Spinlock is needed only for multiprocessor systems. For UP systems, it is not needed and can be omitted. The most well-known operations on semaphores are P and V, which are defined (for UP kernels) as follows.

```
-------------------------------------------------------------------
int P(struct semaphore *s)       |  int V(struct semaphore *s)
{                                |  {
  int SR = int_off();            |    int SR = int_off();
  s->value--;                    |    s->value++;
  if (s->value < 0)              |    if (s->value <= 0)
     block(s);                   |       signal(s);
  int_on(SR);                    |    int_on(SR);
}                                |  }
-------------------------------------------------------------------
int block(struct semaphore *s){  |  int signal(struct semaphore *s){
    running->status = BLOCK;     |  PROC *p = dequeue(&s->queue);
    enqueue(&s->queue, running);|        p->status = READY;
    tswitch();                   |        enqueue(&readyQueue, p);
}                                |  }
-------------------------------------------------------------------
```

A **binary semaphore** (Dijkstra 1965) is a semaphore which can only take on two distinct values, 1 for FREE and 0 for OCCUPIED. P/V operations on binary semaphores are defined as

```
--------------------- P/V on binary semaphores -----------------
int P(struct semaphore *s)       |  int V(struct semaphore *s)
{                                |  {
  int SR = int_off();            |    int SR = int_off();
  if (s->value == 1)             |    if (s->queue == 0)
     s->value = 0;               |       s->value = 1;
  else                           |    else
     block(s);                   |       signal(s);
  int_on(SR);                    |    int_on(SR);
}                                |  }
-------------------------------------------------------------------
```

Binary semaphores may be regarded as a special case of counting semaphores. Since counting semaphores are more general, we shall not use, nor discuss, binary semaphores.

5.10 Applications of Semaphores

Semaphores are powerful synchronizing tools which can be used to solve all kinds of process synchronization problems in both UP and MP systems. The following lists the most common usage of semaphores. To simplify the notations, we shall denote s.value = n by s = n, and P(&s)/V(&s) by P(s)/V(s), respectively.

5.10.1 Semaphore Lock

A **critical region (CR)** is a sequence of operations on shared data objects which can only be executed by one process at a time. Semaphores with an initial value = 1 can be used as locks to protect CRs of long durations. Each CR is associated with a semaphore s = 1. Processes access the CR by using P/V as lock/unlock, as in

```
                    struct semaphore s = 1;
Processes:          P(s);          // acquire semaphore to lock the CR
                                   // CR protected by lock semaphore s
                    V(s);          // release semaphore to unlock the CR
```

With the semaphore lock, the reader may verify that only one process can be inside the CR at any time.

5.10.2 Mutex lock

A **mutex** (Pthreads 2015) is a lock semaphore with an additional owner field, which identifies the current owner of the mutex lock. When a mutex is created, its owner filed is initialized to 0, i.e. no owner. When a process acquires a mutex by mutex_lock(), it becomes the owner. A locked mutex can only be unlocked by its owner. When a process unlocks a mutex, it clears the owner field to 0 if there are no processes waiting on the mutex. Otherwise, it unblocks a waiting process from the mutex queue, which becomes the new owner and the mutex remains locked. Extending P/V on semaphores to lock/unlock of mutex is trivial. We leave it as an exercise for the reader. A major difference between mutexes and semaphores is that, whereas mutexes are strictly for locking, semaphores can be used for both locking and process cooperation.

5.10.3 Resource Management using Semaphore

A semaphore with initial value n > 0 can be used to manage n identical resources. Each process tries to get a unique resource for exclusive use. This can be achieved as follows.

```
                struct semaphore s = n;
Processes: P(s);
                    use a resource exclusively;
           V(s);
```

As long as s > 0, a process can succeed in P(s) to get a resource. When all the resources are in use, requesting processes will be blocked at P(s). When a resource is released by V(s), a blocked process, if any, will be allowed to continue to use a resource. At any time the following invariants hold.

s >= 0 : s = the number of resources still available;
s < 0 : |s| = number of processes waiting in s queue

5.10.4 Wait for Interrupts and Messages

A semaphore with initial value 0 is often used to convert an external event, such as hardware interrupt, arrival of messages, etc. to unblock a process that is waiting for the event. When a process waits for an event, it uses P(s) to block itself in the semaphore's waiting queue. When the awaited event occurs, another process or an interrupt handler uses V(s) to unblock a process from the semaphore queue, allowing it to continue.

5.10.5 Process Cooperation

Semaphores can also be used for process cooperation. The most widely cited cases involving process cooperation are the producer-consumer problem and the reader-writer problem (Silberschatz et al. 2009; Stallings 2011; Tanenbaum et al. 2006; Wang 2015).

5.10.5.1 Producer-Consumer Problem

A set of producer and consumer processes share a finite number of buffers. Each buffer contains a unique item at a time. Initially, all the buffers are empty. When a producer puts an item into an empty buffer, the buffer becomes full. When a consumer gets an item from a full buffer, the buffer becomes empty, etc. A producer must wait if there are no empty buffers. Similarly, a consumer must wait if there are no full buffers. Furthermore, waiting processes must be allowed to continue when their awaited events occur. Figure 5.8 shows a solution of the Producer-Consumer problem using semaphores.

In Fig. 5.8, processes use mutex semaphores to access the circular buffer as CRs. Producer and consumer processes cooperate with one another by the semaphores full and empty.

5.10.5.2 Reader-Writer Problem

A set of reader and writer processes share a common data object, e.g. a variable or a file. The requirements are: an active writer must exclude all others. However, readers should be able to read the data object concurrently if there is no active writer. Furthermore, both readers and writers should not wait indefinitely (starve). Figure 5.9 shows a solution of the Reader-Writer Problem using semaphores.

In Fig. 5.9, the semaphore rwsem enforces FIFO order of all incoming readers and writers, which prevents starvation. The (lock) semaphore rsem is for readers to update the nreader variable in a critical region. The first reader in a batch of readers locks the wsem to prevent any writer from writing while there are active readers. On the writer side, at most one writer can be either actively writing or waiting in wsem queue. In either case, new writers will be blocked in the rwsem queue. Assume that there is no writer blocked at rwsem. All new readers can pass through both P(rwsem) and P(rsem), allowing them to read the data concurrently. When the last reader finishes, it issues V(wsem) to allow any writer blocked at wsem to continue. When the writer finishes, it unlocks both wsem and rwsem. As soon as a writer waits at rwsem, all new comers will be blocked at rwsem also. This prevents readers from starving writers.

5.10.6 Advantages of Semaphores

As a process synchronization tool, semaphores have many advantages over sleep/wakeup.

(1). Semaphores combine a counter, testing the counter and making decision based on the testing outcome all in a single indivisible operation. The V operation unblocks only one waiting process, if any, from the semaphore queue. After passing through the P operation on a semaphore, a process is guaranteed to have a resource. It does not have to retry to get the resource again as in the case of using sleep and wakeup.
(2). The semaphore's value records the number of times an event has occurred. Unlike sleep/wakeup, which must obey the sleep-first-wakeup-later order, processes can execute P/V operations on semaphores in any order.

5.10.7 Cautions of Using Semaphores

Semaphores use a locking protocol. If a process can not acquire a semaphore in P(s), it becomes blocked in the semaphore queue, waiting for someone else to unblock it via a V(s) operation. Improper usage of semaphores may lead to problems. The most well-known problem is deadlock (Silberschatz et al. 2009; Tanenbaum et al. 2006). Deadlock is a condition in which a

```
        DATA buf[N]                /* N buffer cells */
        int head = tail = 0;       /* index to buffer cells */
        SEMAPHORE empty = N; full = 0 pmutex =1; cmutex = 1;

------- Producer -------------------- Consumer -------------
 while(1){                    |      while(1){
    produce an item;          |
    P(empty);                 |         P(full);
      P(pmutex);              |           P(cmutex);
        buf[head++] = item;   |           item = buf[tail++];
        head %= N;            |           tail %= N;
      V(pmutex);              |           V(cmutex);
    V(full);                  |         V(empty);
 }                            |    }
 ------------------------------------------------------------
```

Fig. 5.8 Producer-consumer problem solution

```
        SEMAPHORE rwsem = 1; wsem = 1; rsem = 1;
        Int nreader = 0   /* number of active Readers */
-------------------------------------------------------------
ReaderProcess                  |  WriterProcess
{                              |  {
  while(1){                    |     while(1){
    P(rwsem);                  |       P(rwsem);
    P(rsem);                   |       P(wsem);
      nreader++;               |         /* write data */
      if (nreader==1)          |       V(wsem);
         P(wsem);              |       V(rwsem);
    V(rsem);                   |     }
    V(rwsem);                  |   }
      /* read data */          |
    P(rsem);                   |
      nreader--;               |
      if (nreader==0)          |
          V(wsem);             |
    V(rsem);                   |
-------------------------------------------------------------
```

Fig. 5.9 Reader-writer problem solution

set of processes mutually wait for one another forever, so that none of the processes can proceed. In multitasking systems, deadlocks must not be allowed to occur. Methods of dealing with deadlocks include **deadlock prevention**, **deadlock avoidance**, and **deadlock detection and recovery**. Among the various methods, only deadlock prevention is practical and used in real operating systems. A simple but effective way to prevent deadlocks is to ensure that processes request different semaphores in a unidirectional order, so that cross or circular locking can never occur. The reader may consult (Wang 2015) for how to deal with deadlocks in general.

5.10.8 Use Semaphores in Embedded Systems

We demonstrate the use of semaphores in embedded systems by the following examples.

5.10.8.1 Device Drivers Using Semaphores

In the keyboard driver of Sect. 5.6.2.1, instead of using sleep/wakeup, we may use semaphore for synchronization between processes and the interrupt handler. To do this, we simply redefine the KBD driver's data variable as a semaphore with the initial value 0.

```
typedef volatile struct kbd{ // base = 0x10006000
  char *base;            // base address of KBD, as char *
  char buf[BUFSIZE];     // input buffer
  int head, tail;
  struct semaphore data; // data.value=0; data.queue=0;
}KBD;

KBD kbd;

int kgetc() // main program calls kgetc() to return a char
{
  char c;
  KBD *kp = &kbd;
```

```
  P(&kp->data);          // P on KBD's data semaphore
  lock();
    c = kp->buf[kp->tail++]; // get a c and update tail index
    kp->tail %= BUFSIZE;
  unlock();              // enable IRQ interrupts
  return c;
}
```

(3). kbd_handler(): Rewrite KBD interrupt handler to unblock a process, if any. Since process cannot interfere with interrupt handler, there is no need to protect the data variables inside the interrupt handler.

```
kbd_handler()
{
  struct KBD *kp = &kbd;
  scode = *(kp->base+KDATA);   // read scan code in data register
  if (scode & 0x80)            // ignore key releases
     return;
  if (kp->data.value==BUFSIZE) // input buffer FULL
      return;                 // ignore current key
  c = unsh[scode];             // map scan code to ASCII
  kp->buf[kp->head++] = c;     // enter key into CIRCULAR buf[ ]
  kp->head %= BUFSIZE;
  V(&kp->data);
}
```

Note that the interrupt handler only issues V() to unblock waiting process but it should never block or wait. If the input buffer is full, it simply discards the current input key and returns. As can be seen, the logic of the new driver using semaphore is much clearer and the code size is also reduced significantly.

5.10.8.2 Event-Driven Embedded System Using Semaphore

The Example Program C5.4 uses sleep/wakeup for process synchronization. In the next example program, C5.5, we shall use P/V on semaphores for process synchronization. For the sake of brevity, we only show the modified KBD driver and the kbd process code. For clarity, the modifications are shown in bold faced lines.

```
struct kbd{
  char *base;
  char buf[BUFSIZE];  // #define BUFSIZE 128
  int head, tail;
  struct semaphore data, line;
} kbd;
kbd_init()
{
  struct kbd *kp = &kbd;
  kp->base = 0x10006000; // KBD base in Versatilepb
  kp->head = kp->tail = 0;
  kp->data.value = 0; kp->data.queue = 0;
  kp->line.value = 0; kp->line.queue = 0;
}
int kbd_handler()
{
```

```
  struct kbd *kp = &kbd;
  // same code as before: enter ASCII key into input buffer;
  V(&kp->data);
  if (c=='\r')    // return key: has an input line
     V(&kp->line);
}
int kgetc() // process call kgetc() to return a char
{
  char c;
  KBD *kp = &kbd;
  P(&kp->data);
  lock();                   // disable IRQ interrupts
   c = kp->buf[kp->tail++]; // get a c and update tail index
   kp->tail %= BUFSIZE;
  unlock();                 // enable IRQ interrupts
  return c;
}
int kgets(char *line)        // get a string
{
  char c;
  while((c= kgetc()) != '\r')
     *line++ = c;
  *line = 0;
}
int kbd_task()
{
  char line[128];
  struct kbd *kp = &kbd;
  while(1){
     P(&kp->line);          // wait for a line
     kgets(line);
     printf("line = %s\n", line);
  }
}
int main()
{  // initialization code SAME as before
   printf("P0 create tasks\n");
   kfork((int)kbd_task,  1);
   while(1){ // P0 runs whenever no task is runnable
     while(!readyQueue); // loop if readyQueue empty
     tswitch();
   }
}
```

Figure 5.10 shows the outputs of running the C5.5 program.

5.11 Other Synchronization Mechanisms

Many OS kernels use other mechanisms for process synchronization. These include

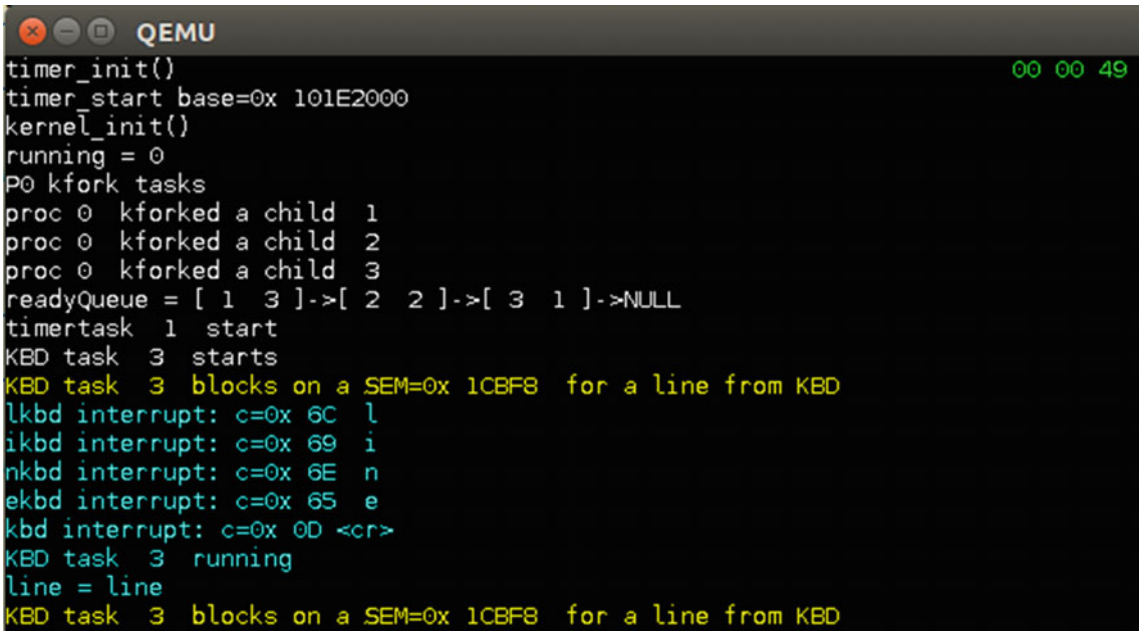


Fig. 5.10 Event-driven multitasking system using semaphores

5.11.1 Event Flags in OpenVMS

OpenVMS (formerly VAX/VMS) (OpenVMS 2014) uses event flags for process synchronization. In its simplest form, an event flag is a single bit, which is in the address spaces of many processes. Either by default or by explicit syscall, each event flag is associated with a specific set of processes. OpenVMS provides service functions for processes to manipulate their associated event flags by

```
set_event(b)   : set b to 1 and wakeup waiter(b) if any;
clear_event(b): clear b to 0;
test_event(b) : test b for 0 or 1;
wait_event(b) : wait until b is set;
```

Naturally, access to an event flag must be mutually exclusive. The differences between event flags and Unix events are:

. A Unix event is just a value, which does not have a memory location to record the occurrence of the event. A process must sleep on an event first before another process tries to wake it up later. In contrast, each event flag is a dedicated bit, which can record the occurrence of an event. Therefore, when using event flags the order of set_event and wait_event does not matter. Another difference is that Unix events are only available to processes in kernel mode, event flags in OpenVMS can be used by processes in user mode.
. Event flags in OpenVMS are in clusters of 32 bits each. A process may wait for a specific bit, any or all of the events in an event cluster. In Unix, a process can only sleep for a single event.
. As in Unix, wakeup(e) in OpenVMS also wakes up all waiters on an event.

5.11.2 Event Variables in MVS

IBM's MVS (2010) uses event variables for process synchronization. An event variable is a structure

```
struct event_variable{
        bit w;            // wait flag initial = 0
        bit p;            // post flag initial = 0
        struct proc *ptr; // pointer to waiting PROC
} e1, e2,..., en;         // event variables
```

Each event variable e can be awaited by at most one process at a time. However, a process may wait for any number of event variables. When a process calls wait(e) to wait for an event, it does not wait if the event already occurred (post bit=1). Otherwise, it turns on the w bit and waits for the event. When an event occurs, another process uses post(e) to post the event by turning on the p bit. If the event's w bit is on, it unblocks the waiting process if all its awaited events have been posted.

5.11.3 ENQ/DEQ in MVS

In addition to event variables IBM's MVS (2010) also uses ENQ/DEQ for resource management. In their simplest form, ENQ(resource) allows a process to acquire the exclusive control of a resource. A resource can be specified in a variety of ways, such as a memory area, the contents of a memory area, etc. A process blocks if the resource is unavailable. Otherwise, it gains the exclusive control of the resource until it is released by a DEQ(resource) operation. Like event variables, a process may call ENQ(r1,r2,…rn) to wait for all or a subset of multiple resources.

5.12 High-Level Synchronization Constructs

Although P/V on semaphores are powerful synchronization tools, their usage in concurrent programs is scattered. Any misuse of P/V may lead to problems, such as deadlocks. To help remedy this problem, many high-level process synchronization mechanisms have been proposed.

5.12.1 Condition Variables

In Pthreads (Buttlar et al. 1996; Pthreads 2015), threads may use condition variables for synchronization. To use a condition variable, first create a mutex, m, for locking a CR containing shared variables, e.g. a counter. Then create a condition variable, con, associated with the mutex. When a thread wants to access the shared variable, it locks the mutex first. Then it checks the variable. If the counter value is not as expected, the thread may have to wait, as in

```
int count                       // shared variable of threads
pthread_mutex_lock(m);          // lock mutex first
 if (count is not as expected)
    pthread_cond_wait(con, m); // wait in con and unlock mutex
pthread_mutex_unlock(m);        // unlock mutex
```

pthread_cond_wait(con, m) blocks the calling thread on the condition variable, which automatically and atomically unlocks the mutex m. While a thread is blocked on the condition variable, another thread may use pthread_cond_signal (con) to unblock a waiting thread, as in

```
pthread_lock(m);
   change the shared variable count;
   if (count reaches a certain value)
      pthread_cond_signal(con); // unblock a thread in con
pthread_unlock(m);
```

When an unblocked thread runs, the mutex m is automatically and atomically locked, allowing the unblocked thread to resume in the CR of the mutex m. In addition, a thread may use pthread_cond_broadcast(con) to unblock all threads that are

waiting for the same condition variable, which is similar to wakeup in Unix. Thus, mutex is strictly for locking, condition variables may be used for threads cooperation.

5.12.2 Monitors

A monitor (Hoare 1974) is an Abstract Data Type (ADT), which includes shared data objects and all the procedures that operate on the shared data objects. Like an ADT in object-oriented programming (OOP) languages, instead of scattered codes in different processes, all codes which operate on the shared data objects are encapsulated inside a monitor. Unlike an ADT in OOP, a monitor is a CR which allows only one process to execute inside the monitor at a time. Processes can only access shared data objects of a monitor by calling monitor procedures, as in

MONITOR m.procedure(parameters);

The concurrent programming language compiler translates monitor procedure calls as entering the monitor CR, and provides run-time protection automatically. When a process finishes executing a monitor procedure, it exits the monitor, which automatically unlocks the monitor, allowing another process to enter the monitor. While executing inside a monitor, if a process becomes blocked, it automatically exits the monitor first. As usual, a blocked process will be eligible to enter the monitor again when it is SINGALed up by another process. Monitors are similar to condition variables but without an explicit mutex lock, which makes them somewhat more "abstract" than condition variables. The goal of monitor and other high-level synchronization constructs is to help users write "synchronization correct" concurrent programs. The idea is similar to that of using strong type-checking languages to help users write "syntactically correct" programs. These high-level synchronizing tools are used mostly in concurrent programming but rarely used in real operating systems.

5.13 Process Communication

Process communication refers to schemes or mechanisms that allow processes to exchange information. Process communication can be accomplished in many different ways, all of which depend of process synchronization.

5.13.1 Shared Memory

The simplest way for processes communication is through shared memory. In most embedded systems, all processes run in the same address space. It is both natural and easy to use shared memory for process communication. To ensure processes access the shared memory exclusively, we may use either locking semaphore or mutex to protect the shared memory as a critical region. If some processes only read but do not modify the shared memory, we may use the reader-writer algorithm to allow concurrent readers. When using shared memory for process communication, the mechanism only guarantees processes read/write shared memory in a controlled manner. It is entirely up to the user to define and interpret the meaning of the shared memory contents.

5.13.2 Pipes

Pipes are unidirectional inter-process communication channels for processes to exchange streams of data. A pipe has a read end and a write end. Data written to the write end of a pipe can be read from the read end of the pipe. Since their debut in the original Unix, pipes have been incorporated into almost all OS, with many variations. Some systems allow pipes to be bidirectional, in which data can be transmitted in both directions. Ordinary pipes are for related processes. Named pipes are FIFO communication channels between unrelated processes. Reading and writing pipes are usually synchronous and blocking. Some systems support non-blocking and asynchronous read/write operations on pipes. For simplicity, we shall consider a pipe as a finite-sized FIFO communication channel between a set of processes. Reader and writer processes of a pipe are synchronized in the following manner. When a reader reads from a pipe, if the pipe has data, the reader reads as much as it needs (up to the pipe size) and returns the number of bytes read. If the pipe has no data but still has writers, the

reader waits for data. When a writer writes data to a pipe, it wakes up the waiting readers, allowing them to continue. If the pipe has no data and also no writer, the reader returns 0. Since readers wait for data if the pipe still has writers, the 0 return value means only one thing, namely the pipe has no data and also no writer. In that case, the reader can stop reading from the pipe. When a writer writes to a pipe, if the pipe has room, it writes as much as it needs to or until the pipe is full. If the pipe has no room but still has readers, the writer waits for room. When a reader reads data from the pipe to create more rooms, it wakes up the waiting writers, allowing them to continue. However, if a pipe has no more readers, the writer must detect this as a broken pipe error and aborts.

5.13.2.1 Pipes in Unix/Linux

In Unix/Linux, pipes are an integral part of the file system, just like I/O devices, which are treated as special files. Each process has three standard file streams; stdin for inputs, stdout for outputs and stderr for displaying error messages, which is usually associated with the same device as stdout. Each file stream is identified by a file descriptor of the process, which is 0 for stdin, 1 for stdout and 2 for stderr. Conceptually, a pipe is a two-ended FIFO file which connects the stdout of a writer process to the stdin of a reader process. This is done by replacing the file descriptor 1 of the writer process with the write-end of the pipe, and replacing the file descriptor 0 of the reader process with the read-end of the pipe. In addition, the pipe uses state variables to keep track of the status of the pipe, allowing it to detect abnormal conditions such no more writers and broken pipe, etc.

5.13.2.2 Pipes in Embedded Systems

Most embedded systems either do not support a file system or the file system may not be Unix-compatible. Therefore, processes in an embedded system may not have opened files and file descriptors. Despite this, we still can implement pipes for process communication in embedded systems. In principle, pipes are similar to the producer-consumer problem, except for the following differences.

. In the producer-consumer problem, a blocked producer process can only be signaled up by another consumer process, and vice versa. Pipes use state variables to keep track of the numbers of reader and writer processes. When a pipe writer detects the pipe has no more readers, it returns with a broken pipe error. When a reader detects the pipe has no more writers and also no data, it returns 0.
. The producer-consumer algorithm uses semaphores for synchronization. Semaphores are suitable for processes to write/read data of the same size. In contrast, pipe readers and writers do not have to read/write data of the same size. For example, writers may write lines but readers read chars, and vice versa.
. The V operation on a semaphore unblocks at most one waiting process. Although rare, a pipe may have multiple writers and readers at both ends. When a process at either end changes the pipe status, it should unblock all waiting processes on the other end. In this case, sleep/wakeup are more suitable than P/V on semaphores. For this reason, pipes are usually implemented using sleep/wakeup for synchronization.

In the following, we shall show how to implement a simplified pipe for process communication. The simplified pipe behaves as named pipes in Linux. It allows processes to write/read a sequence of bytes through the pipe, but it does not check or handle abnormal conditions, such as broken pipe. Full implementation of pipes as file streams will be shown later in Chap. 8 when we discuss general purpose embedded operating systems. The simplified pipe is implemented as follows.

(1). The pipe object: a pipe is a (global) data structure

```
typedef struct pipe{
    char  buf[PSIZE];       // circular data buffer
    int   head, tail;       // circular buf index
    int   data, room;       // number of data & room in pipe
    int   status;           // FREE or BUSY
}PIPE;
PIPE pipe[NPIPE];           // global PIPE objects
```

When the system starts, all the pipe objects are initialized to FREE.

(1). PIPE *create_pipe(): this creates a PIPE object in the (shared) address space of all the processes. It allocates a free PIPE object, initializes it and returns a pointer to the created PIPE object.
(2). Read/write pipe: For each pipe, the user must designate a process as either a writer or a reader, but not both. Writer processes call.

int write_pipe(PIPE *pipePtr, char buf[ ], int n);

to write n bytes from buf[ ] to the pipe. The return value is the number of bytes written to the pipe. Reader processes call

int read_pipe(PIPE *pipePtr, char buf[ ], int n);

which tries to read n bytes from the pipe. The return value is the actual number of bytes read. The following shows the pipe read/write algorithms, which use sleep/wakeup for process synchronization.

```
/*---------- Algorithm of pipe_read  -------------*/
int read_pipe(PIPE *p, char *buf, int n)
{   int r = 0;
    if (n<=0)
       return 0;
    validate PIPE pointer p; // p->status must not be FREE
    while(n){
       while(p->data){
         *buf++ = p->buf[p->tail++] // read a byte to buf
          tail %= PSIZE;
          p->data--; p->room++; r++; n--;
          if (n==0)
              break;
       }
       wakeup(&p->room);    // wakeup writers
       if (r)               // if has read some data
          return r;
       // pipe has no data
       sleep(&p->data);     // sleep for data
    }
}
```

```
/*---------- Algorithm of write_pipe -----------*/
int write_pipe(PIPR *p, char *buf, int n)
{   int r = 0;
    if (n<=0)
       return 0;
    validate PIPE pointer p; // p->status must not be FREE
    while(n){
       while(p->room)
          p->buf[p->head++] = *buf++; // write a byte to pipe;
          p->head %= PSIZE;
          p->data++; p->room--; r++; n--;
          if (n==0)
             break;
       }
       wakeup(&p->data);    // wakeup readers, if any.
       if (n==0)
          return r;         // finished writing n bytes
       // still has data to write but pipe has no room
```

```
            sleep(&p->room);     // sleep for room
        }
    }
```

Note that when a process tries to read n bytes from a pipe, it may return less than n bytes. If the pipe has data, it reads either n bytes or the number of available bytes in the pipe, whichever is smaller. It waits only if the pipe has no data. Thus, each read returns at most PSIZE bytes.

(3). When a pipe is no longer needed, it may be freed by destroy_pipe(PIPE *pipePtr), which deallocates the PIPE object and wake up all the sleeping processes on the pipe.

5.13.2.3 Demonstration of Pipes

The sample system C5.6 demonstrates pipe in an embedded system with static processes.

When the system starts, the initialization code creates a pipe pointed by kpipe. When the initial process P0 runs, it creates two processes, P1 as the pipe writer, and P2 as the pipe reader. For demonstration purpose, we set the pipe's buffer size to a rather small value, PSIZE=16, so that if the writer tries to write more than 16 bytes, it will wait for rooms. After reading from the pipe, the reader wakes up the writer, allowing it to continue. In the demonstration program, P1 first gets a line from the UART0 port. Then it tries to write the line to the pipe. It waits for room if the pipe is full. P2 reads from the pipe and displays the bytes read. Although each time P2 tries to read 20 bytes, the actual number of bytes read is at most PSIZE. For the sake of brevity, we only show the t.c file of the sample program.

```
/*********** t.c file of Pipe Program C5.6 ***********/
PIPE *kpipe;           // global PIPE pointer
#include "queue.c"
#include "pv.c"
#include "kbd.c"
#include "uart.c"
#include "vid.c"
#include "exceptions.c"
#include "kernel.c"
#include "timer.c"
#include "pipe.c"      // pipe implementation

int pipe_writer()      // pipe writer task code
{
  struct uart *up = &uart[0];
  char line[128];
  while(1){
    uprintf("Enter a line for task1 to get : ");
    printf("task%d waits for line from UART0\n", running->pid);
    ugets(up, line);
    uprints(up, "\r\n");
    printf("task%d writes line=[%s] to pipe\n", running->pid, line);
    write_pipe(kpipe, line, strlen(line));
  }
}
int pipe_reader()       // pipe reader task code
{
  char line[128];
  int i, n;
  while(1){
    printf("task%d reading from pipe\n", running->pid);
    n = read_pipe(kpipe, line, 20);
    printf("task%d read n=%d bytes from pipe : [", running->pid, n);
```

```
    for (i=0; i<n; i++)
      kputc(line[i]);
    printf("]\n");
  }
}

int main()
{
   fbuf_init();
   kprintf("Welcome to Wanix in ARM\n");
   uart_init();
   kbd_init();
   pipe_init();           // initialize PIPEs
   kpipe = create_pipe(); // create global kpipe
   kernel_nit();
   kprintf("P0 kfork tasks\n");
   kfork((int)pipe_writer, 1);  // pipe writer process
   kfork((int)pipe_reader, 1);  // pipe reader process
   while(1){
     if (readyQueue)
        tswitch();
   }
}
```

Figure 5.11 shows the sample outputs of running the pipe program C5.6.

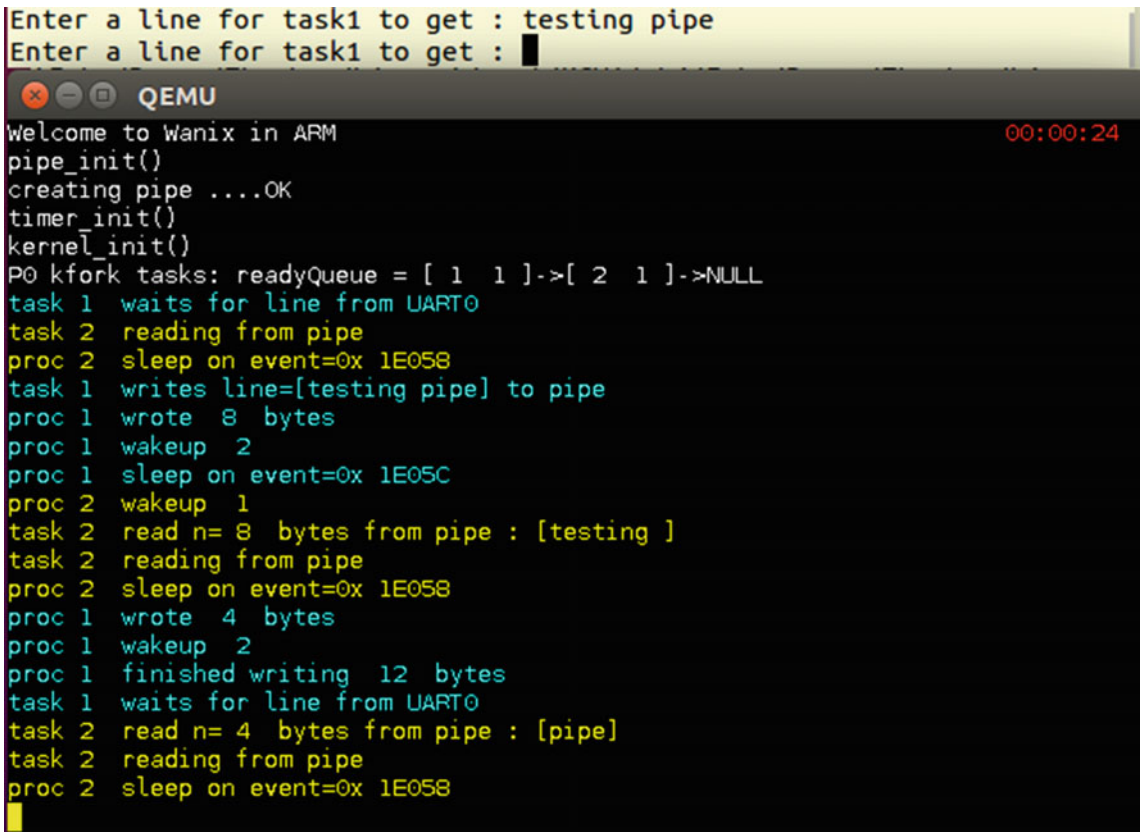


Fig. 5.11 Demonstration of pipe

5.13.3 Signals

Like interrupts to a CPU, signals are (software) interrupts to a process (Unix 1990), which diverts the process from its normal executions to do signal processing. In an ordinary OS, processes execute in one of two distinct modes; kernel mode or user mode. The CPU checks for pending interrupts at the end of each instruction, which is invisible to processes executing on the CPU. Similarly, a process checks for pending signals only in kernel mode, which is invisible to the process in user mode. In most embedded systems, all processes execute in the same address space, so they do not have a separate user mode. If we use signals for process communication, each process must check for pending signals explicitly in the process processing loop, which is equivalent to polling for events. Thus, for embedded systems with only a single address space, signals are unsuited to process communication.

5.13.4 Message Passing

Message passing allows processes to communicate by exchanging messages. Message passing has a wide range of applications. In operating systems, it is a general form of Inter-Process Communication (IPC) (Accetta et al. 1986). In computer networks, it is the basis of server-client oriented programming. In distributed computing, it is used for parallel processes to exchange data and synchronization. In operating system design, it is the basis of so called microkernel, etc. In this section, we shall show the design and implementation of several message passing schemes using semaphores.

The goal of message passing is to allow processes to communicate by exchanging messages. If processes have distinct (user mode) address spaces, they can not access each other's memory area directly. In that case, message passing must go through the kernel. If all processes only execute in the same address space of a kernel, message passing allows processes to exchange information in a controlled manner but hides the synchronization details from the processes. The contents of a message can be designed to suit the needs of the communicating processes. For simplicity, we shall assume that message contents are text strings of finite length, e.g. 128 bytes. To accommodate the transfer of messages, we assume that the kernel has a finite set of message buffers, which are defined as

```
typedef struct mbuf{
        struct mbuf *next;      // pointer to next mbuf
        int pid;                // sender pid
        int priority;           // message priority
        char contents[128];     // message contents
}MBUF;
MBUF mbuf[NMBUF];               // NMBUF = number of mbufs
```

Initially, all message buffers are in a free mbufList. To send a message, a process must get a free mbuf first. After receiving a message, it releases the mbuf for reuse. Since the mbufList is accessed by many processes, it is a critical region (CR), which must be protected. So we define a semaphore mlock = 1 for processes to access the mbufList exclusively. The algorithm of get_mbuf() and put_mbuf() is

```
   MBUF *get_mbuf()  // return a free mbuf pointer or NULL if none
{
   P(mlock);
     MBUF *mp = dequeue(mbuflList); // return first mbuf pointer
   V(mlock);
   return mp;
}
int put_mbuf(MBUF *mp)        // free a used mbuf to mbuflist
{
   P(mlock);
     enqueue(mbufList)
   V(mlock);
}
```

Instead of a centralized message queue, we assume that each PROC has a private message queue, which contains mbufs delivered to, but not yet received by, the process. Initially, every PROC's mqueue is empty. The mqueue of each process is also a CR because it is accessed by all the sender processes as well as the process itself. So we define another semaphore PROC.mlock = 1 for protecting the process message queue.

5.13.4.1 Asynchronous Message Passing

In the asynchronous message passing scheme, both send and receive operations are non-blocking. If a process can not send or receive a message, it returns a failed status, in which case the process may retry the operation again later. Asynchronous communication is intended mainly for loosely-coupled systems, in which interprocess communication is infrequent, i.e. processes do not exchange messages on a planned or regular basis. For such systems, asynchronous message passing is more suitable due to its greater flexibility. The algorithms of asynchronous send-receive operations are as follows.

```
int a_send(char *msg, int pid) // send msg to target pid
{
  MBUF *mp;
  // validate target pid, e.g. proc[pid] must be a valid processs
  if (!(mp = get_mbuf()))    // try to get a free mbuf
     return -1;              // return -1 if no mbuf
  mp->pid = running->pid;    // running proc is the sender
  mp->priority = 1;      // assume SAME priority for all messages
  copy(mp->contents, msg);   // copy msg to mbuf
  // deliver mbuf to target proc's message queue
  P(proc[pid].mlock);        // enter CR
    enter mp into PROC[pid].mqueue  by priority
  V(proc[pid].lock);         // exit CR
  V(proc[pid].message);  // V the target proc's messeage semaphore
  return 1;                  // return 1 for SUCCESS
}

int a_recv(char *msg)    // receive a msg from proc's own mqueue
{
  MBUF *mp;
  P(running->mlock);         // enter CR
  if (running->mqueue==0){   // check proc's mqueue
     V(running->mlock);      // release CR lock
     return -1;
  }
  mp = dequeue(running->mqueue); // remove first mbuf from mqueue
  V(running->mlock);         // release mlock
  copy(msg, mp->contents);   // copy contents to msg
  int sender=mp->pid;        // sender ID
  put_mbuf(mp);              // release mbuf as free
  return sender;
}
```

The above algorithms work under normal conditions. However, if all processes only send but never receive, or a malicious process repeatedly sends messages, the system may run out of free message buffers. When that happens, the message facility would come to a halt since no process can send anymore. One good thing about the asynchronous protocol is that there cannot be any deadlocks because it is non-blocking.

5.13.4.2 Synchronous Message Passing

In the synchronous message passing scheme, both send and receive operations are blocking. A sending process must "wait" if there is no free mbuf. Similarly, a receiving process must "wait" if there is no message in its message queue. In general, synchronous communication is more efficient than asynchronous communication. It is well suited to tightly-coupled systems

in which processes exchange messages on a planned or regular basis. In such a system, processes can expect messages to come when they are needed, and the usage of message buffers is carefully planned. Therefore, processes can wait for messages or free message buffers rather than relying on retries. To support synchronous message passing, we define additional semaphores for process synchronization and redesign the send-receive algorithm as follows.

```
SEMAPHORE nmbuf = NMBUF; // number of free mbufs
SEMAPHORE PROC.nmsg = 0; // for proc to wait for messages

MBUF *get_mbuf()         // return a free mbuf pointer
{
   P(nmbuf);             // wait for free mbuf
   P(mlock);
     MBUF *mp = dequeue(mbufList)
   V(mlock);
   return mp;
}
int put_mbuf(MBUF *mp)   // free a used mbuf to freembuflist
{
   P(mlock);
     enqueue(mbufList, mp);
   V(mlock);
   V(nmbuf);
}

int s_send(char *msg, int pid)// synchronous send msg to target pid
{
   // validate target pid, e.g. proc[pid] must be a valid processs
   MBUF *mp = get_mbuf();     // BLOCKing: return mp must be valid
   mp->pid = running->pid;    // running proc is the sender
   copy(mp->contents, msg);   // copy msg from sender space to mbuf
   // deliver msg to target proc's mqueue
   P(proc[pid].mlock);        // enter CR
     enqueue(proc[pid].mqueue, mp);
   V(proc[pid].lock);         // exit CR
   V(proc[pid].nmsg);         // V the target proc's nmsg semaphore
}

int s_recv(char *msg) // synchronous receive from proc's own mqueue
{
   P(running->nmsg);          // wait for message
   P(running->mlock);         // lock PROC.mqueue
     MBUF *mp = dequeue(running->mqueue); // get a message
   V(running->mlock);         // release mlock
   copy(mp->contents, msg);   // copy contents to Umode
   put_mbuf(mp);              // free mbuf
}
```

The above s_send/s_recv algorithm is correct in terms of process synchronization, but there are other problems. Whenever a blocking protocol is used, there are chances of deadlock. Indeed, the s_send/s_recv algorithm may lead to the following deadlock situations.

(1). If processes only send but do not receive, all processes would eventually be blocked at P(nmbuf) when there are no more free mbufs.
(2). If no process sends but all try to receive, every process would be blocked at its own nmsg semaphore.

(3). A process Pi sends a message to another process Pj and waits for a reply from Pj, which does exactly the opposite. Then Pi and Pj would mutually wait for each other, which is the familiar cross-locked deadlock.

As for how to handle deadlocks in message passing, the reader may consult Chap. 6 of (Wang 2015), which also contains a server-client based message passing protocol.

5.13.4.3 Demonstration of Message Passing

The sample Program C5.7 demonstrates synchronous message passing.

(1). Type.h file : added MBUF type

```
struct semaphore{
  int value;
  struct proc *queue;
};
typedef struct mbuf{
  struct mbuf *next;
  int priority;
  int pid;
  char contents[128];
}MBUF;
typedef struct proc{
  // same as before, but added
  MBUF *mQueue;
  struct semaphore mQlock;
  struct semaphore nmsg;
  int    kstack[SSIZE];
}PROC;
```

(2). Message.c file

```
/******** message.c file ************/
#define NMBUF 10
struct semaphore nmbuf, mlock;
MBUF mbuf[NMBUF], *mbufList; // mbufs buffers and mbufList
int menqueue(MBUF **queue, MBUF *p){// enter p into queue by priority}
MBUF *mdequeue(MBUF **queue){// return first queue element}
int msg_init()
{
  int i;  MBUF *mp;
  printf("mesg_init()\n");
  mbufList = 0;
  for (i=0; i<NMBUF; i++)               // initialize mbufList
    menqueue(&mbufList, &mbuf[i]); // all priority=0, so use menqueue()
  nmbuf.value = NMBUF; nmbuf.queue = 0; // counting semaphore
  mlock.value = 1;     mlock.queue = 0; // lock semaphore
}
MBUF *get_mbuf()        // allocate a mbuf
{
  P(&nmbuf);
  P(&mlock);
    MBUF *mp = mdequeue(&mbufList);
  V(&mlock);
  return mp;
```

```
}
int put_mbuf(MBUF *mp) // release a mbuf
{
  P(&mlock);
    menqueue(&mbufList, mp);
  V(&mlock);
  V(&nmbuf);
}
int send(char *msg, int pid) // send msg to partet pid
{
  if (checkPid()<0)          // validate receiving pid
     return -1;
  PROC *p = &proc[pid];
  MBUF *mp = get_mbuf();
  mp->pid = running->pid;
  mp->priority = 1;
  strcpy(mp->contents, msg);
  P(&p->mQlock);
    menqueue(&p->mQueue, mp);
  V(&p->mQlock);
  V(&p->nmseg);
  return 0;
}
int recv(char *msg)          // recv msg from own msgqueue
{
  P(&running->nmsg);
  P(&running->mQlock);
    MBUF *mp = mdequeue(&running->mQueue);
  V(&running->mQlock);
  strcpy(msg, mp->contents);
  int sender = mp->pid;
  put_mbuf(mp);
  return sender;
}
```

(3). t.c file:

```
#include "type.h"
#include "message.c"
int sender() // send task code
{
  struct uart *up = &uart[0];
  char line[128];
  while(1){
    ugets(up, line);
    printf("task%d got a line=%s\n", running->pid, line);
    send(line, 4);
    printf("task%d send %s to pid=4\n", running->pid,line);
  }
}
int receiver() // receiver task code
{
  char line[128];
  int pid;
  while(1){
```

```
    printf("task%d try to receive msg\n", running->pid);
    pid = recv(line);
    printf("task%d received: [%s] from task%d\n",
            running->pid, line, pid);
  }
}
int main()
{
   msg_init();


   kprintf("P0 kfork tasks\n");
   kfork((int)sender, 1);       // sender process
   kfork((int)receiver, 1);     // receiver process
   while(1){
     if (readyQueue)
        tswitch();
   }
}
```

Figure 5.12 shows the sample outputs of running the C5.7 Program.

5.14 Uniprocessor (UP) Embedded System Kernel

An embedded system kernel comprises dynamic processes, all of which execute in the same address space of the kernel. The kernel provides functions for process management, such as process creation, synchronization, communication and termination. In this section, we shall show the design and implementation of uniprocessor (UP) embedded system kernels. There are two distinct types of kernels; non-preemptive and preemptive. In a non-preemptive kernel, each process runs until it gives up CPU voluntarily. In a preemptive kernel, a running process can be preempted either by priority or by time-slice.

5.14.1 Non-preemptive UP Kernel

A Uniprocessor (UP) kernel is non-preemptive if each process runs until it gives up the CPU voluntarily. While a process runs, it may be diverted to handle interrupts, but control always returns to the point of interruption in the same process at the end of interrupt processing. This implies that in a non-preemptive UP kernel only one process runs at a time. Therefore, there is no need to protect data objects in the kernel from the concurrent executions of processes. However, while a process runs, it may be diverted to execute an interrupt handler, which may interfere with the process if both try to access the same data object. To prevent interference from interrupt handlers, it suffices to disable interrupts when a process executes a piece of critical code. This simplifies the system design.

The Example Program C5.8 demonstrates the design and implementation of a non-preemptive kernel for uniprocessor embedded systems. We assume that the system hardware consists of two timers, which can be programmed to generate timer interrupts with different frequencies, a UART, a keyboard and a LCD display. The system software consists of a set of concurrent processes, all of which execute in the same address space but with different priorities. Process scheduling is by non-preemptive priority. Each process runs until it goes to sleep, blocks itself or terminates. Timer0 maintains the time-of-day and displays a wall-clock on the LCD. Since the task of displaying the wall clock is short, it is performed by the timer0 interrupt handler directly. Two periodic processes, timer_task1 and timer_task2, each of which calls the pause(t) function to suspend itself for a number of seconds. After registering a pause time in the PROC structure, the process changes status to PAUSE, enters itself into a pauseList and gives up the CPU. On each second, Timer2 decrements the pause time of every process in the pauseList by 1. When the time reaches 0, it makes the paused process ready to run again. Although this can be accomplished by the sleep-wakeup mechanism, it is intended to show that periodic tasks can be implemented in the

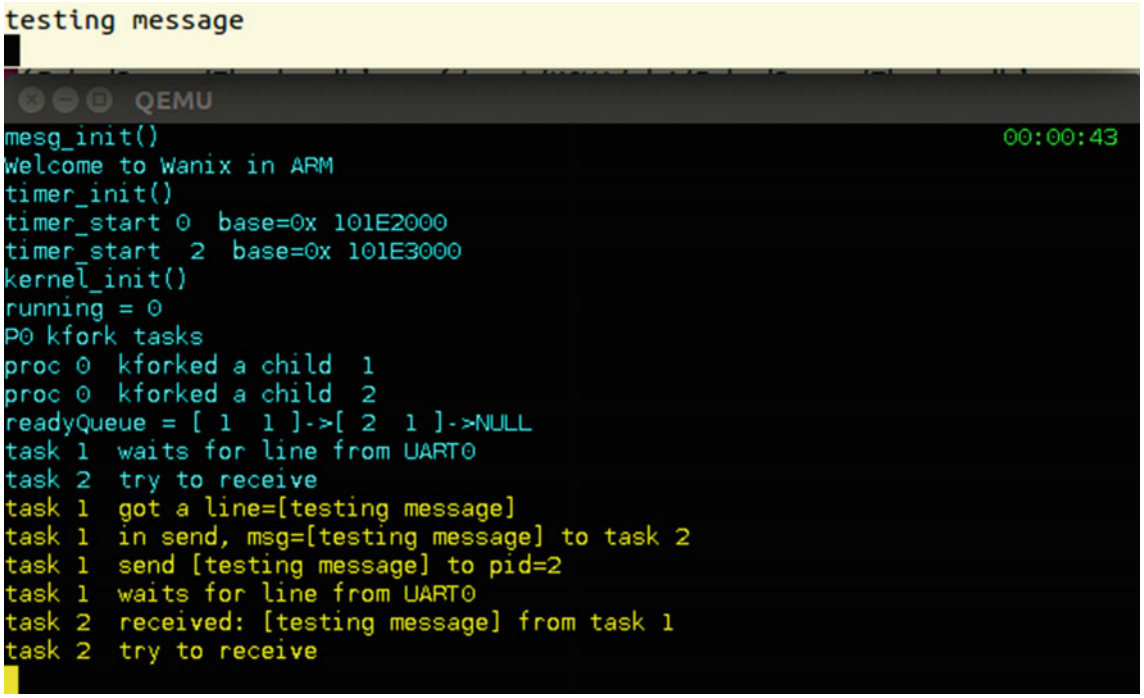


Fig. 5.12 Demonstration of message passing

general framework of timer service functions. In addition, the system supports two sets of cooperative processes, which implement the producer-consumer problem to demonstrate process synchronization using semaphores. Each producer process tries to get an input line from UART0. Then it deposits the chars into a shared buffer, char pcbuffer[N] of size N bytes. Each consumer process tries to get a char from the pcbuff[N] and displays it to the LCD. Producer and consumer processes share the common data buffer as a pipe. For brevity, we only show the relevant code segments of the system.

```
(1). /********* timer.c file of C5.8 **********/
PROC *pauseList;          // a list of pausing processes
typedef struct timer{
  u32 *base;              // timer's base address;
  int tick, hh, mm, ss;   // per timer data area
  char clock[16];
}TIMER; TIMER timer[4];   // 4 timers;
void timer_init()
{
  printf("timer_init()\n");
  pauseList = 0;         // pauseList initially 0
  // initialize all 4 timers
}
void timer_handler(int n) // n=timer unit
{
    int i; TIMER *t = &timer[n];
    t->tick++; t->ss = t->tick;
    if (t->ss == 60){
      t->ss=0; tp->mm++;
      if (t->mm == 60){
        t->mm = 0; t->hh++;
      }
    }
```

```
    if (n==0){ // timer0: display wall-clock directly
       for (i=0; i<8; i++)   // clear old clock area
           unkpchar(t->clock[i], n, 70+i);
       t->clock[7]='0'+(t->ss%10); t->clock[6]='0'+(t->ss/10);
       t->clock[4]='0'+(t->mm%10); t->clock[3]='0'+(t->mm/10);
       t->clock[1]='0'+(t->hh%10); t->clock[0]='0'+(t->hh/10);
       for (i=0; i<8; i++)    // display new wall clock
           kpchar(t->clock[i], n, 70+i);
    }
    if (n==2){// timer2: process PAUSed PROCs in pauseList
       PROC *p, *tempList = 0;
       while ( p = dequeue(&pauseList) ){
          p->pause--;
          if (p->pause == 0){ // pause time expired
           p->status = READY;
             enqueue(&readyQueue, p);
          }
          else
           enqueue(&tempList, p);
       }
       pauseList = tempList;  // updated pauseList
    }
    timer_clearInterrupt(n);
}
int timer_start(int n)
{
  TIMER *tp = &timer[n];
  kprintf("timer_start %d base=%x\n", n, tp->base);
  *(tp->base+TCNTL) |= 0x80;  // set enable bit 7
}
int timer_clearInterrupt(int n) {
  TIMER *tp = &timer[n];
  *(tp->base+TINTCLR) = 0xFFFFFFFF;
}

typedef struct uart{
  char *base;         // base address; as char *
  u32 id;             // uart number 0-3
  char inbuf[BUFSIZE];
  int  inhead, intail;
  struct semaphore indata, uline;
  char outbuf[BUFSIZE];
  int  outhead, outtail;
  struct semaphore outroom;
  int txon;           // 1=TX interrupt is on
}UART; UART uart[4];  // 4 UART structures

int uart_init()
{
  int i; UART *up;
  for (i=0; i<4; i++){ // only use UART0
      up = &uart[i];
      up->base = (char *)(0x101f1000 + i*0x1000);
      *(up->base+0x2C) &= ~0x10; // disable FIFO
      *(up->base+0x38) |= 0x30;
      up->id = i;
```

```
      up->inhead = up->intail = 0;
      up->outhead = up->outtail = 0;
      up->txon = 0;
      up->indata.value = 0; up->indata.queue = 0;
      up->uline.value = 0;  up->uline.queue = 0;
      up->outroom.value = BUFSIZE; up->outroom.queue = 0;
  }
}
void uart_handler(UART *up)
{
  u8 mis = *(up->base + MIS);  // read MIS register
  if (mis & 0x10)  do_rx(up);
  if (mis & 0x20)  do_tx(up);
}
int do_rx(UART *up)            // UART rx interrupt handler
{
  char c;
  c = *(up->base+UDR);
  up->inbuf[up->inhead++] = c;
  up->inhead %= BUFSIZE;
  V(&up->indata);
  if (c=='\r'){
     V(&up->uline);
  }
}
int do_tx(UART *up)            // UART tx interrupt handler
{
  char c; u8 mask;
  if (up->outroom.value >= SBUFSIZE){ // outbuf[ ] empty
    // disable TX interrupt; return
    *(up->base+IMSC) = 0x10;          // mask out TX interrupt
    up->txon = 0;
    return;
  }
  c = up->outbuf[up->outtail++];
  up->outtail %= BUFSSIZE;
  *(up->base + UDR) = (int)c;
  V(&up->outroom);
}
int ugetc(UART *up)
{
  char c;
  P(&up->indata);
  lock();
    c = up->inbuf[up->intail++];
    up->intail %= BUFSIZE;
  unlock();
  return c;
}
int ugets(UART *up, char *s)
{
  while ((*s = (char)ugetc(up)) != '\r'){
    uputc(up, *s);
    s++;
  }
```

```
  *s = 0;
}
int uputc(UART *up, char c)
{
  if (up->txon){ // if TX is on => enter c into outbuf[]
     P(&up->outroom);  // wait for room
     lock();
      up->outbuf[up->outhead++] = c;
      up->outhead %= 128;
     unlock();
     return;
  }
  int i = *(up->base+UFR);         // read FR
  while( *(up->base+UFR) & 0x20 ); // loop while FR=TXF
  *(up->base + UDR) = (int)c;      // write c to DR
  *(up->base+IMSC) |= 0x30;
  up->txon = 1;
}
```

(3). /************* t.c file of C5.8*****************/**

```
#include "type.h"
#include "string.c"
#include "queue.c"
#include "pv.c"
#include "vid.c"
#include "kbd.c"
#include "uart.c"
#include "timer.c"
#include "exceptions.c"
#include "kernel.c"

// IRQ interrupts handler entry point
void irq_chandler()
{
    int vicstatus, sicstatus;
    // read VIC SIV status registers to find out which interrupt
    vicstatus = VIC_STATUS;
    sicstatus = SIC_STATUS;
    if (vicstatus & (1<<4))   // bit4: timer0
       timer_handler(0);
    if (vicstatus & (1<<5))   // bit5: timer2
       timer_handler(2);
    if (vicstatus & (1<<12))  // Bit12: uart0
       uart_handler(&uart[0]);
    if (vicstatus & (1<<13))  // bit13: uart1
       uart_handler(&uart[1]);
    if (vicstatus & (1<<31)){ // SIC interrupts=bit_31=>KBD at bit 3
       if (sicstatus & (1<<3))
          kbd_handler();
    }
}
int pause(int t)
{
  lock();     // disable IRQ interrupts
  running->pause = t;
  running->status = PAUSE;
```

```
  enqueue(&pauseList, running);
  tswitch();
  unlock();   // enable IRQ interrupts
}
int timer1_task()
{
  int t = 5;
  printf("timer1_task %d start\n", running->pid);
  while(1){
     pause(t);
     printf("proc%d run once in %d seconds\n", running->pid, t);
  }
}
int timer2_task()
{
  int t = 7;
  printf("timer2_task %d start\n", running->pid);
  while(1){
    pause(t);
    printf("proc%d run once in %d seconds\n", running->pid, t);
  }
}

/****** SHARED buffer of Producers and Consumers *****/
#define PRSIZE 16
char pcbuf[PRSIZE];
int head, tail;
struct semaphore full, empty, mutex;
int producer()
{
  char c, *cp;
  struct uart *up = &uart[0];
  char line[128];
  while(1){
    ugets(up, line);
    cp = line;
    while(*cp){
      printf("Producer %d P(empty=%d)\n", running->pid, empty.value);
      P(&empty);
      P(&mutex);
        pcbuf[head++] = *cp++;
        head %= PRSIZE;
      V(&mutex);
      printf("Producer %d V(full=%d)\n", running->pid, full.value);
      // show full.queue
      V(&full);
    }
  }
}
int consumer()
{
  char c;
  while(1){
    printf("Consumer %d P(full=%d)\n", running->pid, full.value);
    P(&full);
```

```
    P(&mutex);
      c = pcbuf[tail++];
      tail %= PRSIZE;
    V(&mutex);
    printf("Consumer %d V(empty=%d) ", running->pid, empty.value);
    // show empty.queue
    V(&empty);
  }
}
int main()
{
   fbuf_init();
   uart_init();
   kbd_init();
   printf("Welcome to Wanix in ARM\n");
   //Configure VIC for IRQ interrupts: same as before
   kernel_init();
   timer_init();
   timer_start(0);
   timer_start(2);
   /* initialize data buffer for producer-consumer */
   head = tail = 0;
   full.value = 0;       full.queue = 0;
   empty.value = PRSIZE; empty.queue = 0;
   mutex.value = 1;      mutex.queue = 0;
   printf("P0 kfork tasks\n");
   kfork((int)timer1_task, 1);
   kfork((int)timer2_task, 1);
   kfork((int)producer,    2);
   kfork((int)consumer,    2);
   kfork((int)producer,    3);
   kfork((int)consumer,    3);
   printQ(readyQueue);
   while(1){
     if (readyQueue)
        tswitch();
   }
}
```

5.14.2 Demonstration of Non-preemptive UP Kernel

Figure 5.13 shows the outputs of running the program C5.8, which demonstrate a non-preemptive UP kernel.

5.14.3 Preemptive UP Kernel

In a preemptive UP kernel, while a process runs, CPU can be taken away from it to run another process. Process preemption is triggered by events that require rescheduling of processes. For example, when a higher priority process becomes ready to run or, if using time-sliced process scheduling, when a process has exhausted its time quantum. The preemption policy can be either restrictive or nonrestrictive (fully preemptive). In **restrictive preemption**, while a process is executing a piece of critical code that can not be interfered by other processes, the kernel may disable interrupts or the process scheduler to prevent process switch, thus deferring process preemption until it is safe to do so. In **nonrestrictive preemption**, process

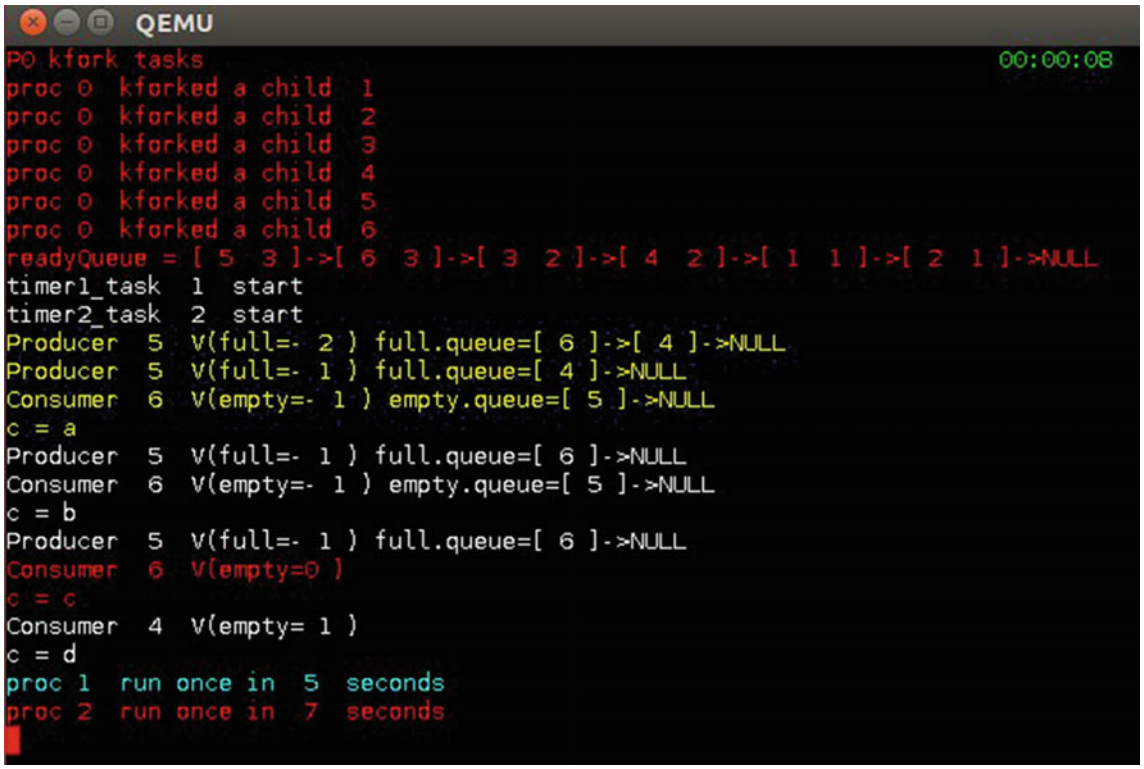


Fig. 5.13 Demonstration of non-preemptive UP kernel

switch takes place immediately, regardless what the current running process is doing. This implies that, in a fully preemptive UP kernel, processes run logically in parallel. As a result, all shared kernel data structures must be protected as critical regions. This makes a fully preemptive UP kernel logically equivalent to a MP kernel since both must support the concurrent executions of multiple processes. The only difference between a MP kernel and a fully preemptive UP kernel is that, whereas processes in the former may run on different CPUs in parallel, processes in the latter can only run concurrently on the same CPU, but their logical behavior are the same. In the following, we shall only consider fully preemptive UP kernels. MP kernels will be covered in Chap. 9 on multiprocessor systems.

We demonstrate the design and implementation of a fully preemptive UP kernel by an example. The system hardware components are the same as in the sample program C5.8. The system software consists of a set of concurrent processes with different priorities, all of which execute in the same address of the kernel. Process scheduling policy is by fully preemptive priority. In order to support full preemption, we first identify the shared data structures in the kernel that must be protected. These include

(1). **PROC *freeList**: which is used for dynamic task creation and termination.
(2). **PROC *readyQueue**: which is used for process scheduling.
(3). **PROC *sleepList, *pauseList**: which are used for sleep/wakeup operations.

For each shared kernel data structure, we implement its access functions as critical regions, each protected by a mutex lock. In addition, we define the following global variables to control process preemption.

(4). **int swflag**: switch process flag, cleared to 0 when a process is scheduled to run, set to 1 whenever a reschedule event occurs, e.g. when a ready process is added to the readyQueue.
(5). **int intnest**: IRQ interrupts nesting counter, initially 0, increment by 1 when enter an IRQ handler, decrement by 1 when exit an IRQ handler. We assume IRQ interrupts are processed in IRQ handlers directly, i.e. not by pseudo interrupt processing tasks. Process switch may occur only at the end of interrupt processing. For nested interrupts, process switch is deferred until the end of all nested interrupts processing.

The fully preemptive UP kernel consists of the following components.

(1). ts.s file of C5.9:

```
/*************** ts.s file of Program C5.9 ***************/
    .set vectorAddr, 0x10140030
reset_handler:              // SAME AS BEFORE
irq_handler:
   sub   lr, lr, #4         // adjust lr
   stmfd sp!, {r0-r12, lr} // save context in IRQ mode stack
   mrs   r12, spsr          // copy spsr into r12
   stmfd sp!, {r12}         // save spsr in IRQ stack
   msr cpsr, #0x93          // to SVC mode
   ldr  r1, =vectorAddr     // read vectorAddr to ACK interrupt
   ldr  r0, [r1]
   msr cpsr, #0x13    // SVC mode with IRQ enabled
   bl  irq_chandler   // handle IRQ in SVC mode: return 1 for tswitch
   msr cpsr, #0x92
   ldr  r1, =vectorAddr // issue EOI
   str  r0, [r1]
   cmp r0, #0
   bgt do_switch
   ldmfd sp!, {r12}     // get spsr
   msr   spsr, r12      // restore spsr
   ldmfd sp!, {r0-r12, pc}^
do_switch:
   msr cpsr, #0x93
   bl  tswitch          // switch task in SVC mode
   msr cpsr, #0x92
   ldmfd sp!, {r12}     // get spsr
   msr   spsr, r12      // restore spsr
   ldmfd sp!, {r0-r12, pc}^
```

Explanations of the ts.s file of Program C5.9:

Reset_handler: As usual, reset_handler is the entry point. It sets SVC mode stack pointer to the high end of proc[0] and copies the vector table to address 0. Next, it changes to IRQ mode to set the IRQ mode stack pointer. Then it calls main() in SVC mode. During system operation, all processes run in SVC mode in the same address of the kernel.

Irq_handler: Process switch is usually triggered by interrupts, which may wake up sleeping processes, make a blocked process ready to run, etc. Thus, irq_handler is the most important piece of assembly code relevant to process preemption. So we only show the irq_handler code. As pointer out in Chap. 3, the ARM CPU can not handle nested interrupts in IRQ mode. To handle nested interrupts, interrupt processing must be done in a different privileged mode. In order to support process preemption due to interrupts, we choose to handle IRQ interrupts in SVC mode. The reader may consult Chap. 3 for how to handle nested interrupts. The irq_handler code interacts with irq_chnadler() in C to support preemption.

```
int irq_chandler()
{
  void *(*f)();          // f is a function pointer
  intnest++;             // interrupt nest counter++
  f =(void *)*((int *)(VIC_BASE_ADDR+0x30)); // read ISR address
  (*f)();                // call the ISR function
  intnest--;             // interrupt nest counter--
  if (intnest==0 && swflag){ // if end of nested IRQs & swflag is set
     swflag = 0;
```

```
    return 1;          // return 1 to switch task
  }
  return 0;            // return 0 for no task switch
```

The algorithm of nested IRQ handler for full preemption is as follows.

/******* Algorithm of IRQ handler for full preemption ********/**

(1). Upon entry, adjust return lr; save working registers, lr and spsr in IRQ stack
(2). Change to SVC mode, read vector interrupt controller to ACK interrupt
(3). Call irq_chandler() in SVC mode with IRQ interrupts enabled

/************* irq_chandler() in C *****************/

(4). Increment interrupt nesting counter by 1;
(5). Call ISR in SVC mode; ISR may set the switch task flag by V()
(6). Decrement interrupt nesting counter by 1
(7). Return 1 to switch task if end of IRQs and swflag is set, else retrun 0

/************* back to irq_handler in assembly *******/

(8). Issue EOI for interrupt
(9). If irq_chandler() return 0: normal return; else switch task in SVC mode
(10). When the switched out task resumes, return via IRQ stack

(2). Modified kernel functions for preemption

For brevity, we only show the modified kernel functions in support of process preemption.

These include kwakeup, V on semaphore and mutex_unlock, all of which may make a sleeping or blocked process ready to run and change the readyQueue. In addition, kfork may also create a new process with a higher priority than the current running process. All these functions call reschedule(), which may switch process immediately or defer process switch until the end of IRQ interrupts processing.

```
int reschedule()
{
  int SR = int_off();
  if (readyQueue && readyQueue->priority >= running->priority){
    if (intnest==0){ // not in IRQ handler: preempt immediately
      printf("%d PREEMPT %d NOW\n", readyQueue->pid, running->pid);
      tswitch();
    }
    else{            // still in IRQ handler: defer preemption
      printf("%d DEFER PREEMPT %d\n", readyQueue->pid, running->pid);
      swflag = 1;    // set need to switch task flag
    }
  }
  int_on(SR);
}
int kwakeup(int event)
{
  PROC *p, *tmp=0;
  int SR = int_off();
  while((p = dequeue(&sleepList))!=0){
    if (p->event==event){
       p->status = READY;
```

```
        enqueue(&readyQueue, p);
      }
      else{
        enqueue(&tmp, p);
      }
    }
    sleepList = tmp;
    reschedule();  // if wake up any higher priority procs
    int_on(SR);
}
int V(struct semaphore *s)
{
    PROC *p; int cpsr;
    int SR = int_off();
    s->value++;
    if (s->value <= 0){
      p = dequeue(&s->queue);
      p->status = READY;
      enqueue(&readyQueue, p);
      printf("timer: V up task%d pri=%d; running pri=%d\n",
             p->pid, p->priority, running->priority);
      resechedule();
    }
    int_on(SR);
}
int mutex_unlock(MUTEX *s)
{
    PROC *p;
    int SR = int_off();
    printf("task%d unlocking mutex\n", running->pid);
    if (s->lock==0 || s->owner != running){ // unlock error
      int_on(SR); return -1;
    }
    // mutex is locked and running task is owner
    if (s->queue == 0){ // mutex has no waiter
       s->lock = 0;     // clear lock
       s->owner = 0;    // clear owner
    }
    else{ // mutex has waiters: unblock one as new owner
       p = dequeue(&s->queue);
       p->status = READY;
       s->owner = p;
       printf("%d mutex_unlock: new owner=%d\n", running->pid,p->pid);
       enqueue(&readyQueue, p);
       resechedule();
    }
    int_on(SR);
    return 0;
}
int kfork(int func, int priority)
{
    // create new task with priority as before
    mutex_lock(&readyQueuelock);
     enqueue(&readyQueue, p);
    mutex_unlock(&readyQueuelock);
```

```
  reschedule();
  return p->pid;
}
```

(3). t.c file:

```
/*************** t.c file of Program C5.9**************/
#include "type.h"
MUTEX *mp;               // global mutex
struct semaphore s1;     // global semaphore
#include "queue.c"
#include "pv.c"          // P/V on semaphores
#include "mutex.c"       // mutex functions
#include "kbd.c"
#include "uart.c"
#include "vid.c"
#include "exceptions.c"
#include "kernel.c"
#include "timer.c"  // timer drivers
int copy_vectors(){ // same as before }
int irq_chandler(){ // shown above }
int task3()
{
    printf("PROC%d running: ", running->pid);
    mutex_lock(mp);
     printf("PROC%d inside CR\n",  running->pid);
    mutex_unlock(mp);
    printf("PROC%d exit\n", running->pid);
    kexit();
}
int task2()
{
    printf("PROC%d running\n", running->pid);
    kfork((int)task3, 3); // create P3 with priority=3
    printf("PROC%d exit:", running->pid);
    kexit();
}
int task1()
{
  printf("proc%d start\n", running->pid);
  while(1){
    P(&s1);  // task1 is Ved up by timer periodically
    mutex_lock(mp);
     kfork((int)task2, 2); // create P2 with priority=2
     printf("proc%d inside CR\n", running->pid);
    mutex_unlock(mp);
    printf("proc%d finished loop\n", running->pid);
  }
}
int main()
{
   fbuf_init();                // LCD driver
   uart_init();                // UARTs driver
   kbd_init();                 // KBD driver
```

```
   kprintf("Welcome to Wanix in ARM\n");
   // configure VIC for vectored nterrupts
   timer_init();
   timer_start(0);              // timer0: wall clock
   timer_start(2);              // timer2: timing events
   mp = mutex_create();         // create global mutex
   s1.value = 0;                // initialize semaphore s1
   s1.queue = (PROC*)0;
   kernel_init();               // initialize kernel and run P0
   kfork((int)task1, 1);        // create P1 with priority=1
   while(1){                    // P0 loop
     if (readyQueue)
        tswitch();
   }
}
```

5.14.4 Demonstration of Preemptive UP Kernel

The sample system C5.9 demonstrates fully preemptive process scheduling. When the system starts, it creates and runs the initial process P0, which has the lowest priority 0. P0 creates a new process P1 with priority=1. Since P1 has a higher priority than P0, it immediately preempts P0, which demonstrates direct preemption without any delay. When P1 runs in task1(), it first waits for a timer event by P(s1=0), which is Ved up by a timer periodically (every 4 seconds). While P1 waits on the semaphore, P0 resumes running. When the timer interrupt handler V up P1, it tries to preempt P0 by P1. Since task switch is not allowed inside interrupt handler, the preemption is deferred, which demonstrates preemption may be delayed by interrupts processing. As soon as interrupt processing ends, P1 will preempt P0 to become running again.

To illustrate process preemption due to blocking, P1 first locks the mutex mp. While holding the mutex lock, P1 creates a process P2 with a higher priority=2, which immediately preempts P1. We assume that P2 does not need the mutex. It creates a process P3 with a higher priority=3, which immediately preempts P2. In the task3() code, P3 tries to lock the same mutex mp, which is still held by P1. Thus, P3 gets blocked on the mutex, which switches to run P2. When P2 finishes, it calls kexit () to terminate, causing P1 to resume running. When P1 unlocks the mutex, it unblocks P3, which has a higher priority than P1, so it immediately preempts P1. After P3 terminates, P1 resumes running again and the cycle repeats. Figure 5.14 shows the sample outputs of running the program C5.9.

It is noted that, in a strict priority system, the current running process should always be the one with the highest priority. However, in the sample system C5.9, when the process P3, which has the highest priority, tries to lock the mutex that is already held by by P1, which has a lower priority, it becomes blocked on the mutex and switches to run the next runnable process. In the sample system, we assumed that the process P2 does not need the mutex, so it becomes the running process when P3 gets blocked on the mutex. In this case, the system is running P2, which does not have the highest priority. This violates the strict priority principle, resulting in what's known as a **priority inversion** (Lampson and Redell 1980), in which a low priority process may block a higher priority process. If P2 keeps on running or it switches to another process of the same or lower priority, process P3 would be blocked for an unknown amount of time, resulting in an **unbounded priority inversion**. Whereas simple priority inversion may be considered as natural whenever processes are allowed to compete for exclusive control of resources, unbounded priority inversion could be detrimental to systems with cirtical timing requirements. The sample program C5.9 actually implements a scheme called **priority inheritance**, which prevents unbounded priority inversion. We shall discuss priority inversion in more detail later in Chap. 10 on real-time systems.

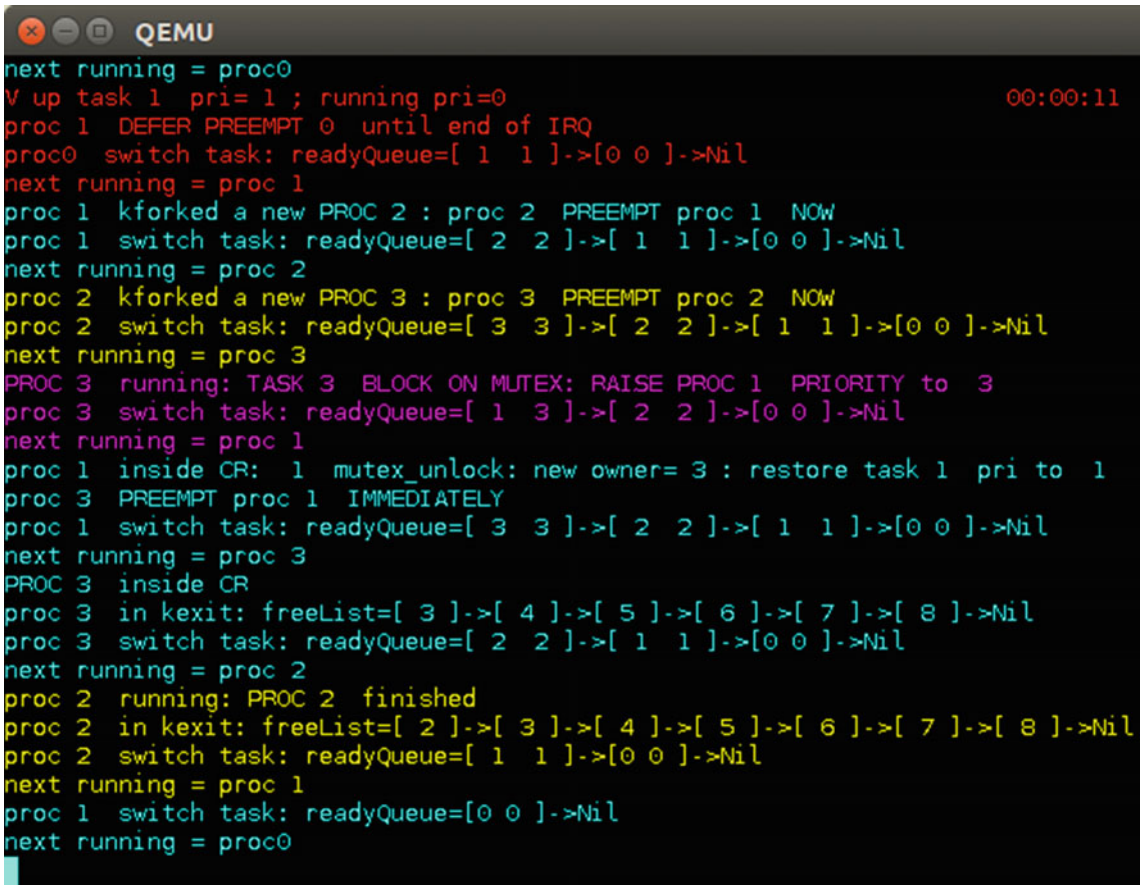


Fig. 5.14 Demonstration of process preemption

5.15 Summary

This chapter covers process management. It introduces the process concept, the principle and technique of multitasking by context switching. It showed how to create processes dynamically and discussed the principles of process scheduling. It covered process synchronization and the various kinds of process synchronization mechanisms. It showed how to use process synchronization to implement event-driver embedded systems. It discussed the various kinds of process communication schemes, which include shared memory, pipes amd message passing. It shows how to integrate these concepts and techniques to implement a uniprocessor (UP) kernel that supports process management with both non-preemptive and preemptve process scheduling. The UP kernel will be the foundation for developing complete operating systems in later chapters.

List of Sample Programs

C5.1: Context switch
C5.2: Multitasking
C5.3. Dynamic Processes
C5.4. Event-driven multitasking system using sleep/wakeup
C5.5: Even-driven multitasking system using semaphores
C5.6: Pipe
C5.7. Message passing
C5.8: Non-preemptive UP kernel
C5.9: Preemptive UP kernel

Problems

1. In the example program C5.2, the tswitch function saves all the CPU registers in the process kstack and it restores all the save registers of the next process when it resumes. Since tswitch() is called as function, it is clearly unnecessary to save/restore R0. Assume that the tswitch function is implemented as

```
tswitch:
// disable IRQ interrupts
stmfd sp!, {r4-r12, lr}
LDR r0, =running     // r0=&running
LDR r1, [r0, #0]     // r1->runningPROC
str sp, [r1, #4]     // running->ksp = sp
bl   scheduler
LDR r0, =running
LDR r1, [r0, #0]     // r1->runningPROC
lDR sp, [r1, #4]
// enable IRQ interrupts
ldmfd sp!, {r4-r12, pc}
```

(1). Show how to initialize the kstack of a new process for it to start to execute the body() function.
(2). Assume that the body() function is written as

int body(int dummy, int pid, int ppid){ }

where the parameters pid, ppid are the process id and the parent process id of the new process. Show how to modify the kfork() function to accomplish this.

2. Rewrite the UART driver in Chap. 3 by using sleep/wakeup to synchronize processes and the interrupt handler.
3. In the example program C5.3, all the processes are created with the same priority (so that they take turn to run).

(1). What would happen if the processes are created with different priorities?
(2). Implement a change_priority(int new_priority) function, which changes the running task's priority to new_priority. Switch process if the current running process no longer has the highest priority.

4. With dynamic processes, a process may terminate when it has completed its task. Implement a kexit() function for tasks to terminate.
5. In all the example programs, each PROC structure has a statically allocate 4KB kstack.

(1). Implement a simple memory manager to allocate/deallocate memory dynamically. When the system starts, reserve a piece of memory, e.g. a 1MB area beginning at 4MB, as a free memory area. The function

char *malloc(int size)

allocates a piece of free memory of size bytes.When a memory area is no longer needed, it is released back to the free memory area by

void mfree(char *address, int size)

Design a data structure to represent the current available free memory. Then implement the malloc() and mfree() functions.

(2). Modify the kstack field of the PROC structure as an integer pointer

```
int *kstack;
```

and modify the kfork() function as

```
int kfork(int func, int priority, int stack_size)
```

which dynamically allocates a memory area of stack_size (in 1KB units) for the new process.

(3). When a process terminates, its stack area must be freed. How to implement this?

6. It is well known that an interrupt handler must never go to sleep, become blocked or wait. Explain why?
7. Misuse of semaphores may result in deadlocks. Survey the literatures to find out how to deal with deadlocks by deadlock prevention, deadlock avoidance, deadlock diction and recovery.
8. The pipe program C5.6 is similar to named pipes (FIFOs) in Linux. Read the Linux man page on fifo to learn how to use named pipes for inter-process communication.
9. Modify the example program C5.8 by adding another set of cooperative produced-consumer processes to the system. Let producers get lines from the KBD, process the chars and pipe them to the consumers, which output the chars to the second UART terminal.
10. Modify the sample program C5.9 to handle nested IRQ interrupts in SYS mode but still allows task preemption at the end of nested interrupt processing.
11. Assume that all processes have the same priority. Modify the sample program C5.9 to support process scheduling by time slice.

References

Accetta, M. et al., "Mach: A New Kernel Foundation for UNIX Development", Technical Conference - USENIX, 1986.
ARM toolchain: http://gnutoolchains.com/arm-eabi, 2016.
Bach, M. J., "The Design of the Unix operating system", Prentice Hall, 1990.
Buttlar, D, Farrell, J, Nichols, B., "PThreads Programming, A POSIX Standard for Better Multiprocessing", O'Reilly Media, 1996.
Dijkstra, E.W., "Co-operating Sequential Processes", in Programming Languages, Academic Press, 1965.
Hoare, C.A.R, "Monitors: An Operating System Structuring Concept", CACM, Vol. 17, 1974.
IBM MVS Programming Assembler Services Guide, Oz/OS V1R11.0, IBM, 2010.
Lampson, B; Redell, D. (June 1980). "Experience with processes and monitors in MESA". Communications of the ACM (CACM) **23** (2): 105–117, 1980.
OpenVMS: HP OpenVMS systems Documentation, http://www.hp.com/go/openvms/doc, 2014.
Pthreads: https://computing.llnl.gov/tutorials/pthreads/, 2015.
QEMU Emulators: "QEMU Emulator User Documentation", http://wiki.qemu.org/download/qemu-doc.htm, 2010.
Silberschatz, A., P.A. Galvin, P.A., Gagne, G, "Operating system concepts, 8th Edition", John Wiley & Sons, Inc. 2009.
Stallings, W. "Operating Systems: Internals and Design Principles (7th Edition)", Prentice Hall, 2011.
Tanenbaum, A. S., Woodhull, A. S., "Operating Systems, Design and Implementation, third Edition", Prentice Hall, 2006.
Versatilepb: Versatile Application Baseboard for ARM926EJ-S, ARM Information Center, 2016.
Wang, K.C., "Design and Implementation of the MTX Operating Systems, Springer International Publishing AG, 2015.

Embedded Real-Time Operating Systems 10

10.1 Concepts of RTOS

A real-time operating system (RTOS) (Dietrich and Walker 2015) is an operating system intended for real-time applications. Real-time applications usually have very stringent timing requirements. First, a RTOS must be able to respond to external events quickly, e.g. within a very short time, known as the **interrupt latency**. Second, it must complete every requested service within a prescribed time limit, known as the **task deadline**. If a real-time system can always meet these critical timing requirements, it is called a **hard real-time** system. If it can only meet the requirements most of the time but not always, it is called a **soft real-time** system. In order to meet the stringent timing requirements, real-time OS are usually designed with the following capabilities.

. Minimum interrupt latency: Interrupt latency is the amount of time from the moment an interrupt is received to the moment the CPU starts to execute the interrupt handler. In order to minimize interrupt latency, a RTOS kernel must not mask out interrupts for long periods of time. This usually means the system must support nested interrupts to ensure processing low priority interrupts do not delay the processing of high priority interrupts.

. Short critical regions: All OS kernels rely on critical regions to protect shared data objects as well as for process synchronization. In a RTOS kernel, all critical regions must be as short as possible.

. Preemptive task scheduling: Preemption means a higher priority task can preempt a lower priority task at any time. In order to meet task deadlines, a RTOS kernel must support preemptive task scheduling. Task switching time must be short also.

. Advanced task scheduling algorithm: Preemptive scheduling is a necessary but not sufficient condition for real-time systems. Without preemptive scheduling, high priority tasks may be delayed by low priority tasks for a variable amount of time, making it impossible to meet task deadlines. However, even with preemptive scheduling, there is no guarantee that the tasks will be able to meet their deadlines. The system must use a suitable scheduling algorithm that helps achieve this goal.

10.2 Task Scheduling in RTOS

In a general purpose OS, the task scheduling policy is usually designed to achieve a balanced system performance among the various conflicting goals, such as throughput, resource utilization and quick response to interactive users, etc. In contrast, in an RTOS, the only goals are to ensure fast response and guarantee task deadlines. The scheduling policy must favor those tasks with the most urgent timing constraints. Such constraints can be translated into task priorities. Therefore, the scheduling algorithm of RTOS must be preemptive based on task priorities. There are two main types of task scheduling algorithms for RTOS, known as Rate-Monotonic Scheduling (RMS) and Earliest-Deadline-first (EDF) scheduling, and their variant forms.

© Springer International Publishing AG 2017
K.C. Wang, *Embedded and Real-Time Operating Systems*,
DOI 10.1007/978-3-319-51517-5_10

10.2.1 Rate-Monotonic Scheduling (RMS)

Rate-Monotonic Scheduling (RMS) (Liu and Layland 1973) is a static priority based scheduling algorithm for real time systems. The RMS model assumes the following conditions.

(1). Periodic tasks: tasks are periodic with deadlines exactly equal to periods.
(2). Static priorities: tasks with shorter periods are assigned higher priorities.
(3). Preemption: the system always runs the highest priority task, which immediately preempts other tasks of lower priorities.
(4). Tasks do not share resources that would cause them to block or wait.
(5). Context switch and other task operation time, e.g. release and startup time, are zero.

RMS analyzes such a system model and derives the condition under which the tasks can be scheduled to meet their deadlines. Liu and Layland proved that for a set of *n* periodic tasks with unique periods, a feasible schedule that will always meet task deadlines exists if the CPU utilization factor U is below a specific bound. The schedulability condition of RMS is

$$\mathbf{U} = \mathbf{SUM}_{\mathbf{i}=1}^{\mathbf{n}}(\mathbf{Ci}/\mathbf{Ti}) <= \mathbf{n}(\mathbf{2} * *(\mathbf{1}/\mathbf{n}) - \mathbf{1})$$

where Ci is the computation time of taski, Ti is the period of taski, and n is the number of tasks to be scheduled. For example, U <= 0.8284 for two tasks. When the number of tasks n becomes large, the value of U tends toward a limit value of

$$\mathbf{ln}(\mathbf{2}) = \mathbf{0.693247}\ldots.$$

This implies that RMS can meet all the task deadlines if CPU utilization is less than 69.32%. Furthermore, RMS is optimal for preemptive uniprocessor systems in the sense that, if any static-priority scheduling algorithm can meet all the deadlines, then so can the RMS algorithm. It is worth noting that the schedulability test of RMS is only a sufficient, but not necessary, condition. For instance, for the task set task1 = (C1=2, T1=4), task2 = (C2=4, T2=8), the value of U is 1.0, which is greater than the RMS bound 0.828, but the tasks are schedulable with 100% CPU utilization, as the following timing diagram shows.

```
time: 0  1  2  3  4  5  6  7  8
      -------------------------
      |task1|     |task1|     |
      |     |task2|     |task2|
      -------------------------
```

In general, if the task periods are **harmonic**, meaning that for each task its period is an exact multiple of every task that has a shorter period, the tasks are schedulable with a higher utilization factor than the RMS bound.

10.2.2 Earliest-Deadline-First (EDF) Scheduling

In the Earliest-Deadline-First (EDF) model (Leung et al. 1982), tasks can be periodic or non-periodic. Each task has a deterministic deadline. Tasks are maintained in a priority queue ordered by their closest deadlines, i.e. shorter deadlines tasks have higher priorities. The EDF scheduling algorithm always runs the task with the closest deadline. Like RMS, EDF is also optimal for preemptive uniprocessor systems. When scheduling periodic tasks with deadlines equal to periods, EDF has a CPU utilization bound of 100%. The schedulability condition of EDF is

$$\mathbf{U} = \mathbf{SUM}_{\mathbf{i}=\mathbf{1}}^{\mathbf{n}}(\mathbf{Ci}/\mathbf{Ti}) <= 1$$

where Ci and Ti are the worst-case computation-time and inter-arrival period of the tasks. Compared with RMS, EDF can guarantee task deadlines with a higher CPU utilization factor but it also has two drawbacks. First, EDF is more difficult to implement because task priorities are no longer static but dynamic. It must keep track of task deadlines and update the task queue whenever a reschedule event occurs. Second, when the system is overloaded (CPU utilization factor > 1) the tasks that

will miss deadlines in RMS are usually the ones with the longer periods (low priorities). In the EDF model, such tasks are largely unpredictable, meaning that any task may miss its deadline. However, there are also comparative analyses of EDF vs. RMS which tends to refute such claims (Buttazzo 2005). Despite this, most real RTOS prefer RMS over EDF, mainly because of the static task priorities and deterministic nature of RMS.

10.2.3 Deadline-Monotonic Scheduling (DMS)

Deadline-Monotonic Scheduling (DMS) (Audsley 1990) is a generalized form of RMS. In RMS, task deadline and period are assumed to be exactly equal. The DMS model relaxes this condition to allow task deadlines to be less than or equal to task periods, i.e.

$$\textbf{task computation time} <= \textbf{deadline} <= \textbf{period}$$

In DMS, tasks with shorter deadlines are assigned higher priorities. For this reason, DMS is also known as Inverse-Deadline Scheduling (IDS). Run-time scheduling is the same as in RMS, i.e. by preemptive task priority. When task deadlines and periods are equal, DMS reduces to RMS as a special case. In (Audsley et al. 1993), the RMS condition is relaxed to allow task deadlines to be less than task periods. It also extended the DMS model to include non-periodic tasks and derived schedulability tests for such cases.

Despite these real time system models and analytical results, both RMS and EDF can only be used as general guidelines in the design of real RTOS. The problem becomes more prominent when it comes to the implementation of real RTOS systems, primarily due to the following reasons. A fundamental flaw in both the RMS and EDF models is that tasks can not share resources that would cause them to block or wait, and task switching incurs zero overhead. These conditions are unrealistic since in a real system resource sharing among tasks is inevitable and task switching time can never be zero. Although there are some attempts to extend both the RMS and EDF models to allow resource sharing by including task blocking time in schedualability analyses, the results usually involve many variables that are difficult to quantify, so they just assume the worst case values. Besides, allowing resource sharing also leads to other problems, such as priority and deadline inversions, which must be handled properly in a real RTOS.

10.3 Priority Inversion

As in any OS, a RTOS kernel must use critical regions (CRs) to protect shared resources. Software tools used to enforce critical regions include Event-Control-Bocks (ECBs), mutexes, semaphores and message queues, etc. All these mechanisms are based on the locking protocol, in which a task becomes blocked if it can not acquire a CR. In addition to the usual problems of locking, such deadlock and starvation, allowing preemption in critical regions leads to a unique problem known as priority inversion (Sha et al. 1990), which can be best described as follows.

Let TL, TM, TH denote tasks of low, medium and high priorities, respectively. Assume that TL has acquired a CR and is executing inside the CR. Next, when TH becomes ready to run, it preempts TL. Assume that TH also needs the same CR, which is still held by TL. So TH blocks itself on the CR, waiting for TL to release the CR. Then, TM becomes ready to run, which does NOT need the CR. Since TM has higher priority than TL, it immediately preempts TL. Now, TM is running but its priority is lower TH, resulting in a priority inversion. In the worst case, TH may be delayed for an unknown amount of time since TM may be preempted by other tasks with priorities between TH and TM, etc. The phenomenon is known as **unbounded priority inversion**. Similarly, in the EDF model, task priorities are assigned dynamically by their closest deadlines. If tasks share resources, which may cause higher priority tasks to block, the same priority inversion problem would occur, resulting in a **deadline-inversion**.

It is noted that in an ordinary OS kernel, priority inversion may also occur but its effect is usually unnoticeable and harmless since all it does is to delay some higher priority tasks for awhile, which eventually acquire their needed CRs and continue. In a RTOS, delaying high priority tasks may cause them to miss deadlines, which may trigger system failure alarms. The most famous example of priority inversion is the system reset problem that occurred in the **Mars Pathfinder mission** (Jones 1997; Reeves 1997). The problem was resolved only after the engineers at JPL reproduced the problem on earth, identified the cause and modified the onboard task scheduler to avoid priority inversion.

10.4 Priority Inversion Prevention

There are many ways to prevent priority inversion. The first one is not to let tasks share resources, as required by both the RMS and EDF models, but this is clearly impractical. The second one is to assign all the tasks with the same priority, which is also impractical. So far, the only practical ways of preventing priority inversion are the following schemes.

10.4.1 Priority Ceiling

In the priority ceiling scheme, it assumes that for each CR the ceiling priority of the CR is greater than the highest priority of all the tasks that may compete for the CR. Whenever a task gains control of a CR, its priority is immediately raised to the ceiling priority of the CR, thus preventing preemption from any other tasks with a priority lower than the ceiling priority. This also implies that a task can lock a CR if its priority is higher than the ceiling priorities of all the CRs locked by other tasks. When a task exits a CR, it reverts back to its original priority. Priority ceiling is easy to implement but it may prevent preemption unnecessarily. For instance, while holding a CR, a low priority task will prevent preemption from any other task lower than the ceiling priority even if it does not need the same CR.

10.4.2 Priority Inheritance

In the priority inheritance (Sha et al. 1990) scheme, while a task holds a CR, if another task with higher priority tries to acquire the same CR, it temporarily raises the priority of the task that holds the CR to that of the requesting task. This ensures that the priority of a task executing inside a CR is always equal to the highest priority of the tasks blocked on the CR. When the executing task exits the CR, it reverts back to its original priority. Priority inheritance is more flexible than priority ceiling but it also incurs more overhead to implement. In the priority ceiling scheme, the priority of a task executing inside a CR is static, which does not change until it exits the CR. In the priority inheritance scheme, whenever a task is about to become blocked on a CR, it must check whether its priority is the highest among all the tasks waiting for the CR and, if so, it must pass the priority to the task that holds the CR. Thus, while executing inside a CR, the priority of a task may change dynamically.

10.5 Survey of RTOS

Unlike general purpose operating systems, which require a wide range of capabilities, real-time operating systems are usually designed to provide only limited functionalities intended for specialized environments. As a result, RTOS are usually much simpler than general purpose OS. For instance, in most RTOS all tasks run in the same address space, so they do not have separate kernel and user modes. Furthermore, most ROTS do not support file systems and user interface, etc. Despite their stringent timing requirements, RTOS are actually easier to develop as compared with general purpose OS. This is evidenced by the large number of RTOS posted on the Internet, ranging from open source ROTS intended for hobbyists to proprietary RTOS intended for commercial markets. In this section, we shall present a brief case study of some of the popular RTOS.

10.5.1 FreeRTOS

FreeRTOS (2016) is an open-source real time kernel designed specifically for small embedded systems. FreeRTOS is essentially a bare-bone kernel, which provides basic support for developing real-time applications. Key features of FreeRTOS are

Tasks: The execution units in FreeRTOS are called tasks. Each task is represented by a Task Control Block (TCB). All tasks executes in the same address space of the kernel. The kernel provides support for task creation, suspension, resumption, change priority and deletion. It also supports co-routines, which are executable units that do not need much stack space.

Scheduling: Task scheduling in FreeRTOS is by preemptive priority. For tasks with the same priority, it also supports cooperative and round-robin with time-slice options. Task switch is not allowed during nested interrupts processing. Task scheduler can be disabled to prevent task switch in long critical regions.

Synchronization: Task synchronization in FreeRTOS is based on the queue operation. It uses both binary and counting semaphores for general task synchronization, and it uses recursive mutexes with priority inheritance to protect critical regions. A recursive mutex can be locked/unlocked by the owner recursively for up to 256 levels deep.

Memory Protection: FreeRTOS has no memory protection in general, but it supports memory protection on some specific ARM boards, e.g. FreeRTOS-MPU supports ARM Cortex-M3 Memory Protection Unit (MPU).

Timer Service: The FreeRTOS kernel supports both timer ticks and software timers.
It also supports a tickless mode which suppresses periodic timer tick interrupts to reduce power consumption.

Portability: The FreeRTOS kernel consists of only a few files, most of which are written in C. It has been ported to several different architectures, including ARM, Intel x86 and PowerPC, etc. DNX (DNX 2015) is a RTOS based on freeRTOS. It adds Unix-like API interface, file system support and new device drivers to the basic freeRTOS kernel.

10.5.2 MicroC/OS (µC/OS)

MicroC/OS (uC/OS) (Labrosse 1999) is a preemptive, real-time multitasking kernel for microprocessors, microcontrollers and Digital signal Processors (DSPs). The current version is uC/OS-III, which is maintained and marketed by Micrium (2016).

(1). Tasks: In uC/OS, the execution units are called tasks, which are essentially threads executing in the same address space of the kernel. The uC/OS kernel provides support for task creation, suspension, resumption, deletion and task statistics. uC/OS-II supports up to 256 tasks. In uC/OS-III, the number of tasks is variable, limited only by the amount of available memory.
(2). Scheduling: Task scheduling is by preemptive static priority. Task priorities are assigned by the user, presumably by the RMS algorithm. In µC/OS-II, all tasks have different priorities (by task ID). µC/OS-III allows multiple tasks to run at the same priority, so it also supports round-robin scheduling by time slice.
(3). Memory Management: The uC/OS kernel does not provide any memory protection. It allows the user to define partitions of memory areas consisting of fixed sized memory blocks. Memory allocation is by fixed sized blocks.
(4). Synchronization: The uC/OS kernel relies on disable/enable interrupts to protect short critical regions, and it uses disable/enable task scheduler to protect long critical regions. Other mechanisms used for task synchronization include event control blocks (ECBs), semaphores, mailboxes and message queues. The uC/OS-III kernel uses mutual-exclusive semaphores (mutexes) with priority inheritance to prevent priority inversion.
(5). Interrupts: The uC/OS kernel supports nested interrupts. Interrupt processing is performed inside ISR routines directly. Task scheduling is disabled during nested interrupts processing. Task switch occurs only when all nested interrupts have ended.
(6). Timer Service: µC/OS requires a periodic time source to keep track of time delays and timeouts. The kernel only provides functions for a task to suspend itself for a specified number of timer ticks. A suspended task becomes ready to run again when its delay time expires. Before the requested time expires, a suspend task can also be resumed by another task. It does not provide general timer service functions, such as interval timers with notification and cancellation, etc.
(7). Port: uC/OS is written mainly in ANSI-C. Its syntax, file naming convention and development environment are based on Microsoft IDE on the Intel-x86 architecture. It's reported that uS/OS-III has been ported to several other platforms, such as the ARM architecture.

10.5.3 NuttX

NuttX (Nutt 2016) is a RTOS with an emphasis on compliance with POSIX and ANSI standards. It includes standard POSIX 1003 APIs and also APIs adopted from other common RTOS. Some of the APIs, such as task_creat, waitpid, vfork, execv, etc. are adapted to suit the embedded environment.

(1). Tasks: The NuttX kernel supports task creation, termination, deletion, init, activate and restart, etc. Tasks can be created by task_creat in one step. Alternatively, they can also be created and started up by task_init followed by task_activate in two steps. Unlike most other RTOS, NuttX tries to comply with the POSIX standard closely. For instance, tasks in NuttX obey the parent-child relation. Child task may inherit file streams, e.g. stdin, stdout and stderr, from the parent, and parent may wait for child termination. It allows tasks to change execution images to different files by execv. On some hardware platforms it even supports vfork, which creates a task skeleton without an execution image. A vforked task may use execv to create its own image from an executable file.
(2). Scheduling: Task scheduling is by preemptive task priority. Each task can set its scheduling policy as either FIFO (First-In-First-Out) or RR (Round-Robin). Tasks with the same priority are scheduled by FIFO or Round-robin with a prescribed time slice. In addition, tasks can also change priority and yield CPU to other tasks of the same priority.
(3). Synchronization: The NuttX kernel uses counting semaphore with priority inheritance, which is available as a configurable option. It supports named message queues of POSIX for inter-task communication. Any task may send/receive messages to/from named message queues. Interrupt handlers may also send messages via named message queues. To prevent task blocking on full message queues, messages can be sent with a timeout option. In NuttX, timeout is implemented by POSIX signals as in Unix.
(4). Signals: In addition to semaphores and message queues, NuttX also uses signals for inter-task communication. It allows any task or interrupt handler to post a signal to any task (by task ID). A signal is a (software) interrupt to a task, causing the task to execute a prescribed signal handler function. Unlike Unix, which has predefined signal handlers, there are no predefined actions for signals in NuttX. The default action for all signals is to ignore the signal. It allows the user to install signal handlers to handle the signals.
(5). Clock and Timer Service: The NuttX kernel supports POSIX compatible timer and interval timer service functions. Each task may create a per-task timer based on a clock, which provides clock ticks. A task may set an interval timer request. When the interval timer expires, a timeout signal is delivered to the task, allowing it to handle the signal by a pre-installed signal handler.
(6). File system and Network Interface: NuttX includes an optional file system, which is not needed for NuttX to run. If file system is enabled, NuttX starts with a pseudo root file system in memory. Real file systems can be mounted on the pseudo root file system. File system interface is by a set of standard POSIX APIs, such as open, close, read, write, etc. It provides a limited network capability by a subset of socket interface functions.

10.5.4 VxWorks

VxWorks (2016) is a proprietary RTOS developed for embedded systems by Wind River. Due to its proprietary nature, we can only gather some general information about the system based on published documents and user guides that are available in the public domain. Key features of the system include the following.

(1). Tasks and Scheduling: multitasking kernel with preemptive and round-robin scheduling and fast interrupt response.
(2). Synchronization: Binary and counting semaphores, mutexes with priority inheritance.
(3). Interprocess Communication: Local and distributed message queues.
(4). Development Environment: As a common practice in embedded systems, VxWorks uses a cross-compiling development environment. Application software is developed on a host system, e.g. Linux. The host provides an integrated development environment (IDE) consisting of editor, compiler toolchain, debugger, and emulators. Applications are cross-compiled to run on the target systems, which include ARM, Intel x86 and PowerPC. In addition to the IDE, VxWorks also include board support packages, TCP/IP networking stack, error detection/reporting and symbolic debugging.
(5). File systems: VxWorks support several file systems, which include the Reliability File System (HRFS), FAT file system (DOSFS), Network File system (NFS) and TFFS for flash devices. These are presumably parts of the IDE in support of the development environment. It is unclear whether any real-time applications may include file system support.

10.5.5 QNX

QNX (2015) is a proprietary Unix-like RTOS aimed primarily at the embedded systems market. A unique feature of QNX is that it is a microkernel based system. The QNX kernel contains only CPU scheduling, interprocess communication, interrupt redirection and timers. All other functions are executed as user processes outside of the microkernel. Key features of QNX are:

(1). Interprocess Communication (IPC) The QNX microkernel supports processes. Each process resides in a unique address space. Processes communicate with one another by exchanging messages through the microkernel. The QNX IPC consists of sending a message from one process to another and waiting for a reply. Because of the overhead in message exchange, most microkernel based systems do not perform well. QNX remedies this problem by using a more efficient message passing mechanism. In QNX, msgSend, which allows a process to send a message and wait for a reply, is a single operation. The message is copied by the kernel from the address space of the sending process to that of the receiving process. If the receiving process is waiting for the message, control of the CPU is transferred to the receiving process at the same time, which eliminates the need for explicitly unblocking the receiving process and invoking the scheduler. This tight integration between message passing and CPU scheduling is a key mechanism that makes the QNX microkernel work. All I/O operations, file system operations, and network operations are based on message passing. Message handling is prioritized by thread priority. Since I/O requests are performed using message passing, high priority threads receive I/O service before low priority threads. Later versions of QNX reduce the number of separate processes and integrate the network stack and other function blocks into single applications to improve the system performance.
(2). Threads: In QNX the smallest execution entities are threads. Each process contains a number of threads, which are independent execution units in the same address space of a process. The QNX microkernel supports Pthreads compliant APIs for threads creation, management and synchronization.
(3). Scheduling: Threads scheduling is by preemptive priority. In addition, it also supports adaptive partition scheduling (APS), which guarantees minimum CPU percentages to selected groups of threads, even though others may have higher priority.
(4). Synchronization: In QNX, IPC is for message passing between processes in different address spaces. It is not for threads within the same address space of a process. For threads synchronization, QNX uses POSIX compliant APIs to support mutexes, condition variables, semaphores, barriers and reader-writer locks, etc. When processes are allowed to share memory, most of the mechanisms are also applicable to threads in different processes.
(5). Boot Loader: Another key component of QNX is the boot loader, which can load an image containing not only the kernel but also any desired collection of user programs and shared libraries. It allows user programs, device drivers and supporting libraries to be built into the same boot image.
(6). Platforms: According to the latest QNX documentation, QNX Neutrino supports SMP and MP with processor affinity, which locks each application to a specific CPU. Due to its microkernel architecture, it should also be easier to adapt QNX to distributed environment.

10.5.6 Real-time Linux

Standard Linux is a general purpose operating system, which is not designed for real-time applications. Despite this, standard Linux has excellent average performance and can even provide millisecond level task scheduling precision. However, it is not designed to provide real-time services that require sub-millisecond precision and reliable timing guarantees. The fundamental cause is because the Linux kernel is not preemptive. Traditionally, the Linux kernel allows task preemption only under certain circumstances:

. When the task is running in user-mode
. When the task returns from a system call or interrupt processing back to user mode
. When the task sleeps or blocks in kernel to explicitly yield control to another process

While a task executes in the Linux kernel, if an event occurs that makes a high priority task ready to run, the high priority task can not preempt the running task until the latter explicitly yields control. In the worst case, the latency of switching to the high priority task could be hundreds milliseconds or more. Thus, in a standard Linux kernel high priority tasks can be

delayed for an arbitrary amount of time, causing the system unable to respond to events quickly or guarantee task deadlines. There are many attempts to modify the Linux kernel to improve its real-time capabilities. In the following, we shall discuss two different approaches to this problem.

10.5.6.1 Real-time Linux Patches

The Linux 2.6 kernel has a configuration option, CONFIG_PREEMPT_VOLUNTARY, which can be enabled when compiling the kernel image. It introduces checks to the most common causes of long latencies in the kernel code, allowing the kernel to yield control to a higher priority task voluntarily. The advantage of this scheme is that it is very easy to implement and it has only limited impact on the system throughput. The disadvantage is that, although it reduces the occurrences of long latencies, it does not totally eliminate them. To further remedy the problem, the Linux 2.6 kernel provides an additional option, CONFIG_PREEMPT, which causes all kernel code outside of spinlock-protected regions and interrupt handlers to be eligible for preemption by higher priority tasks. With this option, worst case latency drops to around single digit milliseconds (Hagen 2005), although some device drivers may have interrupt handlers that still cause much longer latency. To support real-time tasks that require latencies below single-digit milliseconds, current Linux kernel has yet another option, CONFIG_PREEMPT_RT, known as the RT-Preempt patch, which converts the Linux kernel into a fully preemptible kernel by the following means.

. Make in-kernel locking-primitives (spinlocks) preemptible by a re-implementation with real-time mustexs (rt-mutexes). Rt_mutexes extend the semantics of simple mutexes with priority inheritance, in which a low priority owner of a rt-mutex inherits the highest priority of all the tasks waiting for the rt-mutex. In a chain of requests for rt-mutexes, if the owner of a rt-mutex gets blocked on another rt-mutex, it propagates the boosted priority to the owner of the other rt_mutex. The priority boosting is immediately removed once the rt_mutex has been unlocked. Implementation of rt-mutexes is made more efficient by using the kernel's p-list, which keeps track of the highest priority of blocked tasks. On architectures that support the cmp-xhg (compare & exchange) atomic operation, lock/unlock operations on rt-mutex can be further optimized.
. Convert interrupt handlers into preemptible kernel threads. The RT-Preempt patch treats executions of interrupt handlers as kernel pseudo-threads, which have higher priority than all regular threads but they can be preempted (by other pseudo-threads of higher priority), allowing the Linux kernel to support nested interrupts by interrupt priorities.
. Use a high resolution real-time timer. Convert the old Linux timer API into separate infrastructures for high resolution kernel timers plus a watch-dog like timer for timeouts, allowing high resolution POSIX timers in user space.

The performance of Linux 2.6 kernel with the RT-Preempt patch has been studied in (Hagen 2005). The test results showed a significant reduction of jitters in interrupts processing, resulting in a much more responsive and predictable Linux system.

10.5.6.2 RTLinux

RTLinux (Yodaiken Yodaiken 1999) is a RTOS that runs special real-time tasks and interrupt handlers on the same machine as standard Linux. It treats Linux as a preemptive task with the lowest priority, which runs only if there are no runnable real-time tasks, and it can be preempted whenever a real-time task becomes ready to run. The basic principle of RTLinux is rather simple. It places a layer of emulation software, the RTLinux kernel, between Linux and the interrupt controller hardware. All hardware interrupts are caught by the emulator first. It handles real-time related interrupts directly, and it forwards other non-real time related interrupts to the Linux kernel. In the Linux kernel code (on the Intel x86 architecture) all cli (disable interrupts), sti (enable interrupts) and iret (return from interrupts) instructions are replaced with emulating macros S_CLI, S_STI and S_IRET, respectively. The S_CLI macro clears a global variable SFIF to zero, indicating that the Linux kernel has just executed cli to have interrupts disabled. The S_STI macro simulates a real interrupt by creating a stack frame consisting of saved CPU FLAG register, Linux kernel DS register and a return address but executes the S_IRET macro instead. When a Linux interrupt occurs, the emulator checks the SFIF variable. If it is set, i.e. Linux kernel has interrupts enabled, it invokes the Linux interrupt handler immediately. Otherwise, it sets a bit in the SFIF variable to represent a pending Linux interrupt. When the Linux kernel enables interrupts by sti, the S_IRET macro scans the SFIF variable for pending Linux interrupts (non-zero bits). For each pending Linux interrupt, it invokes the corresponding Linux interrupt handler until all pending interrupts are processed.

RTLinux is structured as a small core component and a set of optional components. The core component permits installation of very low latency interrupt handlers that cannot be delayed or preempted by Linux itself and some low level

synchronization and interrupt control routines. Inside RTLinux, real-time tasks are installed as Linux modules which execute in the same address space as the Linux kernel. Communication between Linux process and real-time tasks is by shared memory or dedicated FIFO pipes. Some earlier tests (Yokaiken 1999) showed that the RTLinux core can support real-time tasks with latency in the order of tens of microseconds on older Intel x86 CPUs. However, it was also reported that very frequent real-time interrupts would prevent Linux from running altogether.

10.5.7 Critique of Existing RTOS

In this section, we shall evaluate the various RTOS and formulate a set of general guidelines for the design and implementation of RTOS.

10.5.7.1 RTOS Organizations

Based on the above case studies, we can see that most RTOS are based on the bottom-up approach, in which a RTOS is built with a basic kernel to support tasks, task scheduling, critical regions and task synchronization. Then add additional functionalities to the basic kernel, e.g. execution tracing, events logging, debugging, file system and networking, etc. to improve the capability of the system. Most proprietary RTOS aimed at commercial markets usually also provide an integrated development environment (IDE) to facilitate the development of user applications. The main drawback of the bottom-up approach is the lack of uniformity. Different RTOS may develop and prompt their own proprietary system interfaces, complicating the process of developing user applications. To remedy this problem, many RTOS try to comply with the POSIX 1003.1b real-time extensions. In spite of these efforts, the availability of standard system service functions still varies from system to system.

An alternative approach to RTOS design is the top-down approach, which is aimed at converting an existing operating system, such as Linux, to support real-time operations. The advantage of the approach is obvious. In addition to adding real-time capabilities, it would make all the functionalities of a complete operating system directly accessible. The main drawback of this approach is the large system size, which may be unsuited to small embedded or real-time systems.

10.5.7.2 Tasks in RTOS

In an operating system, processes refer to execution entities with distinct address spaces, and threads are execution units in the same address of a process. In order to provide each process with a unique address space, processes usually executes in two different modes; kernel mode and user mode. While in kernel mode, all processes share the same address space of the kernel. While in user mode, each process executes in a separate address space that is isolated and protected from other processes. This is usually achieved through virtual address mapping by the memory management hardware. In all Unix-like systems, processes are created by the fork-exec paradigm. Fork creates a child process with an identical (user mode) image as the parent. Exec allows a process to change execution image to a different file. In addition, processes also obey the parent-child relation. A parent process may wait for child process termination. When a process terminates, it notifies the parent, which collects the child exit status and eventually releases the child process for reuse. However, the process model is unsuited to simple embedded systems and real-time systems, except in microkernel based systems, such as QNX. In almost all RTOS, tasks are essentially threads since they all execute in the same address space of the system kernel.

10.5.7.3 Memory Management in RTOS

Memory management refers to three distinct aspects: virtual address spaces mapping, dynamic memory allocation during execution and run-time stack overflow checking. In the following, we shall discuss each of the memory management schemes used in most RTOS.

Real Address Space

Tasks in most RTOS execute in the same address space of the kernel. The single address space environment has many advantages. First, it eliminates the overhead associated with virtual address mapping by the memory management hardware. Second, it allows tasks to share memory directly for fast inter-task communication. The main disadvantage of the single address space scheme is the lack of memory protection. Any task may corrupt the shared memory, causing other tasks or the entire system to fail.

Virtual Address Spaces

Most RTOS do not allow tasks to have separate address spaces. So there are no virtual address mapping and memory protection. For security and reliability reasons, memory protection may be necessary. If so, a RTOS should use the simplest memory mapping scheme of the Memory Management Unit (MMU) hardware, for better efficiency. For example, on the Intel architecture, it should use segmentation rather than (2-level) paging. Similarly, on the ARM architecture, it should use one-level paging with large page size rather than two-level paging. Some of the page entries, e.g. shared kernel pages, can be placed in the TLB as lock-down entries to minimize the MMU overhead during task switch.

Dynamic Memory Allocation

Many RTOS allow tasks to allocate memory dynamically during execution. Some RTOS even support the standard malloc()/free() functions of C library for allocate/free memory dynamically, presumably from a heap area of the system. However, a closer examination of real-time system requirements should reveal that allowing dynamic memory allocation at run-time may not be a good idea. Unlike conventional tasks, a key requirement of real-time tasks is that their behavior must be deterministic and predictable. Dynamic memory allocation at run-time would introduce a variable amount of delay to task executions, making them anything but predictable. The only justifiable need for memory allocation is to provide tasks with shared memory for fast inter-task communication. In that case, the needed memory should be allocated either statically or as fixed size blocks rather than in chunks of variable size.

Stack Overflow Checking

All RTOS support task creation. When creating a new task, the user may specify a function to be executed by the task, a task stack size and a pointer to the initial parameters of the function. Most RTOS support stack overflow checking at run-time. This seems to be a good feature but it is really superficial, for the following reasons. First, stack usage of any program should be controlled carefully during program design. Once a program is written, the stack size needed during execution can be estimated by the length of function calls and the amount of local variable spaces in the functions. The actual stack size of a program can be observed through testing. The maximum stack size can be set to the observed size, plus a safety factor, in the final program code. After all, this is the standard practice used by all operating system kernel designers. If every program is developed this way, there should be no reason for any task to run out of stack space during execution. Second, without memory protection hardware, stack overflow checking must be done by software, e.g. check the stack pointer against a preset stack limit value upon entry to every function, but this would introduces additional run-time overhead and delay. Third, even if we include run-time stack overflow checking, it remains unclear what can be done if a task causes a stack overflow. To abort the task may be out of the question. To extend the stack space and allow the task to continue may introduce unacceptable delays. Most RTOS simply leave this question to the user. As usual, the best approach to dealing with stack overflow is to prevent it from occurring during program development through testing.

10.5.7.4 POSIX Compliant APIs

POSIX (2016) specifies a set of standards for Unix-like systems. POSIX.1 specifies the core service functions, such as signals, pipes, file and directory operations. POSIX.1b adds real-time extensions, and POSIX1.c adds threads support. The goal of POSIX standards is to provide a uniform user interface to facilitate the development of portable application programs on Unix-like system. Many real-time systems try to be POSIX compliant. However, strict compliance with the POSIX standards may actually hinder real-time operations. In standard Unix, the execution units are processes. In most real-time systems, tasks are essentially threads since they execute in the same address space of the kernel. It makes sense to provide threads synchronization mechanisms, such as mutex, barrier and condition variables, for task synchronization, but it would be very hard to justify why tasks must use open-close-read-write on file descriptors for I/O. If a real-time task needs I/O, it would be much more efficient to invoke the device drivers directly, rather than going through additional layers of mappings through file descriptors.

10.5.7.5 Synchronization Primitives

All RTOS provide a set of tools for task synchronization. In the following, we discuss the suitability of such tools in RTOS.

. Disable interrupts and task scheduler: Some ROTS allow user programs to disable interrupts when entering critical regions, but this contradicts the short interrupt latency requirement of real-time systems. Many RTOS allow user programs to disable

task scheduler to prevent task switch in long critical regions, but this contradicts preemptive task scheduling. In ROTS, user applications should be shielded from, or even not allowed to do, these low level operations.

. Mutex locks: Almost all RTOS use simple mutexes with priority inheritance to ensure exclusive access to critical regions. Some RTOS supports recursive mutexes, which can be locked/unlocked by the owner recursively. When writing concurrent programs using any kind of locking mechanism, a fundamental requirement is that the program must be free of deadlocks. This is usually achieved by ensuring that the locking order is always unidirectional. It is inconceivable that any task will ever need to acquire the same lock it already holds. Therefore, there is no real need for recursive mutexes.

. Binary and counting semaphores: Semaphores are convenient tools for general task synchronization and cooperation. Many RTOS support both binary and counting semaphores, which require different semantics and implementations. Since counting semaphores are more general than binary semaphores, there is no need for two kinds of semaphores. In some RTOS, when a task tries to acquire a semaphore, it may specify a timeout parameter. If the task gets blocked on the semaphore, it will be unblocked when the timeout value expires. The usefulness of this feature is highly questionable due to many problems it may cause. First, this would introduce additional delays to the timer interrupt handler, which must process the remaining time of all blocked tasks and unblock them when their time expires. Other questions include: what should be the timeout value? What should the task do in case of a timeout? A better solution is to use the conditional CP operation on semaphores. It allows the user to specify an alternative action immediately if a task can not acquire a semaphore, rather than waiting for a timeout.

. Event Control Blocks (ECBs): Unlike mutexes and semaphores, event flags allow tasks to wait for a variable number of events, which add flexibility to the system.

. Inter-task communication: Using shared memory, in conjunction with mutexes to provide protection, is both convenient and efficient for fast inter-task communication. Many RTOS provide static or dynamic message queues for tasks to exchange messages. These are less efficient than shared memory and pipes but they allow more flexibility in application programs. Some RTOS, e.g. NuttX, try to use signals, which would be a poor choice since regular signals are unsuited to interprocess communication (Wang 2015), and extended signals are less efficient than messages.

10.5.7.6 Interrupts Handling in RTOS

One of the fundamental requirements of RTOS is minimal interrupt latency, which implies that a RTOS must allow nested interrupts. All ROTS support nested interrupts, but they may handle nested interrupts in different ways. In most RTOS, interrupts processing are performed inside interrupt handlers directly. Task switching is disabled until all nested interrupt processing have completed. In Linux with real-time patches, interrupts are handled by pseudo-threads in kernel. This allows for faster interrupt response since each interrupt handler only needs to activate a pseudo-thread rather than actually processing the interrupt. In this scheme, nested interrupts processing is pushed to the pseudo-threads level. The choice of whether to handle nested interrupts directly or by pseudo-tasks should be based on the interrupt hardware. For Intel x86 CPUs, which do not have a separate interrupt mode stack, interrupts are handled in the context of the interrupted task, which can use the same stack to process nested interrupts. In this case, it would be better to handle nested interrupts directly inside ISRs. For ARM CPUs, which use a separate IRQ mode stack, the interrupted context must be transferred to a different privileged mode stack before allowing another interrupt. In this case, it would be better to handle nested interrupts by pseudo-tasks.

10.5.7.7 Task Deadlines

Although all RTOS are intended for (or claim to be) hard real-time systems, in reality most RTOS only provide a basic framework for developing real-time applications. As such, none of the published RTOS can actually guarantee task deadlines. The analytical results of RMS and EDF only provide general guidelines for RTOS under the simplest and idealistic conditions. They do not account for the overhead of interrupts processing, task blocking time due to resource sharing, task scheduling and switching time, etc. in a real ROTS system. When using a RTOS to develop real-time applications, it is entirely up to the user to determine whether tasks can meet their deadlines. This is no surprise, considering the wide range of possible real-time applications. To help remedy this problem, most RTOS provide run-time tracing facilities, which allow the user to monitor the time during which interrupts are disabled and/or the amount of time tasks spent inside critical regions, etc. Despite these efforts, performance evaluation of real-time systems essentially boils down to individual case analysis.

10.6 Design Principles of RTOS

Based on the above discussions, we propose a set of general guidelines for the design and implementation of the key components of RTOS.

10.6.1 Interrupts Processing

A RTOS must support nested interrupts. Depending on the interrupt hardware, interrupts processing can be performed in interrupt handlers directly or as pseudo-tasks with higher priorities than ordinary tasks.

10.6.2 Task Management

A RTOS should support task creation. Whenever possible, tasks should be static. Dynamic tasks should be regarded as nonessential. For simple RTOS, tasks should execute in the same address space of the kernel. For RTOS with high security and reliability requirements, tasks should run in user mode in separate virtual address spaces, but the system should use the simplest virtual address mapping scheme for better efficiency.

10.6.3 Task Scheduling

Task scheduling must be preemptive based on task priority. Although RMS is static and simpler, task priority should be based on EDF because it's more realistic in terms of meeting task deadlines.

10.6.4 Synchronization Tools

Use simple mutex with priority inheritance to protect critical regions. Use counting semaphore for task cooperation. For added flexibility, use event flags to allow tasks to wait for a variable number of events.

10.6.5 Task Communication

Use shared memory protected by simple mutxes for direct task communication. Use pipes for tasks to share streams of data. Use synchronous message passing for tasks to exchange messages.

10.6.6 Memory Management

Avoid virtual address mapping if possible. Allow tasks to allocate memory in fixed size blocks, but not in chunks of arbitrary sizes. Support stack overflow checking during development but not in the final system.

10.6.7 File System

If a file system is needed, implement the file system as a RAM disk in memory. Load the file system from a SDC to the RAM disk when the system starts and flushes any changes to the file system back to the SDC periodically.

10.6.8 Tracing and Debugging

A RTOS should provide tracing and debugging facilities to allow user to monitor the progress of tasks, at least during development.

In the following sections, we shall show the design and implementation of two RTOS, one for uniprocessor (UP) systems and the other one for multiprocessor (MP) systems.

10.7 Uniprocessor RTOS (UP_RTOS)

UP_RTOS is a real-time operating system for uniprocessor (UP) systems. It is based on the fully preemptive UP kernel developed in Chap. 5 (Sect. 5.14.3) but with the following extensions to support real-time applications.

(1). Preemptive task scheduling by (static) task priority
(2). Support nested interrupts
(3). Mutexes with priority inheritance for critical regions and resource sharing
(4). Semaphores with priority inheritance for process cooperation
(5). Shared memory, pipes and messages for inter-process communication
(6). A logging task, which records task activities to a SDC for tracing and debugging

The following describes the UP_RTOS kernel.

10.7.1 Task Management in UP_RTOS

UP_RTOS supports a variable number of tasks. The maximum number of tasks (NPROC) in the system is a configurable parameter, which can be set to a value suited to the needs. Tasks can be either static or dynamic. When using static tasks, all tasks are created during system initialization and they exist in the system permanently. When using dynamic tasks, tasks can be created on demand and they terminate after completing their work. Each task is created with a static priority. All tasks execute in the same address space of the kernel. Task creation is by the API

$$\textbf{kfork}(\textbf{int} * \textbf{f}() \, \textbf{task_function}, \, \textbf{int priority});$$

which creates a task to execute a task_function() with a specified priority. Each task is represented by a PROC structure.

```
#define NPROC 256
typedef struct proc{
  struct proc *next;     // pointer to next PROC
  int    *ksp;           // saved sp when not running
  int    status;         // status
  int    pid;            // task ID
  int    pause;          // pausue time
  int    ready_time;     // task release or ready time
  int    priority;       // effective priority
  int    rpriority;      // real priority
  MSG    *mqueue         // message queue
  SEMAPHORE nmsg;        // number of message in messge queue
  MUTEX   mqlock         // message queue lock
  SEMAPHORE wchild;      // wait for ZOMBIE child
  int    kstack[SSIZE]; // 4KB to 8KB task stack area
```

```
}PROC;
PROC proc[NPROC];       // NPROC PROC structures
PROC *readyQueue;       // ready queue by PROC priority
PROC *running;          // current running PROC pointer
```

10.7.2 Task Synchronization in UP_RTOS

UP_RTOS uses sleep/wakeup for task synchronization only in pipes. It uses mutex and counting semaphore for general task synchronization. Mutexes are used as exclusive locks to protect critical regions. Semaphores are used for task cooperation. Both are implemented with priority inheritance to prevent priority inversion. Each mutex already has a owner field, which identifies the current task that holds the mutex lock. In order to support priority inherence in semaphores, we modify the semaphore structure to contain an owner field also, which identifies the current task that holds the semaphore. For simplicity, priority inheritance is only one-level, i.e. it only applies to each mutex or semaphore but is not transitive in a sequence of mutex or semaphore requests. Extension of priority inheritance to nested mutex or semaphore requests is left as an exercise in the Problem section.

10.7.3 Task Scheduling in UP_RTOS

Task scheduling is by preemptive priority. Task preemption is implemented as follows. First, the kernel uses a global counter to keep track of interrupts nesting levels. When entering an interrupt handler, the counter is incremented by 1. When exiting an interrupt handler, the counter is decremented by 1, etc. Task switch is not allowed if the interrupt nesting level is nonzero. Second, the only operations that can make a task ready to run are task creation, mutex_unlock and V on semaphores. Whenever a ready task is entered into the readyQueue, it invokes a reschedule() function to reschedule tasks. The following shows the algorithm of task preemption.

```
/********* Algorithm of task preemption ********/
int intnest; // interrupt nesting counter, initially 0
int swflag;  // switch task flag
reschedule
{
  if (readyQueue->priority > running->priority){
     if (intsest==0)// if not in interrupt handler
          tswitch();  // preempt running task immediately
       else{
          swflag = 1; // defer preemption until end of interrupts
}
interrupt_handler_exit // switch task if swflag set & end of IRQs
{
  if (!intnest && swflag)
     tswtich() in SVC mode;
  }
}
```

In reschedule(), if the current running task is no longer the highest priority and execution is not inside any interrupt handler, it preempts the current running task immediately. If execution is still inside an interrupt handler, it sets a switch task flag, which defers task switch until all nested interrupts processing have ended.

10.7.4 Task Communication in UP_RTOS

The UP_RTOS kernel provides three kinds of mechanisms for inter-task communication.

10.7.4.1 Shared Memory

When the system starts, it initializes a fixed number (32) of 64KB memory regions, e.g. from 32MB to 34MB, for inter-task communication through shared memory. Each shared memory region is represented by a structure

```
#define NPID NPROC/sizeof(int)
struct shmem{
   int procID[NPID];  // bit vector for NPROC task IDs
   MUTEX mutex;       // mutex lock for shared region
   char *address;     // start address of memory region
}shmem[32];
```

When the system starts, it initializes the shmem structures to contain

```
procID = {0};              // no task using the shmem yet
mutex = UNLOCKED;          // for exclusive access to the shared region
address = pointer to a unique 64KB memory area
```

To use a shared memory, a task must first attach itself to a shmem structure by

```
int shmem_attach(struct shmem *mp);
```

shmem_attach() records the task ID bit in procID (for accessibility checking) and returns the number of tasks currently attached to the shared memory. After attaching to a shared memory, tasks may use

```
shmem_read( struct shmem *mp, char buf[ ], int nbytes);
shmem_write(struct shmem *mp, char buf[ ], int nbytes);
```

to read/write data from/to the shared memory. The read/write functions only guarantee each read/write operation is atomic (by the mutex lock of the shared memory). The data format and meaning of the shared memory contents are entirely up to the user to decide.

10.7.4.2 Pipes for Data Streams

This is the same pipe mechanism of Chap. 5 (Sect. 5.13.2). It allows tasks to use pipes to read/write streams of data.

10.7.4.3 Message Queues

This is the same synchronous message passing mechanism of Chap. 5 (Sect. 5.13.4). It allows tasks to communicate by exchange messages. As in the case of shared memory, the user may design the message format and contents to suit the needs.

10.7.5 Protection of Critical Regions

In UP_RTOS, all critical regions are protected by mutex locks. The mutex locking order is always unidirectional, so deadlocks can never occur in the UP_RTOS kernel.

10.7.6 File System and Logging

Logging typically requires writing information to a log file in a file system, which may incur a variable amount of delay time to tasks. We do not see any justifiable needs for tasks in a real-time system to perform file operations. For simplicity and efficiency, we implement logging by a special logging task, which receives logging requests from other tasks by messages and writes the logging information to a storage device, such as a SDC, directly. When the system starts, it creates a logging task with the second lowest priority 1 (higher than the idle task at priority 0). Other tasks uses

$$\textbf{log}(\textbf{char} * \textbf{log_information})$$

to record a line in the log. The logging operation sends a message to the logging task, which formats the log information in the form

timestamp : taskID : line

and writes it to a SDC (1KB) block whenever it runs. It also writes the logged lines to a UART port to let the user see the logging activities on-line. When the system terminates, the log information can be retrieved from the SDC for tracing and analysis.
We show the implementation of the UP_RTOS kernel and demonstrate its capabilities by the following example programs.

C10.1: UP_RTOS with static periodic tasks and round-robin scheduling
C10.2: UP_RTOS with static periodic tasks and preemptive scheduling
C10.3: UP_RTOS with dynamic tasks and preemptive scheduling

10.7.7 UP_RTOS with Static Periodic Tasks and Round-Robin Scheduling

The first UP_RTOS example, denoted by C10.1, demonstrates static periodic tasks with round-robin scheduling. We assume that all tasks are periodic with the same period, hence the same priority in accordance with the RMS scheduling algorithm. As a uniprocessor (UP) system, the UP_RTOS kernel maintains a single readyQueue for task scheduling. In the readyQueue, tasks are ordered by priority. Tasks with the same priority are ordered First-In-first-Out (FIFO). Since all the tasks have the same priority, they are scheduled to run by round-robin. When the system starts, it creates 4 static tasks, all execute the same taskCode() function with the same period. Each task executes an infinite loop, in which a task first blocks itself on a unique semaphore (initialized to 0). A blocked task will be V-ed up by a timer periodically by the task period. When a task is unblocked and becomes ready to run, we get its ready time from a global time and record it in the task PROC structure. When a task runs, it first gets the start time. Then it does some computation, which is simulated by a delay loop. At the end of the execution loop, each task gets the end time and computes its execution and deadline times as follows.

$$\text{Execution time Ci} = \text{end_time} - \text{start_time};$$
$$\text{Deadline time Di} = \text{end_time} - \text{ready_time};$$

It compares the deadline time with task period to see whether it has met the deadline (equal to task period). Then it repeats the loop again. The following lists the code of the C10.1 program. In order to keep the program simple, the system only supports nested interrupts, preemptive task scheduling, but without priority inheritance which will be implemented later.
(1) . ts.s file of C10.1: The main feature of the ts.s file is that it supports nested IRQ interrupts. Task switch is deferred until nested interrupts processing have ended. Details of task preemption are explained later.

```
/************* ts.s file of C10.1 ***************/
      .text
      .code 32
.global vectors_start, vectors_end
.global proc, procsize
.global tswitch, scheduler, running
.global int_off, int_on, lock, unlock
.global swflag, intnest, int_end
.set vectorAddr, 0x10140030 // VIC vector base address
reset_handler:
// set SVC stack to high end of proc[0].kstack
  ldr r0, =proc
  ldr r1, =procsize
  ldr r2, [r1, #0]
```

```
  add r0, r0, r2
  mov sp, r0
// copy vector table to address 0
  bl copy_vectors
// go in IRQ mode, set IRQ stack
  msr cpsr, #0x92
  ldr sp, =irq_stack_top
// call main() in SVC mode with IRQ on: all tasks run in SVC mode
  msr cpsr, #0x13
  bl main
  b .
irq_handler:             // support nested interrupts in SVC mode
  sub      lr, lr, #4
  stmfd sp!, {r0-r12, lr}
  mrs r0, spsr
  stmfd sp!, {r0}        // push SPSR
  ldr r0, =intnest       // r0->intnest
  ldr r1, [r0]
  add r1, #1
  str r1, [r0]           // intnest++
  mov r1, sp             // get irq sp into r1
  ldr sp, =irq_stack_top // reset IRQ stack poiner to IRQ stack top
// switch to SVC mode    // to allow nested IRQs: clear IRQ source
  MSR cpsr, #0x93        // to SVC mode with interrupts OFF
  sub sp, #60            // dec SVC mode sp by 15 entries
  mov r0, sp             // r0=SVC stack top
// copy IRQ stack to SVC stack
  mov r3, #15        // 15 times
copy_stack:
  ldr r2, [r1], #4   // get an entry from IRQ stack
  str r2, [r0], #4   // write to proc's kstack
  sub r3, #1         // copy 15 entries from IRQ stack to PROC's kstack
  cmp r3, #0
  bne copy_stack
// read vectoraddress register: MUST!!! else no interrupts
  ldr  r1, =vectorAddr
  ldr  r0, [r1]  // read vectorAddr register to ACK interrupt
  stmfd sp!, {r0-r3, lr}
  msr cpsr, #0x13     // still in SVC mode but enable IRQ
  bl irq_chandler     // handle interrupt in SVC mode, IRQ off
  msr cpsr, #0x93
  ldmfd sp!, {r0-r3, lr}
  ldr r0, =intnest    // check interrupt nest level
  ldr r1, [r0]
  sub r1, #1
  str r1, [r0]        // intnest--
  cmp r1, #0          // if intnest != 0 => no_switch
  bne no_switch
// intnest==0: END OF IRQs: if swflag=1: switch task
  ldr r0, =swflag
  ldr r0, [r0]
  cmp r0, #0
  bne do_switch       // if swflag=0: no task switch
no_switch:
  ldmfd sp!, {r0}
```

```
  msr   spsr, r0      // restore SPSR
// irq_chandler() already issued EOI
  ldmfd sp!, {r0-r12, pc}^  // return via SVC stack
do_switch:                  // still in IRQ mode
  bl endIRQ                 // show "at end of IRQs"
  bl tswitch                // call tswitch(): resume to here
// will switch task, so must issue EOI
  ldr  r1, =vectorAddr
  str  r0, [r1]       // issue EOI
  ldmfd sp!, {r0}
  msr   spsr, r0
  ldmfd sp!, {r0-r12, pc}^  // return via SVC stack

tswitch:              // for task switch in SVC mode
// disable IRQ interrupts
  mrs r0, cpsr
  orr r0, r0, #0x80  // set I bit to MASK out IRQ interrupts
  msr cpsr, r0
  stmfd     sp!, {r0-r12, lr}
  ldr r0, =running  // r0=&running
  ldr r1, [r0, #0]  // r1->runningPROC
  str sp, [r1, #4]  // running->ksp = sp
  bl  scheduler
  ldr r0, =running
  ldr r1, [r0, #0]  // r1->runningPROC
  ldr sp, [r1, #4]
  // enable IRQ interrupts
  mrs r0, cpsr
  bic r0, r0, #0x80   // clear bit means UNMASK IRQ interrupt
  msr cpsr, r0
  ldmfd     sp!, {r0-r12, pc}

// utility functions: int_on/int_off/lock/unlock: NOT shown
vectors_start:
  LDR PC, reset_handler_addr
  LDR PC, undef_handler_addr
  LDR PC, swi_handler_addr
  LDR PC, prefetch_abort_handler_addr
  LDR PC, data_abort_handler_addr
  B .
  LDR PC, irq_handler_addr
  LDR PC, fiq_handler_addr
reset_handler_addr:          .word reset_handler
undef_handler_addr:          .word undef_handler
swi_handler_addr:            .word swi_handler
prefetch_abort_handler_addr: .word prefetch_abort_handler
data_abort_handler_addr:     .word data_abort_handler
irq_handler_addr:            .word irq_handler
fiq_handler_addr:            .word fiq_handler
vectors_end:
```

Explanations of the ts.s file:

Reset_handler: As usual, reset_handler is the entry point. First, it sets the SVC mode stack pointer to the high end of proc[0] and copies the vector table to address 0. Next, it changes to IRQ mode to set the IRQ mode stack. Then it calls main () in SVC mode. During system operation, all tasks run in SVC mode in the same address of the kernel.

Irq_handler: Task switch is usually triggered by interrupts., which may make a blocked task ready to run. Thus, irq_handler is the most important piece of assembly code relevant to process preemption. Therefore, we only focus on the irq_handler code. As pointer out in Chap. 2, the ARM CPU can not handle nested interrupts in IRQ mode. Nested interrupts processing must be performed in a different privileged mode. In order to support process preemption due to interrupts, we choose to handle IRQ interrupts in SVC mode. The algorithm of irq_handler can be best described by the following algorithm.

/******* Algorithm of IRQ handler for full task preemption ********/**

(1). Upon entry, adjust return lr; save context, including spsr, in IRQ stack
(2). Increment interrupt nesting counter by 1
(3). Switch to SVC mode with interrupts disabled
(4). Transfer context from IRQ stack to proc's SVC stack; reset IRQ stack
(5). Ack and clear interrupt source (prevent infinite loops from the same interrupt source)
(6). Enable IRQ interrupts; save working registers in SVC stack
(7). Call ISR in SVC mode with IRQ interrupts enabled
(8). (return from ISR): disable IRQ interrupts; restore working registers
(9). Decrement interrupt nesting counter by 1
(10). If still inside interrupt handler (counter nonzero): goto no_switch
(11). (end of IRQs): if swflag is set: goto do_switch
(12). no_switch: return via saved context in SVC stack
(13). do_switch: write EOI to interrupt controller; call tswitch() to switch task
(14). (when switched-out task resumes): return via saved interrupt context in SVC stack

(2). uart.c file: UARTs driver: for outputs only by TX interrupts

(3). vid.c file: LCD driver: SAME as before except frame buffer is at 4MB

(4). timer.c file: Use timer0 to activate tasks periodically

```
/** timer.c file of C10.1 **/
#define TLOAD   0x0
#define TVALUE  0x1
#define TCNTL   0x2
#define TINTCLR 0x3
#define TRIS    0x4
#define TMIS    0x5
#define TBGLOAD 0x6

typedef struct timer{
  u32 *base;            // timer's base address
  int tick, hh, mm, ss; // per timer data area
  char clock[16];
}TIMER;
TIMER timer[4];         // 4 timers, use only timer0

void timer_init()
{
  int i;
  TIMER *tp;
  printf("timer_init(): ");
  gtime = 0;
  for (i=0; i<4; i++){ // 4 timers but use only timer0
    tp = &timer[i];
    if (i==0) tp->base = (u32 *)0x101E2000;
    if (i==1) tp->base = (u32 *)0x101E2020;
    if (i==2) tp->base = (u32 *)0x101E3000;
    if (i==3) tp->base = (u32 *)0x101E3020;
    *(tp->base+TLOAD) = 0x0;   // reset
```

```
    *(tp->base+TVALUE)= 0xFFFFFFFF;
    *(tp->base+TRIS)  = 0x0;
    *(tp->base+TMIS)  = 0x0;
    *(tp->base+TLOAD) = 0x100;
    //0x62=|011-0000=|NOTEn|Pe|IntE|-|scal=00|32-bit|0=wrap|
    *(tp->base+TCNTL) = 0x62;
    *(tp->base+TBGLOAD) = 0xF00; // timer count
    tp->tick = tp->hh = tp->mm = tp->ss = 0;
    strcpy((char *)tp->clock, "00:00:00");
  }
}
void timer_handler(int n)
{
    int i;
    TIMER *t = &timer[n];
    gtime++;                 // increment global time
    t->tick++;               // for local wall clock
    if (t->tick >= 64){
      t->tick=0; t->ss++;
      if (t->ss == 60){
         t->ss=0; t->mm++;
       if (t->mm==60){
          t->mm=0; t->hh++;
       }
      }
    }
// display a wall clock
    if (t->tick == 0){ // display wall clock
         for (i=0; i<8; i++){
            unkpchar(t->clock[i], 0, 70+i);
         }
         t->clock[7]='0'+(t->ss%10); t->clock[6]='0'+(t->ss/10);
         t->clock[4]='0'+(t->mm%10); t->clock[3]='0'+(t->mm/10);
         t->clock[1]='0'+(t->hh%10); t->clock[0]='0'+(t->hh/10);
         for (i=0; i<8; i++){
             kpchar(t->clock[i], 0, 70+i);
         }
    }
// unblock tasks by period
    if ((gtime % period)==0){          // period = N*T timer ticks
       for (i=1; i<=4; i++){
          V(&ss[i]);                   // activate task i
          proc[i].start_time = gtime;  // task ready to run time
      }
     }
    timer_clearInterrupt(n);
}
void timer_start(int n) // timer_start(0), 1, etc.
{
  TIMER *tp = &timer[n];
  printf("timer_start %d\n", n);
  *(tp->base+TCNTL) |= 0x80;    // set enable bit 7
}
```

```
int timer_clearInterrupt(int n) // timer_start(0), 1, etc.
{
  TIMER *tp = &timer[n];
  *(tp->base+TINTCLR) = 0xFFFFFFFF;
}
```

(5). Kernel files of C10.1:

```
/*********** pv.c file of C10.1 ***********/
extern PROC *running;
extern PROC *readyQueue;
extern int swflag;
extern int intnest;
int P(struct semaphore *s) // no priority inheritance
{
  int SR = int_off();
  s->value--;
  if (s->value < 0){
     running->status = BLOCK;
     enqueue(&s->queue, running);
     int_on(SR);
     tswitch();
  }
  int_on(SR);
}
int V(struct semaphore *s) // no priority inherence
{
  PROC *p; int cpsr;
  int SR = int_off();
  s->value++;
  if (s->value <= 0){
     p = dequeue(&s->queue);
     p->status = READY;
     enqueue(&readyQueue, p);
     printf("V up task%d pri=%d; running pri=%d\n",
             p->pid, p->priority, running->priority);
     reschedule();   // may preempty running task
  }
  int_on(SR);
}

/*************** kernel.c file of C10.1 **************/
#define NPROC 256
PROC proc[NPROC], *running, *freeList, *readyQueue;
int procsize = sizeof(PROC);
int swflag = 0;  // switch task flag
int intnest;     // interrupt nesting level

int kernel_init()
{
  int i, j;
  PROC *p;
  kprintf("kernel_init()\n");
  for (i=0; i<NPROC; i++){
    p = &proc[i];
    p->pid = i;
```

```
    p->status = READY;
    p->run_time = 0;
    p->next = p + 1;
  }
  proc[NPROC-1].next = 0;
  freeList = &proc[0];
  sleepList = 0;
  readyQueue = 0;
  intnest = 0;
  running = getproc(&freeList); // create and run P0
  running->priority = 0;
  printf("running = %d\n", running->pid);
}

int scheduler()
{
  printf("task%d switch task: ", running->pid);
  if (running->status==READY)
     enqueue(&readyQueue, running);
  printQ(readyQueue);
  running = dequeue(&readyQueue);
  printf("next running = task%d pri=%d realpri=%d\n",
        running->pid, running->priority, running->realPriority);
  color = RED+running->pid;
  swflag = 0;
}

int reschedule() // called from inside V()with IRQ disabled
{
  if (readyQueue && readyQueue->priority > running->priority){
     if (intnest==0){
        printf("task%d PREEMPT task%d IMMEDIATELY\n", readyQueue->pid,
                running->pid);
        tswitch();
     }
     else{
        printf("task%d DEFER PREEMPT task%d ", readyQueue->pid,
                running->pid);
        swflag = 1;  // IRQ are disabled, so no need to lock/unlock
     }
  }
}

// kfork() create a new task and enter into readyQueue
PROC *kfork(int func, int priority)
{
  int i;
  PROC *p = getproc(&freeList);
  if (p==0){
     kprintf("kfork failed\n");
     return (PROC *)0;
  }
  p->ppid = running->pid;
  p->parent = running;
  p->status = READY;
  p->realPriority = p->priority = priority;
```

```
  p->run_time = 0;
  p->ready_time = 0;
  // set kstack to resume to execute func()
  for (i=1; i<15; i++)
      p->kstack[SSIZE-i] = 0;
  p->kstack[SSIZE-1] = (int)func;
  p->ksp = &(p->kstack[SSIZE-14]);
  enqueue(&readyQueue, p);
  printf("task%d create a child task%d\n", running->pid, p->pid);
  reschedule();
  return p;
}
/****************** t.c file of C10.1 ***************/
// set T=5 timer ticks, task period = 8*T
#define T 5
#define period 8*T

#include "type.h"
#include "string.c"
#include "queue.c"
#include "pv.c"
#include "uart.c"
#include "vid.c"
#include "exceptions.c"
#include "kernel.c"
#include "timer.c"
// globals
struct semaphore ss[5];   // semaphores for task to block
volatile u32 gtime;       // global time
UART *up0, *up1;
int tcount = 0;           // number of timer interrupts in low IRQ
void copy_vectors(void) { // same as before }

int enterint() // for nested IRQs: clear interrupt source
{
  int status, ustatus, scode;
  status = *((int *)(VIC_BASE_ADDR)); // read status register
  if (status & (1<<4)){ // timer0 at IRQ 4
     tcount++;
  }
  if (status & (1<<12)){ // uart0 at IRQ 12
     ustatus = *(up0->base + UDS);  // read UDS register
  }
  if (status & (1<<13)){ // uart1 at IRQ 13
    ustatus = *(up1->base + UDS);  // read UDS register
  }
}
int endIRQ() { printf("until END of IRQ\n"); }
int int_end(){ printf("task switch at end of IRQ\n"); }

// use vectored interrupts of PL190
void timer0_handler()
{
   timer_handler(0);
}
```

```
void uart0_handler()
{
  uart_handler(&uart[0]);
}
void uart1_handler()
{
  uart_handler(&uart[1]);
}
int vectorInt_init()
{
  printf("vectorInterrupt_init()\n");
  *((int *)(VIC_BASE_ADDR+0x100)) = (int)timer0_handler;
  *((int *)(VIC_BASE_ADDR+0x104)) = (int)uart0_handler;
  *((int *)(VIC_BASE_ADDR+0x108)) = (int)uart1_handler;
  //(2). write to intControlRegs = E=1|IRQ# =  1xxxxx
  *((int *)(VIC_BASE_ADDR+0x200)) = 0x24;  //0100100 at IRQ 4
  *((int *)(VIC_BASE_ADDR+0x204)) = 0x2C;  //0101100 at IRQ 12
  *((int *)(VIC_BASE_ADDR+0x208)) = 0x2D;  //0101101 at IRQ 13
  // write 32-bit 0's to IntSelectReg to generate IRQ interrupts
  *((int *)(VIC_BASE_ADDR+0x0C)) = 0;
}
void irq_chandler()
{
  int (*f)();                          // f is a function pointer
  f =(void *)*((int *)(VIC_BASE_ADDR+0x30)); // get ISR address
  f();                                 // call the ISR function
  *((int *)(VIC_BASE_ADDR+0x30)) = 1; // write to vectorAddr as EOI
}
int delay(int pid)          // delay loop: simulate task compute time
{
  int i, j;
  for (i=0; i<1000; i++){
    // may use pid for different task delay time
    for (j=0; j<1000; j++); // change this line for different dealys
  }
}

/*********** Static Periodic Tasks ***********/
int taskCode()
{
  int pid = running->pid;
  u32 time1, time2, t, ready_time, complete_time;
  while(1){
    lock(); // reset task semaphore and ready_time
     ss[running->pid].value = 0;
     ss[running->pid].queue = 0;
     running->start_time = 0;
    unlock();

    // block on per-proc semaphore until Ved up by timer
    P(&ss[running->pid]);

    ready_time = running->start_time;
     printf("%d ready=%d", pid, ready_time); // to LCD
    uprintf("%d ready=%d", pid, ready_time); // to UART0
    time1 = gtime;
```

```
     printf("start=%d", time1);
    uprintf("start=%d", time1);
       delay(pid);
    time2 = gtime;

    t = time2 - time1;
    complete_time = time2 – ready_time;

     printf("end=%d%d[%d %d]", time2, t, ready_time, complete_time);
    uprintf("end=%d%d[%d %d]", time2, t, ready_time, complete_time);

    if (complete_time > period){ // if task missed deadline
        printf(" PANIC:miss deadline!\n");
       uprintf(" PANIC:miss deadline!\n");
    }
    else{
       printf(" OK\n");
      uprintf(" OK\n");
    }
}
int main()
{
   int i;
   PROC *p;
   color = WHITE;
   fbuf_init();
   uart_init();
   up0 = &uart[0];
   up1 = &uart[1];
   kprintf("Welcome to UP_RTOS in ARM\n");
   /* enable timer0,1, uart0,1 interrupts */
   VIC_INTENABLE = 0;
   VIC_INTENABLE |= (1<<4);    // timer0,1 at bit4
   VIC_INTENABLE |= (1<<12);   // UART0 at bit12
   VIC_INTENABLE |= (1<<13);   // UART2 at bit13
   vectorInt_init();
   timer_init();
   timer_start(0);
   kernel_init();
   printf("P0 kfork tasks\n"); // create 4 tasks
   for (i=1; i<=4; i++){
      ss[i].value = 0;         // initialize task semaphhores
      ss[i].queue = (PROC *)0;
      kfork((int)taskCode, 1); // all SAME priority = 1
   }
   unlock();
   while(1){                   // idle task P0 loop
     if (readyQueue)
        tswitch();
   }
}
```

Deadline Analysis of the C10.1 system: With 4 tasks of equal period, according to the schedulability condition of RMS, all the tasks should be able to meet their deadlines if

$$(\mathbf{C1}+\mathbf{C2}+\mathbf{C3}+\mathbf{C4})/\mathbf{period} < \mathbf{4}*(\mathbf{2}**(\mathbf{1}/\mathbf{4})-\mathbf{1}) = \mathbf{0.7568} \tag{10.1}$$

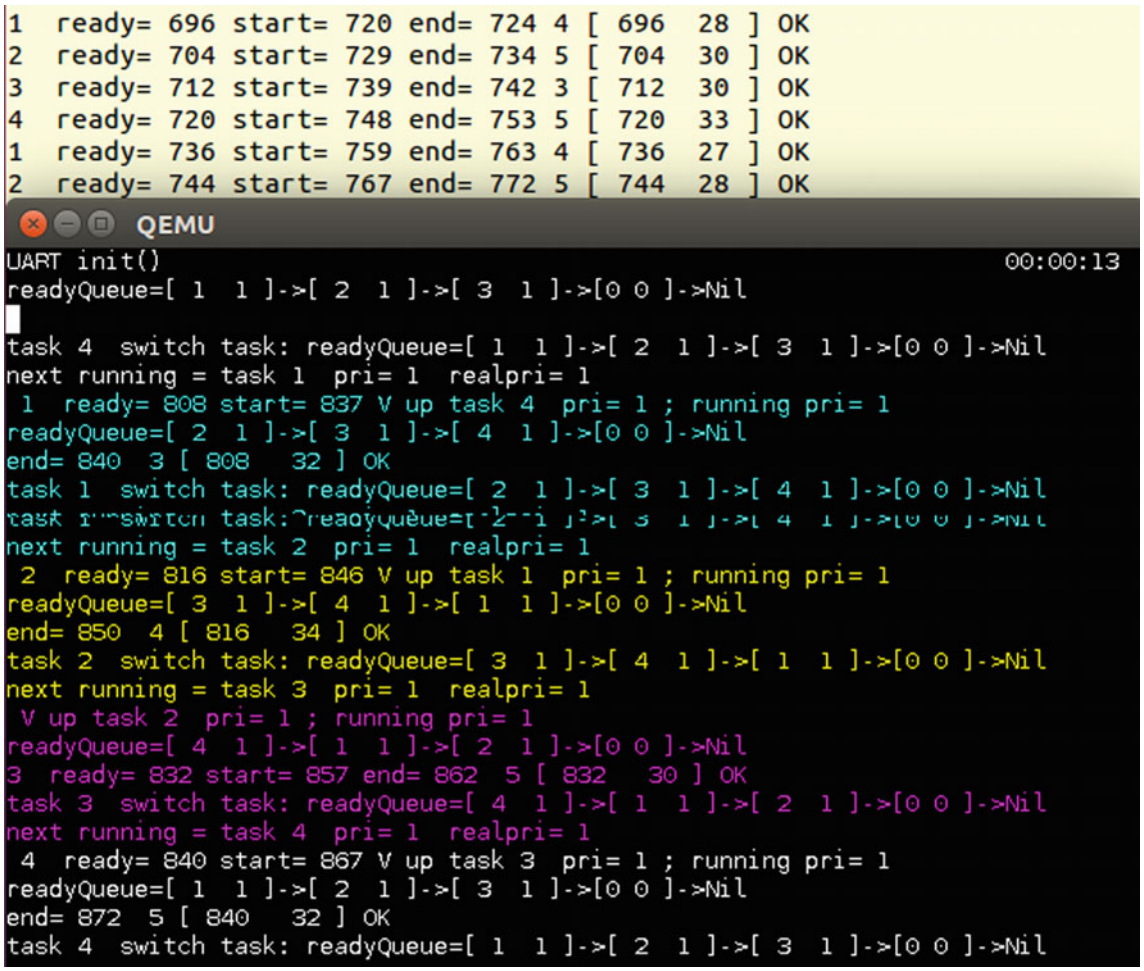


Fig. 10.1 UP_RTOS with round-robin scheduling

If we assume all task compute times Ci=C are the same, then the condition (10.1) becomes

$$\mathbf{C/period < 0.189} \tag{10.2}$$

Figure 10.1 shows the sample outputs of running the example program C10.1. The figure shows that the individual task compute time varies from 3 to 5 timer ticks. The task compute time includes the overhead due to interrupts processing by both timer and I/O interrupts, as well as task scheduling and switching time. In the test program C10.1, we set all the task compute time to C = 5 timer ticks and task period = 8*C, so that C/period = 1/8=0.125, which is within the schedulability bound of RMS. In this case, all the tasks indeed can meet their deadlines, as Figure 10.1 shows.

The condition (10.2) suggests that we should expect all the tasks to meet their deadlines if C/period < 0.189. However, the test results showed otherwise. For instance, if we set the task period to 7*C (C/period = 0.1428), which is still within the RMS bound, some of the tasks will start to miss their deadlines, as shown in Figure 10.2.

In fact, the test results showed that, with period = 6*C (C/period = 0.167), almost all the tasks will miss their deadlines. The example shows that, while it is easy to design and implement a ROTS kernel to support periodic tasks, there is no guarantee that the task can meet their deadlines even if the system follows the principles of RMS strictly.

10.7.8 UP_RTOS with Static Periodic Tasks and Preemptive Scheduling

The second example of UP_RTOS, denoted by C10.2, demonstrates static periodic tasks with preemptive task scheduling. When the system starts, it creates 4 static tasks with different periods. As in RMS, each task is assigned a static priority inversely proportional to its period. Specifically, the task periods and priorities are assigned as follows, where the task periods are in units of T timer ticks.

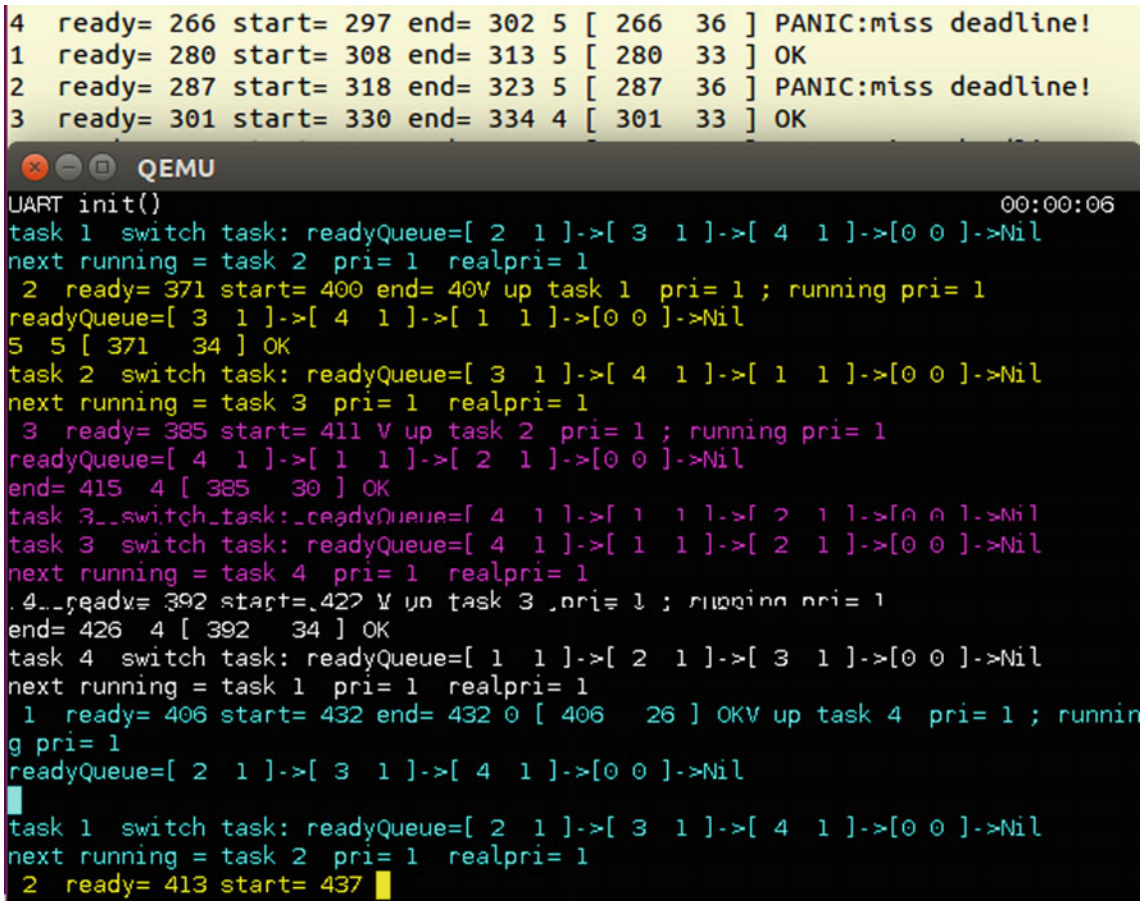


Fig. 10.2 Demonstration of tasks missing deadlines

Task	Period	Priority
P1	5T	1
P2	4T	2
P3	3T	3
P4	2T	4

Each task executes an infinite loop, in which it first blocks itself on a unique semaphore initialized to 0. Each blocked task will be V-ed up by the timer periodically according to the task period. As in the program C10.1, the ready time of a task is the time when the task becomes ready to run, i.e. when it is activated by the timer or when it is entered into the readyQueue. When a task runs, it gets the start time form a global time. Then it does some computation, which is simulated by a delay loop. At the end of the execution loop, each task gets the end time and tests whether it has met the deadline (equal to task period). Then it repeats the loop again. The following shows the code of the C10.2 program. For the sake of brevity, we only show the modified code of C10.2.

(1). ts.s file of C10.2. Same as in C10.1.
(2). uart.c file: UARTs driver: for outputs only by TX interrupts
(3). vid.c file: LCD driver: SAME as before except frame buffer at 4MB
(4). timer.c file: In the timer interrupt handler, tasks are V-ed up by task periods. It also sets the ready_time of each task when the task is entered into the readyQueue.

```
/** timer.c file of C10.2 **/
#define TLOAD   0x0
```

```
#define TVALUE  0x1
#define TCNTL   0x2
#define TINTCLR 0x3
#define TRIS    0x4
#define TMIS    0x5
#define TBGLOAD 0x6
typedef struct timer{
  u32 *base;            // timer's base address
  int tick, hh, mm, ss; // per timer data area
  char clock[16];
}TIMER;
TIMER timer[4];         // 4 timers, use only timer0
void timer_init()
{
  int i;
  TIMER *tp;
  printf("timer_init(): ");
  gtime = 0;
  for (i=0; i<4; i++){ // 4 timers but use only timer0
    tp = &timer[i];
    if (i==0) tp->base = (u32 *)0x101E2000;
    if (i==1) tp->base = (u32 *)0x101E2020;
    if (i==2) tp->base = (u32 *)0x101E3000;
    if (i==3) tp->base = (u32 *)0x101E3020;
    *(tp->base+TLOAD) = 0x0;   // reset
    *(tp->base+TVALUE)= 0xFFFFFFFF;
    *(tp->base+TRIS)  = 0x0;
    *(tp->base+TMIS)  = 0x0;
    *(tp->base+TLOAD) = 0x100;
    //0x62=|011-0000=|NOTEn|Pe|IntE|-|sca1=00|32-bit|0=wrap|
    *(tp->base+TCNTL) = 0x62;
    *(tp->base+TBGLOAD) = 0xF00; // timer count
    tp->tick = tp->hh = tp->mm = tp->ss = 0;
    strcpy((char *)tp->clock, "00:00:00");
  }
}
void timer_handler(int n)
{
    int i;
    TIMER *t = &timer[n];
    gtime++;                // increment global time
    t->tick++;              // for local wall clock
    if (t->tick >= 64){
      t->tick=0; t->ss++;
      if (t->ss == 60){
         t->ss=0; t->mm++;
       if (t->mm==60){
          t->mm=0; t->hh++;
       }
      }
    }
    if (t->tick == 0){ // display wall clock
         for (i=0; i<8; i++){
            unkpchar(t->clock[i], 0, 70+i);
         }
```

```
        t->clock[7]='0'+(t->ss%10); t->clock[6]='0'+(t->ss/10);
        t->clock[4]='0'+(t->mm%10); t->clock[3]='0'+(t->mm/10);
        t->clock[1]='0'+(t->hh%10); t->clock[0]='0'+(t->hh/10);
        for (i=0; i<8; i++){
            kpchar(t->clock[i], 0, 70+i);
        }
    }
    if ((gtime % (2*T))==0){     // activate P4 every 2T
       V(&ss[4]);
       proc[4].ready_time = gtime;
    }
    if ((gtime % (3*T))==0){     // activate P3 every 3T
       V(&ss[3]);
       proc[3].ready_time = gtime;
    }
    if ((gtime % (4*T))==0){     // activate P2 every 4T
       V(&ss[2]);
       proc[2].ready_time = 0;
    }
    if ((gtime % (5*T))==0){     // activate P1 every 5T
       V(&ss[1]);
       proc[1].ready_time = gtime;
    }
    timer_clearInterrupt(n);
}
void timer_start(int n) // timer_start(0), 1, etc.
{
  TIMER *tp = &timer[n];
  printf("timer_start %d\n", n);
  *(tp->base+TCNTL) |= 0x80;    // set enable bit 7
}
int timer_clearInterrupt(int n) // timer_start(0), 1, etc.
{
  TIMER *tp = &timer[n];
  *(tp->base+TINTCLR) = 0xFFFFFFFF;
}
```

(5). Kernel files of C10.2: Same as in C10.1

(6). t.c file of C10.2

```
/*************** t.c file of C10.2 ***************/
#define T 32
#include "type.h"
#include "string.c"
#include "queue.c"
#include "pv.c"
#include "uart.c"
#include "vid.c"
#include "exceptions.c"
#include "kernel.c"
#include "timer.c"
// globals
```

```
struct semaphore ss[5];   // semaphores for task to block
volatile u32 gtime;       // global time
UART *up0, *up1;
int tcount = 0;           // number of timer interrupts in low IRQ
void copy_vectors(void) { // same as before }

int enterint() // for nested IRQs: clear interrupt source
{
  int status, ustatus, scode;
  status = *((int *)(VIC_BASE_ADDR)); // read status register
  if (status & (1<<4)){ // timer0 at IRQ 4
     tcount++;
  }
  if (status & (1<<12)){ // uart0 at IRQ 12
     ustatus = *(up0->base + UDS);  // read UDS register
  }
  if (status & (1<<13)){ // uart1 at IRQ 13
    ustatus = *(up1->base + UDS);  // read UDS register
  }
}
int endIRQ() { printf("until END of IRQ\n"); }
int int_end(){ printf("task switch at end of IRQ\n"); }

// use vectored interrupts of PL190
void timer0_handler()
{
   timer_handler(0);
}
void uart0_handler()
{
  uart_handler(&uart[0]);
}
void uart1_handler()
{
  uart_handler(&uart[1]);
}
int vectorInt_init()
{
  printf("vectorInterrupt_init()\n");
  *((int *)(VIC_BASE_ADDR+0x100)) = (int)timer0_handler;
  *((int *)(VIC_BASE_ADDR+0x104)) = (int)uart0_handler;
  *((int *)(VIC_BASE_ADDR+0x108)) = (int)uart1_handler;
  //(2). write to intControlRegs = E=1|IRQ# =  1xxxxx
  *((int *)(VIC_BASE_ADDR+0x200)) = 0x24;  //0100100 at IRQ 4
  *((int *)(VIC_BASE_ADDR+0x204)) = 0x2C;  //0101100 at IRQ 12
  *((int *)(VIC_BASE_ADDR+0x208)) = 0x2D;  //0101101 at IRQ 13
  // write 32-bit 0's to IntSelectReg to generate IRQ interrupts
  *((int *)(VIC_BASE_ADDR+0x0C)) = 0;
}
void irq_chandler()
{
  int (*f)();                          // f is a function pointer
  f =(void *)*((int *)(VIC_BASE_ADDR+0x30)); // get ISR address
  f();                                 // call the ISR function
  *((int *)(VIC_BASE_ADDR+0x30)) = 1; // write to vectorAddr as EOI
}
```

```
int delay(int pid) // delay loop: simulate task compute time
{
  int i, j;
  for (i=0; i<1000; i++){
    for (j=0; j<1000; j++); // change this line for different dealys
  }
}
int taskCode()
{
  int pid = running->pid;
  u32 time1, time2, t, period, read_ytime, ctime;
  period = (6 - running->pid)*T; // task period=5,4,3,2
  while(1){
    lock();
     ss[running->pid].value = 0;
     ss[running->pid].queue = 0;
     proc[pid].ready_time = 0;
    unlock();

    P(&ss[running->pid]); // block on semaphore; Ved up by period

    readytime = running->ready_time;
    time1 = gtime;
     printf("P%dready=%dstart=%d", pid, readytime, time1);
    uprintf("P%dready=%dstart=%d", pid, readytime, time1);
       delay(pid); // simulate task compute time
    time2 = gtime;
    t = time2 - time1;
    ctime = time2 - readytime; // task comcpletion time
     printf("end=%d%d[%d%d]", time2, t, period, ctime);
    uprintf("end=%d%d[%d%d]", time2, t, period, ctime);
    if (dtime > period){
        printf(" PANIC! miss deadline!\n"); // to LCD
       uprintf(" PANIC! miss deadline!\n"); // to UART0
    }
    else{
       printf(" OK\n");
      uprintf(" OK\n");
    }
  }
}
int main()
{
   int i;
   PROC *p;
   fbuf_init();
   uart_init();
   up0 = &uart[0];
   up1 = &uart[1];
   kprintf("Welcome to UP_RTOS in ARM\n");
   /* enable timer0,1, uart0,1 SIC interrupts */
   VIC_INTENABLE = 0;
   VIC_INTENABLE |= (1<<4);  // timer0,1 at bit4
   VIC_INTENABLE |= (1<<5);  // timer2,3
   VIC_INTENABLE |= (1<<12); // UART0 at bit12
```

```
   VIC_INTENABLE |= (1<<13); // UART2 at bit13
   VIC_INTENABLE |= (1<<31); // SIC to VIC's IRQ31
   vectorInt_init();
   timer_init();
   timer_start(0);
   kernel_init();
   kprintf("P0 kfork tasks\n");
   for (i=1; i<=4; i++){       // create 4 static tasks
     ss[i].value = 0;
     ss[i].queue = (PROC *)0;
     kfork((int)taskCode, i);  // priority = 1,2,3,4
   }
   printQ(readyQueue);
   unlock();
   while(1){   // P0 loop
     if (readyQueue)
        tswitch();
   }
}
```

Deadline Analysis of the C10.2 system: With 4 tasks and task periods of 2T to 5T, we can derive an upper bound on the task compute time as follows. Assume that Ci is the compute time of taski. According to the schedulability condition of RMS, we expect all the tasks can meet their deadlines if

```
          C1/5T + C2/4T + C3/3T + C4/2T < 4(2**(1/4) – 1)
or        12C1 + 15C2 + 20C3 + 30C4 < 45.41T
```

This condition can be used to estimate the maximum compute times of the tasks without missing deadlines. For instance, if we assume that the compute time of all the tasks are the same, i.e. C=Ci for all i, then the individual task compute time upper bound is

$$\mathbf{C < (45.41/77)T = 0.59T} \tag{10.3}$$

However, if we take into account the overhead due to interrupts processing, task blocking time and task switching time, etc. the allowable task compute time must be much smaller. Rather than trying to adjust the compute time of individual tasks, we simply use different values of T in the testing program and observe the run-time task behavior. In the testing program, the unit of task period T is set to 32 timer ticks initially.

Figure 10.3 shows the sample outputs of running the C10.2 program. The figure shows that all 4 tasks are activated to run periodically. Average task compute time is about 5 timer ticks or 0.156T, which is significantly lower than the theoretical bound of 0.59T. In this case, all tasks can indeed meet their deadlines. However, if we shorten T to a smaller value, some of the tasks will start to miss their deadlines. For example, with T=16 timer ticks, C=0.31T, which is still well below the RMS bound of 0.59T, only task4 will run because it is activated so quickly that it's always the ready task with the highest priority. In this case, task1 to taks3 will always miss their deadlines because they can never be scheduled to run. The example shows that, with preemptive task scheduling, it is even harder to determine whether tasks can meet their deadlines based on the theoretical models of RTOS. This is an area of RTOS which definitely needs more study, both analytically and empirically.

10.7.9 UP_RTOS with Dynamic Tasks Sharing Resources

The third example of UP_RTOS, denoted by C10.3, demonstrates dynamic tasks sharing resources. Resources are simulated by mutex locks, which support priority inheritance to prevent priority inversion. In the C10.3 system, tasks are created dynamically, which may terminate after completing their work. Task scheduling is by preemptive (static) task priority. The following lists the implementation code of the C10.3 program.

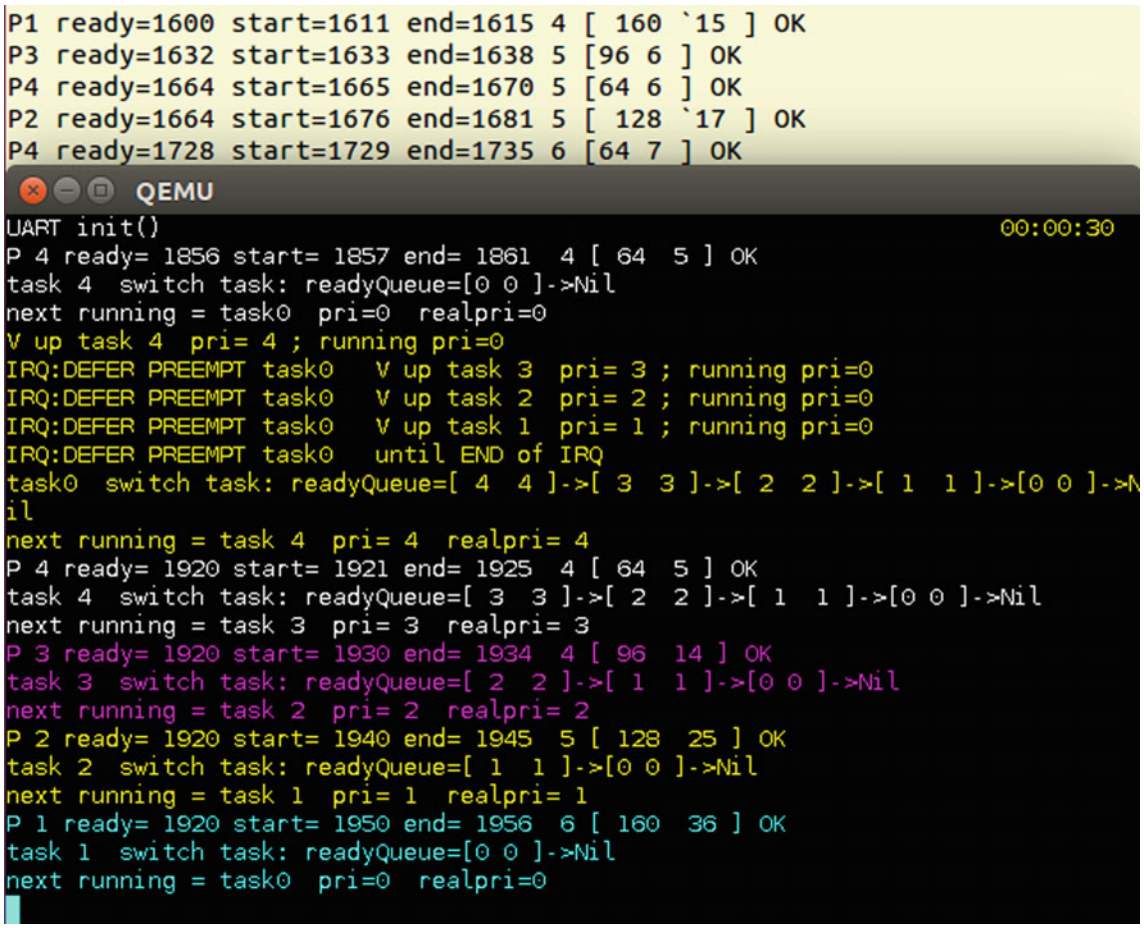


Fig. 10.3 UP_RTOS with periodic tasks and preemptive scheduling

(1). ts.s file of C10.3: the ts.s file is identical to that of C10.1

(2). Kernel C files of C10.3

For brevity, we only show the modified kernel functions in support of task preemption. These include V operation on semaphores and mutex_unlock, which may make a blocked task ready to run and change the readyQueue. In addition, kfork may create a new task with a higher priority than the current running task. All these functions call reschedule(), which may switch task immediately or defer task switch until the end of IRQ interrupts processing.

```
int reschedule()
{
  int SR = int_off();
  int pid = running->pid;
  if (readyQueue && readyQueue->priority > running->priority){
    if (intnest==0){ // not in IRQ handler: preempt immediately
       printf("%d PREEMPT %d IMMEDIATELY\n", readyQueue->pid, pid);
       tswitch();
    }
    else{            // still in IRQ handler: defer preemption
      printf("%d DEFER PREEMPT %d\n", readyQueue->pid, running->pid);
      swflag = 1;    // set need to switch task flag
    }
  }
  int_on(SR);
}
```

```
int P(struct semaphore *s)
{
  int SR = int_off();
  s->value--;
  if (s->value < 0){           // block running task
     running->status = BLOCK;
     enqueue(&s->queue, running);
     running->status = BLOCK;
     enqueue(&s->queue, running);
     // priority inheritance
     if (running->priority > s->owner->priority){
        s->owner->priority = running->priority; // boost owner priority
        reorder_readyQueue(); // re-order readyQueue
     }
     tswitch();
  }
  s->owner = running;         // as owner of semaphore
  int_on(SR);
}
int V(struct semaphore *s)
{
  PROC *p; int cpsr;
  int SR = int_off();
  s->value++;
  s->owner = 0;               // clear  s owner field
  if (s->value <= 0){         // s queue has waiters
     p = dequeue(&s->queue);
     p->status = READY;
     s->owner = p;            // new owner
     enqueue(&readyQueue, p);
     running->priority = running->rpriority; // restore owner priority
     printf("timer: V up task%d pri=%d; running pri=%d\n",
             p->pid, p->priority, running->priority);
     resechedule();
  }
  int_on(SR);
}
int mutex_lock(MUTEX *s)
{
  PROC *p;
  int SR = int_off();
  printf("task%d locking mutex %x\n", running->pid, s);
  if (s->lock==0){ // mutex is in unlocked state
       s->lock = 1;
       s->owner = running;
  }
  else{ // mutex is already locked: caller BLOCK on mutex
     running->status = BLOCK;
     enqueue(&s->queue, running);
     // priority inheritance
     if (running->priority > s->owner->priority){
        s->owner->priority = running->priority; // boost owner priority
        reorder_readyQueue(); // re-order readyQueue
     }
     tswitch(); // switch task
  }
```

```
  int_on(SR);
}
int mutex_unlock(MUTEX *s)
{
  PROC *p;
  int SR = int_off();
  printf("task%d unlocking mutex\n", running->pid);
  if (s->lock==0 || s->owner != running){ // unlock error
     int_on(SR); return -1;
  }
  // mutex is locked and running task is owner
  if (s->queue == 0){ // mutex has no waiter
     s->lock = 0;     // clear lock
     s->owner = 0;    // clear owner
  }
  else{ // mutex has waiters: unblock one as new owner
     p = dequeue(&s->queue);
     p->status = READY;
     s->owner = p;
     enqueue(&readyQueue, p);
     running->priority = running->rpriority; // restore owner priority
     resechedule();
   }
   int_on(SR);
   return 1;
}
PROC *kfork(int func, int priority)
{
  // create new task p with priority as before
  p->rpriority = p->priority = priority;
  // with dynamic tasks, readyQueue must be protected by a mutex
  mutex_lock(&readyQueuelock);
   enqueue(&readyQueue, p); // enter new task into readyQueue
  mutex_unlock(&readyQueuelock);
  reschedule();
  return p;
}
```

t.c file of C10.3

```
/*************** t.c file of C10.3 ***************/
#include "type.h"
MUTEX *mp;                  // global mutex
struct semaphore s1;        // global semaphore
#include "queue.c"
#include "mutex.c"          // modified mutex operations
#include "pv.c"             // modified P/V operations
#include "kbd.c"
#include "uart.c"
#include "vid.c"
#include "exceptions.c"
#include "kernel.c"
#include "timer.c"
#include "sdc.c"            // SDC driver
#include "mesg.c"           // message passing
```

```
int klog(char *line){ send(line, 2) } ;// send msg to log task #2
int endIRQ(){ printf("until END of IRQ\n") }
void copy_vectors(void) { // same as before }

// use vectored interrupts of PL190
int timer0_handler(){ timer_handler(0) }
int uart0_handler() { uart_handler(&uart[0]) }
int uart1_handler() { uart_handler(&uart[1]) }
int v31_handler()
{
  int sicstatus = *(int *)(SIC_BASE_ADDR+0);
  if (sicstatus & (1<<3)) { kbd_handler() }
  if (sicstatus & (1<<22)){ sdc_handler() }
}
int vectorInt_init()      // add KBD and SDC interrupts
{
  printf("vectorInt_init() ");
  /***** write to vectorAddr regs *************************/
  *((int *)(VIC_BASE_ADDR+0x100)) = (int)timer0_handler;
  *((int *)(VIC_BASE_ADDR+0x104)) = (int)uart0_handler;
  *((int *)(VIC_BASE_ADDR+0x108)) = (int)uart1_handler;
  *((int *)(VIC_BASE_ADDR+0x10C)) = (int)kbd_handler;
  *((int *)(VIC_BASE_ADDR+0x10C)) = (int)v31_handler;
  *((int *)(VIC_BASE_ADDR+0x110)) = (int)sdc_handler;
  /***** write to intControlRegs: E=1|IRQ# = 1xxxxx *********/
  *((int *)(VIC_BASE_ADDR+0x200)) = 0x24; //0100100 at IRQ 4
  *((int *)(VIC_BASE_ADDR+0x204)) = 0x2C; //0101100 at IRQ 12
  *((int *)(VIC_BASE_ADDR+0x208)) = 0x2D; //0101101 at IRQ 13
  *((int *)(VIC_BASE_ADDR+0x20C)) = 0x3F; //0111111 at IRQ 31
  *((int *)(VIC_BASE_ADDR+0x210)) = 0x36; //0110110 at IRQ 22
  /***** write 0 to IntSelectReg to generate IRQ interrupts */
  *((int *)(VIC_BASE_ADDR+0x0C)) = 0;
}
int irq_chandler()
{
  int (*f)();                          // f is a function pointer
  f =(void *)*((int *)(VIC_BASE_ADDR+0x30)); // read vectorAddr reg
  f();                                 // call the ISR function
  *((int *)(VIC_BASE_ADDR+0x30)) = 1; // write to vectorAddr as EOI
}

int delay(){ // delay loop to simulate task computation }

int task5()
{
    printf("task%d running: ", running->pid);
    klog("start");
    mutex_lock(mp);
     printf("task%d inside CR\n",  running->pid);
     klog("inside CR");
    mutex_unlock(mp);
    klog("terminate");
    kexit(0);
}
```

```
int task4()
{
    klog("create task5");
    kfork((int)task5, 5);  // create P5 with priority=5
    delay();
    printf("task%d terminte\n", running->pid);
    klog("terminate");
    kexit(0);
}
int task3()                // Ved up by timer periodically
{
  int pid, status;
  while(1){
    printf("task%d waits for timer event\n", running->pid);
    P(&s1);
    klog("running");
    mutex_lock(mp);
     printf("task%d inside CR\n", running->pid);
     klog("create task4");
     kfork((int)task, 4);  // create P4 while holding mutex lock
     delay();
    mutex_unlock(mp);
    pid = kwait(&status);
  }
}
int task2()  // logging task at priority 1
{
   int blk = 0;
   char line[1024];           // 1KB log line
   printf("task%d: ", running->pid);
   while(1){
     //printf("task%d:recved line=%s\n", running->pid, line);
     r = recv(line);
     put_block(blk, line);    // write log line to SDC
     blk++;
     uprintf("%s\n", line);   // display to UART0 on-line
     printf("log: ");
   }
}
int task1()
{
   int status, pid;
   pid = kfork((int)task2, 2);
   pid = kfork((int)task3, 3);
   while(1)  // task1 waits for any ZOMBIE children
      pid = kwait(&status);
}
int main()
{
   fbuf_init();               // LCD
   uart_init();               // UARTs
   kbd_init();                // KBD
   printf("Welcome to UP_RTOS in ARM\n");
```

```
   /* configure VIC and SIC for interrupts */
   VIC_INTENABLE = 0;
   VIC_INTENABLE |= (1<<4);   // timer0,1 at bit4
   VIC_INTENABLE |= (1<<12);  // UART0 at bit12
   VIC_INTENABLE |= (1<<13);  // UART2 at bit13
   VIC_INTENABLE |= (1<<31);  // SIC to VIC's IRQ31
   UART0_IMSC = 1<<4;         // enable UART0 RXIM interrupt
   SIC_INTENABLE = (1<<3);    // KBD int=bit3 on SIC
   SIC_INTENABLE |= (1<<22);  // SDC int=bit22 on SIC
   SIC_ENSET = (1<<3);        // KBD int=3 on SIC
   SIC_ENSET |= 1<<22;        // SDC int=22 on SIC
   vectorInt_init();          // initialize vectored interrupts
   timer_init();
   timer_start(0);            // timer0
   sdc_init();                // initialize SDC driver
   msg_init();                // initialize message buffers
   mp = mutex_create();       // mutex and semaphore
   s1.value = 0; s1.queue = 0;
   kernel_init();             // initialize kernel, run P0
   kfork((int)task1, 1);      // create P1 with priority=1
   while(1){                  // P0 loop
     if (readyQueue)
        tswitch();
   }
}
```

When the C10.3 system starts, it creates and runs the initial task P0, which has the lowest priority 0. P0 creates a new task P1 with a priority=2. Since P1 has a higher priority, it preempts P0 directly, which demonstrates immediate task preemption without any delay. When P1 runs, it creates a child task P2 with a priority=1 as the logging task. In this case, it does not switch process. Then it creates another child task P3 with a priority=3. Since P3 has a higher priority, it immediately preempts P1. When P1 runs again, it executes in a loop, waiting for any ZOMBIE child and frees the child PROC for reuse. When P3 runs, it waits for a timer event, which will be V-ed up periodically by the timer. When P3 runs again, it locks up a mutex and creates P4 with priority=4, which immediately preempts P3. P4 creates a child P5 with priority 5, which immediately preempts P4. When P5 runs, it tries to acquire the same mutex lock, which is still held by P3. So P5 becomes blocked on the mutex but it raises the priority of P3 to 5, thus preventing priority inversion. When P3 unlocks the mutex, it unblocks P5 and restores its own priority back to 3, allowing P4 to run next rather than P3. These demonstrate priority inheritance in UP_RTOS. After unlocking the mutex, P3 has two options. It may repeat the loop or terminate. If P3 repeats the loop, it must wait for its child to terminate and release child PROC structure for reuse. Otherwise, the number of available PROC structures would be one less each time P3 runs. Alternatively, P3 may terminate. In that case, P1 will dispose of all the terminated PROCs and creates a new instance of P3 again. In the demonstration system C10.3, task P2 is the logging task. It runs with the second lowest priority so that it does not impede the executions of other tasks. P2 repeatedly try to receive messages, which are logging requests from other tasks. For each longing request, P2 formats the log request into the form

timestamp : pid:log information

and writes the line to a 1KB block in a SDC by invoking the SDC driver directly. Figure 10.4 shows the sample outputs of running the UP_RTOS system C10.3.

The top part of Fig. 10.4 shows the on-line log information displayed by the logging task to a UART port. The logging information are also saved to a SDC as permanent records. In the demonstration program, the SDC is a virtual SD, which is not an ordinary file. Its contents can be read by the get_block() operation of the SDC driver for inspection.

Schedulability and Deadline Analysis: The schedulability of RTOS comprising tasks sharing resources has been studied by many people. With resource sharing, task priorities are no longer static. So such analyses do not apply to the simple RMS model, but they are applicable to both EDF and DMS models. The following is a (simplified) schedulability analysis based on the work of (Audsley et al. 1993). First, we define the following terms.

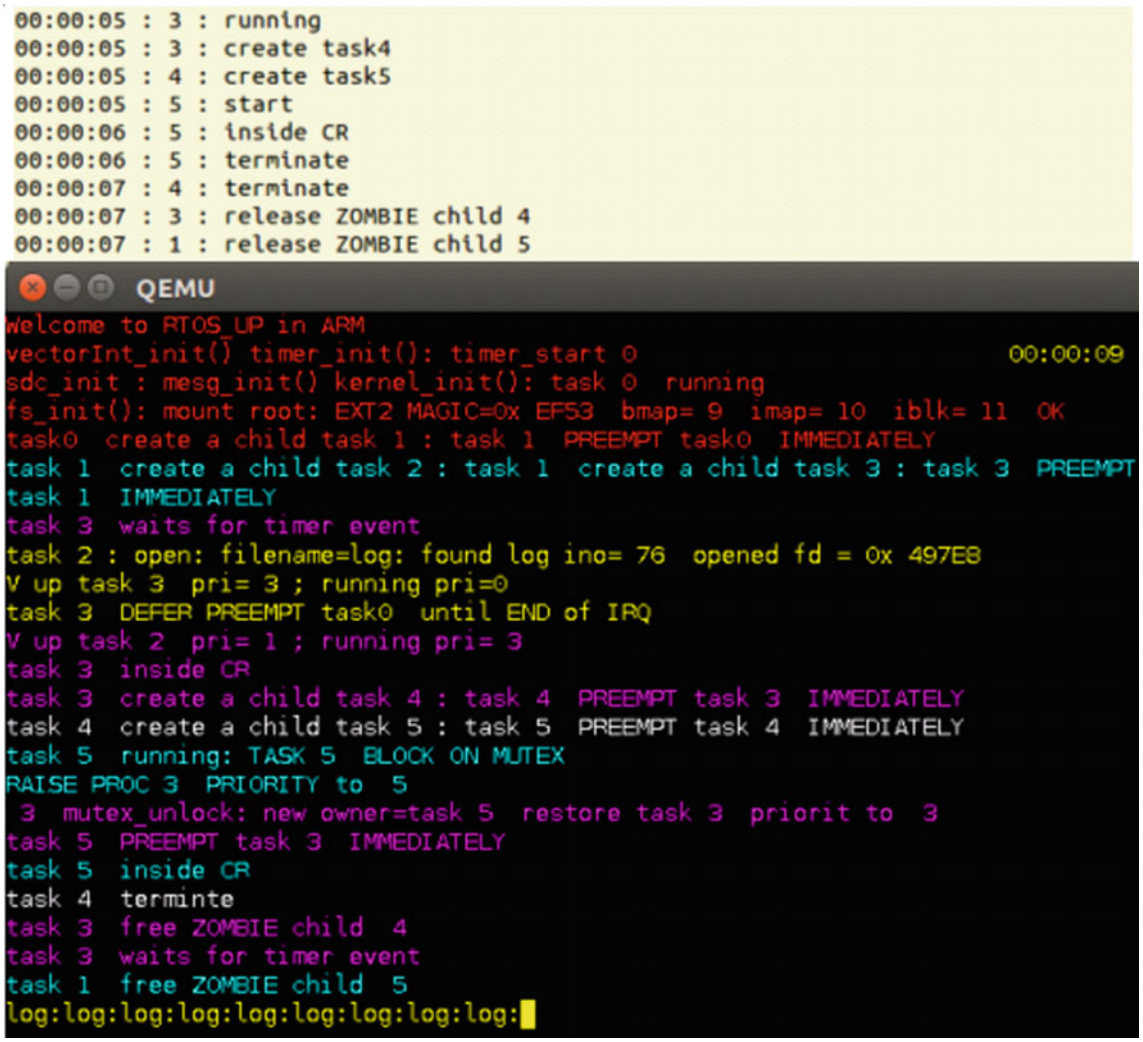


Fig. 10.4 UP_RTOS with dynamic tasks sharing resources

Ci = the worst-case computation time of task i on each release
Ti = the lower bound on the time between successive arrivals of task i. For periodic tasks, Ti is equal to the task period
Di = the deadline requirement of task i, measured relative to a given release time of task i
Bi = the worst case blocking time task i can experience due to priority inheritance. Bi is normally equal to the longest critical section of lower priority tasks accessing semaphores with ceilings higher than or equal to the priority of task i
Ii = the worst case interference a task i can experience from other tasks. Interference on task i is defined as the time higher priority tasks can preempt task i and execute, hence prevent task i from executing
Ri = the worst case response time for a task i measured from the time the task is released. For a schedulable task, Ri < Di or Ri < Ti, if deadline Di is not specified
hp(i) = the set of tasks of higher base priority than the base priority of task i, i.e. tasks which could preempt task i

Based on the terms defined above, the response time of task i can be expressed as

$$\mathbf{Ri} = \mathbf{Ci} + \mathbf{Bi} + \mathbf{Ii} \tag{10.4}$$

The worst case computation time **Ci** can be determined by running each task in an isolated environment. The worst case blocking time **Bi** is equal to the longest critical section of any lower priority task accessing a semaphore or mutex. The worst case interference of task i from task j is [Ri/Tj]Cj, which accounts for the time task i being blocked by task j due to task preemption. The total interference of task i from all other tasks j is given by

$$\mathbf{Ii} = \underset{\mathbf{j\ in\ hp(i)}}{\mathbf{SUM}}\{[\mathbf{Ri}/\mathbf{Tj}]\mathbf{Cj}\}; \tag{10.5}$$

Combining equations (10.4) and (10.5) yields the (recurrence) equation

$$\mathbf{Ri}(\mathbf{n}+\mathbf{1}) = \mathbf{Ci} + \mathbf{Bi} + \underset{\mathbf{j\ in\ hp(i)}}{\mathbf{SUM}}\{[\mathbf{Ri}(\mathbf{n})/\mathbf{Tj}]\mathbf{Cj}\} \tag{10.6}$$

The recurrence equation can be solved by iteration. The iteration starts with Ri(0) = 0 and terminates when Ri(n+1) = Ri (n). The iteration can be terminated earlier if either Ri(n+1) > Di or Ri(n+1) > Ti, indicating that taski will fail the deadline. Furthermore, it has been shown (Joseph and Pandya 1986) that the iteration is guaranteed to converge if the CPU utilization is <100%. After computing the response times of all the tasks, it is a simple matter to test whether they can meet their deadlines. All the tasks can meets their deadlines if

$$\mathbf{Ri} < \mathbf{Di}\ \mathbf{or}\ \mathbf{Ri} < \mathbf{Ti}(\mathbf{for\ periodic\ tasks})\ \mathbf{for\ all\ i}$$

The analytic results can be used as a guideline when designing a RTOS with resource sharing. To determine whether tasks in a real RTOS can meet their deadlines must still be verified empirically. For the example RTOS system C10.3, we leave the schedulability analysis and verification of task deadlines as an exercise in the Problem section.

10.8 Multiprocessor RTOS (SMP_RTOS)

In Chap. 9, we discussed SMP operations for the ARM MPcore architecture in detail. In this section, we shall show how to design and implement a RTOS for SMP systems. While the design principle is general, which is applicable to any SMP system, we shall use the ARM A9-MPcore architecture as the platform for implementation and testing. Rather than presenting a complete system in one step, we shall develop the SMP_RTOS system in incremental steps. In each step, we shall focus on a problem that is unique to real-time operations in a SMP environment, and demonstrate the solution by a sample system. The steps are:

(1). Develop a SMP kernel for task management with improved concurrency
(2). Adapt UP_RTOS for SMP
(3). Implement preemptive task scheduling by priority in SMP
(4). Implement nested interrupts in SMP
(5). Implement priority inheritance in SMP
(6). Integrate the results into a complete SMP_RTOS

10.8.1 SMP_RTOS Kernel for Task Management

First, we show the design and implementation of an initial SMP_ROTS kernel for task management. The initial SMP_RTOS kernel only supports dynamic task creation, task termination and task synchronization. It should exploit the parallel processing capability of multiple CPUs to improve concurrency. To begin with, we shall assume the following operating environment of tasks.

. Each task is a single execution entity. There are no additional threads within a task.
. All tasks execute in the same address space, so no need for virtual address mapping.
. Tasks do not use signals for inter-task communication and do not need file system support, so they do not have any resources, as in a general purpose OS.

These are reasonable assumptions for real-time applications, which lead to a simplified PROC structure for real-time tasks. The following shows the simplified PROC structure in the SMP_RTOS kernel.

10.8.1.1 PROC Structure in SMP_RTOS

```
#define SSIZE 1024
typedef struct proc{
  struct proc *next;
  int    *ksp;             // saved stack pointer
  int    status;           // FREE|READY|BLOCK|ZOMBIE, etc.
  int    priority;         // effective priority
  int    pid;
  int    ppid;
  struct proc *parent;
  int    exitCode;         // exit code
  int    ready_time;
  struct mbuf *mqueue;     // message queue
  struct mutex mlock;      // message queue mutex lock
  struct semaphore nmsg;   // wait for message semaphore
  int    cpuid;            // which CPU this proc is running on
  int    rpriority;        // for priority inheritance
  int    timeslice;        // for time sliced scheduling
  struct semaphore wchild; // wait for ZOMBIE children
  int    kstack[SSIZE];
}PROC;
```

In the simplified PROC structure, the fields of virtual address space, user mode context and resources, e.g. file descriptors and signals, are all eliminated. For clarity, the added fields for real-time operations are shown in bold faced lines.

10.8.1.2 Kernel Data Structures: Kernel data structures are defined as follows.

```
#define NCPU    4
#define NPROC 256
```

Each task is represented by a PROC structure. The total number of PROC structures, NPROC, is a configuration parameter, which can be set to suit the actual system needs. Initially, all the PROC structures are in a freeList. When creating a task, we allocate a free PROC from the freeList. When a task terminates, its PROC structure is eventually released back to the freeList for reuse. The freeList is protected by a spinlock during allocation/deallocation of free PROCs.

```
PROC *freeList;   // contain all free PROCs
int freelock = 0; // spinlock for freeList
```

In a UP kernel, it suffices to use a single running pointer pointing at the current executing PROC. In a SMP kernel, we shall identify the tasks executing on different CPUs by the following means.

```
#define running run[get_cpuid()]
PROC *run[NCPU];  // pointer to running PROC on each CPU
```

In a UP kernel, it suffices to use a single queue for task scheduling. In a SMP kernel, a single scheduling queue can be a bottleneck, which severely restricts the concurrency of the system. To improve both concurrency and efficiency, we assume that each CPU has a separate scheduling queue protected by a spinlock.

```
PROC *readyQueue[NCPU];  // per CPU task scheduling queues
int  readylock[NCPU];    // spinlocks for readyQueue[cpuid]
```

Each CPU always tries to run tasks from its own readyQueue[cpuid]. If a CPU's readyQueue is empty, it runs a special idle task, which puts the CPU in a power-saving state, waiting for events or interrupts. When the system starts, each CPU runs a special initial task, which is also the idle task on the CPU.

```
PROC iproc[NCPU];        //  per CPU initial/idle task
```

Tasks normally run in Supervisor (SVC) mode, which use the stack in the PROC structure. A task may enter IRQ mode to handle interrupts, or ABT mode to handle data abort exceptions, etc. Each CPU must have separate stacks for interrupts and exceptions processing. Thus, we define per CPU IRQ and exception mode stacks as

```
int irq_stack[NCPU][1024];  // per CPU IRQ mode stack
int abt_stack[NCPU][1024];  // per CPU ABT mode stack
int und_stack[NCPU][1024];  // per CPU UND mode stack
```

In a ROTS kernel, preemptive task scheduling and nested interrupts are essential. With preemptive task scheduling, each CPU needs a switch task flag. With nested interrupts, each CPU must keep track of the interrupt nesting level. Task switch is not allowed while the CPU is executing inside any interrupt handler. Task switch may occur only if all the nested interrupts processing have ended. Thus, we define

```
volatile int intnest[NCPU]; // per CPU interrupt nesting counter
volatile int swflag[NCPU];  // per CPU switch task flag
```

Since each of these variables is accessed only by a single CPU, either from a task running on a CPU or by an interrupt handler on the same CPU, it suffices to protect them by the usual disable/enable interrupts of the CPU.

10.8.1.3 Protection of Kernel Data Structures

Each kernel data structure is protected by a spinlock. Requests for spinlocks are mostly single level. When tasks must acquire multiple spinlocks, we shall ensure that the locking order is always unidirectional, so that deadlocks can not occur.

10.8.1.4 Synchronization Tools

The SMP_RTOS kernel supports the following tools for task synchronization and inter-task communication.

Spinlocks: spinlocks are used to protect critical regions (CRs) of short durations and also in device interrupt handlers.

Mutexes: mutexes are for protecting CRs of long durations. They are implemented with priority inheritance to prevent priority inversion.

Semaphores: semaphores are used only for process cooperation. They are implemented with priority inheritance also.

Shared Memory: tasks execute in the same address space, so they may use shared memory for inter-task communication. Each shared memory area is protected by a mutex lock to ensure exclusive access.

Messages: tasks may send/recv messages through the per-task message queue.

10.8.1.5 Task Management

tasks can be created dynamically to execute a function with a static priority on a specific CPU. Task creation is by the API

```
int pid = kfork((int)func, int priority, int cpu)
```

To be consistent with the traditional Unix/Linux kernel, tasks obey the usual parent-child relation. When a task has completed its work, it may terminate by the API

```
void kexit(int exitValue)
```

When a task with children terminates, it first releases its own terminated children, if any, and send other children to a special task P1, which behaves like the INIT process in a Unix/Linux kernel. A terminated task becomes a ZOMBIE and notifies its parent, which may be the original parent or the adopted parent P1. A parent task may wait for children termination by the API

```
int kwait(int *status)
```

which returns the terminated child pid, its exit status and releases the ZOMBIE child PROC for reuse. Synchronization between the parent and child is by a semaphore in the parent PROC, which prevents race conditions between the parent and the child tasks even if they run in parallel on different CPUs.

Next, we show the implementation of the task management part of the SMP_RTOS kernel. As usual, the kernel code consists of a ts.s file and a set of files in C. The kernel files are compile-linked to a binary executable t.bin, which is run directly on the emulated realview-pbx-a9 VM under QEMU by a sh script.

```
arm-none-eabi-as -mcpu=cortex-a9 ts.s -o ts.o
arm-none-eabi-gcc -c -mcpu=cortex-a9 t.c -o t.o
arm-none-eabi-ld -T t.ld ts.o t.o -Ttext=0x10000 -o t.elf
arm-none-eabi-objcopy -O binary t.elf t.bin
qemu-system-arm -M realview-pbx-a9 -smp 4 -m 512M -sd ../sdc \
                -kernel t.bin -serial mon:stdio
```

Alternatively, we may also write t.bin as a kernel image to the boot/ directory in a SDC file system and use a separate booter to boot up the kernel image from the SDC.

(1) . Startup Sequence: When the realview-pbx-a9 VM starts, CPU0 is the boot processor and all other secondary CPUs (APs) are held in a WFI state, waiting for SGIs from the boot processor to truly start up. The boot processor starts to execute reset_handler in the ts.s file. The actions of CPU0 are

. Set SVC mode stack pointer to the high end of iproc[0], which makes iproc[0].kstack as the initial stack.
. Set IRQ mode stack pointer to the high end of irq_stack[0], which makes irq_stack[0] as its IRQ mode stack. Similarly, it sets up the ABT and UND modes stacks.
. Copy vector table to address 0, then call main() in C to initialize the kernel.

The following shows the reset_handler code of CPU0 and the startup code of the APs.

(2). Reset_handler and apStart Code in Assembly

```
.global reset_handler, apStart           // CPU0, AP code
.global iproc, procsize                  // globals
.global irq_stack, abt_stack, und_stack  // globals
.global tswitch, scheduler
.global int_on, int_off, lock, unlock
.global get_cpsr, get_spsr;
.global slock, sunlock, get_cpuid
.global vectors_start, vectors_end

  reset_handler:

// CPU0: set SVC stack to HIGH END of iproc[0].kstack[]
  ldr r0, =iproc      // r0 points to proc's
  ldr r1, =procsize   // r1 -> procsize
```

```
  ldr r2, [r1, #0]    // r2 = procsize
  add r0, r0, r2
  mov sp, r0          // SVC sp -> high end of iproc[0]
// CPU0: go in IRQ mode to set IRQ stack
  msr cpsr, #0x92
  ldr sp, =irq_stack  // point at int irq_stack[0][1024]
  add sp, sp, #4096   // high end of irq_stack[0]
// CPU0: go in ABT mode to set ABT stack
  msr cpsr, #0x97
  ldr sp, =abt_stack
  add sp, sp, #4096
// CPU0: go in UND mode to set UND stack
  msr cpsr, #0x9B
  ldr sp, =und_stack
  add sp, sp, #4096
// CPU0: go back in SVC mode to set SPSR to SVC mode with IRQ on
  msr cpsr, #0x93
  msr spsr, #0x13
// CPU0: copy vector table to address 0
  bl copy_vectors
  bl  main            // call main() in C
  b   .               // hang if main() ever returns
apStart:              // AP entry point
  LDR r0, =iproc      // r0 points to iproc
  LDR r1, =procsize   // r1 -> procsize
  LDR r2, [r1, #0]    // r2 = procsize
// get CPU ID
  MRC p15, 0, r1, c0, c0, 5   // read CPU ID register to r1
  AND r1, r1, #0x03           // mask in CPU ID bits only
  mul r3, r2, r1      // procsize*CPUID: r3->high end of iproc[CPUID]
  add r0, r0, r3      // r0 += r3
  mov sp, r0          // sp->iproc[CPUID] high end
//AP: go in IRQ mode to set IRQ stack
  MSR cpsr, #0x92
  ldr r0, =irq_stack  // int irq_stack[4][1024] in C
  lsl r2, r1, #12     // r2 = r1*4096
  ADD r0, r0, r2      // r0 -> high end svcstack[cpuid]
  MOV sp, r0          // IRQ sp of CPU ID -> highend of irq[ID][1024]
//AP: go in ABT mode to set ABT stack
  MSR cpsr, #0x97
  LDR sp, =abt_stack
  lsl r2, r1, #12     // r2 = r1*4096
  ADD r0, r0, r2      // r0 -> high end svcstack[cpuid]
  MOV sp, r0          // IRQ sp of CPU ID -> highend of irq[ID][4096]
//AP:  go in UND mode to set UND stack
  MSR cpsr, #0x9B
  LDR sp, =und_stack
  lsl r2, r1, #12     // r2 = r1*4096
  ADD r0, r0, r2      // r0 -> high end svcstack[cpuid]
  MOV sp, r0          // IRQ sp of CPU ID -> highend of irq[ID][4096]
//AP: go back in SVC mode to set SPSR
  MSR cpsr, #0x93
  MSR spsr, #0x13
```

```
//AP: call APstart() with IRQ on
  MSR cpsr, #0x13
  bl APstart          // call APstart() in C
```

(3). The main() and APstart() functions in C

In main(), CPU0 first initializes the device drivers, configures the GIC for interrupts, initialize the kernel data structures and runs the initial task with pid=1000. Then it issues SGI to activate the APs. After all the APs are ready, it creates a P0 as the logging task and the INIT task P1. Then it execute run_task(), trying to run tasks form the readyQueue of the same CPU. Upon receiving a SGI from CPU0, each AP starts to execute the apStart code in assembly. The actions of each AP are similar to those of CPU0, i.e. it sets SVC mode stack pointer to the high end of iproc[cpuid] and IRQ and other modes stacks by cpuid. Then it calls APstart() in C. In APstart(), each AP configures and starts its local timer, updates the global variable ncpu to synchronize with CPU0. Then it calls run_task() also. At this moment, the readyQueues of the APs are still empty. So each AP runs an idle task with pid=1000+cpuid and enters the WFE state. After creating tasks on the APs, there are two ways to let the APs continue. CPU0 may send a SGI to all the APs, causing them to get up from the WFE state, or each AP will get up when its local timer interrupts. The following shows the code segments of run_task(), APstart() and main().

```
int run_task()  // each CPU tries to run task from its own readyQ
{
   int cpuid = get_cpuid();
   while(1){
      slock(&readylock[cpuid]);
      if (readyQueue[cpuid] == 0){  // check own readyQueue
         sunlock(&readylock[cpuid]);
         asm("WFE");                // wait for events/interrupts
      }
      else{
         sunlock(&readylock[cpuid]);
         ttswitch();                // switch task on CPU
     }
  }
}
int APstart()
{
   int cpuid = get_cpuid();
   slock(&aplock);
   color = YELLOW;
   printf("CPU%d in APstart ... ", cpuid);
   config_int(29, cpuid);         // configure local timer
   ptimer_init(); ptimer_start(); // start local timer
   ncpu++;
   running = &iproc[cpuid];       // run initial iproc[cpuid]
   printf("%d running on CPU%d\n", running->pid, cpuid);
   sunlock(&aplock);
   run_task();
}
int main()
{
   // initialize device drivers
   // configure GIC for interrupts
   kernel_init(); //initialize kernel
   printf("CPU0 startup APs: ");
   int *APaddress = (int *)0x10000030;
```

```
   *APaddress = (int)apStart;
   send_sgi(0x0, 0x0F, 0x01);   // send SGI 0 to all Aps
   printf("CPU0 waits for APs ready\n");
   while(ncpu < 4);
   printf("CPU0 continue\n");
   kfork(int)logtask, 1, 0); // create P0 on CPU0 as logging task
   kfork((int)f1,     1, 0); // create P1 on CPU0 as INIT task
   run_task();
}
```

10.8.1.6 Demonstration of Task Management in SMP_RTOS

We demonstrate task management in the initial SMP_RTOS kernel by a sample program, denoted by C10.4. Task management includes task creation, task termination and wait for child task termination. The program consists of the following components.

(1). ts.s file of C10.4:

```
/*********** ts.s file of C10.4: SMP_RTOS kernel **********/
      .text
      .code 32
.global reset_handler, vectors_start, vectors_end
.global proc, procsize, apStart
.global tswitch, scheduler
.global int_on, int_off, lock, unlock
.global get_fault_status, get_fault_addr, get_cpsr, get_spsr;
.global slock, sunlock, get_cpuid
reset_handler:
   // same as shown above
apStart:                    // AP entry point
   // same as shown above
irq_handler:                // IRQ interrupts entry point
  sub  lr, lr, #4
  stmfd sp!, {r0-r12, lr}  // save context
  bl  irq_chandler          // call irq_handler()
  ldmfd sp!, {r0-r12, pc}^ // return
data_handler:
  sub lr, lr, #4
  stmfd sp!, {r0-r12, lr}
  bl  data_abort_handler
  ldmfd sp!, {r0-r12, pc}^
tswitch:                    // tswitch() in SVC mode
  msr   cpsr, #0x93         // interrupts off
  stmfd sp!, {r0-r12, lr}
  ldr r4, =run               // r4=&run
  mrc p15, 0, r5, c0, c0, 5 // read CPUID register to r5
  and r5, r5, #0x3           // only the CPU ID
  mov r6, #4                 // r6 = 4
  mul r6, r5                 // r6 = 4*cpuid
  add r4, r6                 // r4 = &run[cpuid]
  ldr r6, [r4, #0]           // r6->running PROC
  str sp, [r6, #4]           // save sp in PROC.ksp
  bl  scheduler              // call scheduler() in C
```

```
  ldr r4, =run              // r4=&run
  mrc p15, 0, r5, c0, c0, 5 // read CPUID register to r5
  and r5, r5, #0x3          // only the CPU ID
  mov r6, #4                // r6 = 4
  mul r6, r5                // r6 = 4*cpuid
  add r4, r6                // r4 = &run[cpuid]
  ldr r6, [r4, #0]          // r6->running PROC
  ldr sp, [r6, #4]          // restore ksp
  msr cpsr, #0x13           // interrupts on
  ldmfd sp!, {r0-r12, pc}   // resume new running

int_off:                    // int sr=int_off()
  stmfd sp!, {r1}
  mrs r1, cpsr
  mov r0, r1                // r0 = r1
  orr r1, r1, #0x80
  msr cpsr, r1              // mask out IRQ
  ldmfd sp!, {r1}
  mov pc, lr                // return original cpsr
int_on:                     // int_on(SR)
  mrs cpsr, r0
  mov pc, lr
lock:                       // maks out IRQ in cpsr
  mrs r0, cpsr
  orr r0, r0, #0x80
  msr cpsr, r0
  mov pc, lr
unlock:                     // mask in IRQ in cpsr
  mrs r0, cpsr
  bic r0, r0, #0x80
  msr cpsr, r0
  mov pc, lr
  .global send_sgi
// void send_sgi(int ID, int target_list, int filter_list)
send_sgi:                     // send_sgi with CPUTarget list
  and  r3, r0, #0x0F          // Mask off unused bits of ID to r3
  and  r1, r1, #0x0F          // Mask off unused bits of target_filter
  and  r2, r2, #0x0F          // Mask off unused bits of filter_list
  orr  r3, r3, r1, LSL #16    // Combine ID and target_filter
  orr  r3, r3, r2, LSL #24    // and now the filter list
// Get the address of the GIC
  mrc  p15, 4, r0, c15, c0, 0  // Read periph base address
  add  r0, r0, #0x1F00        // Add offset of the sgi_trigger reg
  str  r3, [r0]               // Write to SGI Register(ICDSGIR)
  bx   lr
slock:                        // int slock(&spin)
  ldrex r1, [r0]
  cmp   r1, #0x0
  WFENE
  bne   slock
  mov   r1, #1
  strex r2, r1, [r0]
  cmp   r2, #0x0
```

```
  bne   slock
  DMB                          // barrier
  bx    lr
sunlock:                       // sunlock(&spin)
  mov   r1, #0x0
  DMB
  str   r1, [r0]
  DSB
  SEV
  bx    lr
get_fault_status:  // read and return MMU reg 5
  mrc p15,0,r0,c5,c0,0    // read DFSR
  mov pc, lr
get_fault_addr:            // read and return MMU reg 6
  mrc p15,0,r0,c6,c0,0    // read DFSR
  mov pc, lr
get_cpsr:
  mrs r0, cpsr
  mov pc, lr
get_spsr:
  mrs r0, spsr
  mov pc, lr
get_cpuid:
  mrc p15, 0, r0, c0, c0, 5   // Read CPU ID register
  and r0, r0, #0x03           // Mask off, leaving the CPU ID field
  bx  lr
  .global  enable_scu
// void enable_scu(void): Enables the SCU
enable_scu:
  mrc p15, 4, r0, c15, c0, 0  // Read periph base address
  ldr r1, [r0, #0x0]          // Read the SCU Control Register
  orr r1, r1, #0x1            // Set bit 0 (The Enable bit)
  str r1, [r0, #0x0]          // Write back modifed value
  bx  lr
vectors_start:
  LDR PC, reset_handler_addr
  LDR PC, undef_handler_addr
  LDR PC, svc_handler_addr
  LDR PC, prefetch_abort_handler_addr
  LDR PC, data_abort_handler_addr
  B .
  LDR PC, irq_handler_addr
  LDR PC, fiq_handler_addr
reset_handler_addr:          .word reset_handler
undef_handler_addr:          .word undef_handler
svc_handler_addr:            .word svc_entry
prefetch_abort_handler_addr: .word prefetch_abort_handler
data_abort_handler_addr:     .word data_handler
irq_handler_addr:            .word irq_handler
fiq_handler_addr:            .word fiq_handler
vectors_end:
// other SMP utility functions: NOT SHOWN
```

(2). Kernel.c file of SMP_RTOS

```
#define NCPU     4
#define NPROC  256
#define SSIZE 1024
#define FREE     0
#define READY    1
#define BLOCK    2
#define ZOMBIE   3
typedef struct proc{
  struct proc *next;
  int    *ksp;             // saved stack pointer
  int    status;           // FREE|READY|BLOCK|ZOMBIE, etc.
  int    priority;         // real priority
  int    pid;
  int    ppid;
  struct proc *parent;
  int    exitCode;         // exit code
  int    ready_time;
  struct mbuf *mqueue;     // message queue
  struct mutex mlock;      // message queue mutex lock
  struct semaphore nmsg;   // wait for message semaphore
  int    cpuid;            // which CPU this proc is on
  int    epriority;        // effective priority for PI
  int    timeslice;        // for time sliced scheduling
  struct semaphore wchild; // wait for ZOMBIE children
  int    kstack[SSIZE];
}PROC;

#define running run[get_cpuid()]
// kernel data structures and spinlocks
PROC iproc[NCPU];            // per CPU initial/idle PROCs
PROC *run[NCPU];             // pointer to running PROC on each CPU
PROC proc[NPROC];            // PROC structures
PROC *freeList;              // freeList
PROC *readyQueue[NCPU];      // per CPU readyQueue
int readylock[NCPU];         // per CPU readyQueue spinlcok
int intnest[NCPU];           // per CPU IRQ nesting level
int swflag[NCPU];            // per CPU switch task flag
int freeListlock = 0;        // freeList spinlock
int procsize = sizeof(PROC); // PROC size
int irq_stack[NCPU][1024];   // per CPU IRQ mode stack
int abt_stack[NCPU][1024];   // per CPU ABT mode stack
int und_stack[NCPU][1024];   // per CPU UND mode stack

int kernel_init()
{
  int i;
  PROC *p; char *cp;
  printf("kernel_init(): init iprocs\n");
  for (i=0; i<NCPU; i++){    // initialize per CPU initial PROCs
    p = &iproc[i];
    p->pid = 1000 + i;       // special pid for per CPU idle PROCs
    p->status = READY;
```

```
    p->ppid = p->pid;
    run[i] = p;              // run[i] points to idle iPROC[i]
    readyQueue[i] = 0;       // ready queues
    readylock[i] = 0;        // spinlocks
    intnest[i] = 0;          // IRQ nest levels
    swflag[i] = 0;           // switch task flags
  }
  for (i=0; i<NPROC; i++){
    p = &proc[i];
    p->pid = i;
    p->status = FREE;
    p->priority = 0;
    p->ppid = 0;
    p->next = p + 1;
  }
  proc[NPROC-1].next = 0;    // freeList of PROCs
  freeList = &proc[0];
  sleepList = 0;
  freelock = 0;
  running = run[0];          // initial task on CPU0
}

int ttswitch()               // switch task on CPUID
{
  int cpuid = get_cpuid();
  lock();
  slock(&readylock[cpuid]);
    tswitch();
  sunlock(&readylock[cpuid]);
  unlock();
}

int scheduler()
{
  int pid; PROC *old=running;
  int cpuid = get_cpuid();
  slock(&readylock[cpuid];
  if (running->pid < 1000){  // regular tasks
     if (running->status==READY)
        enqueue(&readyQueue[cpuid], running);
  }
  running = dequeue(&readyQueue[cpuid]);
  if (running == 0)
     running = &iproc[cpuid];// run idle task on CPU
  swflag[cpuid] = 0;
  sunlock(&readylock[cpuid]);
}

int kfork(int func, int priority, int cpu)
{
  int cpuid = get_cpuid();
  PROC *p = getproc(&freeList);  // allocate a free PROC
  if (p==0){ printf("kfork1 failed\n"); return -1 }
  p->ppid = running->pid;
  p->parent = running;
  p->status = READY;
```

```
  p->priority = priority;
  p->cpuid = cpuid
  for (i=1; i<15; i++)            // all 14 entries = 0
      p->kstack[SSIZE-i] = 0;
  p->kstack[SSIZE-1] = (int)func;  // resume to func
  p->ksp = &(p->kstack[SSIZE-14]); // saved ksp
  slock(&readylock[cpu]);
    enqueue(&readyQueue[cpu], p);  // enter p into rQ[cpu]
  sunlock(&readylock[cpu]);
  if (p->priority > running->priority){
    if (cpuid == cpu){             // if on the same CPU
      printf("%d PREEMPT %d on CPU%d\n", p->pid, running->pid, cpuid);
      ttswitch();
    }
  }
  return p->pid;
}

int P(struct semaphore *s)  // P operation in SMP
{
  int SR = int_off();
  slock(&s->lock);
  s->value--;
  if (s->value < 0){
    running->status = BLOCK;
    enqueue(&s->queue, running);
    // priority inheritance only if on the same CPU
    if (running->priority > s->owner->priority){
       if (s->owner->cpu == running->cpu){
          s->owner->priority = running->priority; //raise owner priority
          reorder_readyQueue(); // re-order readyQueue
       }
    }
    sunlock(&s->lock);
    ttswitch();
  }
  else{
    s->owner = running;      // as owner of s
    sunlock(&s->lock);
  }
  int_on(SR);
}

int V(struct semaphore *s)  // V operation in SMP
{
  PROC *p;
  int SR = int_off();
  slock(&s->lock);
  s->value++;
  s->owner = 0;             // clear s owner field
  if (s->value <= 0){       // s queue has waiters
     p = dequeue(&s->queue);
     p->status = READY;
     s->owner = p;          // new owner
```

```
      slock(&readylock[p->cpuid]); // rQ of original CPU
        enqueue(&readyQueue[p->cpuid], p);
      sunlock(&readylock[p->cpuid]);
      running->priority = running->rpriority; // restore owner priority
      resechedule();
   }
   sunlock(&s->lock);
   int_on(SR);
}

int kexit(int value)  // task termination
{
   int i; PROC *p;
   if (running->pid==1){ return -1 } // P1 never dies
   slock(&proclock);              // acquire proclock
   for (i=2; i<NPROC; i++){
     p = &proc[i];
     if ((p->status != FREE) && (p->ppid == running->pid)){
       if (p->status==ZOMBIE){
           p->status = FREE;
           putproc(&freeList, p); // release any ZOMBIE child
       }
       else{
           printf("send child %d to P1\n", p->pid);
           p->ppid = 1;
           p->parent = &proc[1];
     }
   }
   sunlock(&proclock);            // release proclock
   running->exitCode = value;
   running->status = ZOMBIE;
   V(&running->parent->wchild);
   ttswitch();
}

int kwait(int *status)     // wait for any ZOMBIE child
{
   int i, nChild = 0;
   PROC *p;
   slock(&proclock);        // acquire proclock
   for (i=2; i<NPROC; i++){
      p = &proc[i];
      if (p->status != FREE && p->ppid == running->pid)
         nChild++;
   }
   sunlock(&proclock);
   if (!nChild){            // no child error
      return -1;
   }
   P(&running->wchild);     // wait for a ZOMBIE child
   slock(&proclock);
   for (i=2; i<NPROC; i++){
       p = &proc[i];
       if ((p->status==ZOMBIE) && (p->ppid == running->pid)){
       *status = p->exitCode;
```

```
      p->status = FREE;
      putproc(&freeList, p);
      sunlock(&proclock);
      return p->pid;
  }
}
```

(3). t.c file of SMP_RTOS

```
/************* t.c file of SMP_RTOS ***********/
#include "type.h"
int aplock = 0;
int ncpu = 1;
#include "string.c"
#include "queue.c"
#include "pv.c"
#include "uart.c"
#include "kbd.c"
#include "ptimer.c"
#include "vid.c"
#include "exceptions.c"
#include "kernel.c"
#include "sdc.c"
#include "message.c"

int copy_vectors(){ // same as before }
#define GIC_BASE 0x1F000000
int config_gic()
{
  // set int priority mask register
  *(int *)(GIC_BASE + 0x104) = 0xFFFF;
  // set CPU interface control register: enable signaling interrupts
  *(int *)(GIC_BASE + 0x100) = 1;
  // distributor control register to send pending interrupts to CPUs
  *(int *)(GIC_BASE + 0x1000) = 1;
  config_int(29, 0);    // ptimer at 29 to CPU0
  config_int(44, 1);    // UAR0   to CPU1
  config_int(49, 2);    // SDC    to CPU0
  config_int(52, 3);    // KBD    to CPU1
}
int config_int(int N, int targetCPU)
{
  int reg_offset, index, value, address;
  reg_offset = (N>>3)&0xFFFFFFFC;
  index = N & 0x1F;
  value = 0x1 << index;
  address   =  (GIC_BASE + 0x1100) + reg_offset;
  *(int *)address |= value;
  reg_offset = (N & 0xFFFFFFFC);
  index = N & 0x3;
  address   =  (GIC_BASE + 0x1800) + reg_offset + index;
  *(char *)address = (char)(1 << targetCPU);
  address   =  (GIC_BASE + 0x1400) + reg_offset + index;
  *(char *)address = (char)0x88; // prioirty=8
}
```

```
int irq_chandler()
{
  // read ICCIAR of CPU interface in the GIC
  int intID = *(int *)(GIC_BASE + 0x10C);
  if (intID == 29){   // local timer of CPU
     ptimer_handler();
  }
  if (intID == 44){   // uart0
     uart_handler(0);
  }
  if (intID == 49){   // SDC
     sdc_handler();
  }
  if (intID == 52){   // KBD
     kbd_handler();
  }
  *(int *)(GIC_BASE + 0x110) = intID; // issue EOI
}
int kdelay(int d){ // delay loop }
int taskCode()
{
  int cpuid = get_cpuid();
  int pid = running->pid;
  while(1){
    printf("TASK%d ON CPU%d: ", pid, cpuid);
    kdelay(pid*100000); // simulate task computation
    ttswitch();
  }
}
int INIT_task()  // INIT task P1
{
  int i, pid, status;
  // create 2 tasks on different CPUs
  for (i=0; i<2; i++){
      kfork((int)taskCode, 1, 0);
      kfork((int)taskCode, 1, 1);
      kfork((int)taskcode, 1, 2);
      kfork((int)taskCode, 1, 3);
  }
  while(1){ // task1 waits for ZOMBIE children
     pid = kwait(&status);
  }
}
int logtask()   // logging task P0
{
  int pid;
  int cpuid = get_cpuid();
  char msg[128];
  printf("LOGtask%d start on CPU%d\n", running->pid, cpuid);
  while(1){ // task0 waits for message
    pid = recv(msg);
  }
}
```

```
int run_task()  // each CPU tries to run task from its own readyQ
{
   int cpuid = get_cpuid();
   while(1){
      slock(&readylock[cpuid]);
      if (readyQueue[cpuid] == 0){   // check own readyQueue
         sunlock(&readylock[cpuid]);
         asm("WFE");                 // wait for events/interrupts
      }
      else{
         sunlock(&readylock[cpuid]);
         ttswitch();                 // switch task on CPU
     }
  }
}
int APstart()
{
   int cpuid = get_cpuid();
   slock(&aplock);
   color = YELLOW;
   printf("CPU%d in APstart ... ", cpuid);
   config_int(29, cpuid);   // per CPU local timer
   ptimer_init();
   ptimer_start();
   ncpu++;
   running = &iproc[cpuid]; // run per CPU initial task
   printf("%d running on CPU%d\n", running->pid, cpuid);
   sunlock(&aplock);
   run_task();
}
int main()
{
   int cpuid = get_cpuid();
   enable_scu();
   fbuf_init();
   printf("Welcome to SMP_RTOS in ARM\n");
   sdc_init();
   kbd_init();
   uart_init();
   ptimer_init();
   ptimer_start();
   config_gic();
   printf("CPU0 initialize kernel\n");
   kernel_init();
   printf("CPU0 startup APs: ");
   int *APaddr = (int *)0x10000030;
   *APaddr = (int)apStart;
   send_sgi(0x0, 0x0F, 0x01);
   printf("CPU0 waits for APs ready\n");
   while(ncpu < 4);
   printf("CPU0 continue\n");
   kfork(int)logtask, 1, 0);     // P0 on CPU0 as loggin task
   kfork((int)INIT_task,  1, 0); // P1 on CPU0 as INIT   task
   run_task();
}
```

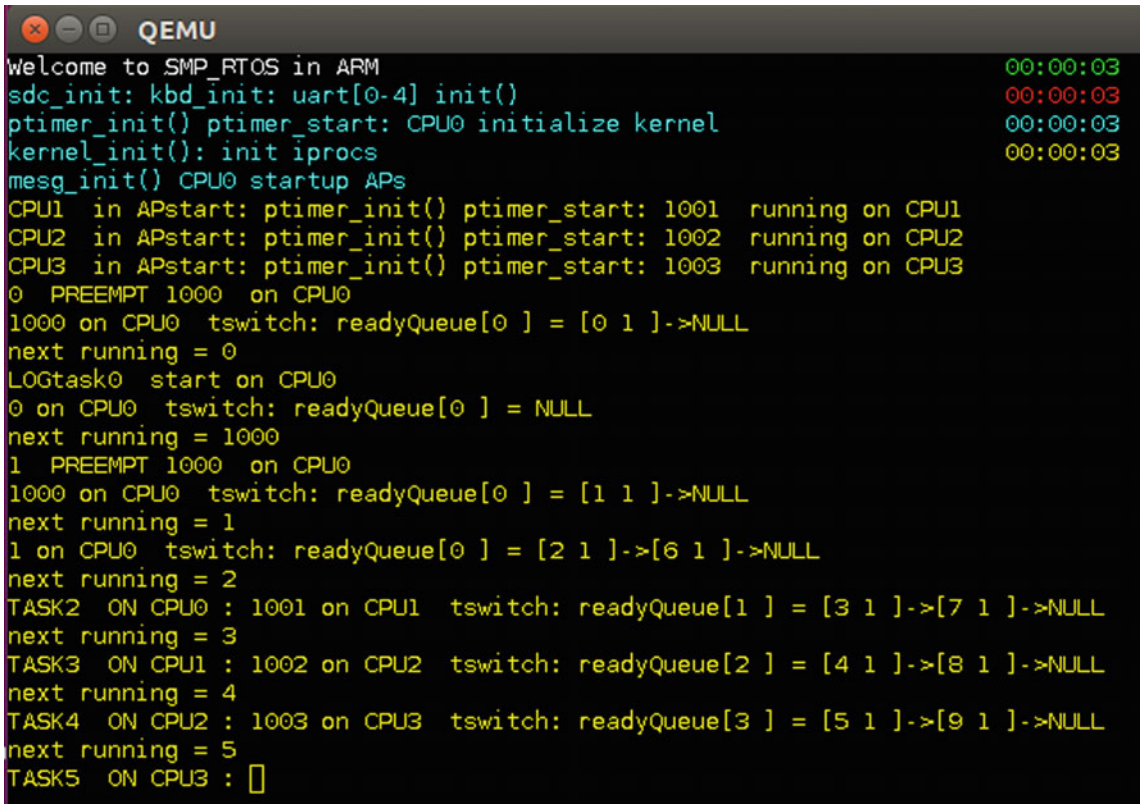


Fig. 10.5 Task management in SMP_RTOS

The object of the initial SMP_RTOS kernel is to demonstrate that the system can create and run tasks on different CPUs. When the system starts, CPU0 creates and runs the initial task P1000, which calls main(). In main(), P1000 initializes device drivers and the system kernel. Then it activates the secondary CPUs (APs). Each AP initializes itself and runs an initial task P1000+cpuid, which calls run_task(), trying to run tasks from its own readyQueue[cpuid]. After activating the APs, P1000 creates a logging task P0 and an INIT task P1 on CPU0. Then P1000 switches task to run the INIT task P1. P1 creates two children tasks on each CPU and waits for any child task to terminate. Each task executes a loop, in which it delays for some time to simulate task computation and calls ttswitch() to switch tasks. In the sample system, tasks run the loop continually. The reader may test the kwait() operation by modifying the taskCode() function to let tasks terminate. Figure 10.5 shows the sample outputs of running the program C10.4, which demonstrates task management in the SMP_RTOS kernel.

10.8.2 Adapt UP_RTOS to SMP

The simplest way to develop a RTOS for multiprocessors is to adapt the UP_RTOS for MP operations. In this approach, the system comprises multiple CPUs. Each CPU executes a set of tasks that are bound to the CPU and operate in the local environment of the CPU. The main advantage of this approach lies in its simplicity. It requires very little work to convert the UP_ROTS into a MP RTOS. The disadvantage is that the resulting system is only a Asymmetric MP system, not a SMP system. We illustrate such a system, denoted by MP_RTOS, by a sample program C10.5. The program consists of the following components.

(1). Ts.s file: same as in UP_RTOS, except the simplified irq_hanler, which supports task switch but no nested interrupts.

```
irq_handler:                // IRQ interrupts entry point
 sub   lr, lr, #4
 stmfd sp!, {r0-r12, lr}  // save all Umode regs in kstack
   mrs   r0, spsr
   stmfd sp!, {r0}
   bl  irq_chandler
```

```
// check return value: 0=nornal, 1=swflag was set => swtich task
   cmp r0, #0
   bgt doswitch
   ldmfd sp!, {r0}
   msr   spsr, r0
   ldmfd sp!, {r0-r12, pc}^
doswitch:
   msr cpsr, #0x93
   stmfd sp!, {r0-r3, lr}
   bl ttswitch
   msr cpsr, #0x93
   ldmfd sp!, {r0-r3, lr}
   msr cpsr, #0x92
   ldmfd sp!, {r0}
   msr   spsr, r0
   ldmfd sp!, {r0-r12, pc}^
```

(2). kernel.c file: same as in UP_RTOS for task management, except we define

```
   struct semaphore sem[NCPU];
```

and initialize each sem.value = 0. Each semaphore is associated with a CPU, which can only be accessed by tasks executing on the same CPU. If a task becomes blocked on a semaphore, it is unblocked by the CPU's local timer periodically.

```
void ptimer_handler()
{
   int cpuid = id = get_cpuid();
   struct ttt *tp = &tt[cpuid];
   slock(&plock);
   // update tick, ss, mm, hh: same as before
   if (tp->tick==0){  // every second: display a wall clock
      // display wall clock: same as before
      if ((tp->ss % 5)==0)  // every 5 seconds, activate a task
         V(&sem[cpuid]);    // V to unblock a waiting task
   }
   sunlock(&plock);
   ptimer_clearInterrupt(); // clear timer interrupt
}
```

(3). t.c file: Same as in UP_RTOS for task management, except for the following modifications. Instead creating task on different CPUs, the initial task of each CPU creates a task to execute INIT_task() on the same CPU. Each task creates another task to execute taskCode(), also on the same CPU. All the tasks have the same priority 1. Each task executes an infinite loop, in which it delays for some time to simulate task computation and then blocks itself on a semaphore associated with the CPU. The local timer of each CPU periodically calls V to unblock the tasks, allowing them to continue. Whenever a ready task is entered into the local scheduling queue of a CPU, it may preempt the current running task on the same CPU.

```
/*********** t.c file of MP_RTOS C10.5 ************/
int run_task(){ // same as before
int kdelay(int d){ // delay }

int taskCode()
{
  int cpuid = get_cpuid();
  while(1){
```

```
    printf("TASK%d ON CPU%d\n", running->pid, cpuid);
    kdelay(running->pid*10000);
    reset_sem(&sem[cpuid]);      // reset sem.value to 0
    P(&sem[cpuid]);
  }
}
int INIT_task()
{
  int cpuid = get_cpuid();
  kfork((int)taskCode, 1, cpuid);
  while(1){
    printf("task%d on CPU%d\n", running->pid, cpuid);
    kdelay(running->pid*10000);
    reset_sem(&sem[cpuid]);     // reset sem.value to 0
    P(&sem[cpuid]);
  }
}

int APstart()
{
   int cpuid = get_cpuid();
   slock(&aplock);
   printf("CPU%d in APstart: ", cpuid);
   config_int(29, cpuid); // need this per CPU
   ptimer_init(); ptimer_start();
   ncpu++;
   running = &iproc[cpuid];
   printf("%d running on CPU%d\n", running->pid, cpuid);
   sunlock(&aplock);
   kfork((int)INIT_task, 1, cpuid);
   run_task();
}
int main()
{
   // initialize device drivers, GIC and kernel: same as before
   kprintf("CPU0 continue\n");
   kfork((int)f1, 1, 0);
   run_task();
}
```

10.8.2.1 Demonstration of MP_RTOS

Figure 10.6 shows the outputs of running the MP_RTOS system.

10.8.3 Nested Interrupts in SMP_RTOS

In a UP RTOS kernel, only one CPU handles all the interrupts. To ensure fast responses to interrupts, it is vital to allow nested interrupts. In a SMP RTOS kernel, interrupts are usually routed to different CPUs to balance the interrupt processing load. Despite this, it is still necessary to allow nested interrupts. For instance, each CPU may use a local timer, which generates timer interrupts with high priority. Without nested interrupts, a low priority device driver may block timer interrupts, causing the timer to lose ticks. In this section, we shall show how to implement nested interrupts in the SMP_RTOS kernel by the sample program C10.6. The principle is the same as in UP_RTOS, except for the following minor difference.

All ARM MPcore processors use the GIC for interrupts control. In order to support nested interrupts, the interrupt handler must read the GIC's interrupt ID register to acknowledge the current interrupt before enabling interrupts. The GIC's interrupt ID register

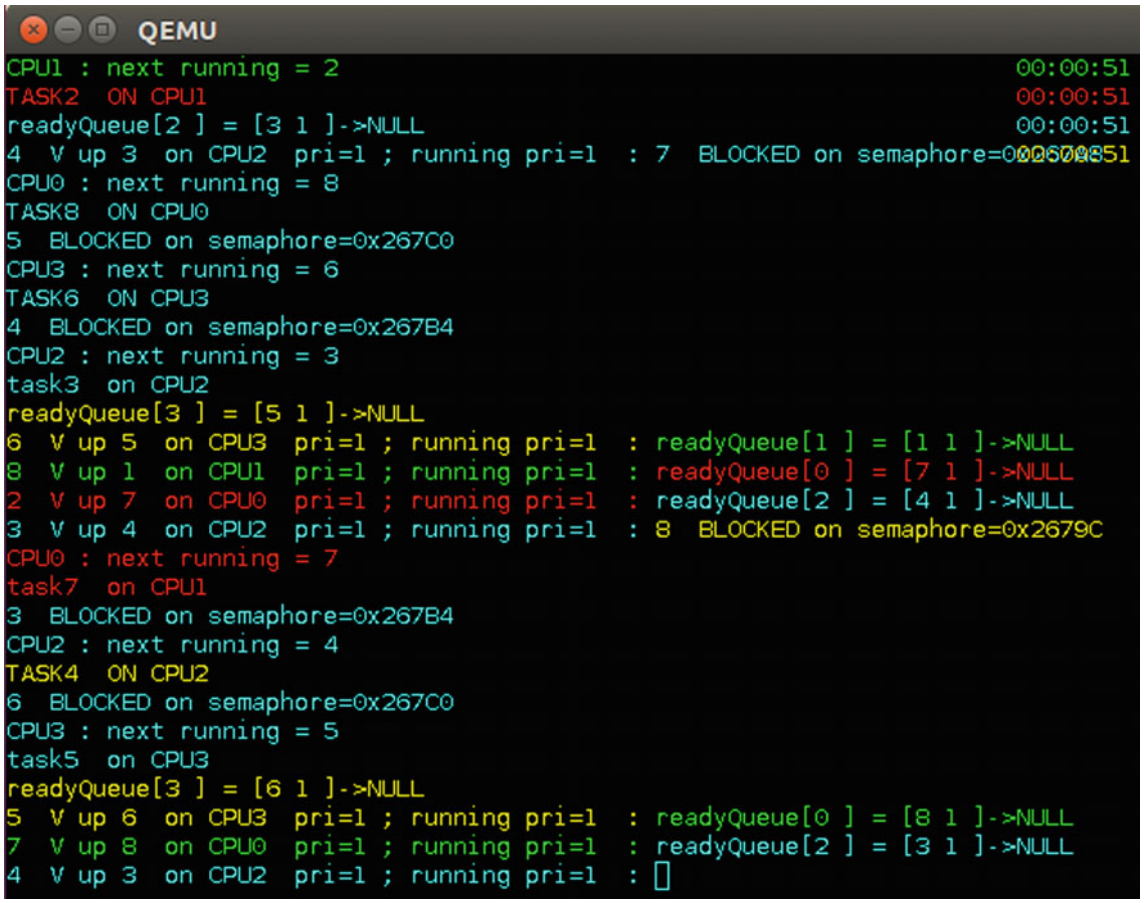


Fig. 10.6 Demonstration of MP_RTOS system

can only be read once. The interrupt handler must use the interrupt ID to invoke the irq_chandler(int intID) in C. For special SGI interrupts (0-15), they should be handled separately since they have high interrupt priorities and do not need any nesting.

```
   .set GICreg, 0x1F00010C  // GIC intID register
   .set GICeoi, 0x1F000110  // GIC EOI   register
irq_handler:
   sub   lr, lr, #4
   stmfd sp!, {r0-r12, lr}
   mrs   r0, spsr
   stmfd sp!, {r0}         // push SPSR
// read GICreg to get intID and ACK current interrupt
   ldr   r1, =GICreg
   ldr   r0, [r1]
// handle SGI intIDs direcly
   cmp   r0, #15           // SGI interrupts
   bgt   nesting
   ldr   r1, =GICeoi
   str   r0, [r1]          // issue EOI
   b     return
nesting:
// to SVC mode
   msr   cpsr, #0x93       // SVC mode with IRQ off
   stmfd sp!, {r0-r3, lr}
   msr   cpsr, #0x13       // SVC mode with IRQ interrupts on
   bl    irq_chandler      // call irq_chandler(intID)
```

```
   msr   cpsr, #0x93       // SVC mode with IRQ off
   ldmfd sp!, {r0-r3, lr}  //
   msr   cpsr, #0x92       // IRQ mode with interrupts off
return:
   ldmfd sp!, {r0}
   msr   spsr, r0
   ldmfd sp!, {r0-r12, pc}^
int irq_chandler(int intID)  // called with interrupt ID
{
  int cpuid = get_cpuid();
  int cpsr = get_cpsr() & 0x9F; // CPSR with |IRQ|mode| mask

  intnest[cpuid]++;          // inc intsest by 1
  if (intID != 29)
     printf("IRQ%d on CPU%d in mode=%x\n", intID, cpuid, cpsr);
  if (intID == 29){          // local timer of CPU
      ptimer_handler();
  }
  if (intID == 44){          // UART0 interrpt
      uart_handler(0);
  }
  if (intID == 49){          // SDC interrupt
      sdc_handler();
  }
  if (intID == 52){          // KBD interrupt
      kbd_handler();
  }
  *(int *)(GIC_BASE + 0x110) = intID; // issue EOF
  intnest[cpuid]--;          // dec intnest by 1
}
```

Figure 10.7 shows the outputs of running the program C10.6, which demonstrates SMP_RTOS with nested interrupts. As the figure shows, all interrupts are handled in SVC mode.

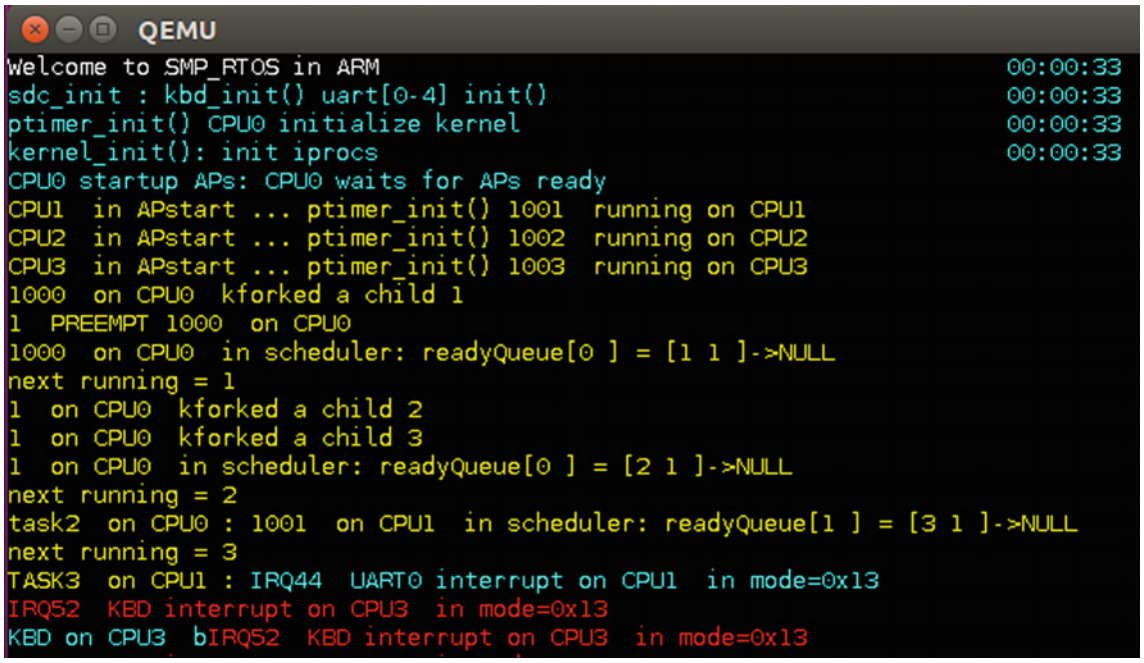


Fig. 10.7 Nested interrupts in SMP_RTOS

10.8.4 Preemptive Task Scheduling in SMP_RTOS

In order to meet task deadlines, every ROTS must support preemptive task scheduling to ensure high priority tasks can be executed as soon as possible. In this section, we shall show how to implement preemptive task scheduling in the SMP_RTOS kernel and demonstrate task preemption by the sample program C10.7. The task preemption problem can be classified into two main cases.

Case 1: When a running task makes another task of higher priority ready to run, e.g. when creating a new task with a higher priority or unblocking a task with a higher priority.

Case 2: When an interrupt handler makes a task runnable, e.g. when unblocking a task with a higher priority or when the current running task has exhausted its time quantum in round-robin scheduling by timeslice.

In a UP kernel, implementing task preemption is straightforward. For the first case, whenever a task makes another task ready to run, it simply checks whether the new task has a higher priority. If so, it invokes task switching to preempt itself immediately. For the second case, the situation is only slightly more complex. With nested interrupts, the interrupt handler must check whether all nested interrupts processing have ended. If so, it can invoke task switching to preempt the current running task. Otherwise, it defers task switching until the end of nested interrupts processing.

In SMP, the situation is very different. In a SMP kernel, an execution entity, i.e. a task or an interrupt handler, executing on one CPU may make another task ready to run on a different CPU. If so, it must inform the other CPU to reschedule tasks for possible task preemption. In this case, it must issue a SGI, e.g. with interrupt ID=2, to the other CPU. Responding to a SGI_2 interrupt, the target CPU can execute a SGI_2 interrupt handler to reschedule tasks. We illustrate SGI based task switch by the sample program C10.7. For the sake of brevity, we only show the relevant code segments.

10.8.5 Task Switch by SGI

```
/************ irq_handler() in ts.s file *************/
irq_handler:                 // IRQ interrupts entry point
   sub   lr, lr, #4
   stmfd sp!, {r0-r12, lr}   // save all Umode regs in kstack
   bl     irq_chandler       // call irq_handler() in C in svc.c file
// return value=1 means swflag is set => swtich task
   cmp   r0, #0
   bne   doswitch
   ldmfd sp!, {r0-r12, pc}^ // return
doswitch:
   msr   cpsr, #0x93
   stmfd sp!, {r0-r3, lr}
   bl    ttswitch           // switch task in SVC mode
   ldmfd sp!, {r0-r3, lr}
   msr   cpsr, #0x92        // return by context in IRQ stack
   ldmfd sp!, {r0-r12, pc}^
/************** t.c file ****************************/
int send_sgi(int ID, int targetCPU, int filter)
{
  int target = (1 << targetCPU); // set CPU ID bit
  if (targetCPU > 3) // > 3 means to all CPUs
     target = 0xF;   // turn on all 4 CPU ID bits
  int *sgi_reg = (int *)(GIC_BASE + 0x1F00);
  *sgi_reg = (filter << 24) | (target<<16) | ID;
}
int SGI2_handler()
{
  int cpuid = get_cpuid();
  printf("%d on CPU%d got SGI_2: set swflag\n", running->pid, cpuid);
```

```
  swflag[cpuid] = 1;
}
int irq_chandler()
{
  int intID = *(int *)(GIC_BASE + 0x10C);
  int cpuid = get_cpuid();
  if (intID == 2){
     SGI2_handler();
  }
  // other intID cases: same as before
  *(int *)(GIC_BASE + 0x110) = (cpuid<<10)|intID; // issue EOI
  if (swflag[cpuid]){ // return 1 for task switch in irq_handler
     swflag[cpuid] = 0;
     return 1;
  }
  return 0;
}
int kdelay(int d){ // delay to simulate task computation }
int f2()
{
  int cpuid = get_cpu_id();
  while(1){
    printf("TASK%d RUNNING ON CPU%d: ", running->pid, cpuid);
    kdelay(running->pid*10000);  // simulate task compute time
  }
}
int f1()
{
  int cpuid = get_cpu_id();
  while(1){
    printf("task%d running on CPU%d: ", running->pid, cpuid);
    kdelay(running->pid*100000); // simulate task compute time
  }
}
int main()
{
  // start up APs: same as before
  printf("CPU0 continue\n");
  kfork((int)f1, 1, 1); // create tasks on CPU1
  kfork((int)f2, 1, 1);
  kfork((int)f1, 1, 2); // create tasks on CPU2
  kfork((int)f2, 1, 2);
  while(1){
    printf("input a line: ");
    kgetline(line); printf("\n");
    send_sgi(2, 1, 0);  // SGI_2 to CPU1
    send_sgi(2, 2, 0);  // SGI_2 to CPU2
  }
}
```

In the sample program C10.7, after starting up the APs, the main program executing on CPU0 creates two sets of tasks on both CPU1 and CPU2, all of the same priority 1. All the tasks execute an infinite loop. Without external inputs, each CPU would continue to execute the same task forever. After creating tasks on different CPUs, the main program prompts for an input line from the keyboard. Then it sends a SGI with intID=2 to the other CPUs, causing them to execute the SGI2_handler. In the SGI2_handler, each CPU turns on its switch task flag and returns a 1. In the irq_handler code, it checks

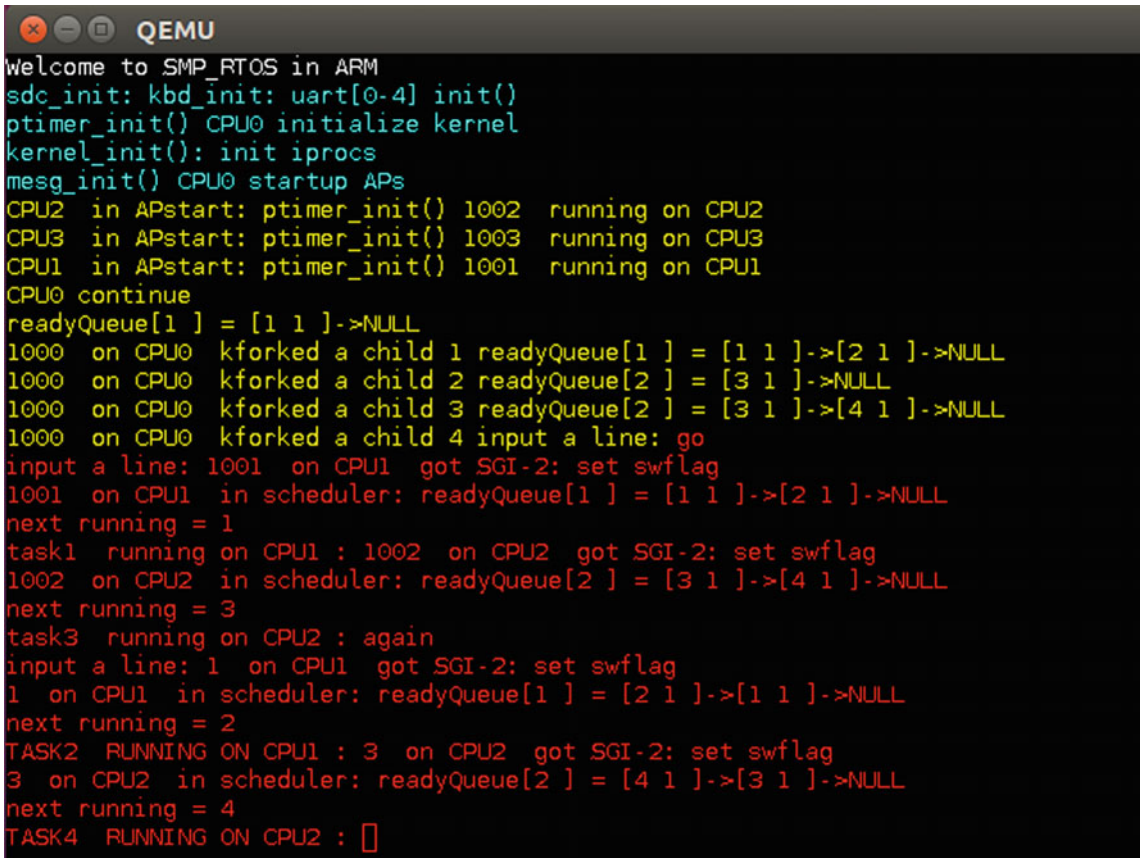


Fig. 10.8 Task switch by SGI in SMP_RTOS

the return value. If the return value is nonzero, indicating a need for task switch, it changes to SVC mode and calls ttswitch() to switches task. When the switched out task resumes, it returns to the original interrupted point by the saved context in IRQ stack. Figure 10.8 shows the outputs of task switching by SGI. As the figure shows, each input line sends SGI-2 to both CPU1 and CPU2, causing them to switch tasks.

10.8.6 Timesliced Task Scheduling in SMP_RTOS

In a RTOS, tasks may have the same priority. Instead of waiting for such tasks to give up CPU voluntarily, it may be desirable or even necessary to schedule them by round-robin with timeslice. In this scheme, when a task is scheduled to run, it is given a timeslice as the maximum amount of time the task is allowed to run. In the timer interrupt handler, it decrements the running task's timeslice periodically. When the running task's timeslice reaches zero, it is preempted to run another task. We demonstrate task scheduling by timeslice in the SMP_RTOS kennel by an example. The example program, C10.8, is the same as C10.7 except for the following modifications.

```
/********** kernel.c file of C10.8 ***********/
int scheduler()
{
  int pid; PROC *old=running;
  int cpuid = get_cpuid();
  if (running->pid < 1000){ // regular tasks only
    if (running->status==READY)
       enqueue(&readyQueue[cpuid], running);
```

```
  }
  running = dequeue(&readyQueue[cpuid]);
  if (running == 0)         // if CPU's readyQueue is empty
     running = &iproc[cpuid]; // run the idle task
  running->timeslice = 4;     // 4 seconds timeslice
  swflag[cpuid] = 0;
  sunlock(&readylock[cpuid]);
}
/********* ptimer.c file of C10.8 ***************/
void ptimer_handler()         // local timer handler
{
   int cpuid = get_cpuid();
   // update tick, display wall clock: same as before
   ptimer_clearInterrupt();   // clear timer interrupt
   if (tp->tick == 0){        // per second
     if (running->pid < 1000){// only regular tasks
        running->timeslice--;
        printf("%dtime=%d ", running->pid, running->timeslice);
        if (running->timeslice <= 0){
           printf("%d on CPU%d time up\n", running->pid, cpuid);
           swflag[cpuid] = 1; // set swflag o switch task at end of IRQ
       }
     }
   }
}
/************* t.c file of C10.8 ***************/
int f3()
{
  int cpuid = get_cpuid();
  while(1){
    printf("TASK%d ON CPU%d\n", running->pid, cpuid);
    kdelay(running->pid * 100000); // simulate computation
  }
}
int f2()
{
  int cpuid = get_cpuid();
  while(1){
    printf("task%d on cpu%d\n", running->pid, cpuid);
    kdelay(running->pid * 100000); // simulate computation
  }
}
int f1()
{
  int pid, status;
  kfork((int)f2, 1, 1);    // create task on CPU1
  kfork((int)f3, 1, 1);    // create task on CPU1
  while(1){
     pid = kwait(&status); // P1 waits for ZOMBIE child
  }
}
int run_task(){ // same as before }
int main()
{
   // start APs" same as before
```

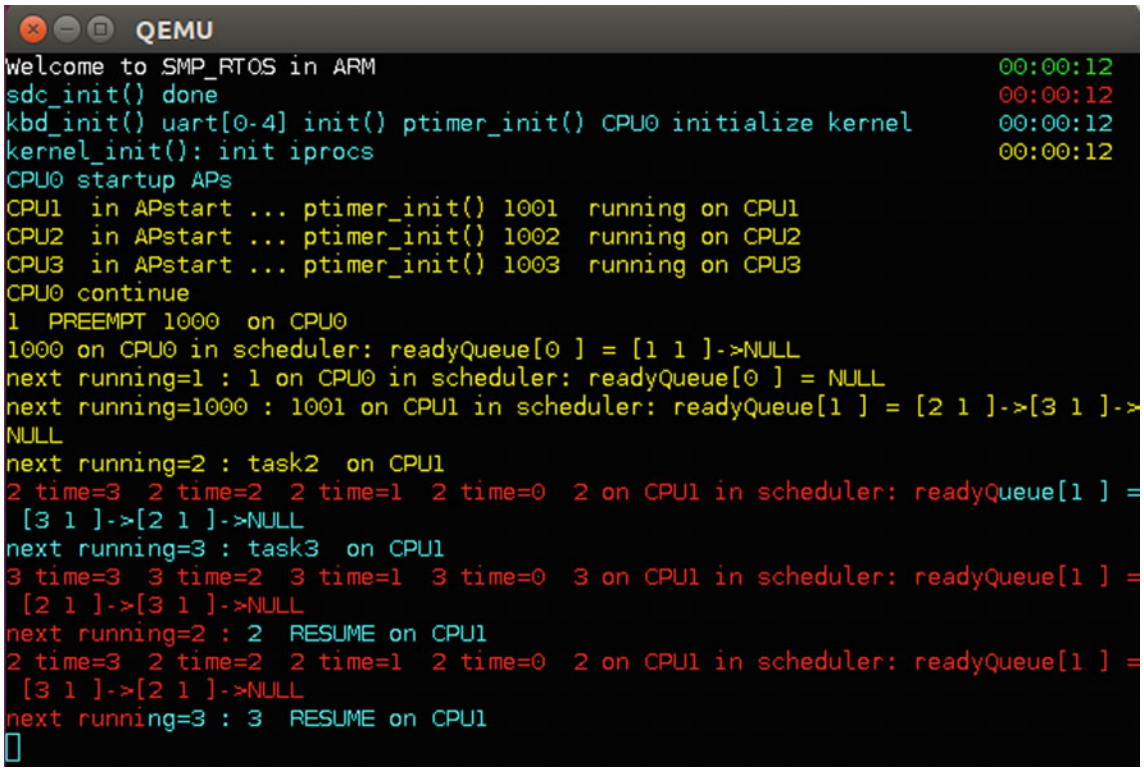


Fig. 10.9 Timesliced task scheduling in SMP_RTOS

```
    printf("CPU0 continue\n");
    kfork((int)f1, 1, 0);  // create task 1 on CPU0
    run_task();
}
```

In the sample program C10.8, the initial task of CPU0 creates task1 to execute f1() on CPU0. Task1 creates task2 and task3 on CPU1 with the same priority. Then task1 waits for any child task to terminate. Since both task2 and task3 have the same priority, without timeslicing, CPU1 would continue to run task2 forever. With timeslicing, task2 and task3 would take turn to run, each runs for a timeslice=4 seconds. Figure 10.9 shows the outputs of running the C10.8 program, which demonstrates timesliced task scheduling.

10.8.7 Preemptive Task Scheduling in SMP_RTOS

Based on the above discussions, we now show the implementation of preemptive task scheduling in the SMP_ROTS kernel.

10.8.7.1 Reschedule Function for Task Preemption

```
int reschedule(PROC *p)
{
  int cpuid = get_cpuid();  // executing CPU
  int targetCPU = p->cpuid; // target CPU
  if (cpuid == targetCPU){  // if same CPU
     if (readyQueue[cpuid]->priority > running->priority){
        if (intnest[cpuid]==0){ // PREEMPTION immediately
           ttswitch();
```

```
        }
        else{                    // defer until end of nested IRQs
           swflag[cpuid] = 1;
        }
     }
  }
  else{                     // different CPU
     send_sgi(intID=2, CPUID=targetCPU, filter=0);
}
}
```

10.8.7.2 Task Creation with Preemption

```
int kfork(int func, int priority, int cpu)
{
   int cpuid = get_cpuid();  // executing CPU
   // create a new task p, enter into readyQueue[cpuid] as before
   reschedule(p);            // invoke resechedule
   return p->pid;
}
```

10.8.7.3 Semaphore with Preemption

```
int V(struct semaphore *s)
{
  PROC *p = 0;
  int SR = int_off();
  slock(&s->lock);
  s->value++;
  if (s->value <= 0){
     p = dequeue(&s->queue);
     p->status = READY;
     slock(&readylock[p->cpuid]);
      enqueue(&readyQueue[p->cpuid], p);
     sunlock(&readylock[p->cpuid]);
  }
  sunlock(&s->lock);
  int_on(SR);
  if (p)              // if has unblocked a waiter
     reschedule();
}
```

10.8.7.4 Mutex with Preemption

```
int mutex_unlock(MUTEX *s)
{
  PROC *p = 0;
  int SR = int_off();
  slock(&s->lock)     // acquire spinlock
  // ASSUME: mutex has waiters: unblock one as new owner
  p = dequeue(&s->queue);
```

```
  p->status = READY;
  s->owner = p;
  slock(&readylock[p->cpuid]);
    enqueue(&readyQueue[p->cpuid], p);
  sunlock(&readylock[p->cpuid]);
  sunlock(&s->lock);  // release spinlock
  int_on(SR);
  if (p)              // if has unblocked a waiter
     reschedule();
}
```

10.8.7.5 Nested IRQ interrupts with Preemption

```
int SGI2_handler()
{
   int cpuid = get_cpuid();
   PROC *p = readyQueue[cpuid];
   if (p && p->priority > running->priority){
        swflag[cpuid] = 1;  // set task switch flag
   }
}
int irq_chandler(int intID) // called with interrupt ID
{
  int cpuid = get_cpuid();
  intnest[cpuid]++;         // inc intsest count by 1
  if (intID == 2){          // SGI_2 interrupt
     SGI2_handler();
  }
  // other IRQ interrupt handlers: same as before
  *(int *)(GIC_BASE + 0x110) = (cpuID<<10)|intID; // issue EOI
  intnest[cpuid]--;         // dec intnest count by 1
}
int checkIRQnesting()// return 1 if end of IRQ nesting && swflag is set
{
   int cpuid = get_cpuid();
   if (intnest[cpuid] == 0 && swflag[cpuid])
      return 1;
   return 0;
}
/*** irq_handler with nested interrupts AND SGI_2 ****/
    .set GICreg, 0x1F00010C  // GIC intID register
    .set GICeoi, 0x1F000110  // GIC EOI   register
irq_handler:
   sub   lr, lr, #4
   stmfd sp!, {r0-r12, lr}
   mrs   r0, spsr
   stmfd sp!, {r0}  // push SPSR
// read GICreg to get intID
   ldr   r1, =GICreg
   ldr   r0, [r1]
// handle SGI intID=0 direcly
   cmp   r0, #0            // SGI 0 used during startup
   bgt   nesting
   ldr   r1, =GICeoi
```

```
   str   r0, [r1]          // issue EOI
   b     return
nesting:                   // to SVC mode
   msr   cpsr, #0x93       // SVC mode with IRQ off
   stmfd sp!, {r0-r3, lr}
   msr   cpsr, #0x13       // SVC mode with interrupts on
   bl    irq_chandler      // irq_chandler return 1: task switch
   msr   cpsr, #0x93       // SVC mode with IRQ off
   bl    checkIRQnesting   // return 1 if OK to switch task
   cmp   r0, #0
   bgt   doswitch
   ldmfd sp!, {r0-r3, lr}
   b     return
doswitch:
   stmfd sp!, {r0-r3, lr}
   bl    ttswitch          // switch task is SVC mode
   ldmfd sp!, {r0-r3, lr}
return:
   msr   cpsr, #0x92       // IRQ mode with interrupts off
   ldmfd sp!, {r0}
   msr   spsr, r0
   ldmfd sp!, {r0-r12, pc}^
/************ end of irq_handler ******************/
```

10.8.8 Priority Inheritance

In the SMP_RTOS kernel, spinlocks, mutexes and semaphores are used for process synchronization. Spinlocks are non-blocking, i.e. they do not cause context switch, so there is no need for priority inheritance. Mutexes and semaphores are blocking, which may cause task switch, so they must support priority inheritance. Implementation of priority inheritance in SMP is similar to that of UP_RTOS, except for the following differences. When a task is about to become blocked on a mutex or semaphore, it may have to boost the priority of the mutex or semaphore owner, which may be running on a different CPU. If the mutex or semaphore owner and the current task are on the same CPU, the situation is the same as in UP_RTOS. Otherwise, the current task must send a SGI to the CPU of the mutex or semaphore owner, causing it to adjust task priority and reschedule tasks on the target CPU. The scheme can be implemented as follows. When a task is about to become blocked on a mutex or semaphore:

(1). Write [pid, priority] to a dedicated shared memory, where pid is the PID of the mutex/semaphore owner and priority is the new priority.
(2). Issue SGI_3 to the CPU of the mutex/semaphore owner task
(3). SGI_3_handler on the target CPU:
Get [pid, priority] from shared memory;
Raise target task's priority and reschedule task

The following shows the implementation of priority inheritance for mutexes in SMP_ROTS. The implementation of priority inheritance for semaphores is similar.

```
volatile int PID[NCPU];  // shared memory for SGI_3 handlers
int mutex_lock(MUTEX *s)
{
  PROC *p;
  int cpuid = get_cpuid();
  int SR = int_off();
```

```
  slock(&s->lock);   // spinlock
  if (s->state==UNLOCKED{  // mutex is in unlocked state
     s->state = LOCKED;
     s->owner = running;
     sunlock(&s->lock);
     int_on(SR);
     return 1;
  }
  /************** mutex already locked ******************/
  if (s->owner == running){ // already locked by this task
      printf("mutex already locked by you!\n");
      sunlock(&s->lock);
      int_on(SR);
      return 1;
  }
  printf("TASK%d BLOCK ON MUTEX: ", running->pid);
  running->status = BLOCK;
  enqueue(&s->queue, running);
  // boost owner priority to running's priority
  if (running->priority > s->owner->priority){
     if (s->owner->cpuid == cpuid){ // same CPU
        s->owner->priority = running->priority; // boost owner priority
     }
     else{ // not the same CPU
        PID[s->owner-cpuid] = s->owner->pid; // owner's pid
        send_sgi(3, s->owner->cpuid, 0); // send SGI to recehdule task
     }
  }
  sunlock(&s->lock);
  tswitch();  // switch task
  int_on(SR);
  return 1;
}
int mutex_unlock(MUTEX *s)
{
  PROC *p;
  int cpuid = get_cpuid();
  int SR = int_off();
  slock(&s->lock);         // spinlock
  if (s->state==UNLOCKED || s->owner != running){
     printf("%d mutex_unlock error\n", running->pid);
     sunlock(&s->lock);
     int_on(SR);
     return 0;
  }
  // caller is owner and mutex was locked
  if (s->queue == 0){      // mutex has no waiter
     s->state = UNLOCKED;  // clear locked state
     s->owner = 0;         // clear owner
     running->priority = running->rpriority;
  }
  else{  // mutex has waiters: unblock one as new owner
     p = dequeue(&s->queue);
     p->status = READY;
     s->owner = p;
```

```
      slock(&readylock[p->cpuid]);
        enqueue(&readyQueue[p->cpuid], p);
      sunlock(&readylock[p->cpuid]);
      running->priority = running->realPriority; // restore priority
      if (p->cpuid == cpuid){  // same CPU
         if (p->priority > running->priority){
            if (intnest==0)    // not inside IRQ handler
               ttswitch();     // preemption now
            else{
               swflag[cpuid] = 1; // defer preemption until IRQ end
      }
      else // caller and p on different CPUs => SGI to p's CPU
          send_sgi(2, p->cpuid, 0); // send SGI to reschedule task
   }
   sunlock(&s->lock);
   int_on(SR);
   return 1;
}
```

10.8.9 Demonstration of the SMP_RTOS System

The SMP_RTOS system integrates all the functionalities discussed above to provide the system with the following capabilities.

(1). A SMP kernel for task management. Tasks are distributed to different CPUs to improve concurrency in SMP.
(2). Preemptive task scheduling by priority and timeslice
(3). Support nested interrupts
(4). Priority inheritance in mutex and semaphore operations
(5). Task communication by shared memory and messages
(6). Inter-processor synchronization by SGI
(7). A logging task for recording task activities to a SDC

We demonstrate the SMP_RTOS system by the sample program C10.9 For the sake of brevity, we only show the t.c file of the system.

```
/******** t.c file of SMP_RTOS demo. system C10.9 ********/
#include "type.h"         // system types and constants
#include "string.c"
struct semaphore s0, s1;  // for task synchroonization
int irq_stack[4][1024], abt_stack[4][1024], und_stack[4][1024];
#include "queue.c"        // enqueue/dequeue functions
#include "pv.c"           // semaphores with priority inheritance
#include "mutex.c"        // mutex with priority inheritance
#include "uart.c"         // UARTs driver
#include "kbd.c"          // KBD driver
#include "ptimer.c"       // local timer driver
#include "vid.c"          // LCD display driver
#include "exceptions.c"   // exceptions handlers
#include "kernel.c"       // kernel init and task schedulers
#include "wait.c"         // kexit() and kwait() functions
#include "fork.c"         // kfork() function
#include "sdc.c"          // SDC driver
```

```
#include "message.c"      // message passing

int copy_vectors(){ // same as before }
int config_gic()  { // same as before }
int config_int(int N, int targetCPU){// same as before }
int irq_chandler(){ // same as before }
int APstart()     { // same as before }

int SGI_handler()   // sgi handler
{
  int cpuid = get_cpuid();
  printf("%d on CPU%d got SGI-2: ", running->pid, cpuid);
  if (readyQueue[cpuid]){ // if readQ nonempty
     printf("set swflag\n");
     swflag[cpuid] = 1;
  }
  else{ // empty readyQ
    printf("NO ACTION\n");
    swflag[cpuid] = 0;
  }
}
int klog(char *line){ send(line, 2) }
char *logmsg = "log information";
int task5()
{
  printf("task%d running: ", running->pid);
  klog("start");
  mutex_lock(mp);
     printf("task%d inside CR\n",  running->pid);
     klog("inside CR");
  mutex_unlock(mp);
  klog("terminate");
  kexit(0);
}
int task4()
{
  int cpuid = get_cpuid();
  while(1){
    printf("%d on CPU%d: ", running->pid, cpuid);
    kfork((int)task5, 5, 2);  // creat task5 task on CPU2
    kdelay(running->pid*10000);
    kexit(0);
  }
}
int task3()
{
  int cpuid = get_cpuid();
  while(1){
    printf("%d on CPU%d: ", running->pid, cpuid);
    klog(logmsg);
    mutex_lock(mp);  // lock mutex
      pid = kfork((int)task4, 4, 2); // create task4 on CPU2
      kdelay(running->pid*10000);
    mutex_unlock(mp);
```

```
    pid = kwait(&status);
  }
}
int task2()  // log task
{
   int r, blk;
   char line[1024];
   printf("log task %d start\n", running->pid);
   blk = 0;   // start write to block 0 of SDC
   while(1){
     printf("LOGtask%d: receiving\n", running->pid);
     r = recv(line);         // receive a log line
     put_block(blk, line);   // write log to SDC (1KB) block
     blk++;
     uprintf("%s", line);    // display log to UART0 on-line
     printf("log: ");
   }
}
int task1()
{
  int status, pid, p3;
  int cpuid = get_cpuid();
  fs_init();
  kfork((int)task2, 2, 1);
  V(&s0); // start up logtask
  kfork((int)task3, 3, 2);
  while(1)
     pid = kwait(&status);
}
int run_task(){ // same as before }

int main()
{
   int cpuid = get_cpuid();
   enable_scu();
   fbuf_init();
   kprintf("Welcome to SMP_RTOS in ARM\n");
   sdc_init();
   kbd_init();
   uart_init();
   ptimer_init();
   ptimer_start();
   msg_init();
   config_gic();
   kprintf("CPU0 initialize kernel\n");
   kernel_init();
   mp = mutex_create();
   s0.lock = s1.lock  = s0.value = s1.value = 0;
   s0.queue = s1.queue = = 0;
   printf("CPU0 startup APs: ");
   int *APaddr = (int *)0x10000030;
   *APaddr = (int)apStart;
   send_sgi(0x0, 0x0F, 0x01);
```

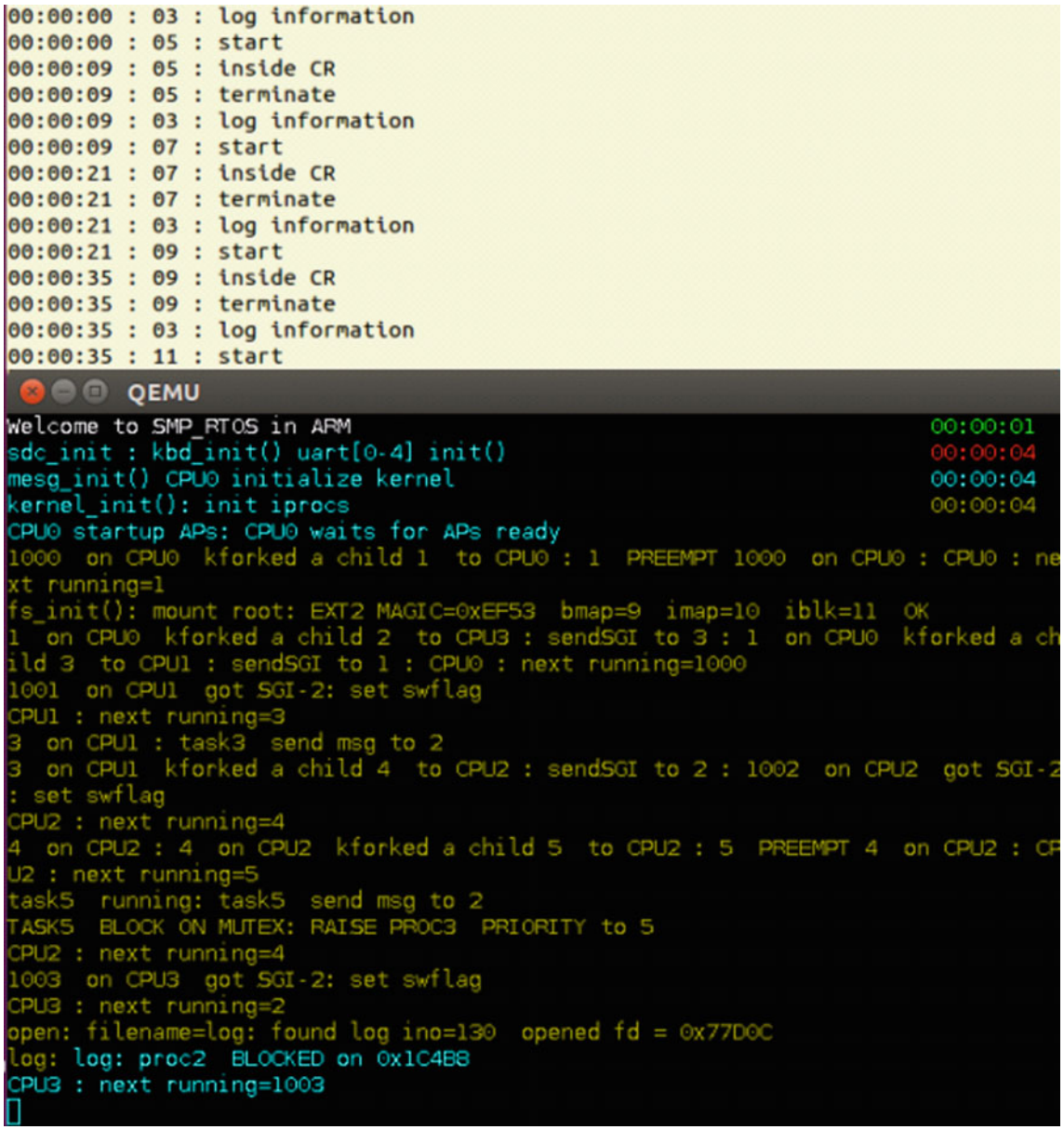


Fig. 10.10 Demonstration of SMP_RTOS

```
    printf("CPU0 waits for APs ready\n");
    while(ncpu < 4);
    kfork((int)task1, 1, 0);
    run_task();
}
```

Figure 10.10 shows running the SMP_RTOS system. The top part of Fig. 10.10 shows the on-line log information generated by the logging task. The bottom part of Fig. 10.10 shows the startup screen of SMP_RTOS.

In the main() function of the SMP_RTOS system, the initial process on CPU0 creates a task P1, which behaves as the INIT process. P1 creates task2 as the logging task on CPU1, and task3 on CPU2. Then P1 executes a loop to wait for any ZOMBIE child. Task3 first locks up a mutex. Then it creates task4, which creates task5, all on CPU2 but with different priorities. When task5 runs, it tries to acquire the same mutex lock, which is still held by task3. In this case, task5 will be blocked on the mutex but it raises the effective priority of task3 to 5, which demonstrates priority inheritance. In addition, the system also shows task preemption and inter-processor synchronization by SGIs. All the tasks may send log information as

messages to the logging task, which invokes the SDC driver directly to write the log information to a (1KB) block of the SDC. It also writes the log information to a UART port to display it on-line. Variation to the logging scheme is listed as a programming project in the Problems section.

10.9 Summary

This chapter covers embedded real-time operating systems (RTOS). It introduces the concepts and requirements of real-time systems. It covers the various kinds of task scheduling algorithms in RTOS, which include RMS, EDF and DMS. It explains the problem of priority inversion due to preemptive task scheduling and shows how to handle priority inversion and task preemption. It includes case studies of several popular real-time OS and presents a set of general guidelines for RTOS design. It shows the design and implementation of a UP_RTOS for uniprocessor (UP) systems. Then it extends the UP_RTOS to a SMP_RTOS, which supports nested interrupts, preemptive task scheduling, priority inheritance and inter-processor synchronization by SGI.

List of Sample Programs

C10.1. UP_RTOS with periodic tasks and round-robin scheduling
C10.2. UP_RTOS with periodic tasks and preemptive scheduling
C10.3. UP_RTOS with dynamic tasks
C10.4. SMP_RTOS kernel for task management
C10.5. MP_RTOS, a localized SMP system.
C10.6. Nested Interrupts in SMP_RTOS
C10.7. Task Preemption by SGI in SMP_RTOS
C10.8. Timesliced task scheduling in SMP_RTOS
C10.9. Demonstration of SMP_RTOS

Problems

1. In the UP_RTOS and SMP_RTOS systems, the logging task saves log information to a SDC directly, bypassing the file system. Modify the programs C10.1 and C10.8 to let the logging task save log information to a file in an (EXT2/3) file system on a SDC. Discuss the advantages and disadvantages of using log files.
2. In the UP_RTOS kernel, priority inheritance in only one-level. Implement priority inherence in a chain of nested requests for mutex or semaphore locks.
3. For the example UP_ROTS system C10.3, perform a response time analysis to determine whether the tasks will meet their deadlines. Verify whether the task can meet their deadlines empirically.
4. In the MP_RTOS system, tasks are bound to separate CPUs for parallel processing. Devise a way to support task migration, which allows tasks to be distributed to different CPUs to balance the processing load.
5. In the SMP_RTOS system, tasks running on different CPUs may interfere with one another since they all execute in the same address space. Use MMU with non-uniform VA to PA mapping to provide each CPU with a separate VA space.
6. In a SMP RTOS, interrupts may be handled directly by ISRs or by pseudo-tasks. Design and implement RTOS systems which employ these interrupts processing schemes and compare their performances.

References

Audsley, N.C: "Deadline Monotonic Scheduling", Department of Computer Science, University of York, 1990
Audsley, N. C, Burns, A., Richardson, M. F., Tindell, K., Wellings, A. J.: "Applying new scheduling theory to static priority pre-emptive scheduling. Software Engineering Journal, 8(5):284–292, 1993.
Buttazzo, G. C.: Counter RMS claims Rate Monotonic vs. EDF: Judgment Day, Real-Time Systems, 29, 5–26, 2005
Dietrich, S., Walker, D., "The evolution of Real-Time Linux", http://www.cse.nd.edu/courses/cse60463/www/amatta2.pdf, 2015
DNX: DNX RTOS, http://www.dnx-rtos.org, 2015
FreeROTS: FreeRTOS, http://www.freertos.org, 2016

Index

A
Abstract Data Type (ADT), 142
Adapt UP Algorithms to SMP, 395
Android, 1, 265
Application Processors (APs), 329
Arithmetic operations, 12
ARM926EJ-S, 7
ARM architecture, 2, 3
ARM assembly programming, 17
ARM Cortex-A9 MPcore processor, 331
ARM Cortex-A15 MPcore processor, 331
ARM CPU registers, 8
ARM exceptions, 47
ARM instructions, 2, 3
ARM LDREX/STREX instructions, 2, 350
ARM memory barriers, 351
ARM Memory Management Unit (MMU), 3, 193
ARM MPcore, 2, 4
ARM MPcore architecture, 2
Arm-none-eabi toolchain, 16
ARM PL011 UART Interface, 67
ARM PL110 Color LCD controller, 33
ARM PL190/192 Interrupt Controller, 52
ARM PL190 Vectored Interrupts Controller, 84
ARM processor modes, 8, 47
ARM programming, 17
ARM SB804 dual timer modules, 56
ARM system emulators, 16
ARM toolchain, 2, 16
ARM Versatilepb, 3, 16
ARM virtual machines, 2, 3
ASCII code, 37
Asymmetric MP (ASMP), 329, 331
Asynchronous events, 98
Asynchronous message passing, 148
Auxiliary Control Register (ACTL), 331

B
Barrel shifter, 12
Base register, 14
Baud rate, 29
Binary data sections, 34
Binary executable, 18
Binary semaphore, 134
BIOS, 329
Block data transfer, 14
BMP file format, 34
Boot kernel image from SDC, 253
Boot Processor (BSP), 329
Branch instructions, 11
Broken pipe error, 143

C
Cache coherence, 329
chdir-getcwd-stat, 307
Clear device interrupt request, 55
Command line parameters, 222
Comparison operations, 12
Conditional locking, 361
Conditional mutex, 361
Conditional semaphore, 362
Conditional spinlock, 361
Condition flags, 10
Condition variables, 141
Context switching, 116
Coprocessor instructions, 15
Cortex A9-MPcore, 7
Cortex-A series, 7
Cortex-M series, 7
Cortex-R series, 7
CPU Interface Registers, 332
Critical region, 67, 293, 349
Critical section, 67
Cross compiler, 16
C startup code, 226
Current Program Status Register (CPSR), 9

D
Data movement operations, 13
Data Memory Barrier (DMB), 351
Data Synchronization Barrier (DSB), 351
Deadline analysis, 425, 432
Deadline-inversion, 403
Deadline-Monotonic Scheduling (DMS), 403
Deadlock avoidance, 137
Deadlock detection and recovery, 137
Deadlock prevention, 137
Deadlock prevention in parallel algorithms, 394
Deadlock prevention in SMP, 394
Delete_dir_entry, 306
Design methodology, 105

© Springer International Publishing AG 2017
K.C. Wang, *Embedded and Real-Time Operating Systems*,
DOI 10.1007/978-3-319-51517-5

Design principles of RTOS, 412
Device driver and interrupt handlers in SMP, 384, 396
Device drivers, 27, 391
Device memory, 362
Device switch table, 315
Disable MMU, 172
Display image files, 33
Display text, 37
Domain Access Control Register (DACR), 169, 173
Dynamic 2-level paging, 238
Dynamic paging, 286
Dynamic priority, 125
Dynamic process creation, 3
Dynamic processes, 121
Dynamic tasks sharing resources, 432

E
Earliest-Deadline-First (EDF) scheduling, 402
ELF executable file, 217
ELF file, 18
Enable MMU, 171
ENQ/DEQ, 141
EOS kernel files, 267
Eoslib, 390
Equivalent states, 107
Escape key, 62
Event-driven model, 97
Event-driven systems, 97
Event-driver embedded systems, 3
Event flags, 140
Event priorities, 103
Event variables, 140
Exception and Signal Processing, 288
Exception handlers, 49, 203
Exceptions vector table, 49
Exec, 3, 221
Execution image, 20
EXT2 file system, 4

F
FAT file system, 214
Fault Address Register, 174
Fault Status Registers, 173
File I/O operations, 296
File locking, 318
File operation levels, 294
File protection, 317
file stream, 297, 301
Finite state machine (FSM), 105
FIQ mode, 9
Flat binary executable file, 217
font file, 37
fork(), 221
Fork-Exec in Unix/Linux, 221
FreeBSD, 265
freeList, 121
FreeRTOS, 404
FSM minimization, 108
FSM state table, 106
Full Descending stacks, 15
Function Call Convention in C, 21

G
General Purpose Input-Output (GPIO), 27
General Purpose Operating System (GPOS), 265
Generic Interrupt Controller (GIC), 2, 4, 330, 332
generic printf() function, 3
GIC programming, 332
Global timers, 357
GoUmode, 379
GPIO programming, 27

H
Hard real-time system, 401
Hardware Interrupt Sequence, 54
Harmonic, 402

I
Immediate Value and Barrel Shifter, 13
Implication chart, 107
Initial ramdisk image, 213
INIT program, 321
Inline assembly, 26
Insert_dir_entry, 305
Instruction pipeline, 10
Instruction Synchronization Barrier (ISB), 351
Intel x86 CPU, 203
Inter-process communication, 3
Inter-Processor Interrupts (IPIs), 329, 330
Inter-processor synchronization, 2
Interrupt controllers, 51
Interrupt Distributor Registers, 332
Interrupt-driven device drivers, 2, 56
Interrupt-driven driver design, 62
Interrupt handlers, 55
Interrupt latency, 401
Interrupt mask, 54, 390
Interrupt priorities, 52
Interrupt priority inversion, 88
Interrupt processing, 51, 54
Interrupts, 2, 51, 329
Interrupt Service Routines (ISRs), 55
Interrupt Set-enable registers, 339
Interrupts routing, 330
Inverse-Deadline Scheduling (IDS), 403
IOAPIC, 329
I/O-bound vs. compute-bound, 124
I/O buffer management, 318, 396
I/O by polling, 29
I/O redirections, 4
IRQ and Exception handlers, 378
IRQ mode, 9

K
Kernel Mapped High (KMH), 193
Kernel Mapped Low (KML), 193
Kernel mode, 297
Kernel Mode Pgdir and Page Tables, 285
Kexit, 122
Keyboard driver, 61
kfork, 122
KMH memory mapping, 241

Kmode, 194
kwait, 220

L
LCD display driver, 33
LDRES/STRES instructions, 4
Library I/O functions, 295, 297
Light-weighted processes, 231
linker script, 18
link-unlink, 308
Linux, 1
LOAD and store instructions, 14
Local timers, 357
Login program, 322
Long jump, 23

M
Makefile, 267, 390
Mailman's algorithm, 339
Mars Pathfinder mission, 403
Mealy model, 105
Memory barriers, 2, 4
Memory management in SMP, 362
Memory Management Unit (MMU), 2, 169
Memory map, 27
Memory-mapped-I/O, 27
Message passing, 147, 283
Microcontroller, 1
MicroC/OS (μC/OS), 405
Microkernel, 147, 407
mkdir-creat-mknod, 306
mk script, 18
MMU registers, 170
Monitors, 142
Moore model, 105
mount_root, 305
Mount-umount, 316
MTX kernel files, 390
Multiply instructions, 14
Multiprocessing, 329
Multiprocessor (MP) System Model, 105
Multiprocessor RTOS (SMP_RTOS), 440
Multi-Sector Data Transfer, 82
Multitasking, 3, 113
Multitasking in SMP, 371
Multitasking kernel, 2
Mutex, 135
Mutex in SMP, 353

N
Named pipes in Linux, 143
Nested Interrupts, 87
Nested Interrupts in SMP_RTOS, 458
Nested interrupts processing, 3
NetBSD, 265
Non-Nested Interrupt Handler, 55
Non-preemption, 125
Non-preemptive UP Kernel, 152
Nonrestrictive preemption, 158
Non-shared memory, 362
Non-uniform VA Spaces, 365
NuttX, 405

O
Objcopy, 18
Objdump, 18
One-Level Paging, 178
One-Level Paging with High VA Space, 184
Open-close-lseek, 310
Opendir-Readdir, 315
Operating Systems, 6

P
P, 133
Page descriptors, 169
Page Table Descriptor, 175
Parallel algorithms, 2
Parallel Computing, 368
Periodic events, 98
Peripheral Base Address, 330
Peripheral timers, 357
Pipes, 142
PL050 KMI, 17
PL110 LCD controller, 17
PL181 MultiMedia Card Interface, 17
PL011 UART, 17
PL190 Vectored Interrupt Controller, 17
PMTX, 266
Porting Linux to ARM, 266
POSIX Compliant APIs, 410
Preemption, 125
Preemptive Task Scheduling in SMP_RTOS, 461, 465
Preemptive task switching, 401
Preemptive UP Kernel, 158
Primary and Secondary Interrupt Controllers, 53
Priority Ceiling, 404
Priority inheritance, 164, 404, 468
Priority inversion, 164, 403
Process concept, 113
Process Control Block (PCB), 113
Process Cooperation, 135
Processes with Individual Domains, 212
Process Family Tree, 220
Process Image File Loader, 216
Process management, 3
Process Management in EOS, 269
Process Management in SMP_EOS, 391
Process Models, 103
Process scheduler, 125
Process Scheduling, 124, 125
Process Synchronization, 126
Process Termination, 218, 219
PROC structure, 113, 115
Producer-Consumer Problem, 135, 136
PSR Transfer Instructions, 15
Pthreads, 141

Q
QEMU, 2
QEMU monitor commands, 19
QNX, 407

R
Race condition, 349
RAM Disk, 213

Raspberry PI, 265
Raspberry PI-2, 4
Raspbian, 265
Rate-Monotonic Scheduling (RMS), 402
Raw data section, 213
Reactive systems, 97
Readelf, 18
Reader-Writer Problem, 136, 137
Read_pipe, 144
Read-Write Special Files, 315
ReadyQueue, 121, 395
Real and Effective uid, 317
Real-time Linux, 407
Real-time Linux Patches, 408
Real-time operating system (RTOS), 2, 4, 401
Real-Time (RT) System Model, 105
Real-time systems, 1
Realview-pbx-a9, 331
Reduced Instruction Set Computing (RISC), 7
Reset_handler, 378
Resource management, 2
Response time vs. throughput, 125
Restrictive preemption, 158
Rmdir, 308
Round-robin, 125
RTLinux, 408
Run-time stack usage, 2

S
Saved program status register (SPSR), 9
Scan codes, 62
Schedulability, 402
Schedulability and Deadline, 438
Scheduler, 115, 401
SD Card Driver, 74
SD Card Protocols, 74
SDC Driver in SMP, 384
SDC File System, 248
Section Descriptor, 175
Secure Digital (SD) Cards, 73
Semaphores, 133, 359
Send EOI, 55
Serial I/O, 29
Serial Peripheral Interface (SPI), 74
SEV (Send-Event), 351
SGI register (GICD_SGIR), 330
Shared memory, 3, 142, 362
Sh Program, 322
Signal catcher, 288, 289, 291, 325
Signal Handling in EOS Kernel, 290
Signal processing, 2
Signals, 147, 288, 289
Sleep and Wakeup, 126
Sleep/wakeup in SMP, 392
SMP Booting on Virtual Machines, 341
SMP_EOS, 389
SMP_RTOS, 2
Snoop Control Unit (SCU), 332
Soft real-time system, 401
Software Generated Interrupts (SGIs), 2, 330
Software Interrupt (SWI), 15
Spinlocks, 390, 391
Stack and Subroutines, 15
Stack frame, 23
Stack frame pointer, 23
Stack Operations, 14
Standard C startup file crt0, 224
Startup Sequence of ARM MPcores, 340
StateChart Model, 110
State diagram, 109
State Machine Model, 105
State table minimization, 107
Static 2-Level Paging, 236
Status Registers, 9
Super-Loop Model, 95
Super-loop system model, 3
Survey of RTOS, 404
SVC_entry, 379
SVC Handler, 202
SVC mode, 9
SVGA mode, 33
SWAP, 350
Switch Pgdir, 286, 379
symlink-readlink, 309
Symmetric Multiprocessing (SMP), 2
Synchronous Message Passing, 148
Syscall, 194
SYS_FLAGSSET register, 330
System call, 2, 194, 295
System Call Entry and Exit, 202

T
Target CPU registers, 339
Task Control Block (TCB), 113, 404
Task deadline, 401
Task scheduling algorithms, 2
Task Scheduling in RTOS, 401
TCP/IP Telnet, 33
Test-and-Set (TS), 349
Threads, 3, 231
Threads Synchronization, 234
Timer Driver, 56
Timer Request Queue, 292, 293
Time-sharing, 125
Timesliced Task Scheduling in SMP_RTOS, 463
Translation Lookaside Buffer (TLB), 169
Translation of Large Page References, 178
Translation of Page References, 176
Translation of Section References, 176
Translation of Small Page References, 177
Translation Table, 175
Translation Table Base Register (TTBR), 173
Translation table walking logic, 169
Trap stack frame, 291
Tswitch, 378
Two-Level Paging, 183
Two-Level paging scheme, 169
Two-Level Paging with High VA Space, 189
Two-Stage Booting, 260

U
UART Driver, 29, 67
Ubuntu (15.10) Linux, 2
Umode, 194
Unbounded priority inversion, 164, 403
UND mode, 9
Uniform VA Spaces, 363
Uniprocessor RTOS (UP_RTOS), 413
Uniprocessor (UP) Kernel Model, 3, 104

Uniprocessor (UP) systems, 2
UP_RTOS, 2
UP_RTOS with Static Periodic Tasks, 416, 426
User Commands, 295
User interface, 2, 321
User Mode Image, 194
User Mode Page Tables, 286
User Mode Processes, 193

V
V, 133
Vectored Interrupts, 84
Vector Interrupt Controllers (VICs), 3
Vfork, 3, 228
VGA mode, 33
Virtual Address Space Mapping, 193
Virtual Address Translations, 174
Virtual machines, 2
VxWorks, 406

W
Wait-for-Event (WFE), 351
Wait-for-Interrupts (WFI), 351
Write_pipe, 144
Write Regular Files, 313

X
XCHG/SWAP Operations, 350

Z
ZOMBIE child, 220, 270